FPT UNIVERSITY

FPT UNIVERSITY
SoftwareDesignDocument

Frevia

GroupBar	
Group Members	Pham Kha Vy - CE182254 Nguyen Tien Thuan - CE181024 Tran Phat Tai - CE181632 Vo Nhut Tin - CE180619 Nguyen Lu Nhat Huy - CE181196
Supervisor	Mrs. Cao Hong Nga
Ext Supervisor	
Capstone Project code	FRE

- Can Tho, October 2026 -

Table of Contents

1 Overall Description **7**

1.1 Assumptions 7

1.2 Design Constraints 8

1.2.1 Platform Compatibility 8

1.2.2 Integration Complexity 8

1.2.3 Performance Requirements 8

1.3 Technology Suggestion 9

1.3.1 NestJS Framework 10

1.3.2 Next.js and React 19 Frontend 10

1.3.3 Prisma ORM and PostgreSQL 10

1.3.4 Supporting Services 10

2 System Architecture Design **11**

2.1 Overall Architecture 11

2.2 System Architecture 12

2.2.1 API Module Structure 12

2.3 Package Diagram 14

3 System Detailed Design **15**

3.1 Forum Management 15

3.1.1 UC-14.01 – View Forum Category List 17

3.1.2 UC-14.02 – View Forum Category Detail 18

3.1.3 UC-14.03 – View Post List 19

3.1.4 UC-14.04 – View Post Detail 20

3.1.5 UC-14.05 – Create Post 21

3.1.6 UC-14.06 – Update Post 22

3.1.7 UC-14.07 – Delete Post 23

3.1.8 UC-14.08 – Like Post 24

3.1.9 UC-14.09 – Unlike Post 25

3.1.10 UC-14.10 – View Post Likes 26

3.1.11 UC-14.11 – View Post Comments 27

3.1.12 UC-14.12 – Create Comment 28

3.1.13 UC-14.13 – Update Comment 29

3.1.14 UC-14.14 – Delete Comment 30

3.1.15 UC-14.15 – Like Comment 31

3.1.16 UC-14.16 – Unlike Comment 32

3.1.17 UC-14.17 – Report Content 33

3.2 Contract Management 34

3.2.1 UC-13.01 – Create Contract . . . 35
3.2.2 UC-13.02 – Update Contract . . . 36
3.2.3 UC-13.03 – Complete Contract . . . 37
3.2.4 UC-13.04 – View Contract List . . . 38
3.2.5 UC-13.05 – View Contract Detail . . . 39
3.2.6 UC-13.06 – Sign Contract . . . 40
3.2.7 UC-13.07 – Cancel Contract . . . 41
3.2.8 UC-13.08 – View Contract Files . . . 42
3.2.9 UC-13.09 – Upload Contract File . . . 43
3.2.10 UC-13.10 – Delete Contract File . . . 44
3.3 Authentication . . . 45
3.3.1 UC-01 – Login with Google . . . 46
3.3.2 UC-02 – Login . . . 47
3.3.3 UC-03 – Logout . . . 48
3.3.4 UC-04 – Register . . . 49
3.3.5 UC-05 – Forgot Password . . . 50
3.4 Role Management . . . 50
3.4.1 UC-06.01 – View Role List . . . 52
3.4.2 UC-06.02 – View Role Detail . . . 53
3.4.3 UC-06.03 – Create Role . . . 54
3.4.4 UC-06.04 – Update Role . . . 55
3.4.5 UC-06.05 – Delete Role . . . 56
3.4.6 UC-06.06 – Update Role Permissions . . . 57
3.5 Manage Conversation . . . 57
3.5.1 UC-27.01 – View Conversation List . . . 59
3.5.2 UC-27.02 – Start Conversation . . . 60
3.5.3 UC-27.03 – View Conversation Detail . . . 61
3.5.4 UC-27.04 – Send Message . . . 62
3.5.5 UC-27.05 – Mark Message as Read . . . 63
3.5.6 UC-27.06 – Delete/Recall Message . . . 64
3.5.7 UC-27.07 – Pin/Unpin Conversation . . . 65
3.5.8 UC-27.08 – Send File . . . 66
3.6 Milestone Management . . . 66
3.6.1 UC-24.01 – View Milestone List . . . 68
3.6.2 UC-24.02 – View Milestone Detail . . . 69
3.6.3 UC-24.03 – Create Milestone . . . 70
3.6.4 UC-24.04 – Update Milestone . . . 71
3.6.5 UC-24.05 – Delete Milestone . . . 72

3.6.6 UC-24.06 – Request Milestone Changes . . . 73
3.6.7 UC-24.07 – Approve Milestone . . . 74
3.6.8 UC-24.08 – Update Milestone Progress . . . 75
3.7 Milestone File Management . . . 76
3.7.1 UC-25.01 – View Milestone Files . . . 77
3.7.2 UC-25.02 – Upload Milestone File . . . 78
3.7.3 UC-25.03 – Delete Milestone File . . . 79
3.8 Milestone Submission Management . . . 80
3.8.1 UC-26.01 – Submit Milestone . . . 81
3.8.2 UC-26.02 – Delete Milestone Submission . . . 82
3.8.3 UC-26.03 – View Milestone Submission List . . . 83
3.8.4 UC-26.04 – View Milestone Submission Detail . . . 84
3.9 Permission Management . . . 85
3.9.1 UC-15.01 – View Permission List . . . 86
3.9.2 UC-15.02 – View Permission Detail . . . 87
3.10 Session Management . . . 87
3.10.1 UC-16.01 – View Session List . . . 89
3.10.2 UC-16.02 – View Session Detail . . . 90
3.10.3 UC-16.03 – Revoke Session . . . 91
3.11 Forum Category Management . . . 92
3.11.1 UC-17.01 – Create Forum Category . . . 93
3.11.2 UC-17.02 – Update Forum Category . . . 94
3.11.3 UC-17.03 – Delete Forum Category . . . 95
3.11.4 UC-17.06 – Restore Forum Category . . . 96
3.11.5 UC-14.01 – View Forum Category List . . . 97
3.11.6 UC-14.02 – View Forum Category Detail . . . 98
3.12 Manage Moderation . . . 98
3.12.1 UC-34.01 – View Moderation Queue . . . 99
3.12.2 UC-34.02 – View Moderation Item Detail . . . 100
3.12.3 UC-34.03 – Resolve Moderation Item . . . 101
3.13 Manage ID Verification . . . 102
3.13.1 UC-35.01 – View ID Verification Request List . . . 103
3.13.2 UC-35.02 – View ID Verification Request Detail . . . 104
3.13.3 UC-35.03 – Approve ID Verification . . . 105
3.13.4 UC-35.04 – Reject ID Verification . . . 106
3.14 Manage Forum Trash . . . 107
3.14.1 UC-36.01 – View Deleted Posts/Comments List . . . 108
3.14.2 UC-36.02 – View Deleted Posts/Comments Detail . . . 109

3.14.3 UC-36.03 – Restore Deleted Posts/Comments . . . 110
3.15 Dispute Management . . . 111
3.15.1 UC-33.01 – Open Dispute . . . 112
3.15.2 UC-33.02 – View Dispute List . . . 113
3.15.3 UC-33.03 – View Dispute Detail . . . 114
3.15.4 UC-33.04 – Respond to Dispute . . . 115
3.15.5 UC-33.05 – Respond to Dispute Resolution . . . 116
3.15.6 UC-33.06 – Propose Dispute Resolution . . . 117
3.16 User Management . . . 118
3.16.1 UC-28.01 – View User List . . . 119
3.16.2 UC-28.02 – View User Detail . . . 120
3.16.3 UC-28.03 – Create User . . . 121
3.16.4 UC-28.04 – Update User . . . 122
3.16.5 UC-28.05 – Update Client Profile . . . 123
3.16.6 UC-28.06 – Update Freelancer Basic Info . . . 124
3.16.7 UC-28.07 – Update Freelancer Skills . . . 125
3.16.8 UC-28.08 – Update Freelancer Portfolio . . . 126
3.17 Payment Management . . . 127
3.17.1 UC-43.01 – Connect Stripe Account . . . 128
3.17.2 UC-43.02 – View Payout History . . . 129
3.17.3 UC-43.03 – View Payout Detail . . . 130
3.17.4 UC-43.04 – Fund Milestone . . . 131
3.17.5 UC-43.05 – Refund Payment . . . 132
3.17.6 UC-43.06 – Pay Platform Fee . . . 133
3.17.7 UC-43.07 – Release Payment . . . 134
3.17.8 UC-43.08 – Pay Dispute Fee . . . 135
3.17.9 UC-43.09 – View Payment History . . . 136
3.17.10 UC-43.10 – View Payment Detail . . . 137
3.17.11 UC-43.11 – Execute Dispute Resolution Payment . . . 138
3.17.12 UC-43.12 – Handle Stripe Webhook Event . . . 139
3.17.13 UC-43.13 – Sync Payment Intent Status . . . 140
3.18 Banner Management . . . 141
3.18.1 UC-37.01 – View Banner List . . . 142
3.18.2 UC-37.02 – View Banner Detail . . . 143
3.18.3 UC-37.03 – Create Banner . . . 144
3.18.4 UC-37.04 – Update Banner . . . 145
3.18.5 UC-37.05 – Delete Banner . . . 146
3.19 Skill Management . . . 147

3.19.1 UC-38.01 – View Skill List . . . 148
3.19.2 UC-38.02 – View Skill Detail . . . 149
3.19.3 UC-38.03 – Create Skill . . . 150
3.19.4 UC-38.04 – Update Skill . . . 151
3.19.5 UC-38.05 – Delete Skill . . . 152
3.19.6 UC-38.06 – Restore Skill . . . 153
3.20 Expert Profile Management . . . 153
3.20.1 UC-28.09 – Update Expert Profile . . . 155
3.21 Job Category Management . . . 156
3.21.1 UC-46.01 – View Job Category List . . . 157
3.21.2 UC-46.02 – View Job Category Detail . . . 158
3.21.3 UC-46.03 – Create Job Category . . . 159
3.21.4 UC-46.04 – Update Job Category . . . 160
3.21.5 UC-46.05 – Delete Job Category . . . 161
4 Class Specification . . . 162
4.1 ForumController Class . . . 162
4.2 ForumService Class . . . 164
4.3 ForumRepository Class . . . 168
4.4 ForumLikeController Class . . . 172
4.5 ForumLikeService Class . . . 172
4.6 ForumLikeRepository Class . . . 173
4.7 ForumCommentController Class . . . 175
4.8 ForumCommentService Class . . . 177
4.9 ForumCommentRepository Class . . . 178
4.10 ForumReportController Class . . . 181
4.11 ForumReportService Class . . . 183
4.12 ForumReportRepository Class . . . 185
Supporting DTO classes for reports . . . 187
4.13 ContractController Class . . . 188
4.14 ContractService Class . . . 190
4.15 ContractRepository Class . . . 192
4.16 SharedFileController Class . . . 194
4.17 SharedFileService Class . . . 195
4.18 SharedFileRepository Class . . . 197
4.19 AuthController Class . . . 198
4.20 AuthService Class . . . 201

4.21 AuthRepository Class 205
4.22 RolesController Class 208
4.23 RolesService Class 209
4.24 RolesRepository Class 211
4.25 ConversationController Class 213
4.26 ConversationService Class 215
4.27 ConversationRepository Class 219
4.28 ConversationsGateway Class 222
4.29 MilestoneController Class 224
4.30 MilestoneService Class 227
4.31 MilestoneRepository Class 228
4.32 MilestoneFileController Class 231
4.33 MilestoneFileService Class 232
4.34 MilestoneFileRepository Class 234
4.35 MilestoneSubmissionController Class 235
4.36 MilestoneSubmissionService Class 237
4.37 MilestoneSubmissionRepository Class 239
4.38 PermissionsController Class 241
4.39 PermissionsService Class 242
4.40 PermissionsRepository Class 243
4.41 SessionsController Class 244
4.42 SessionsService Class 245
4.43 SessionsRepository Class 247
4.44 Forum Category Administration (Shared Classes) 248
4.45 ForumAdminController Class 248
4.46 ForumAdminService Class 252
4.47 ForumAdminRepository Class 256
4.48 IdentityVerificationsAdminController Class 260
4.49 IdentityVerificationsAdminService Class 262
4.50 IdentityVerificationsAdminRepository Class 264
4.51 DisputeController Class 265
4.52 DisputeAdminController Class 268
4.53 DisputeService Class 269
4.54 DisputeRepository Class 273
4.55 UsersController Class 277

4.56 UsersService Class 280
4.57 UsersRepository Class 283
4.58 PaymentController Class 286
4.59 PaymentService Class 291
4.60 PaymentRepo Class 296
4.61 StripeService Class 299
4.62 BannersAdminController Class 303
4.63 BannersAdminService Class 304
4.64 BannersAdminRepository Class 306
4.65 SkillsAdminController Class 307
4.66 SkillsAdminService Class 309
4.67 SkillsAdminRepository Class 311
4.68 AdminExpertProfileController Class 312
4.69 ExpertProfileService Class 313
4.70 ExpertProfileRepository Class 315
4.71 JobCategoriesAdminController Class 316
4.72 JobCategoriesAdminService Class 318
4.73 JobCategoriesAdminRepository Class 319

1 Overall Description

1.1 Assumptions

This system is designed based on the following assumptions:

- The Node.js runtime and the frameworks chosen for the project (NestJS 11 for the back-end and Next.js 16 for the frontend) will continue to be actively supported and maintained, ensuring long-term stability and compatibility with evolving technologies.
- The RESTful API architecture will remain a scalable and efficient choice for separating the Next.js presentation layer from the NestJS business layer.
- PostgreSQL, accessed through Prisma ORM 7, will continue to provide reliable, scalable, and secure relational data storage suitable for handling user identity, jobs, contracts, escrow payments, and forum data.
- Third-party services used by the platform — Stripe (payments and escrow), Cloudinary (file storage), Google (OAuth login), the AI moderation/analysis service, and mail delivery via Nodemailer — will stay available and API-compatible during the project lifetime.
- Modern web browsers (e.g., Chrome, Edge, Firefox, Safari) will continue to support the front-end standards required by Next.js/React 19, including HTML5, CSS3 (TailwindCSS), JavaScript ESM, and responsive rendering with React Server Components.
- The application will be deployed in environments with stable internet access to support real-time messaging (Socket.io) and online payment processing.
- Users (Guests, Clients, Freelancers, Experts, and Admins) are familiar with basic web application navigation: browsing job boards, filling forms, managing profiles, and following payment flows.

This system design is also based on the following risks:

- Any breaking change in the major versions of Next.js, NestJS, or Prisma could require refactoring of the user interface components or of the data access layer.
- Cross-browser inconsistencies or outdated device software may affect rendering or the real-time messaging experience.
- Cybersecurity threats such as credential stuffing, token hijacking, or database breaches could compromise sensitive personal and financial information if not proactively managed (the design therefore includes bcrypt-hashed passwords, JWT access/refresh tokens, session revocation, and role-based access control).

- Outages or policy changes of the integrated third-party services (Stripe, Cloudinary, Google OAuth, AI moderation) could temporarily disrupt payments, file uploads, login, or content moderation.
- Network connectivity issues could disrupt access to job proposals, contract collaboration, or checkout flows that rely on server interaction.

1.2 Design Constraints

This system should comply with the following constraints:

1.2.1 Platform Compatibility

The Frevia application must be compatible with modern web browsers (e.g., Chrome, Edge, Firefox, Safari) for all user roles including Guest, Client, Freelancer, Expert, and Admin. The frontend is implemented with Next.js 16 (App Router, React Server Components) styled with TailwindCSS 4 and shadcn/ui components; the backend is a NestJS 11 REST API. Responsive web design principles must be applied so that a consistent user experience is provided across desktop, tablet, and mobile browsers. Internationalization is achieved with next-intl so the interface can be served in multiple languages from the same code base.

1.2.2 Integration Complexity

The Frevia platform integrates a variety of components: the Next.js web application, the NestJS REST API, the PostgreSQL database (via Prisma 7 with the pg adapter), Stripe Checkout and Stripe Connect for escrow payments and payouts, Cloudinary for image and document storage, Socket.io for real-time chat and notifications, Google OAuth for federated login, e-mail for OTP delivery, and AI services for content moderation, CV parsing, and identity document verification. Communication between the client and the server is carried out through versioned REST endpoints secured with JWT access tokens and refresh tokens stored in dedicated session records; role-based access control restricts each operation to the allowed roles. Design decisions must keep these integrations loosely coupled behind services and repositories so that a provider can be replaced without rewriting business logic.

1.2.3 Performance Requirements

The application must remain responsive under concurrent usage by many freelancers and clients. Job browsing, proposal listing, forum feeds, and conversation histories must load promptly; therefore the design mandates server-side pagination for every list operation, database

indexes on frequently filtered columns, optimized Prisma queries executed inside transactions for counted lists, and streaming React Server Components to reduce time-to-first-byte. Real-time features (messaging, notifications) must propagate events with minimal delay via Socket.io. Payment operations must reconcile eventual states through Stripe webhooks and explicit synchronization endpoints.

1.3 Technology Suggestion

Frevia adopts a RESTful API architecture to support secure, stateless, and scalable communication between the Next.js frontend and the NestJS backend. This approach provides:

- Modular, reusable API endpoints for account management, job posting, proposals, contracting, milestone escrow, dispute resolution, community forum, and administration.
- Easy extension for mobile applications or third-party integrations in the future.
- Secure access using JWT tokens with role-based authorization for every endpoint.
- Uniform use of standard HTTP methods (GET, POST, PATCH, PUT, DELETE), making the system intuitive to maintain.
- A clear separation between the presentation layer and the business logic layer.

All request/response payloads are validated and typed with shared Zod schemas packaged in `@shared/types`, which is consumed by both the frontend and the backend so that a single source of truth prevents contract drift between the two applications.

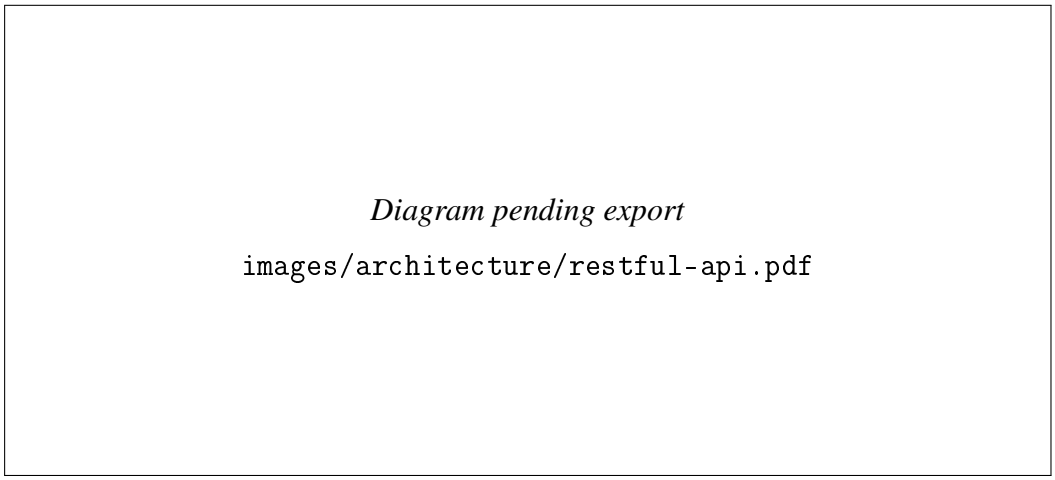

Figure 1: RESTful API communication between the Next.js frontend and the NestJS backend

1.3.1 NestJS Framework

The backend of Frevia is built with NestJS 11, a TypeScript-first, modular web framework running on Node.js. It is selected for its:

- Rich dependency-injection and module ecosystem, matching the Controller–Service–Repository layering.
- First-class decorator-based REST routing with guards and interceptors for authentication, roles, permissions, and response serialization.
- Excellent support for Zod schema validation (nestjs-zod) and typed DTOs.
- Component structure for WebSockets (Socket.io gateways) used by the messaging module.
- Straightforward integration with Prisma ORM and PostgreSQL for persistence.

1.3.2 Next.js and React 19 Frontend

The web client uses Next.js 16 with the App Router. React Server Components render public pages (job lists, job details, forum) directly from the server, while TanStack Query drives interactive islands (forms, likes, real-time chat). react-hook-form together with Zod resolvers validate user input with the same schemas the API uses.

1.3.3 Prisma ORM and PostgreSQL

Persistence relies on PostgreSQL accessed through Prisma 7. The schema defines 62 models covering identity and access, profiles, jobs, proposals, contracts and milestones, payments and disputes, reviews, messaging, the community forum, and platform configuration. Prisma's type-safe client keeps data access code verifiable at compile time.

1.3.4 Supporting Services

Payments and payouts use Stripe Checkout and Stripe Connect with webhook-driven reconciliation; binaries are stored in Cloudinary; emails (OTP, notifications) are sent with Nodemailer; Google login uses OAuth 2.0; environment configuration is loaded through Infisical so no secret is committed to the repository.

2 System Architecture Design

2.1 Overall Architecture

Frevia follows a three-tier, deployment-separated architecture (Figure 2):

- **Presentation tier — Next.js 16 web application:** server components render public pages (job board, job detail, community forum, expert directory) and private dashboards; client components handle interactivity such as forms, likes, and chat. All server access goes through a single typed HTTP client that attaches the JWT access token and refreshes it transparently when it expires.
- **Application tier — NestJS REST API:** a modular monolith organized into 31 route modules. Every module is internally layered as Controller → Service → Repository. Controllers validate and serialize with shared Zod DTOs; services hold business rules and authorization checks; repositories encapsulate all Prisma queries.
- **Data tier — PostgreSQL via Prisma 7:** the relational store for 62 domain models. Binary files are never stored in the database but in Cloudinary; real-time transport for chat and notifications is handled by a Socket.io gateway beside the REST API.

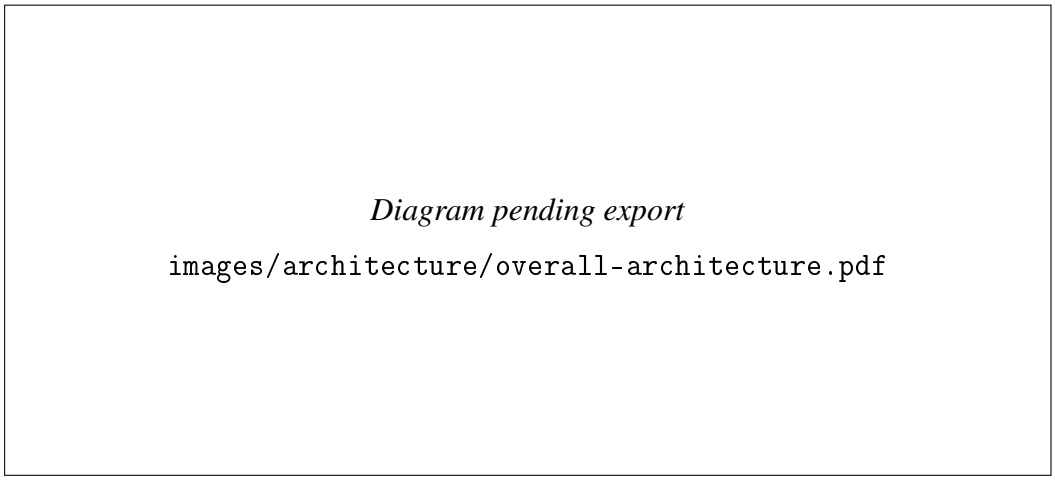

Figure 2: Overall architecture of the Frevia system

The two applications are deployed independently (the web app on Vercel, the API on a Node.js host) and communicate only through the documented REST contract validated by the shared Zod schemas package.

2.2 System Architecture

The internal architecture of the API follows a strict layered pattern, enforced identically in every module (Figure 3):

- **Controller layer** — thin Routing Layer. Exposes REST endpoints, applies guards (authentication, roles, permissions), binds validated request DTOs, and delegates to a service. Controllers never query the database directly.
- **Service layer** — business logic layer. Implements use cases, ownership/authorization checks, state-machine transitions (proposal, contract, dispute), and cross-aggregation logic; it throws typed HTTP exceptions (`NotFoundException 404`, `ForbiddenException 403`, `ConflictException 409`) that a global exception filter converts to a uniform error response.
- **Repository layer** — data access layer. The only component allowed to touch Prisma; complex list queries (pagination + counting) are wrapped in `$transaction` for consistency.
- **Shared kernel** — the `@shared/types` workspace package contains Zod models, constants, permission names, and error codes shared with the web app.

Security flow: every request carries a JWT access token; the auth guard validates it and resolves the active role; role and permission guards authorize module-specific operations; refresh tokens are stored server-side in a `Session` row and can be revoked from the session management feature.

AI-assisted pipeline: user-generated content (forum posts/comments, CVs, identity documents) is passed through AI moderation or AI analysis services; forum content can be auto-approved when clean, or queued in `ModerationQueue` for administrator approval when suspicious.

2.2.1 API Module Structure

Table 1 summarizes the 31 route modules of the API and their responsibilities.

Table 1: API route modules and responsibilities

Module	Responsibility
auth	registration, OTP, login, refresh token, logout, forgot password, Google OAuth, current account, role switching/joining
sessions	device session listing, detail, revocation

Module	Responsibility
account-profile	general account profile, avatar, identity verification upload/AI check, client profile, social links, favorite freelancers, following
users	administrator user management and skill catalog
roles, permissions	RBAC role and permission administration
profiles	professional profile, CV upload/AI analysis, skills
portfolios	portfolio item CRUD on profiles
expert-profile	expert profile management and public expert directory
profile-revisions	profile change request approval workflow
identity-verifications-admin	identity verification review
job-categories	public job category browsing
job-categories-admin	job category administration (soft delete + restore)
skills-admin	skill catalog administration (soft delete + restore)
browse-job	public job listing and job detail by slug
manage-job	client job management, bookmarks, skill search
proposals	proposal lifecycle: draft, submit, withdraw, interview, reject, accept
contract	contract lifecycle: create, update, sign, complete, cancel
milestone	milestone management inside a contract
milestone-file, milestone-submission	milestone attachments and work submission review
shared-file	contract shared workspace files
payments	Stripe Connect onboarding, platform fee, milestone funding, release, refund, dispute fee, transactions, webhook
disputes	dispute lifecycle (user side and admin arbitration side)
reviews	contract reviews and responses
conversations	direct messaging with real-time delivery
notifications	notification feed management
forums	forum categories and posts (public side)
forums-comment, forums-like, forums-reports	comments, likes, content reports
forums-admin	AI moderation queue, pending posts, trash restore, forum statistics
saved-search	saved job search filters
banners, banners-admin	advertisement banner delivery and administration

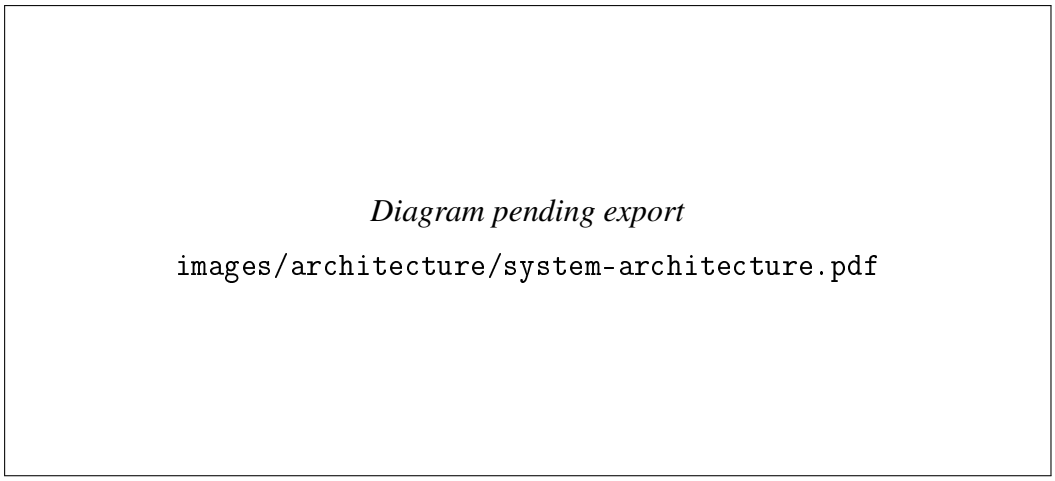

Figure 3: Layered system architecture with security and AI moderation flows

2.3 Package Diagram

The repository is a pnpm monorepo managed by Turborepo with three workspaces (Figure 4):

- `apps/web` — the Next.js application. Sub-packages group screens by actor (guest browsing, client, freelancer, admin), shared UI components (`@repo/ui`), server actions, and the i18n message catalogs used by next-intl.
- `apps/api` — the NestJS application. `src/routes/*` contains the 31 domain modules described above; `src/shared` contains cross-cutting infrastructure: authentication guards and decorators, permission constants, global HTTP exception filter, Zod serializer interceptor, Cloudinary storage service, and Prisma client setup. Each domain module folder is organized as `<module>.controller.ts`, `<module>.service.ts`, `<module>.repository.ts`, DTO classes, and module-specific error factories (e.g., `forums.error.ts`).
- `packages/shared-type` — the single source of truth for API contracts: Zod request/response models (`model/*`), enumeration/constant definitions, and permission name lists, imported by both `apps/web` and `apps/api`.

Dependencies only point downwards: `web` and `api` both depend on `shared-type`; no route module imports another route module — cross-module data needs flow through services at the Prisma schema level.

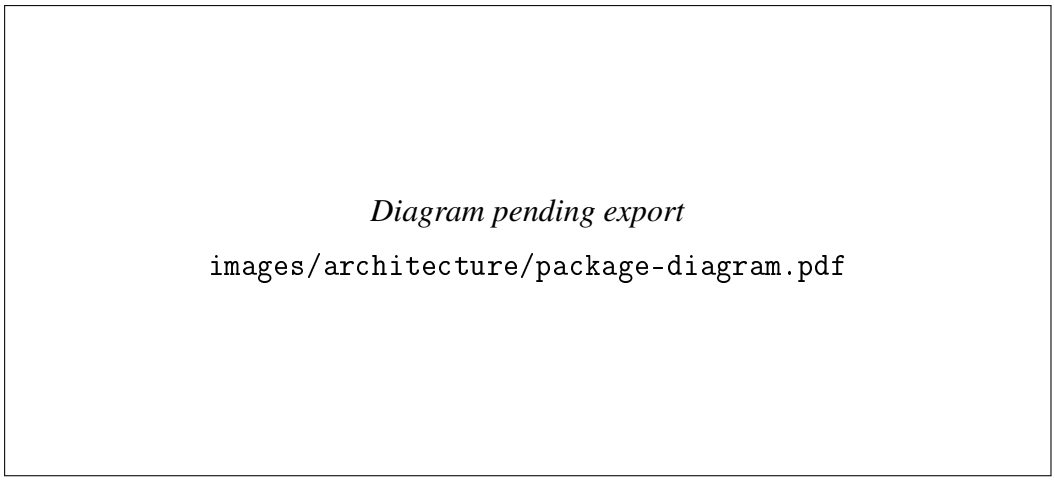

Figure 4: Package diagram of the Frevia monorepo

3 System Detailed Design

3.1 Forum Management

Forum Management covers the UC-14 community forum: browsing forum categories and posts, publishing and maintaining posts and comments, toggling likes on posts and comments, and reporting content that breaks the forum rules.

a. Class Diagram

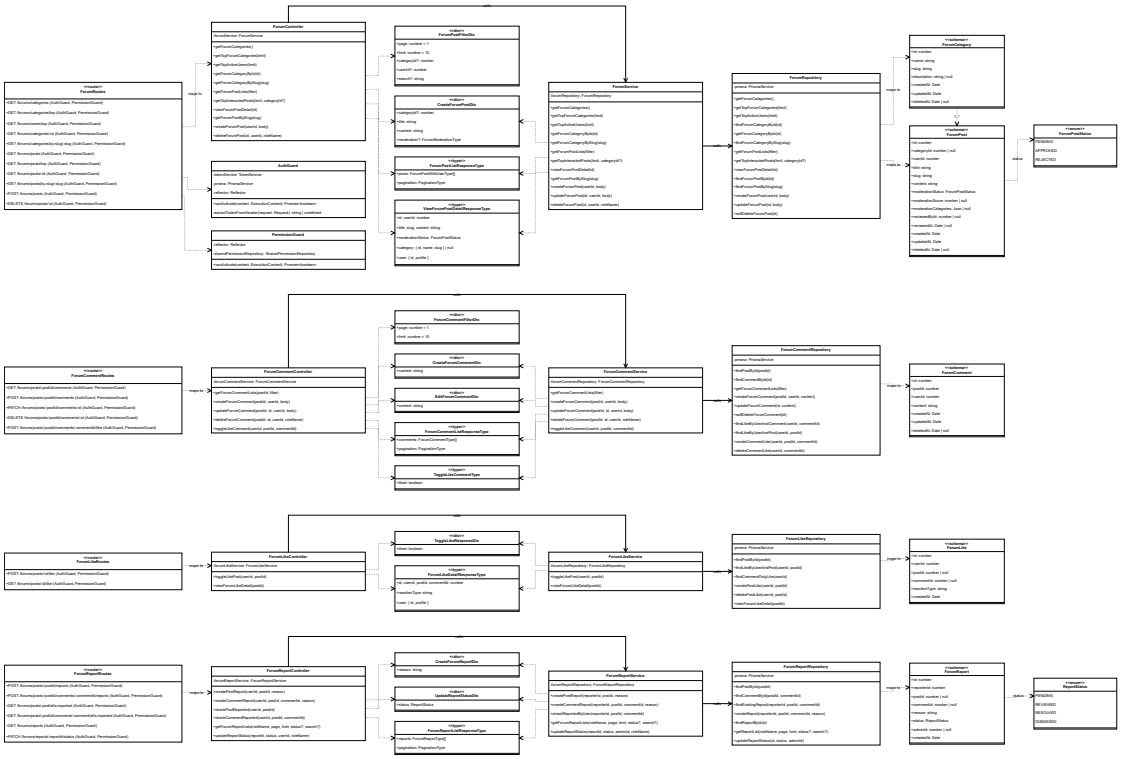

Figure 5: Class diagram of Forum Management

b. Sequence Diagram(s)

3.1.1 UC-14.01 – View Forum Category List

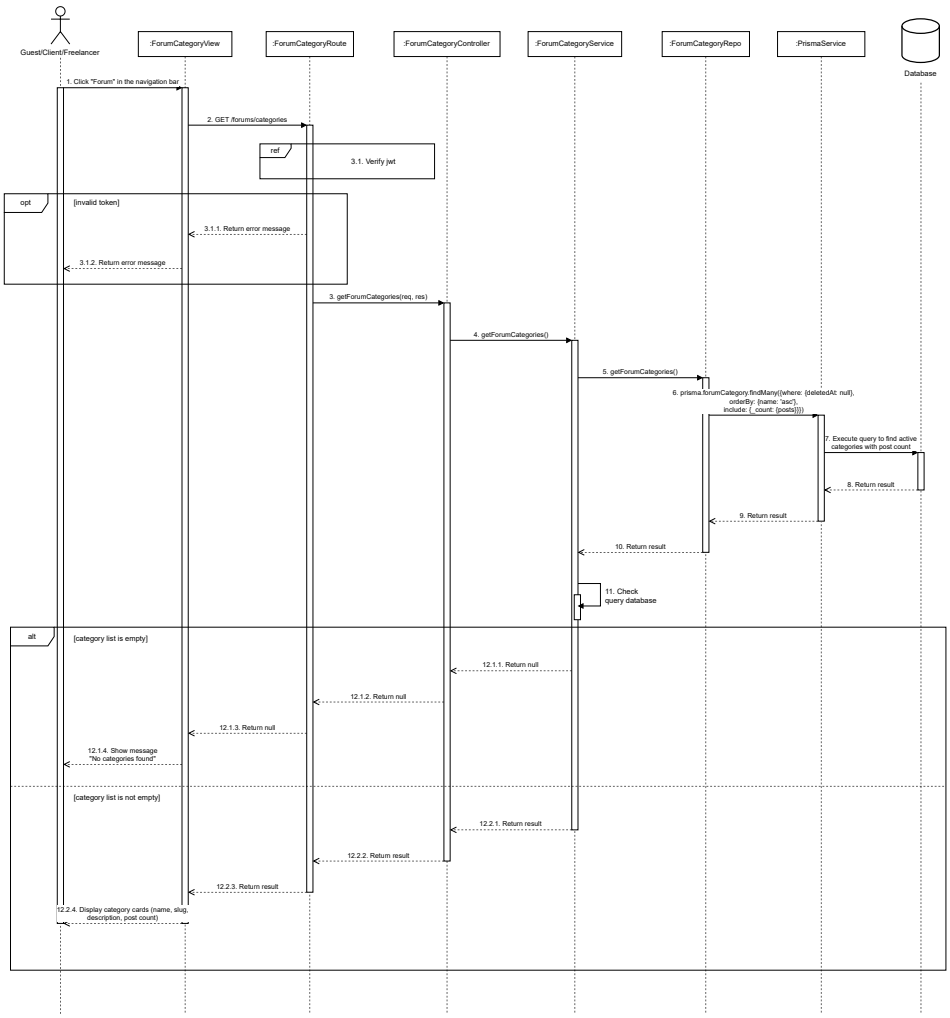

Figure 6: Sequence diagram of UC-14.01 View Forum Category List

3.1.2 UC-14.02 – View Forum Category Detail

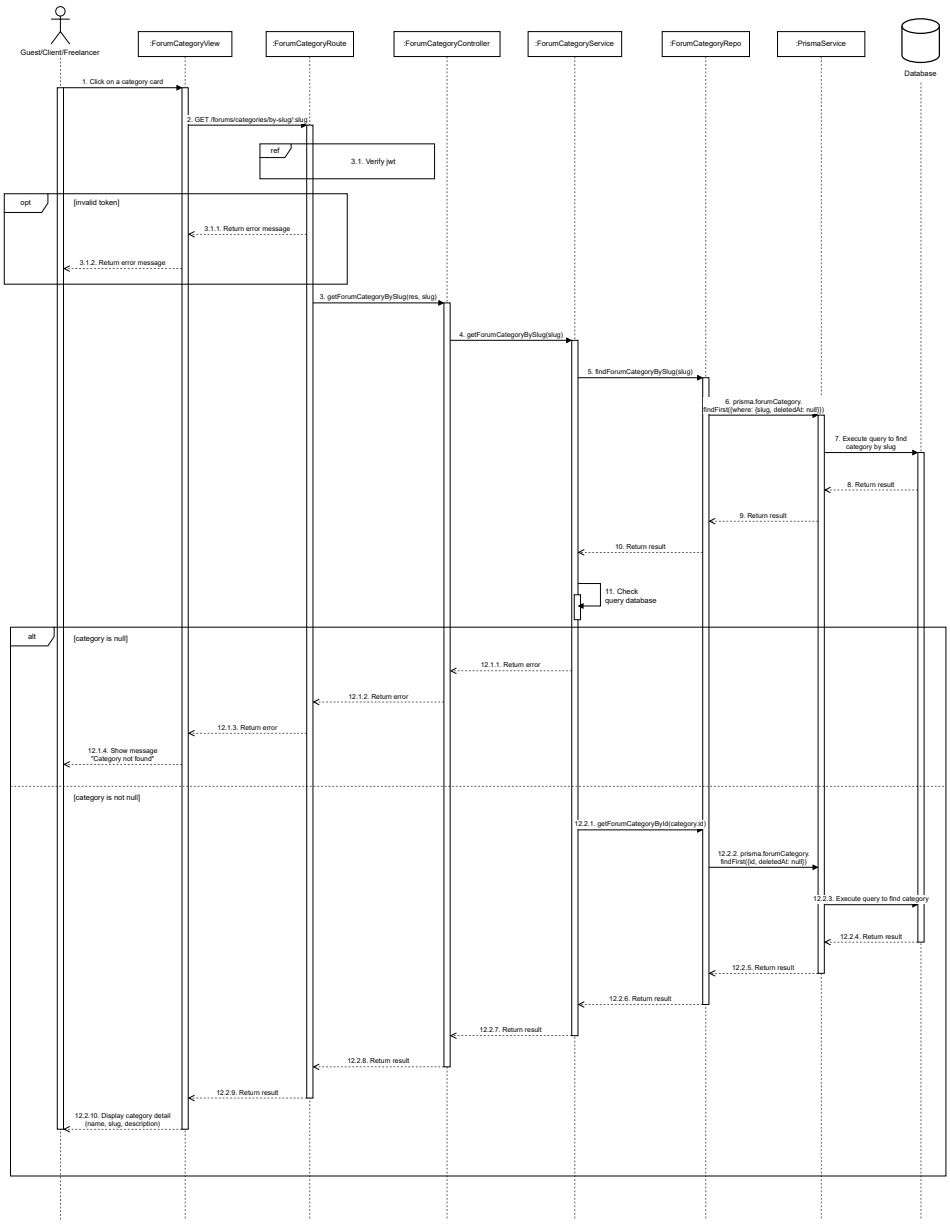

Figure 7: Sequence diagram of UC-14.02 View Forum Category Detail

3.1.3 UC-14.03 – View Post List

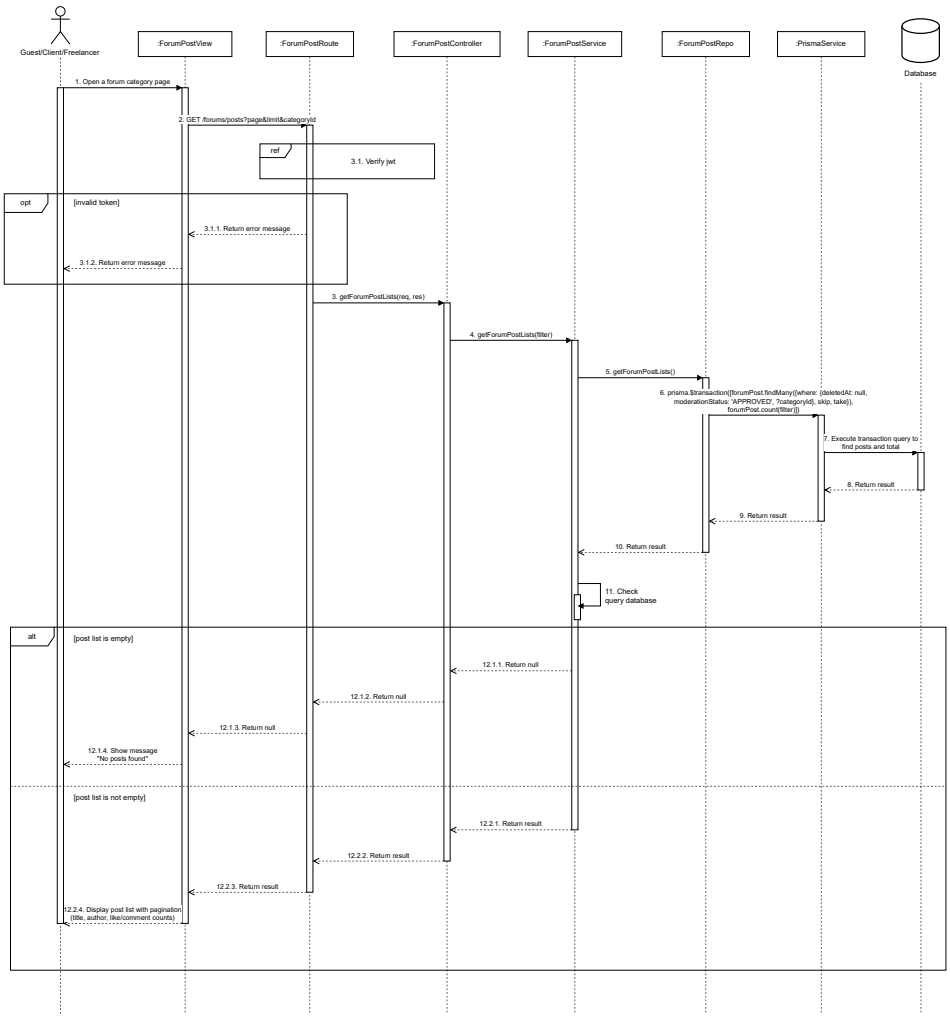

Figure 8: Sequence diagram of UC-14.03 View Post List

3.1.4 UC-14.04 – View Post Detail

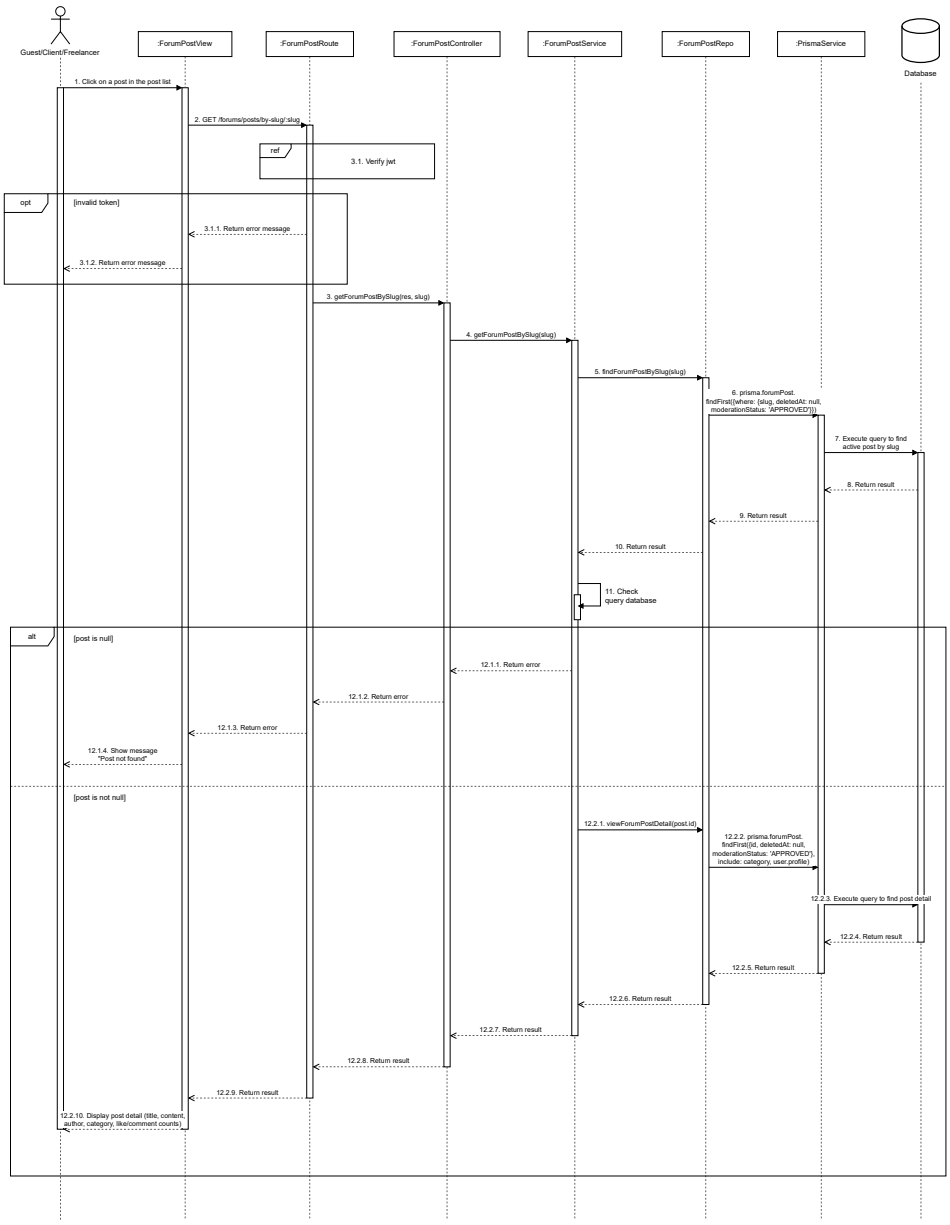

Figure 9: Sequence diagram of UC-14.04 View Post Detail

3.1.5 UC-14.05 – Create Post

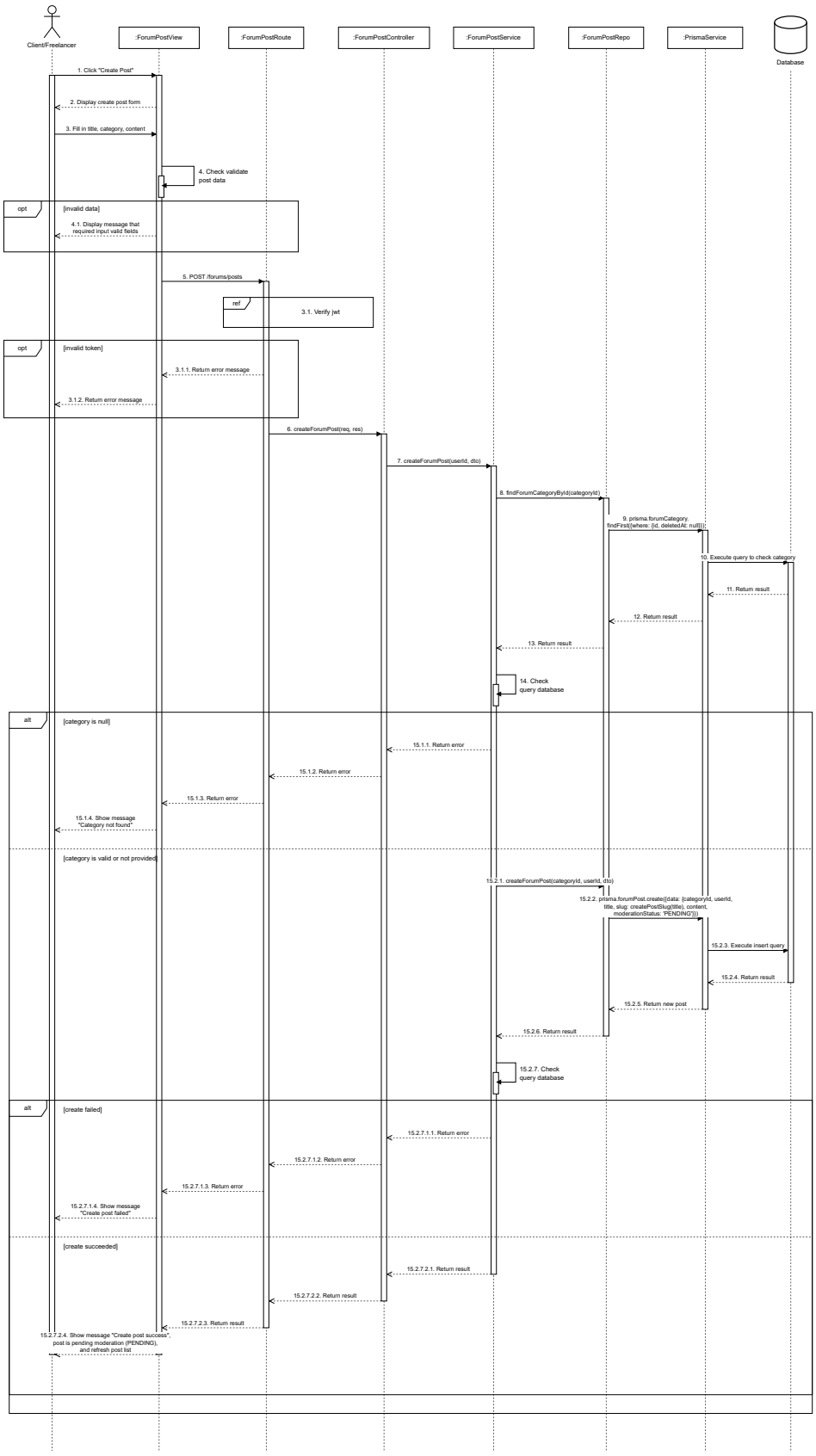

Figure 10: Sequence diagram of UC-14.05 Create Post

3.1.6 UC-14.06 – Update Post

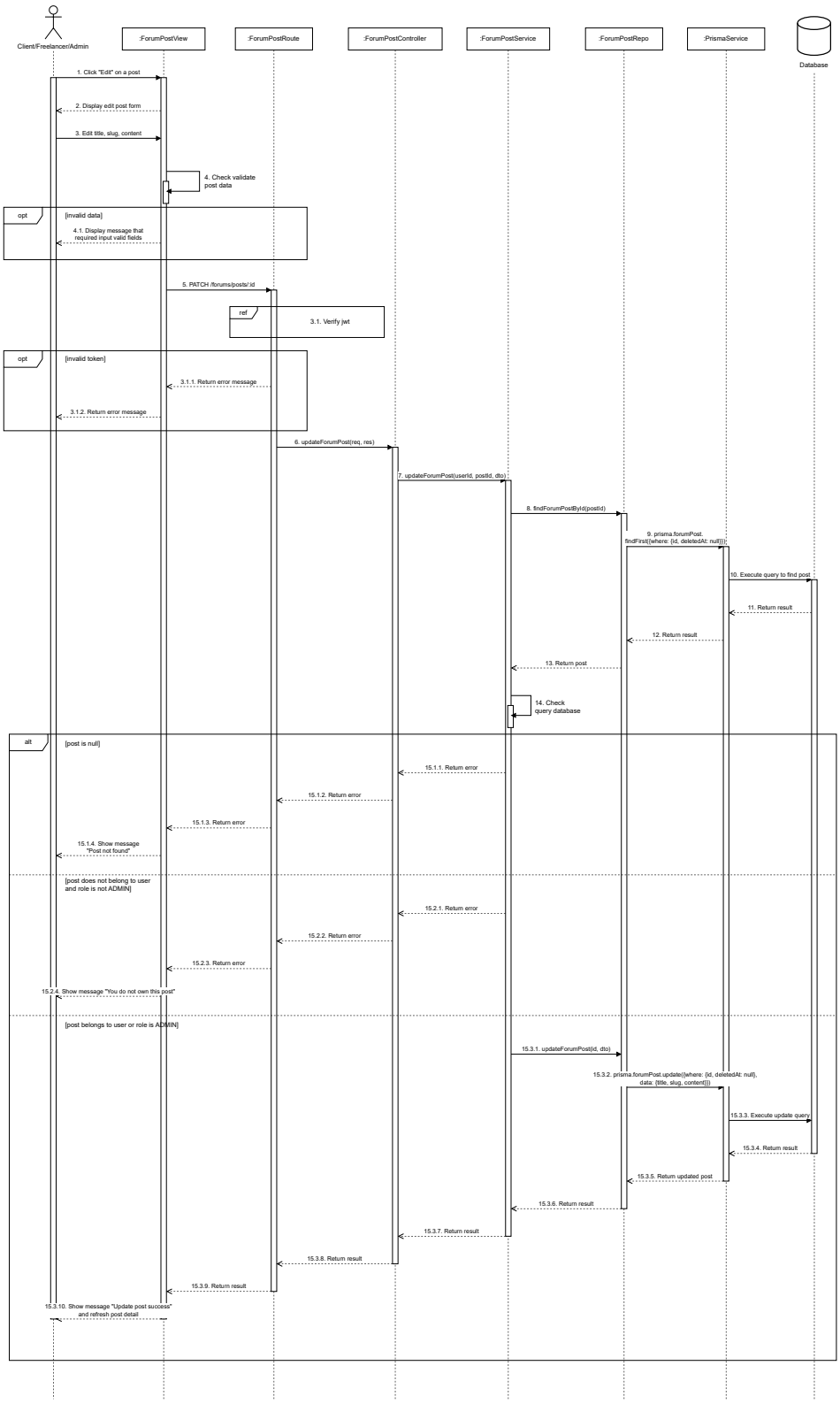

Figure 11: Sequence diagram of UC-14.06 Update Post

3.1.7 UC-14.07 – Delete Post

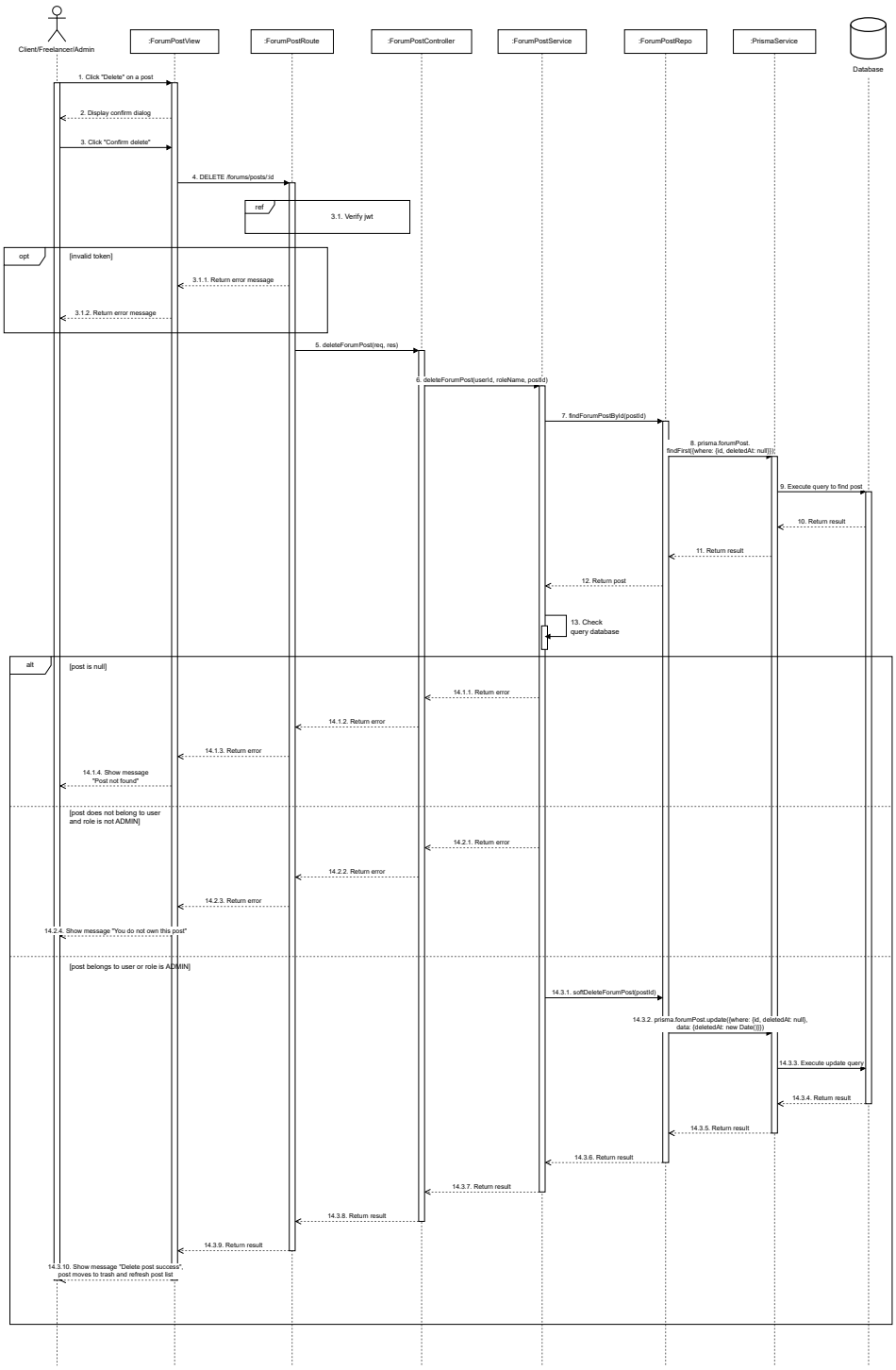

Figure 12: Sequence diagram of UC-14.07 Delete Post

3.1.8 UC-14.08 – Like Post

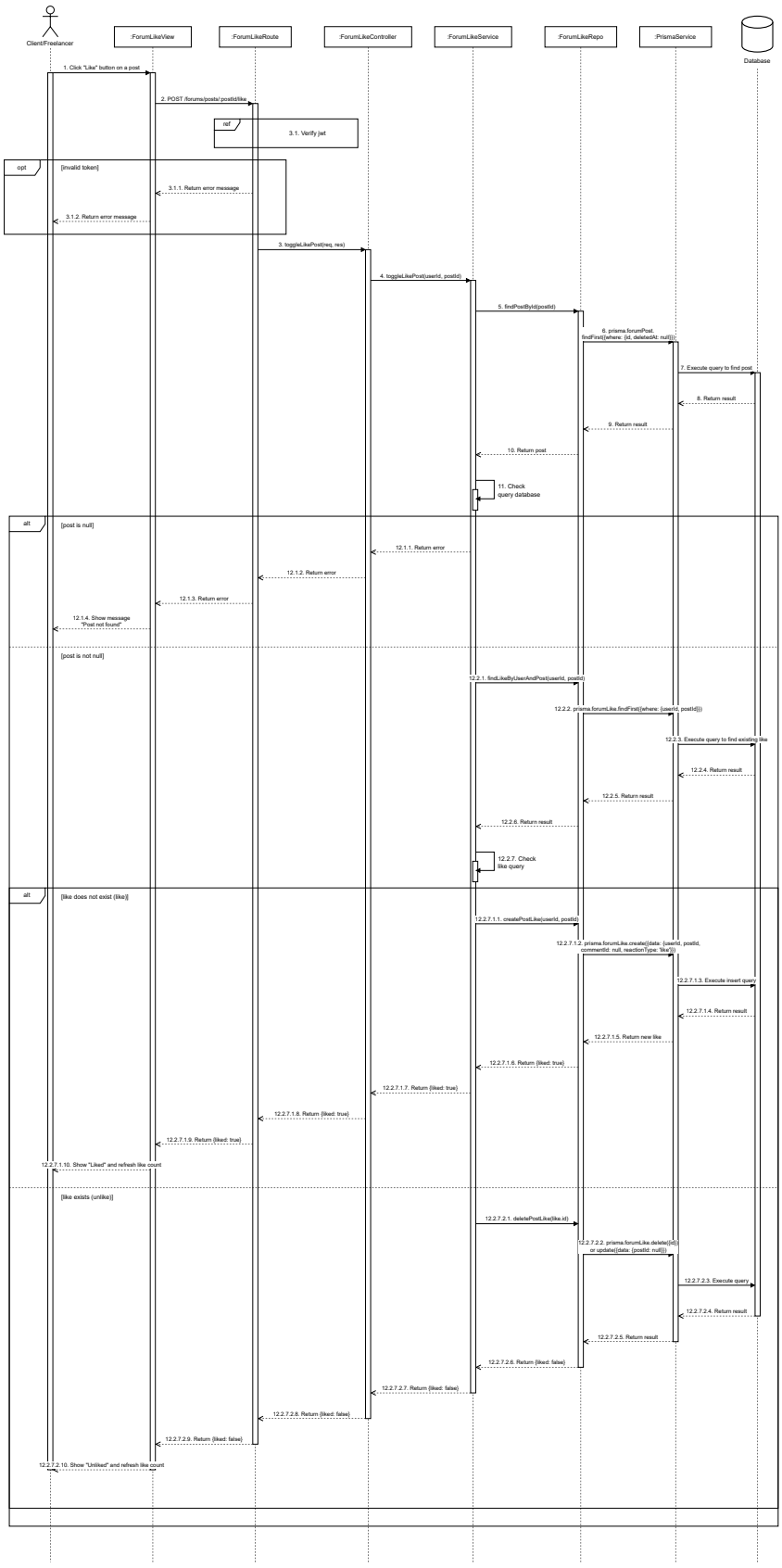

Figure 13: Sequence diagram of UC-14.08 Like Post

3.1.9 UC-14.09 – Unlike Post

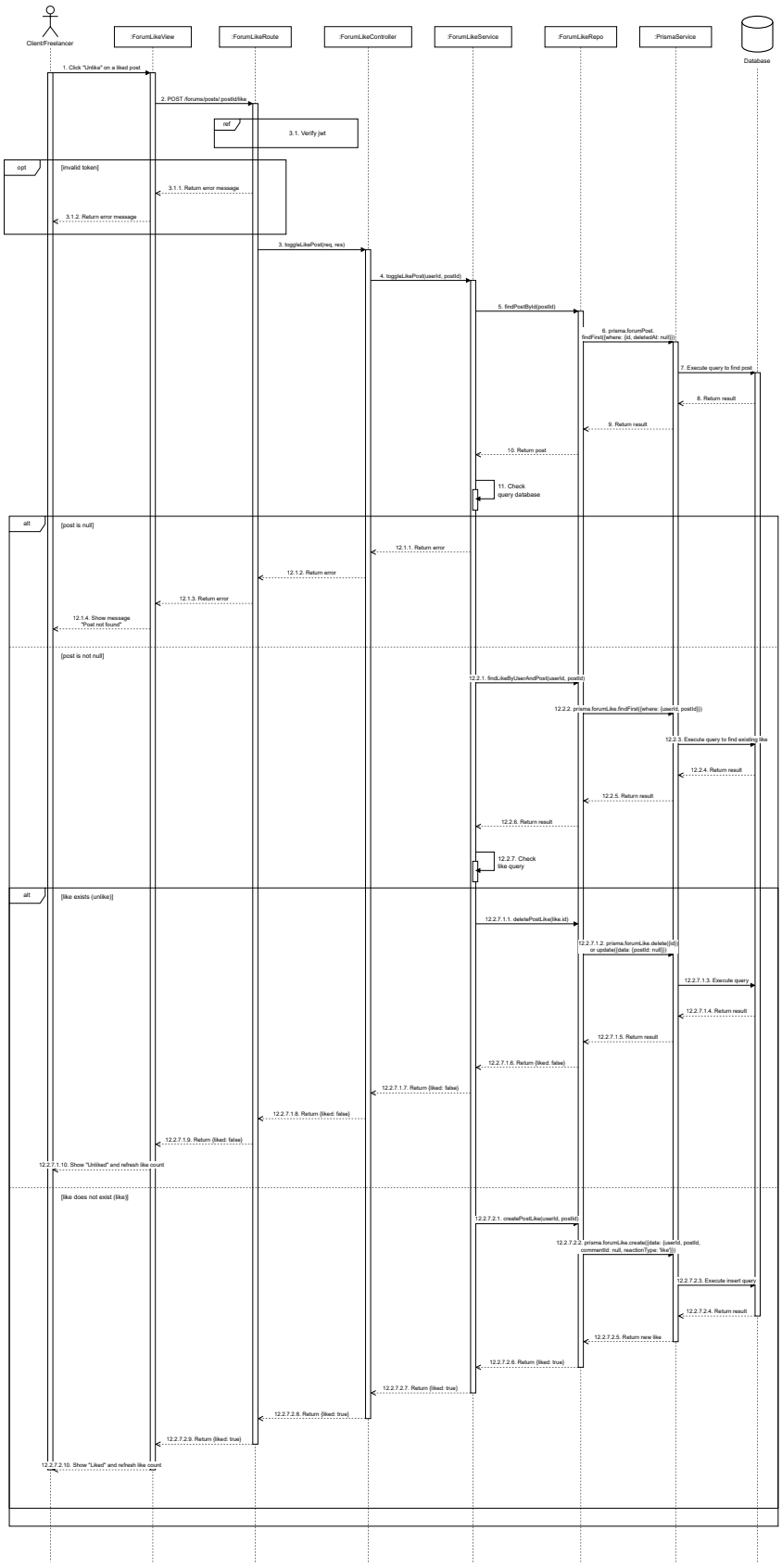

Figure 14: Sequence diagram of UC-14.09 Unlike Post

3.1.10 UC-14.10 – View Post Likes

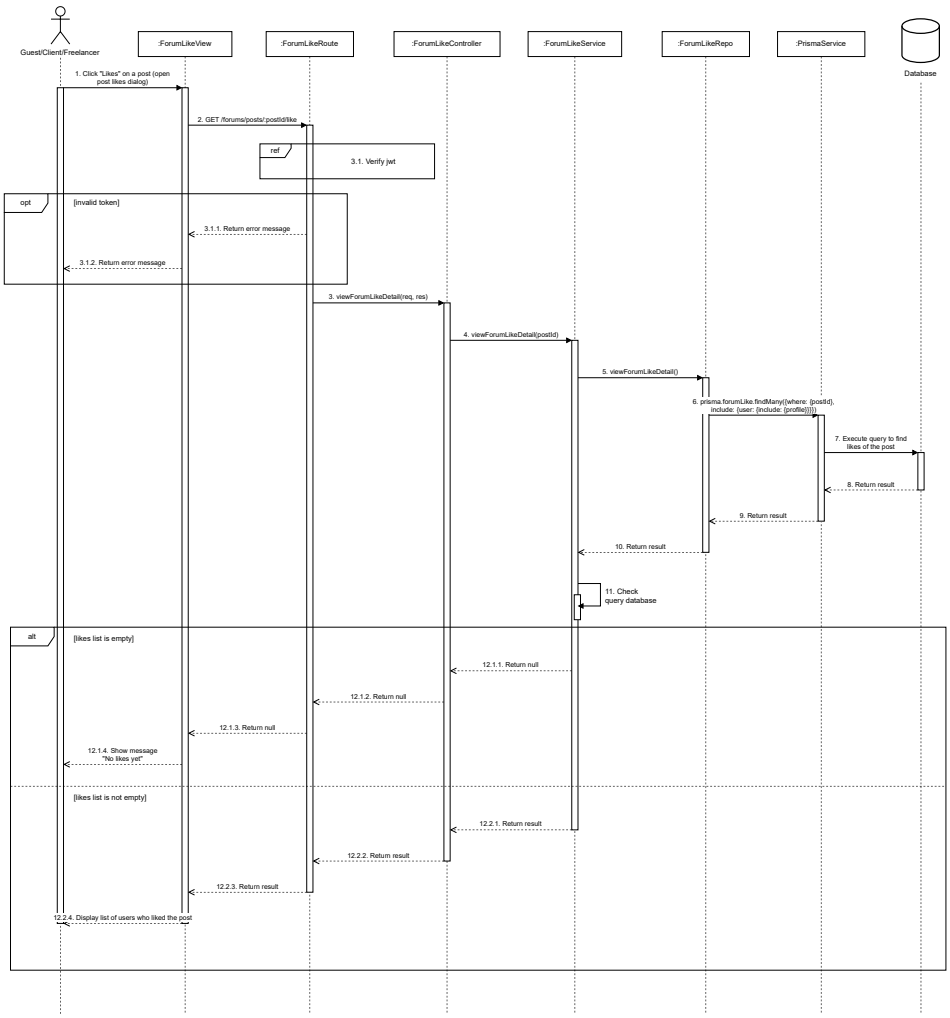

Figure 15: Sequence diagram of UC-14.10 View Post Likes

3.1.11 UC-14.11 – View Post Comments

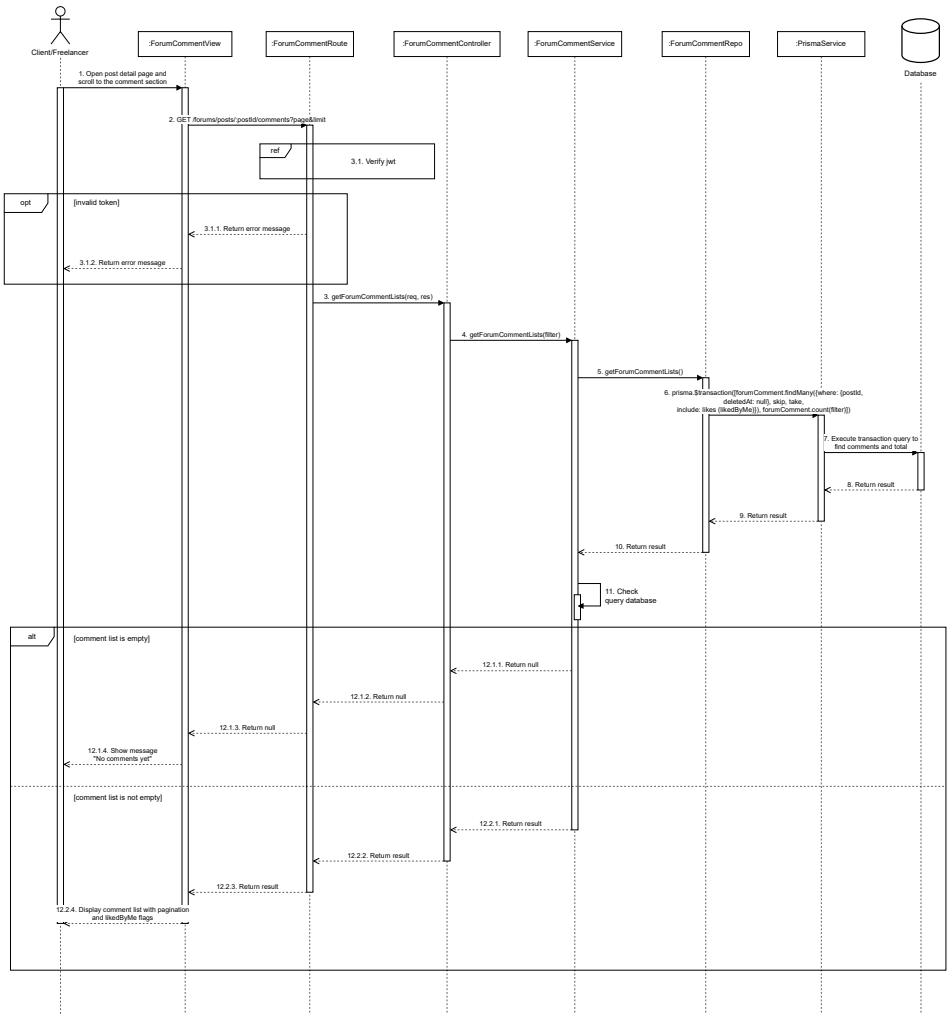

Figure 16: Sequence diagram of UC-14.11 View Post Comments

3.1.12 UC-14.12 – Create Comment

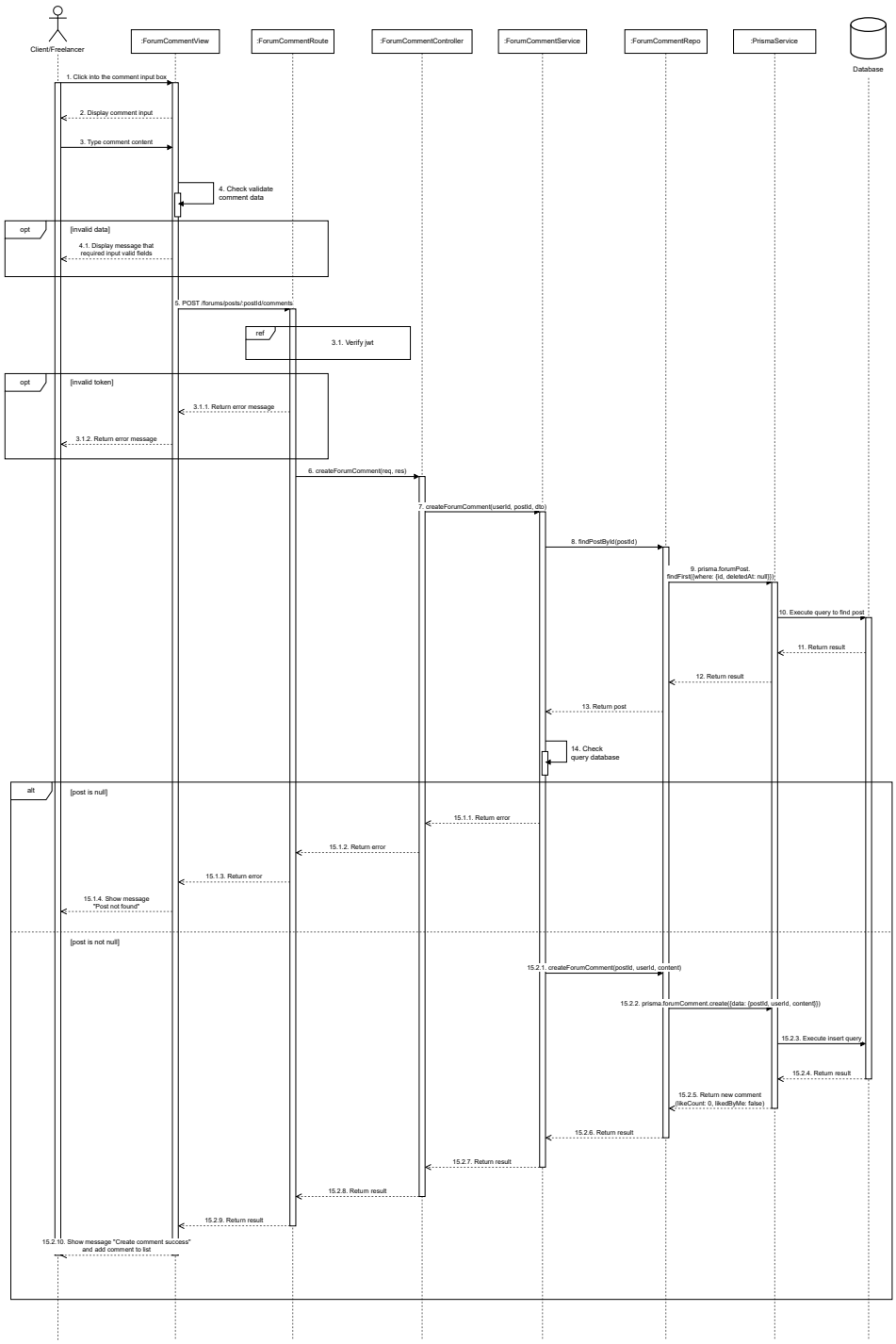

Figure 17: Sequence diagram of UC-14.12 Create Comment

3.1.13 UC-14.13 – Update Comment

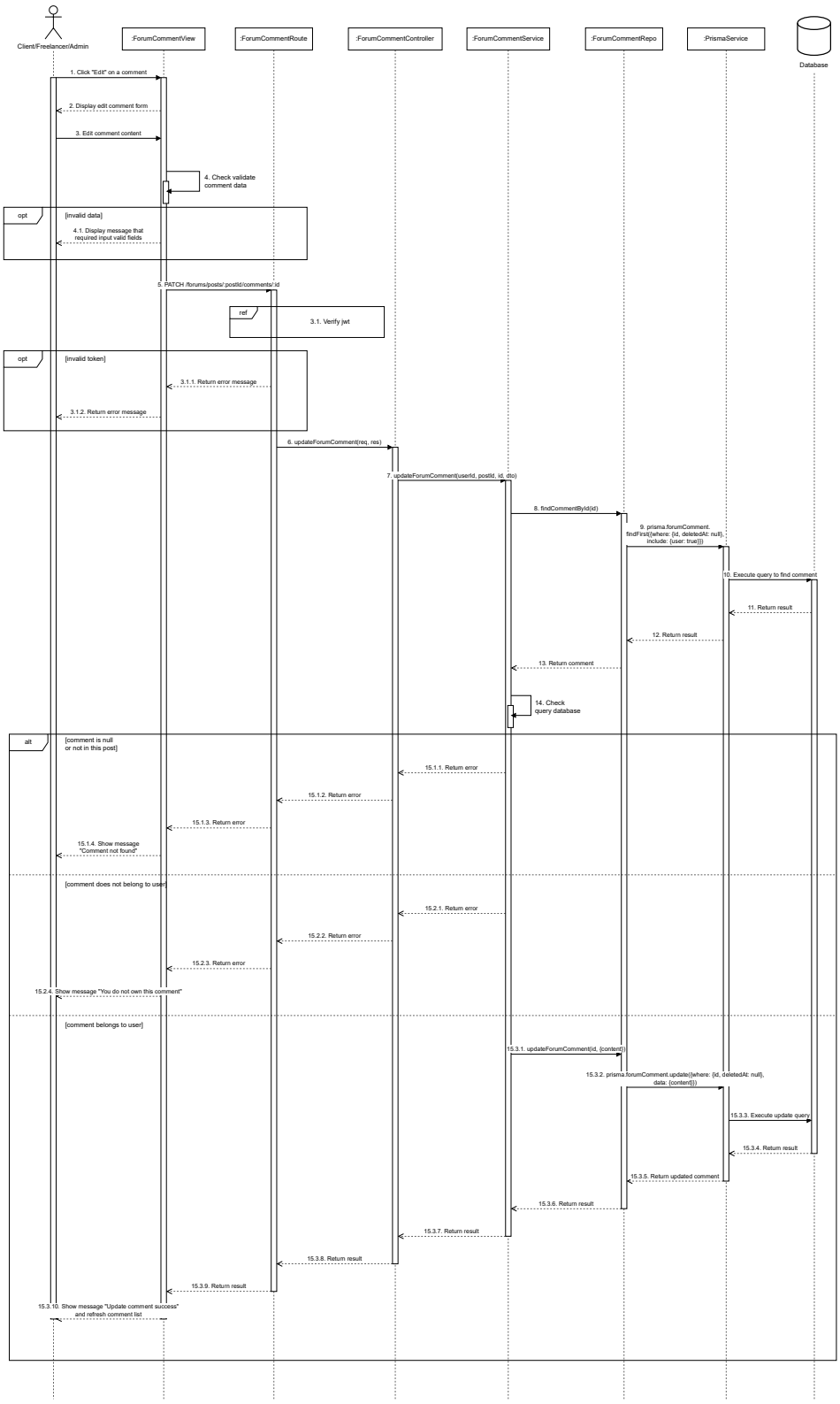

Figure 18: Sequence diagram of UC-14.13 Update Comment

3.1.14 UC-14.14 – Delete Comment

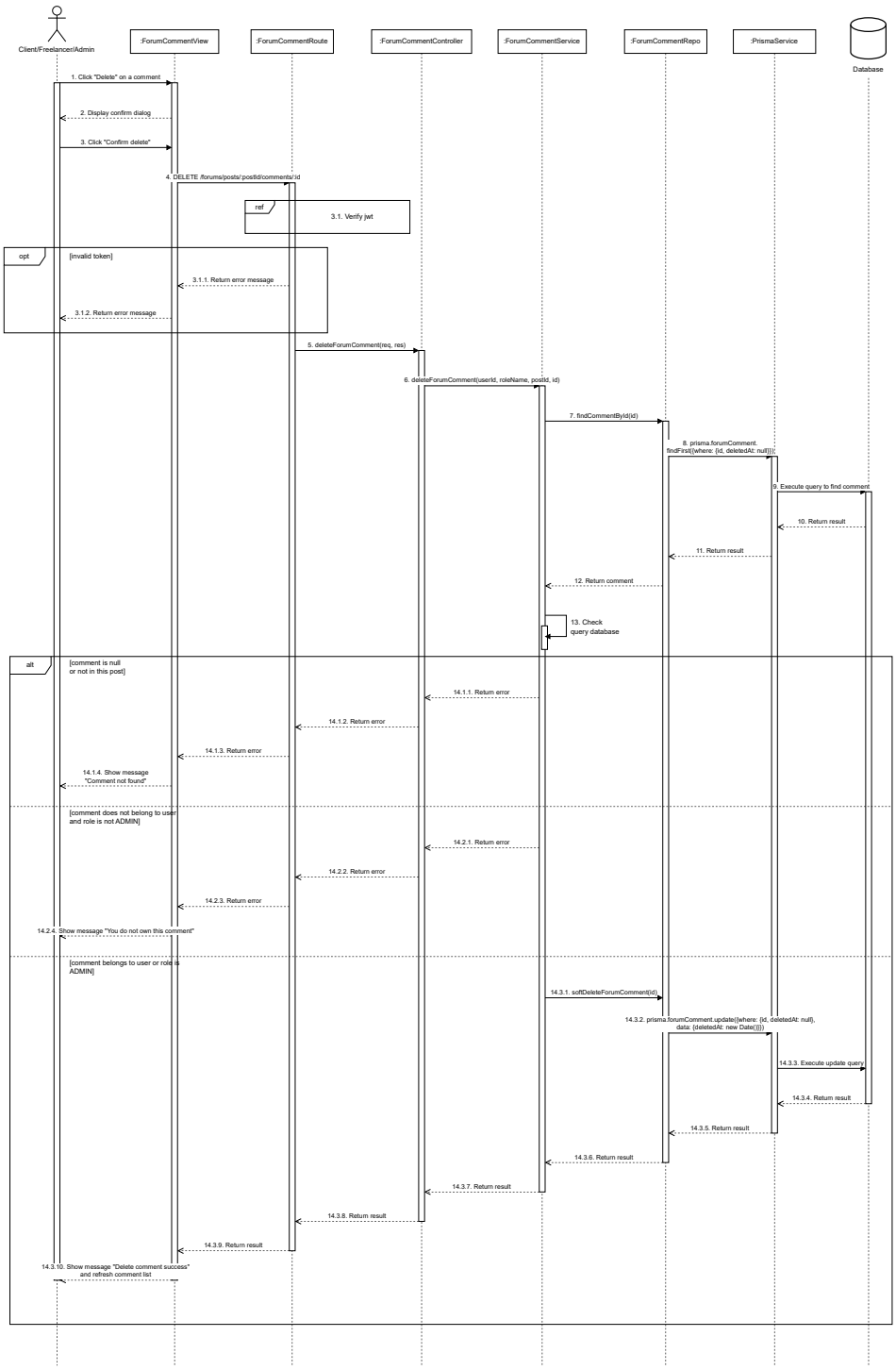

Figure 19: Sequence diagram of UC-14.14 Delete Comment

3.1.15 UC-14.15 – Like Comment

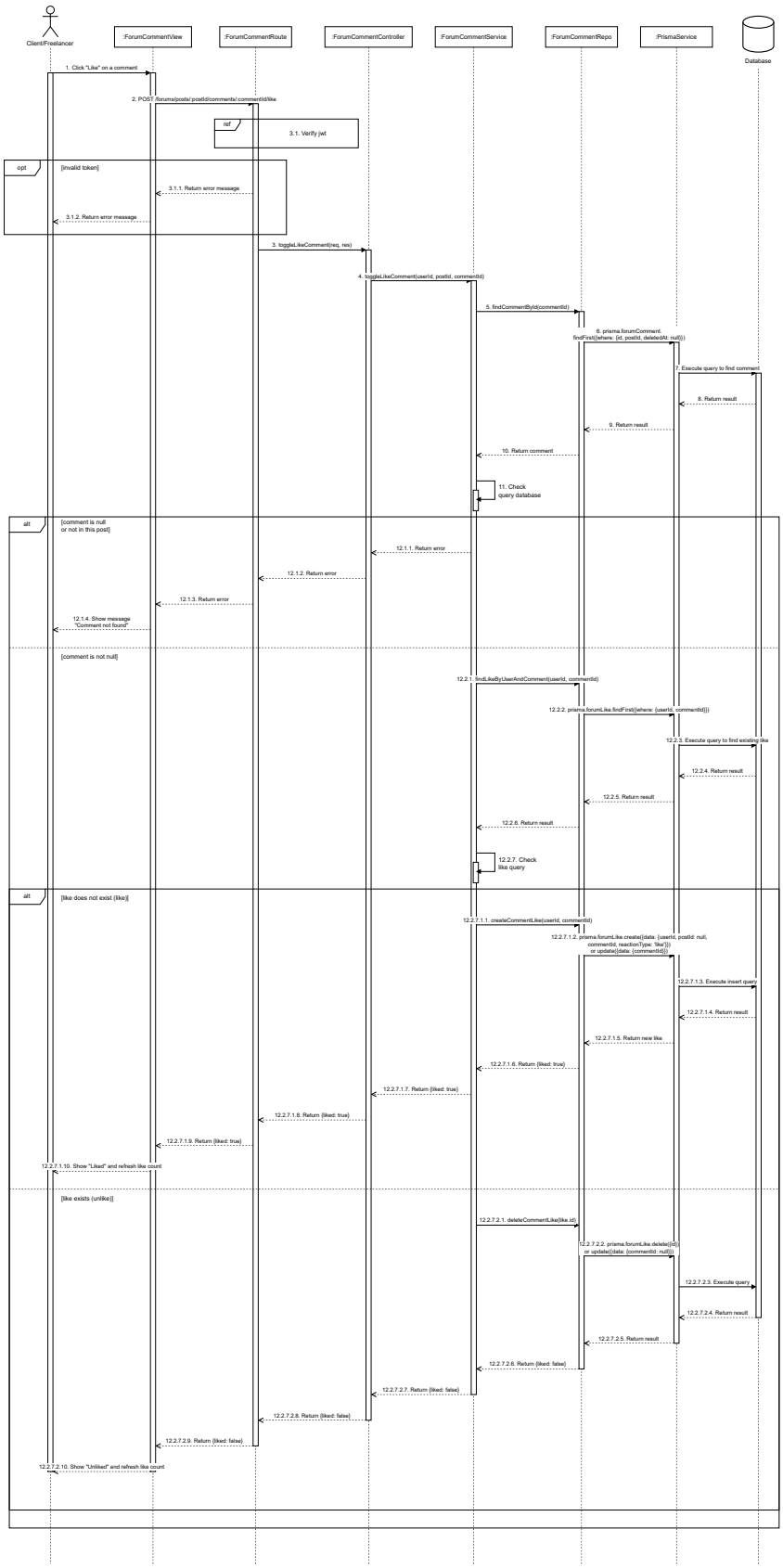

Figure 20: Sequence diagram of UC-14.15 Like Comment

3.1.16 UC-14.16 – Unlike Comment

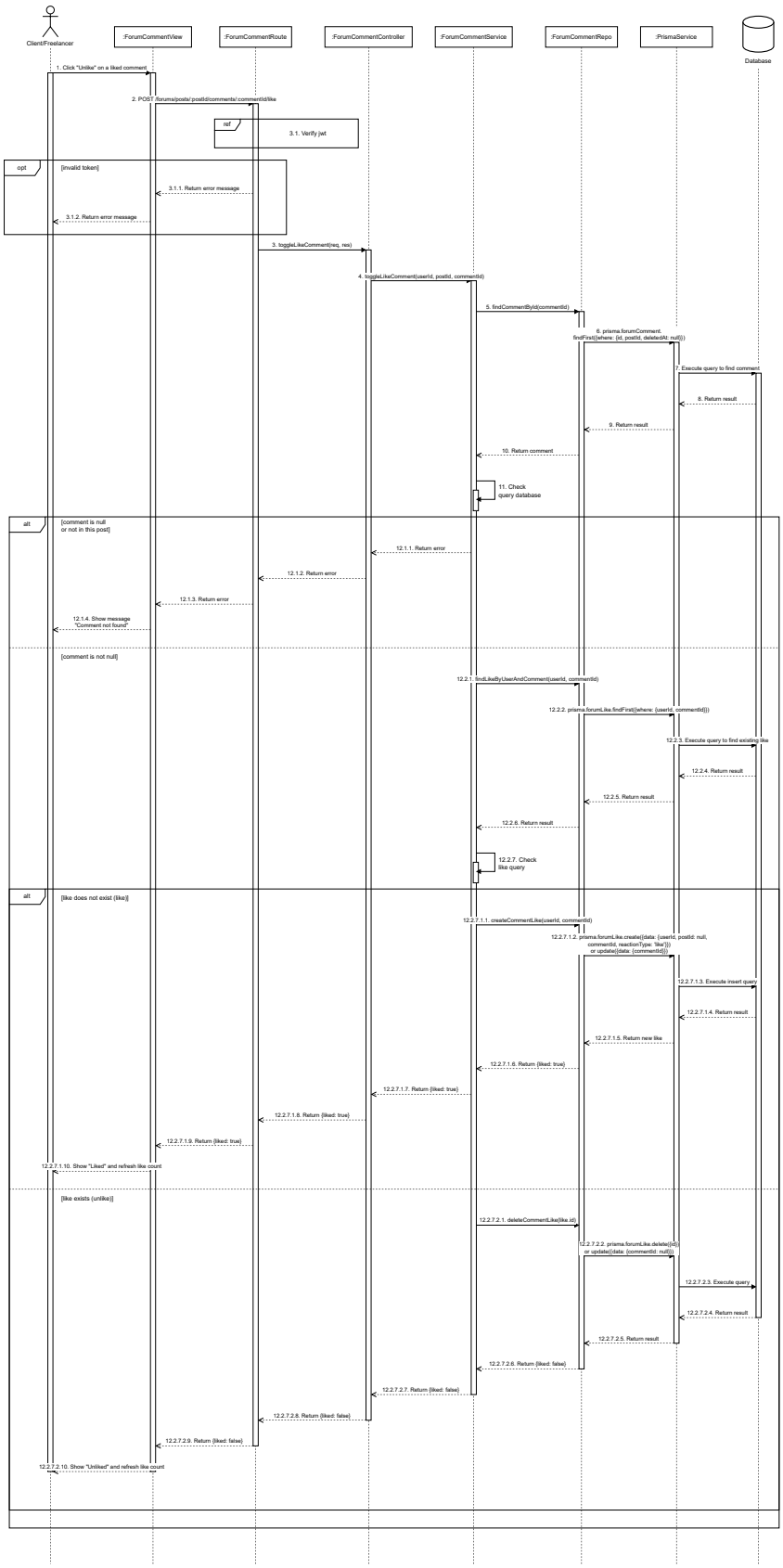

Figure 21: Sequence diagram of UC-14.16 Unlike Comment

3.1.17 UC-14.17 – Report Content

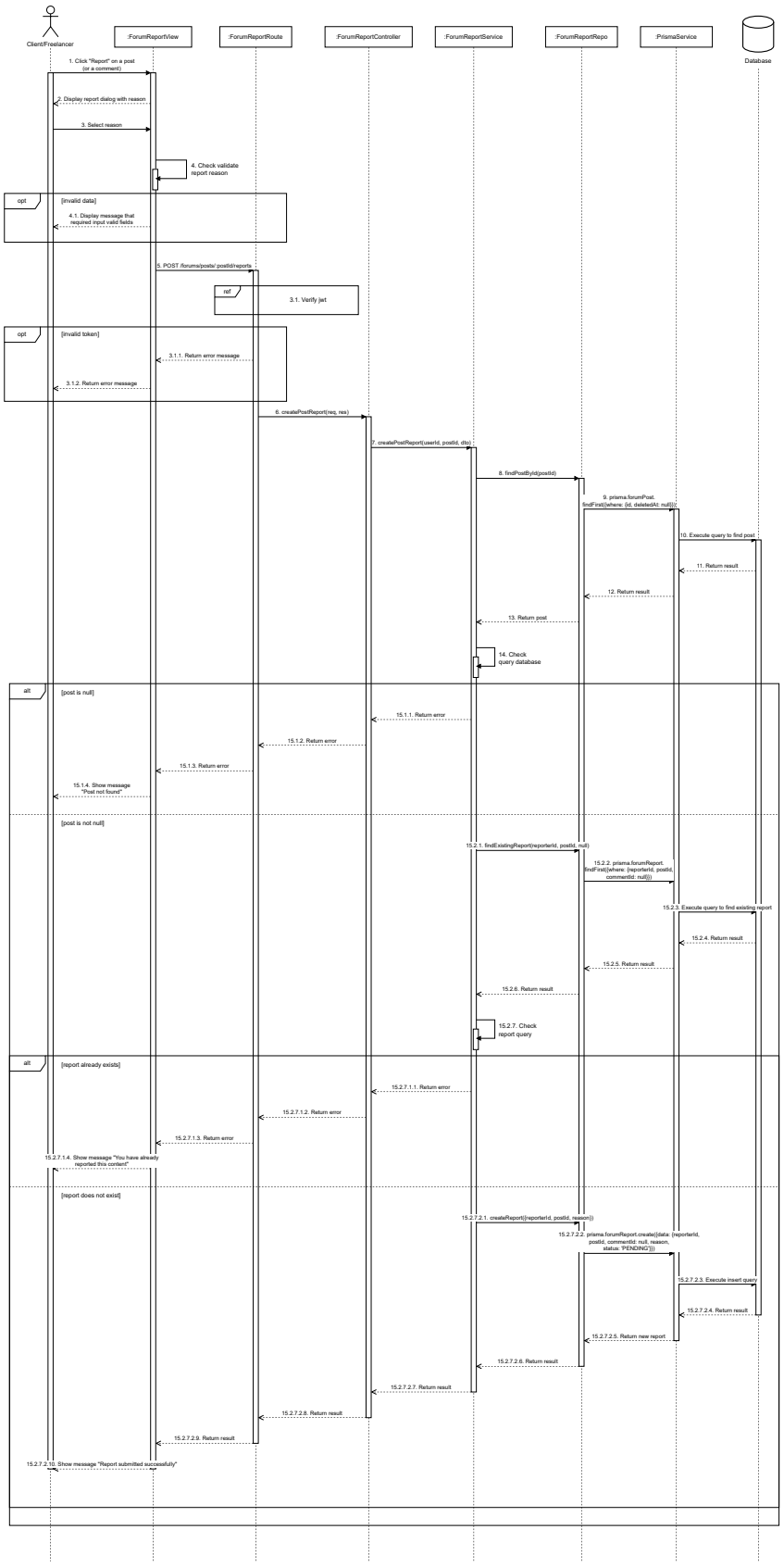

Figure 22: Sequence diagram of UC-14.17 Report Content

3.2 Contract Management

Contract Management covers the UC-13 contract lifecycle: creating and updating fixed-price contracts upon proposal acceptance, handling dual-signing between client and freelancer, marking contracts as completed or cancelled, and managing contract shared files including viewing, uploading, and deleting contract attachments.

a. Class Diagram

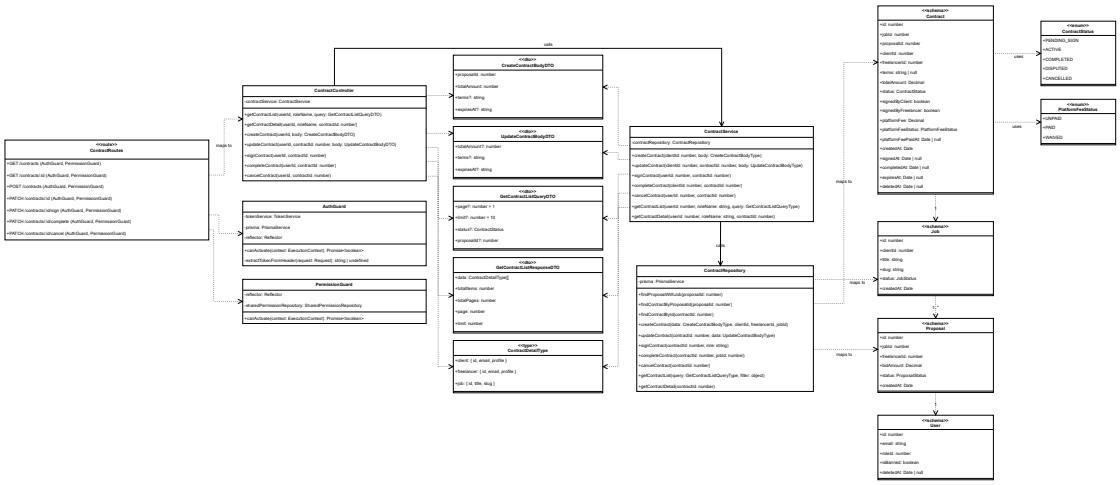

Figure 23: Class diagram of Contract Management

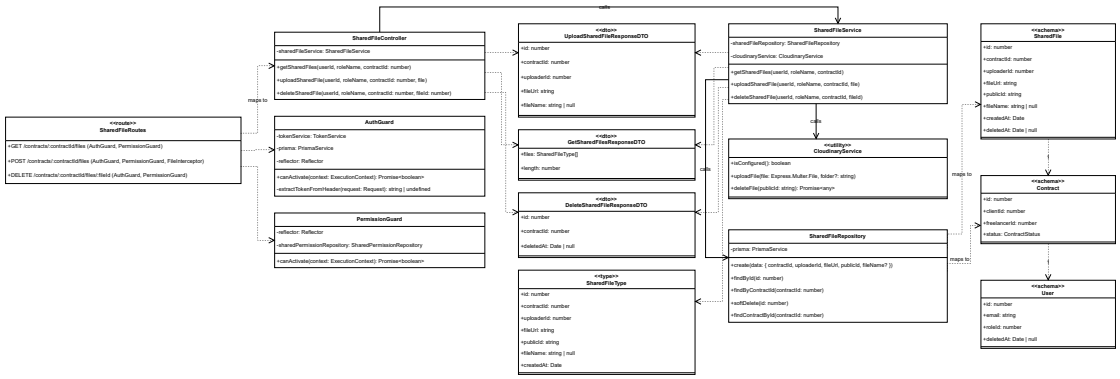

Figure 24: Class diagram of Contract Shared File Management

b. Sequence Diagram(s)

3.2.1 UC-13.01 – Create Contract

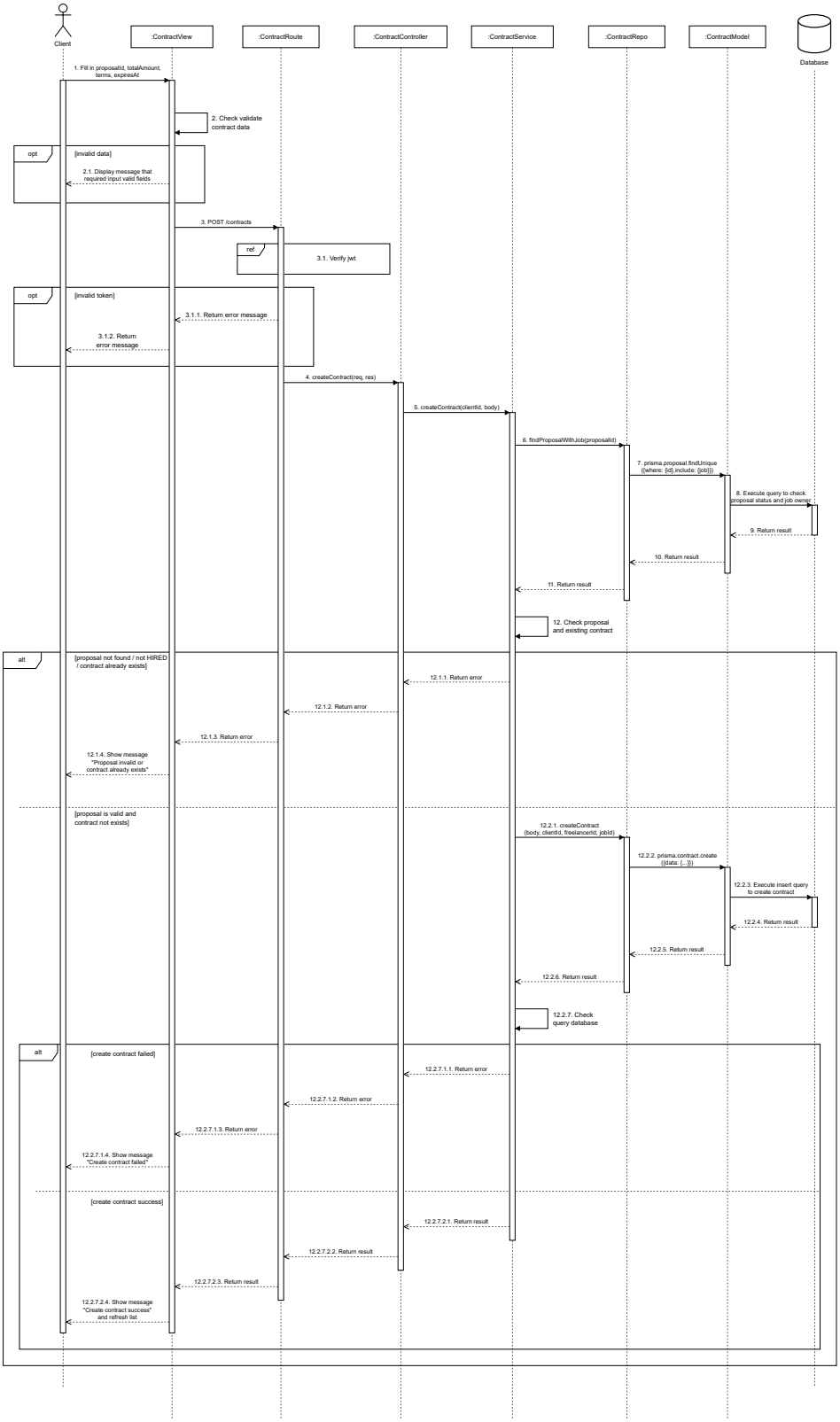

Figure 25: Sequence diagram of UC-13.01 Create Contract

3.2.2 UC-13.02 – Update Contract

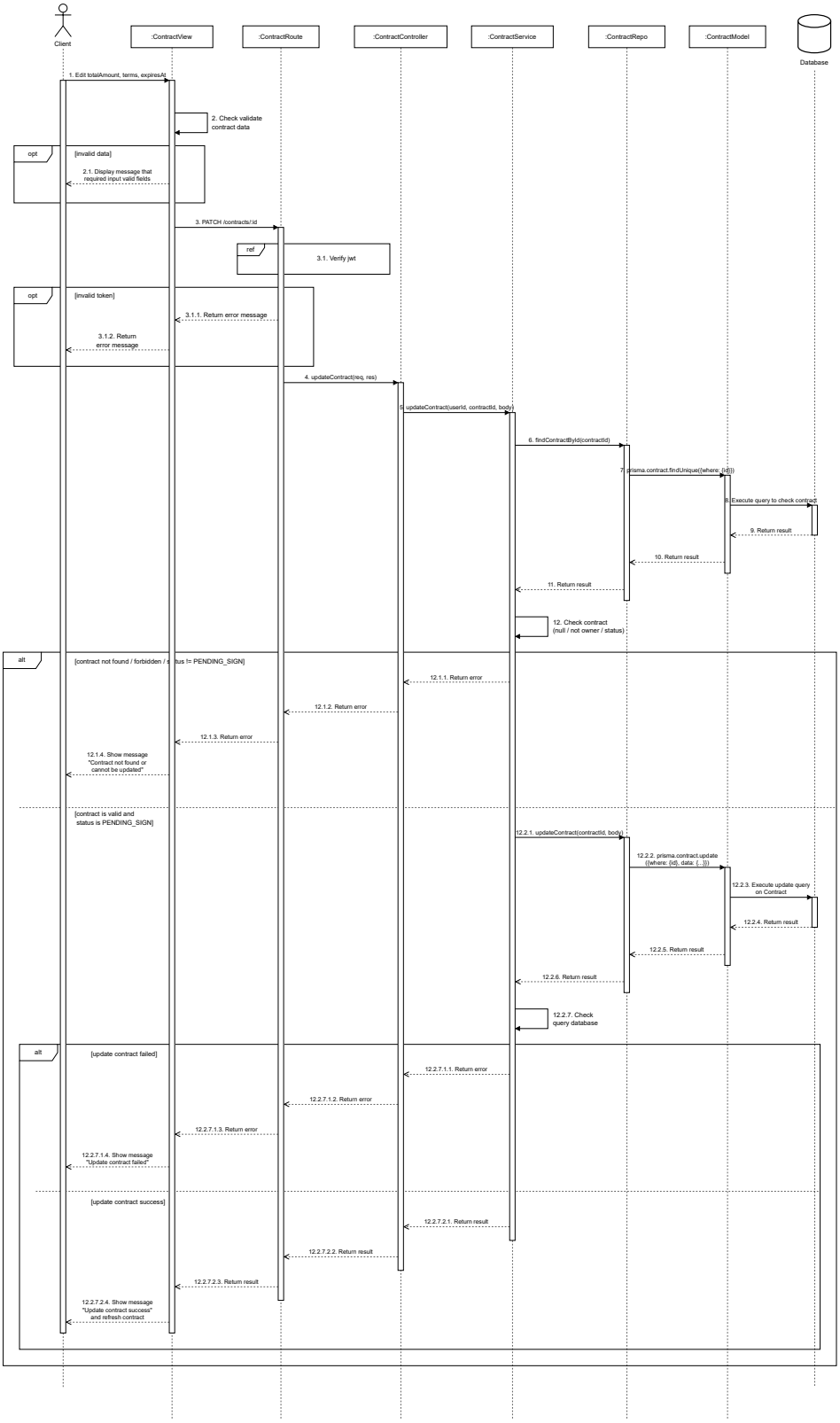

Figure 26: Sequence diagram of UC-13.02 Update Contract

3.2.3 UC-13.03 – Complete Contract

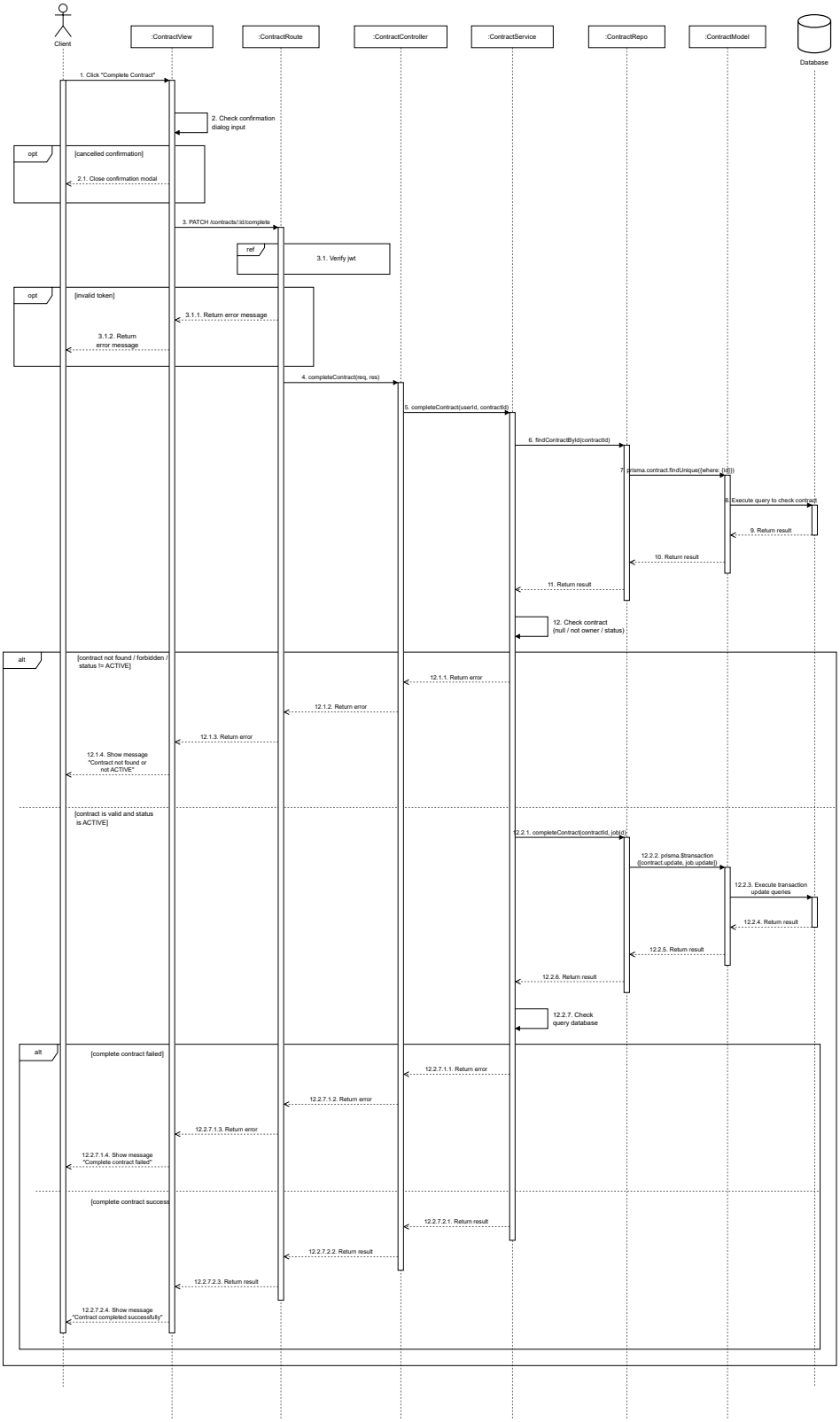

Figure 27: Sequence diagram of UC-13.03 Complete Contract

3.2.4 UC-13.04 – View Contract List

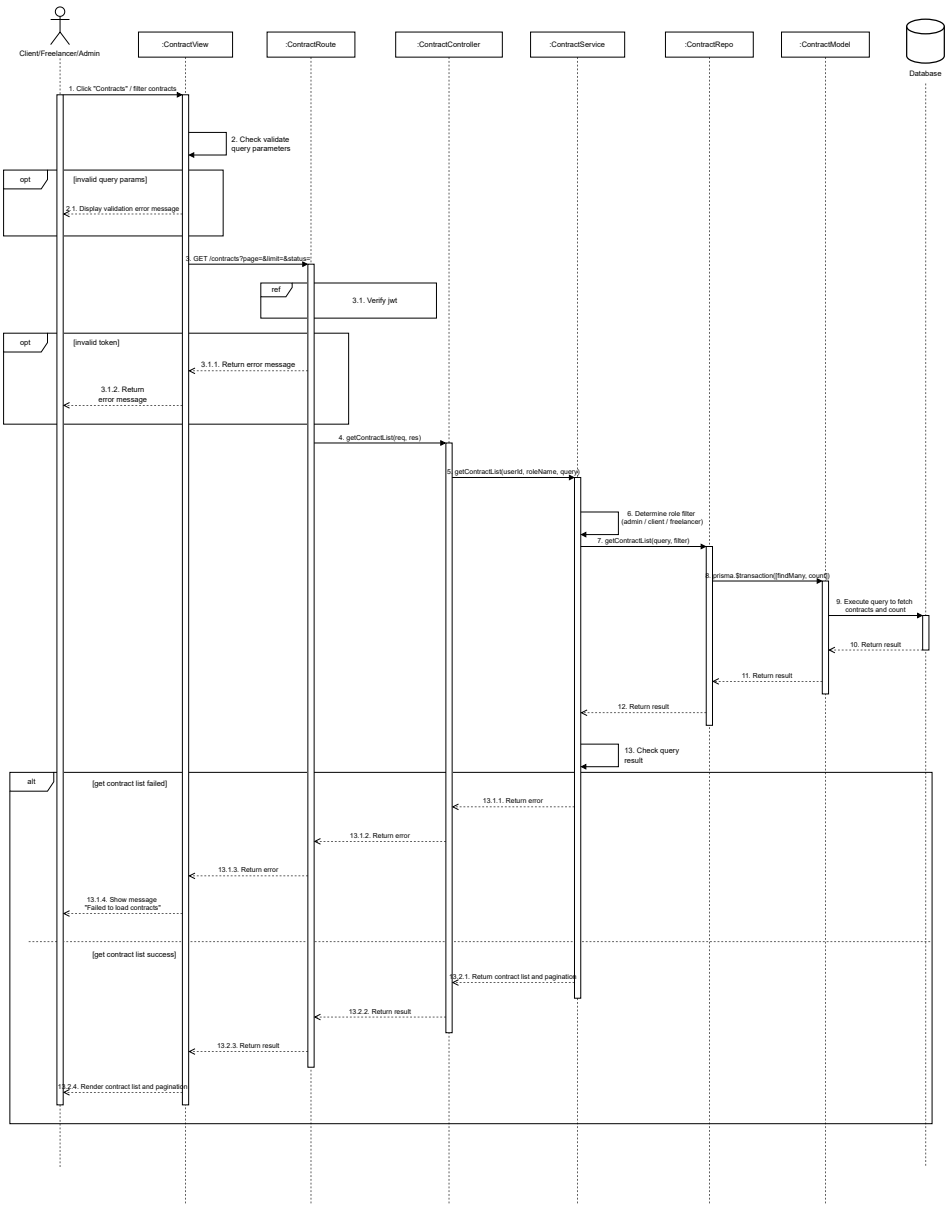

Figure 28: Sequence diagram of UC-13.04 View Contract List

3.2.5 UC-13.05 – View Contract Detail

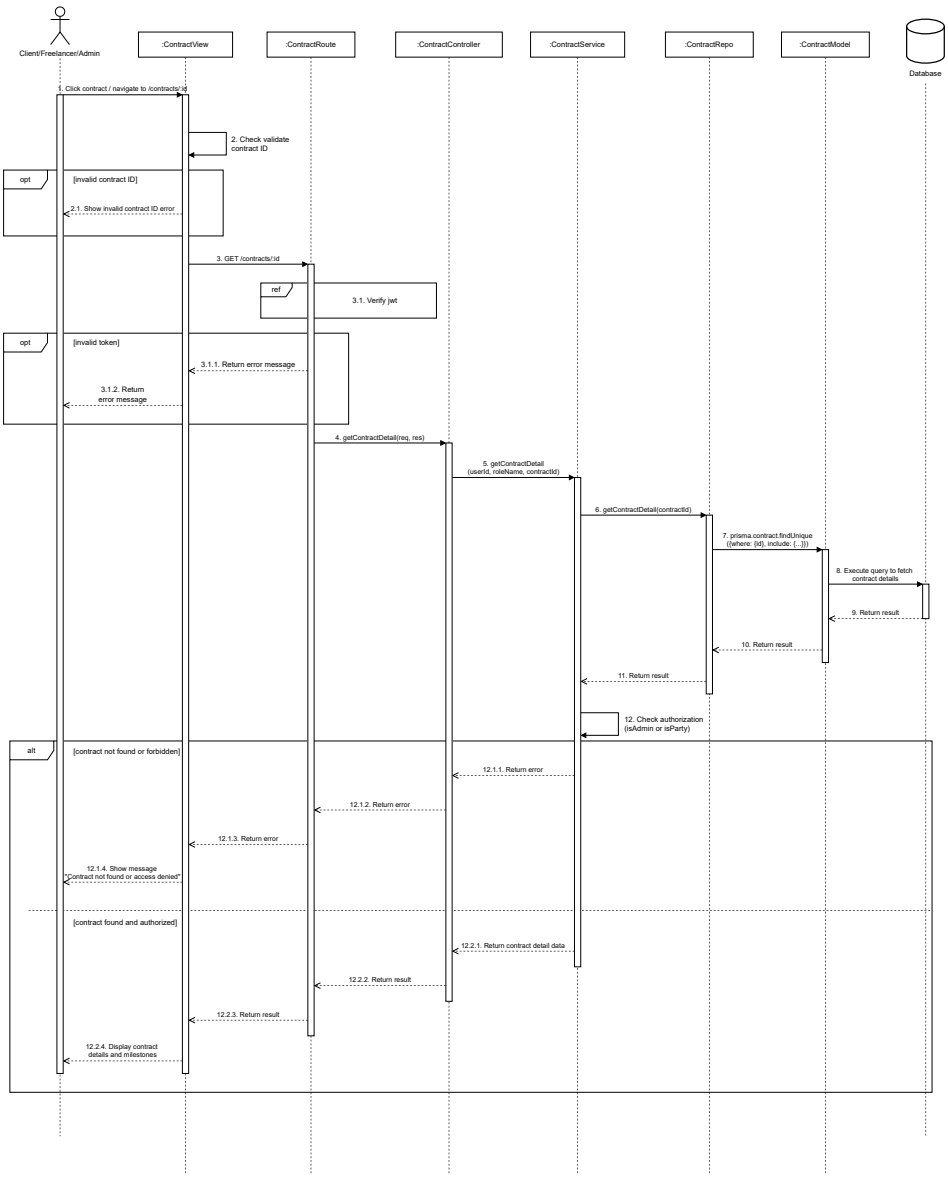

Figure 29: Sequence diagram of UC-13.05 View Contract Detail

3.2.6 UC-13.06 – Sign Contract

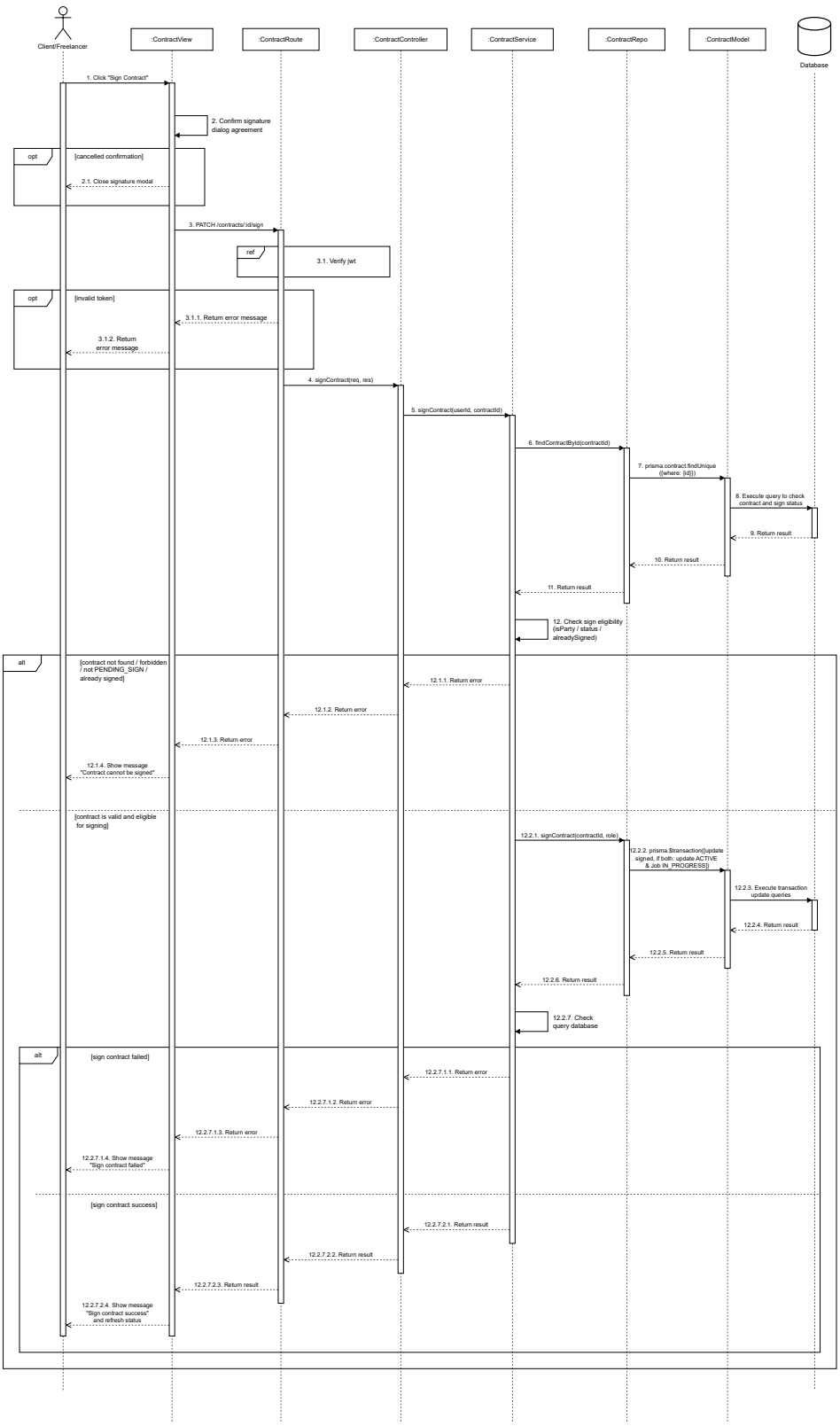

Figure 30: Sequence diagram of UC-13.06 Sign Contract

3.2.7 UC-13.07 – Cancel Contract

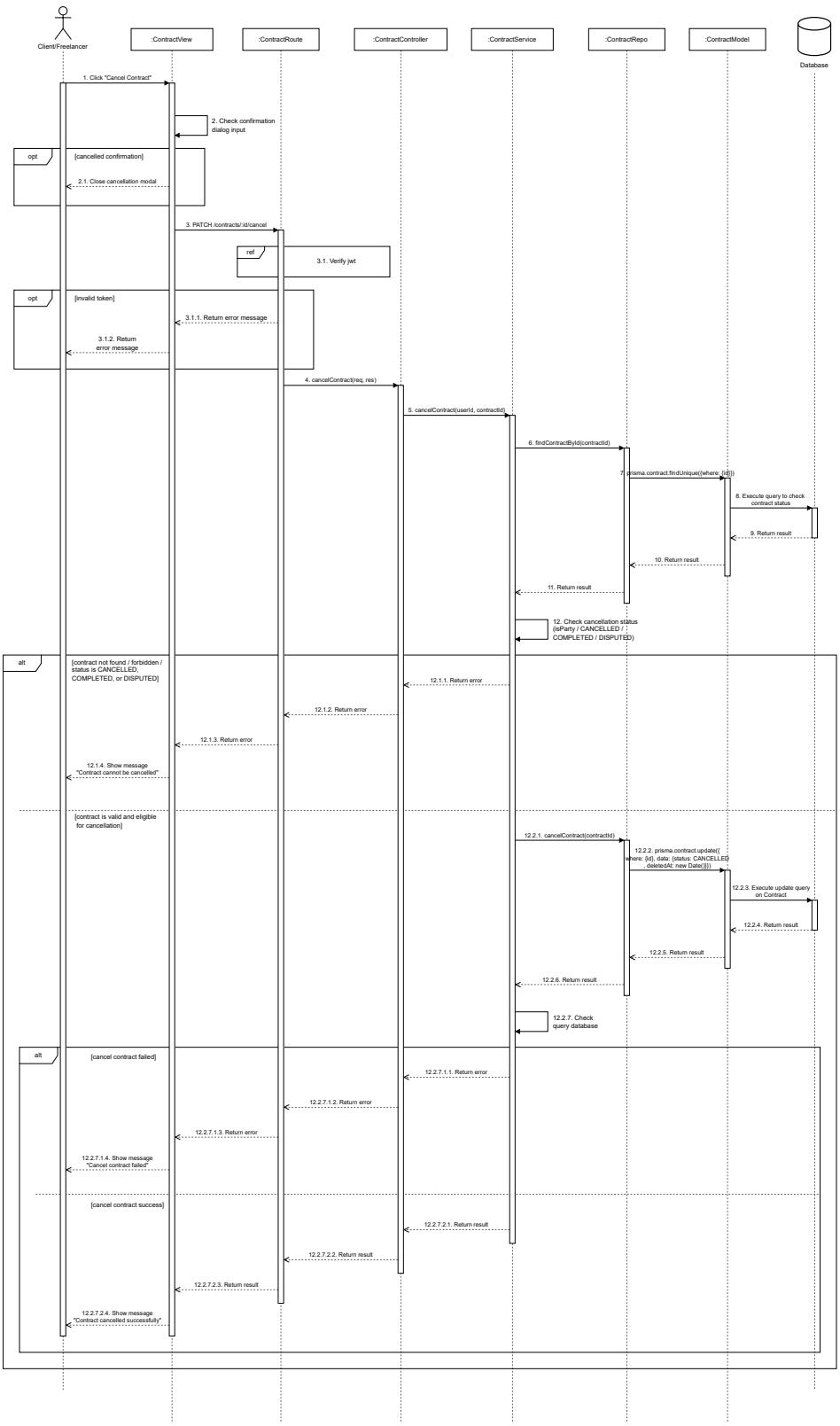

Figure 31: Sequence diagram of UC-13.07 Cancel Contract

3.2.8 UC-13.08 – View Contract Files

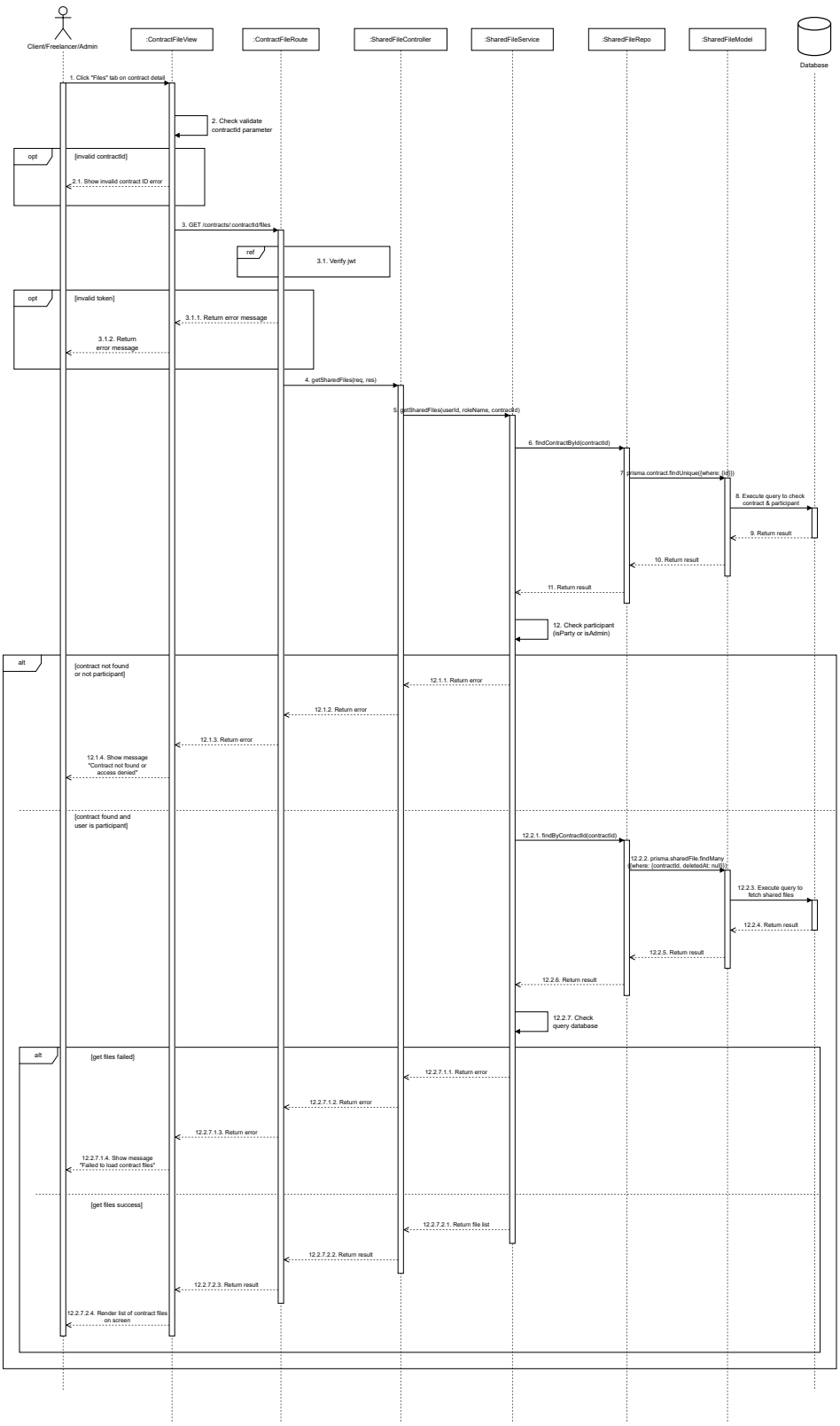

Figure 32: Sequence diagram of UC-13.08 View Contract Files

3.2.9 UC-13.09 – Upload Contract File

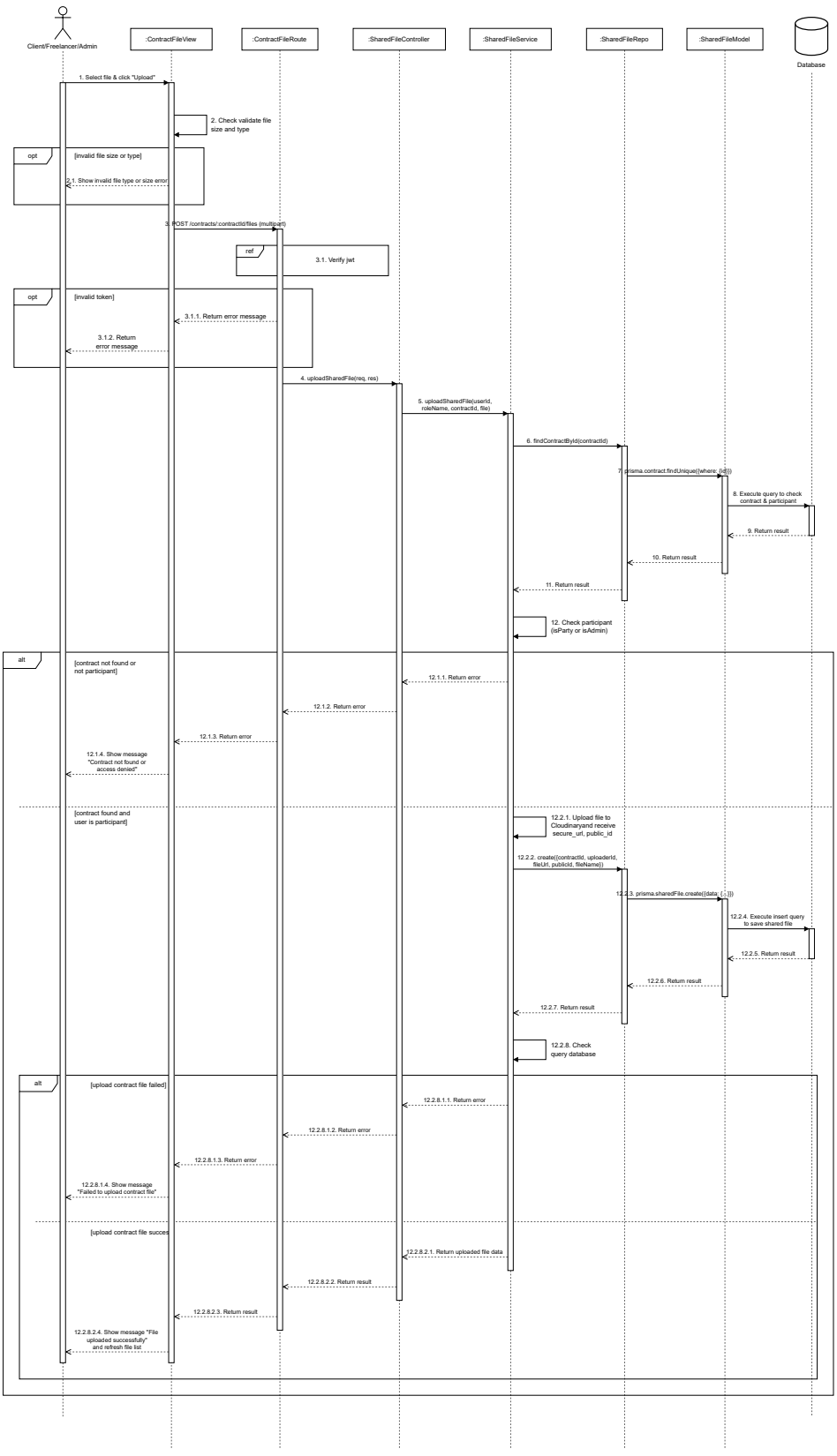

Figure 33: Sequence diagram of UC-13.09 Upload Contract File

3.2.10 UC-13.10 – Delete Contract File

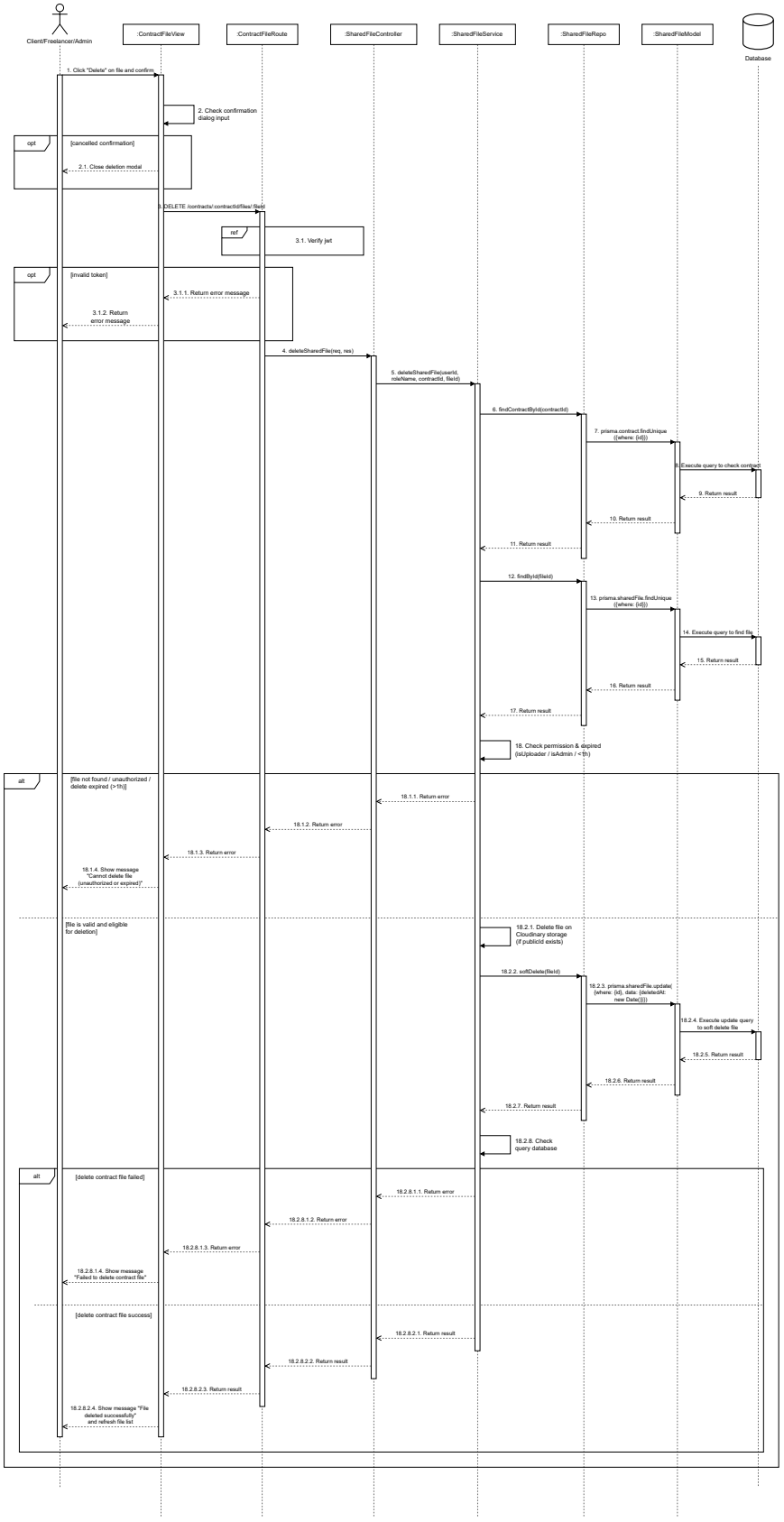

Figure 34: Sequence diagram of UC-13.10 Delete Contract File

3.3 Authentication

The authentication module covers account registration, email/password and Google OAuth login, logout, password recovery, and the JWT/session flow shared by all protected endpoints.

a. Class Diagram

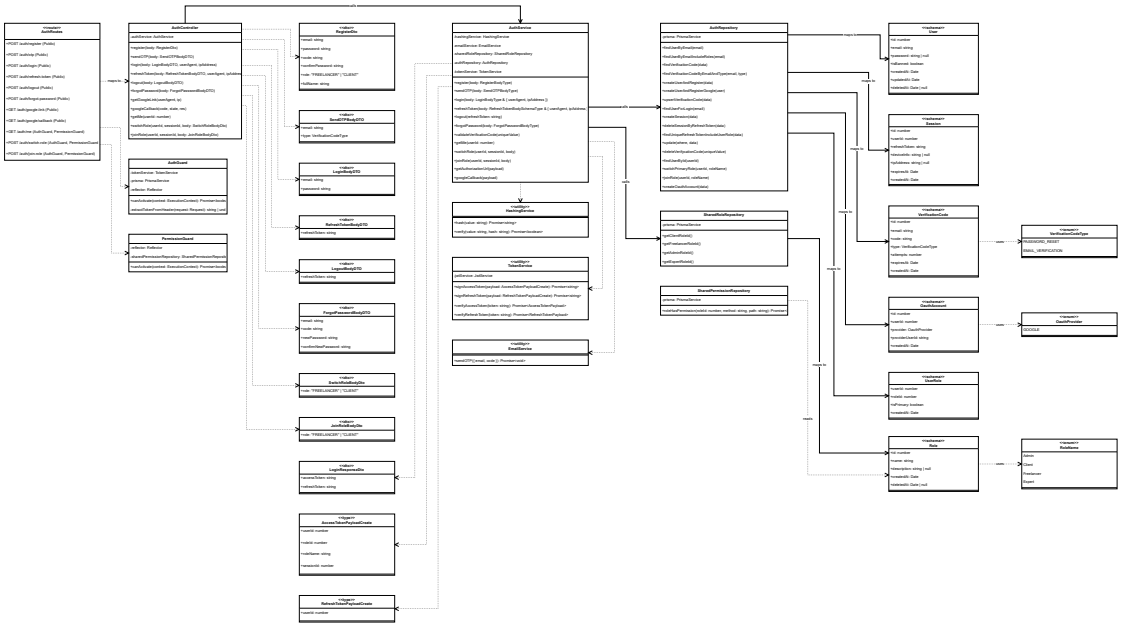

Figure 35: Class diagram of Authentication

b. Sequence Diagram(s)

3.3.1 UC-01 – Login with Google

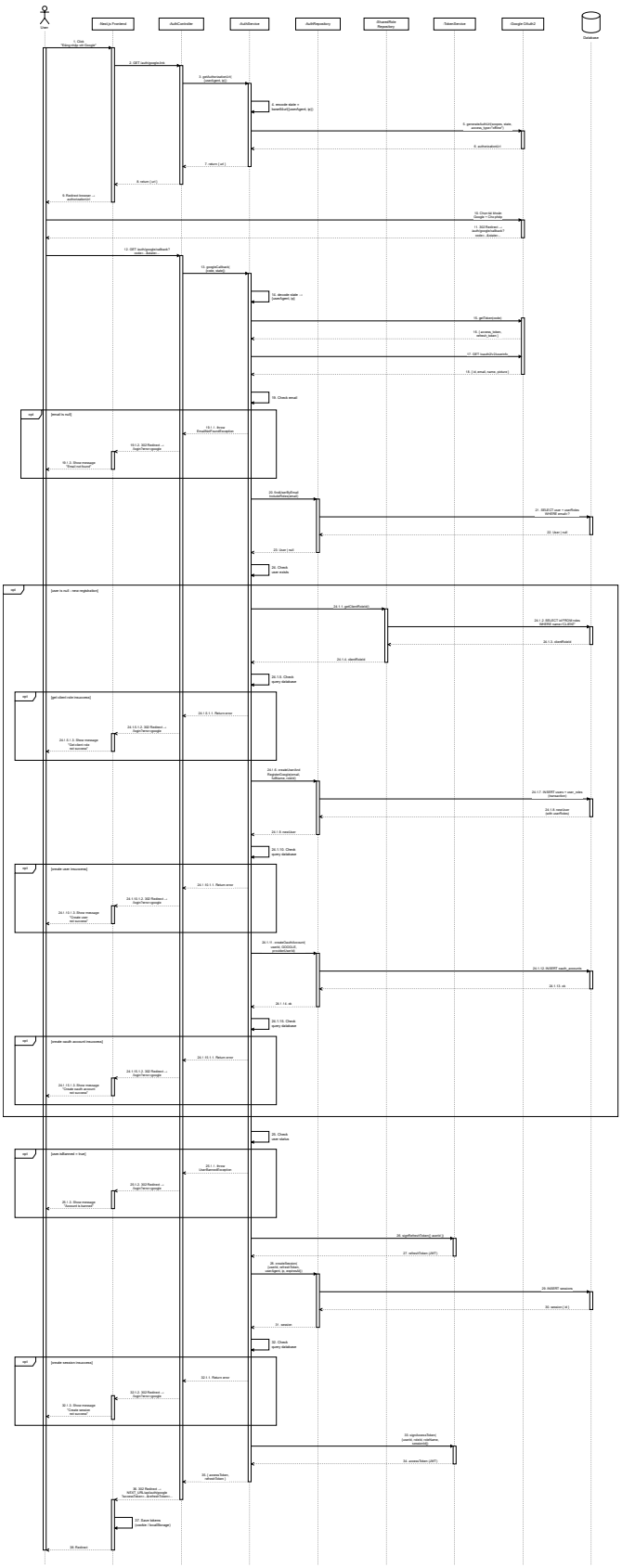

Figure 36: Sequence diagram of UC-01 Login with Google

3.3.2 UC-02 – Login

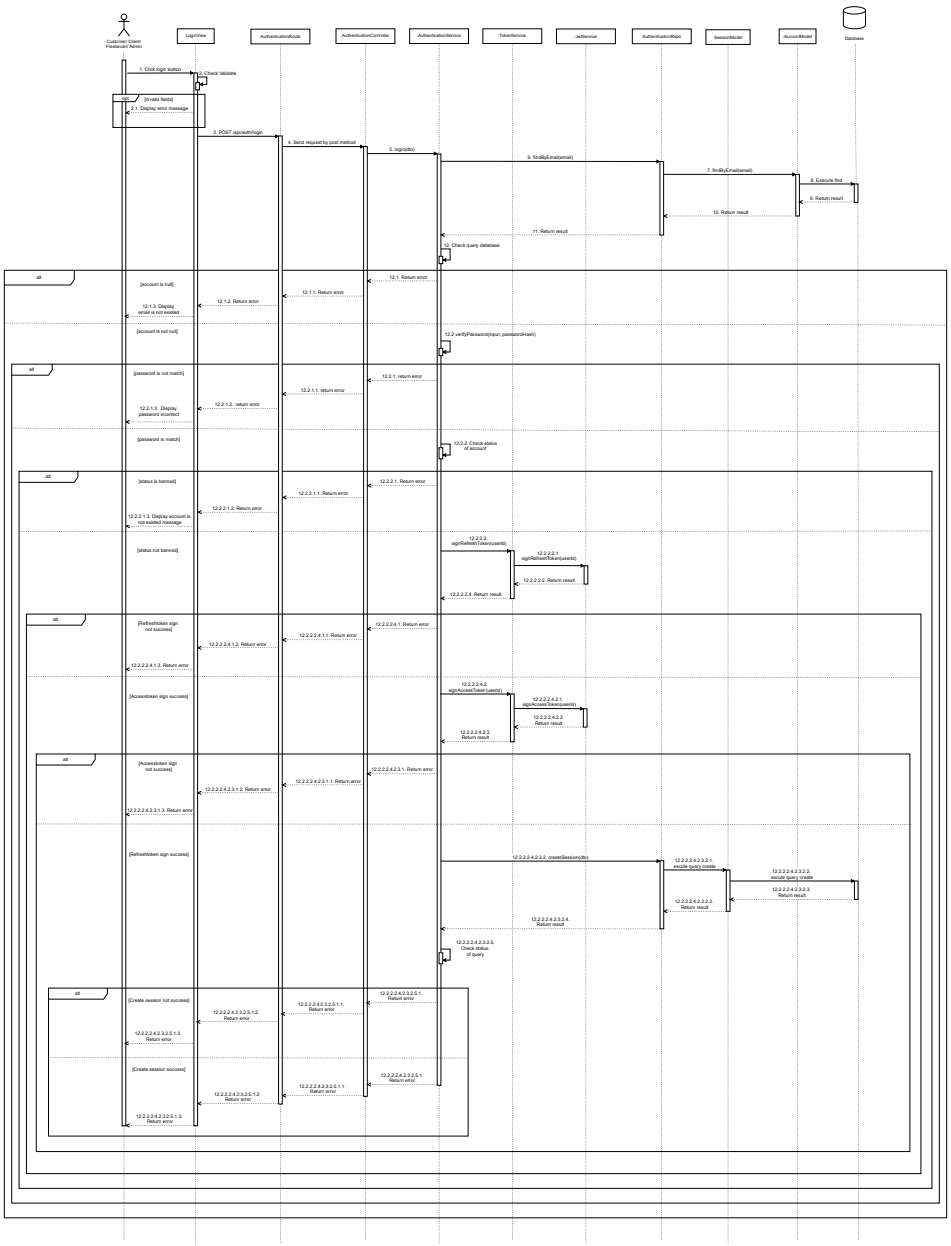

Figure 37: Sequence diagram of UC-02 Login

3.3.3 UC-03 – Logout

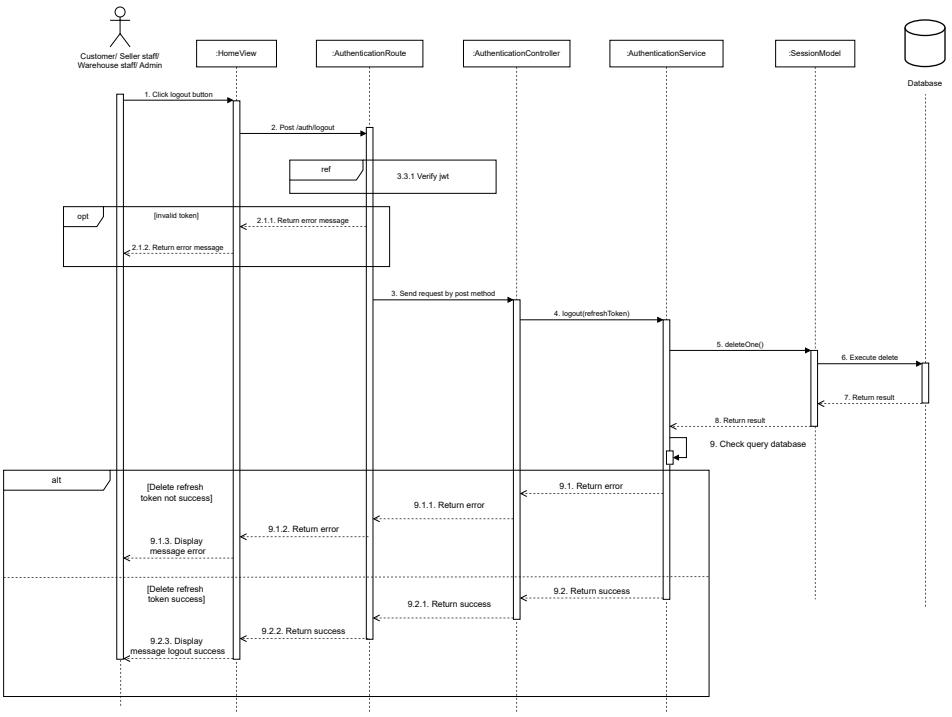

Figure 38: Sequence diagram of UC-03 Logout

3.3.4 UC-04 – Register

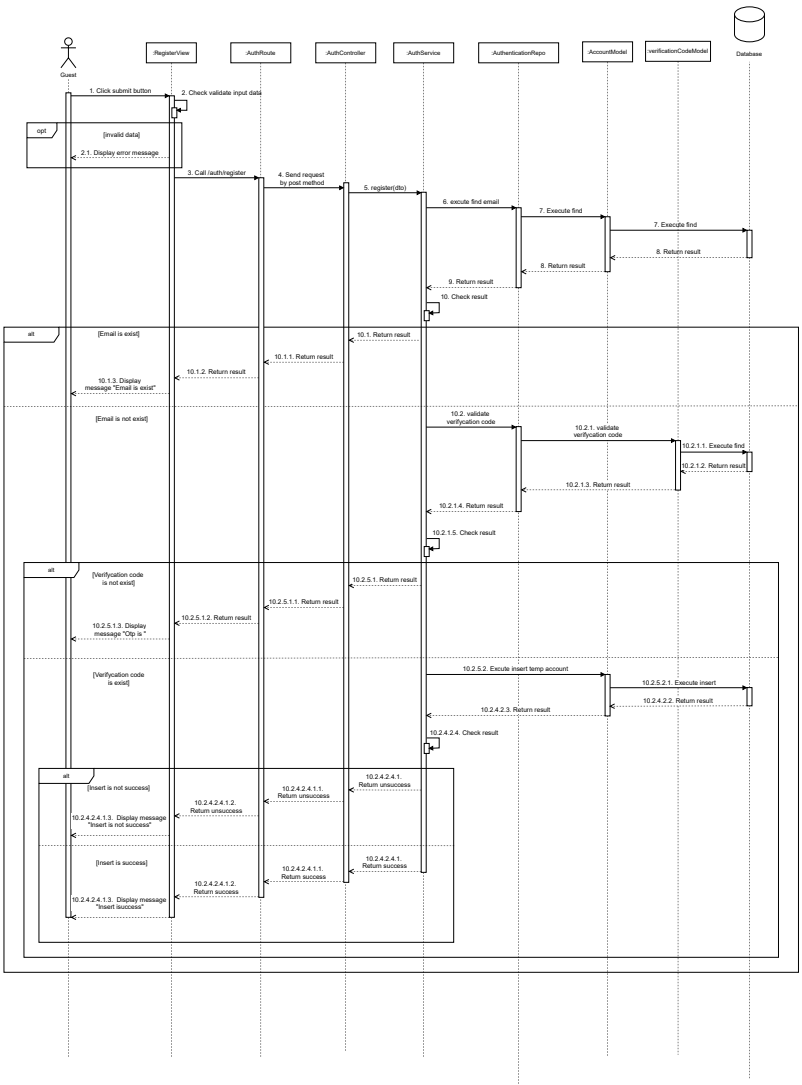

Figure 39: Sequence diagram of UC-04 Register

3.3.5 UC-05 – Forgot Password

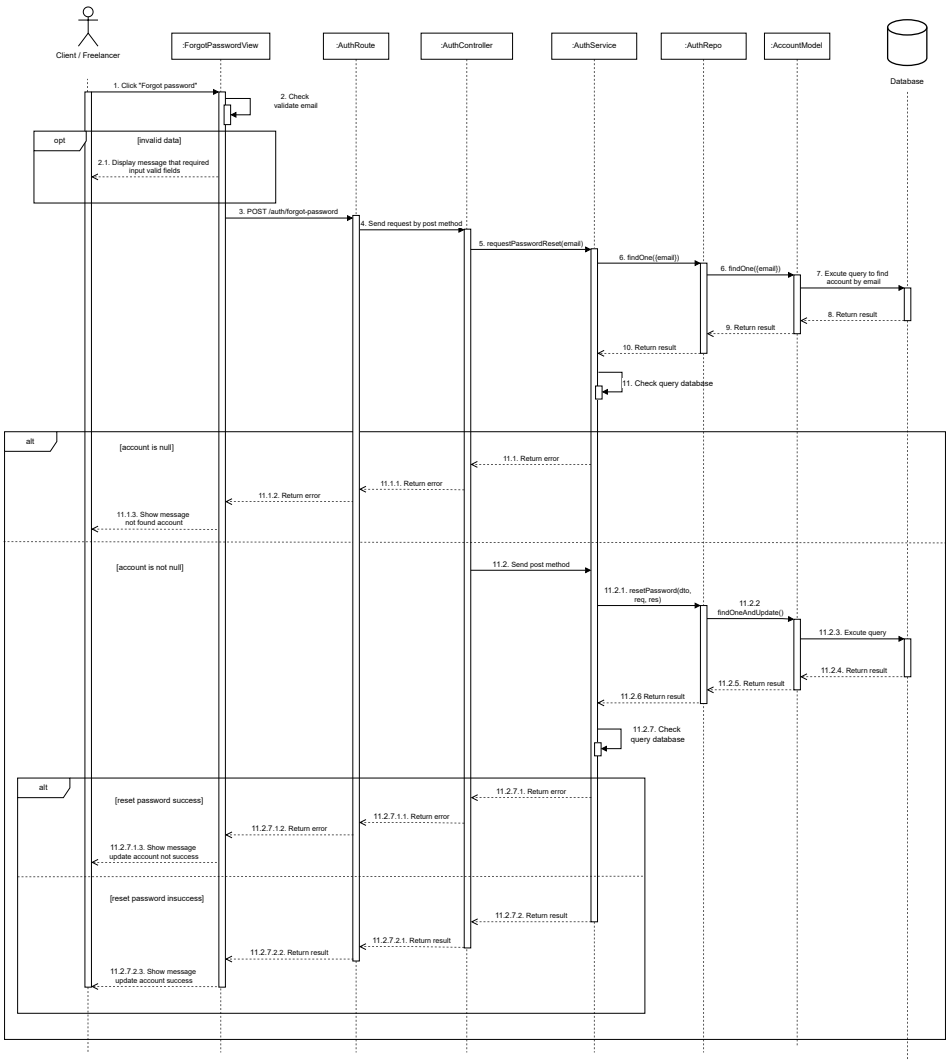

Figure 40: Sequence diagram of UC-05 Forgot Password

3.4 Role Management

Role Management lets an administrator inspect the role catalogue, maintain role metadata, and replace the permission set assigned to a role.

a. Class Diagram

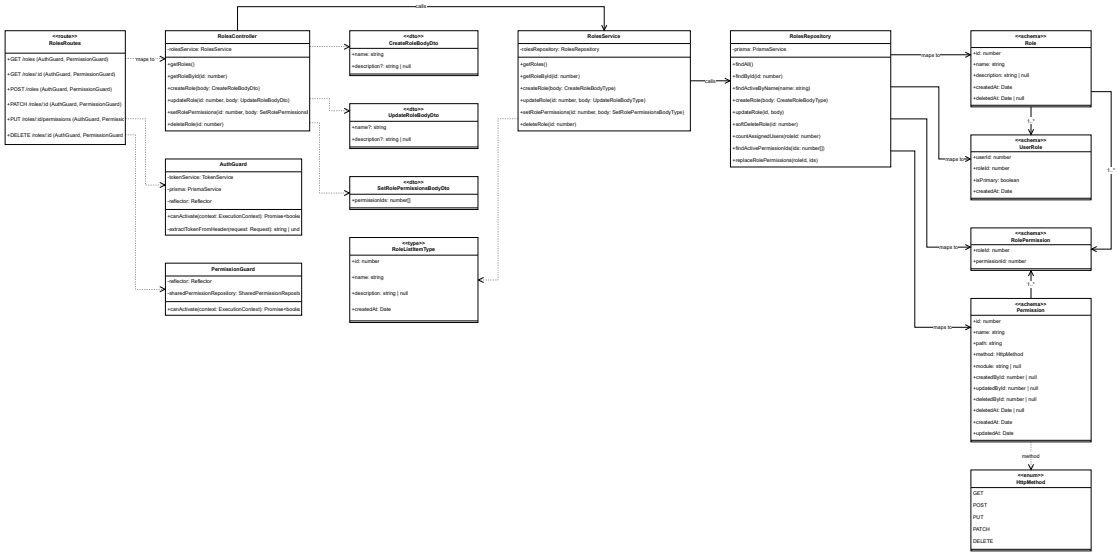

Figure 41: Class diagram of Role Management

b. Sequence Diagram(s)

3.4.1 UC-06.01 – View Role List

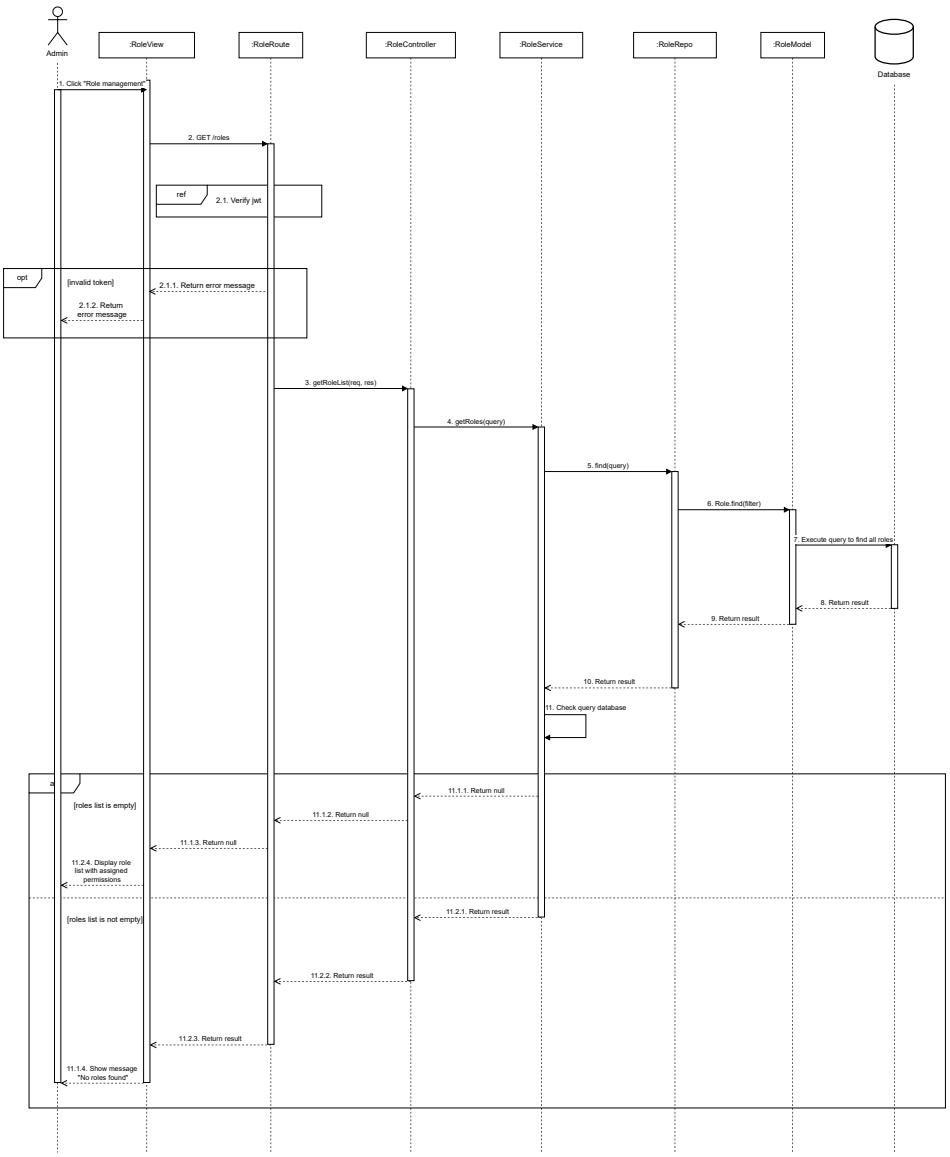

Figure 42: Sequence diagram of UC-06.01 View Role List

3.4.2 UC-06.02 – View Role Detail

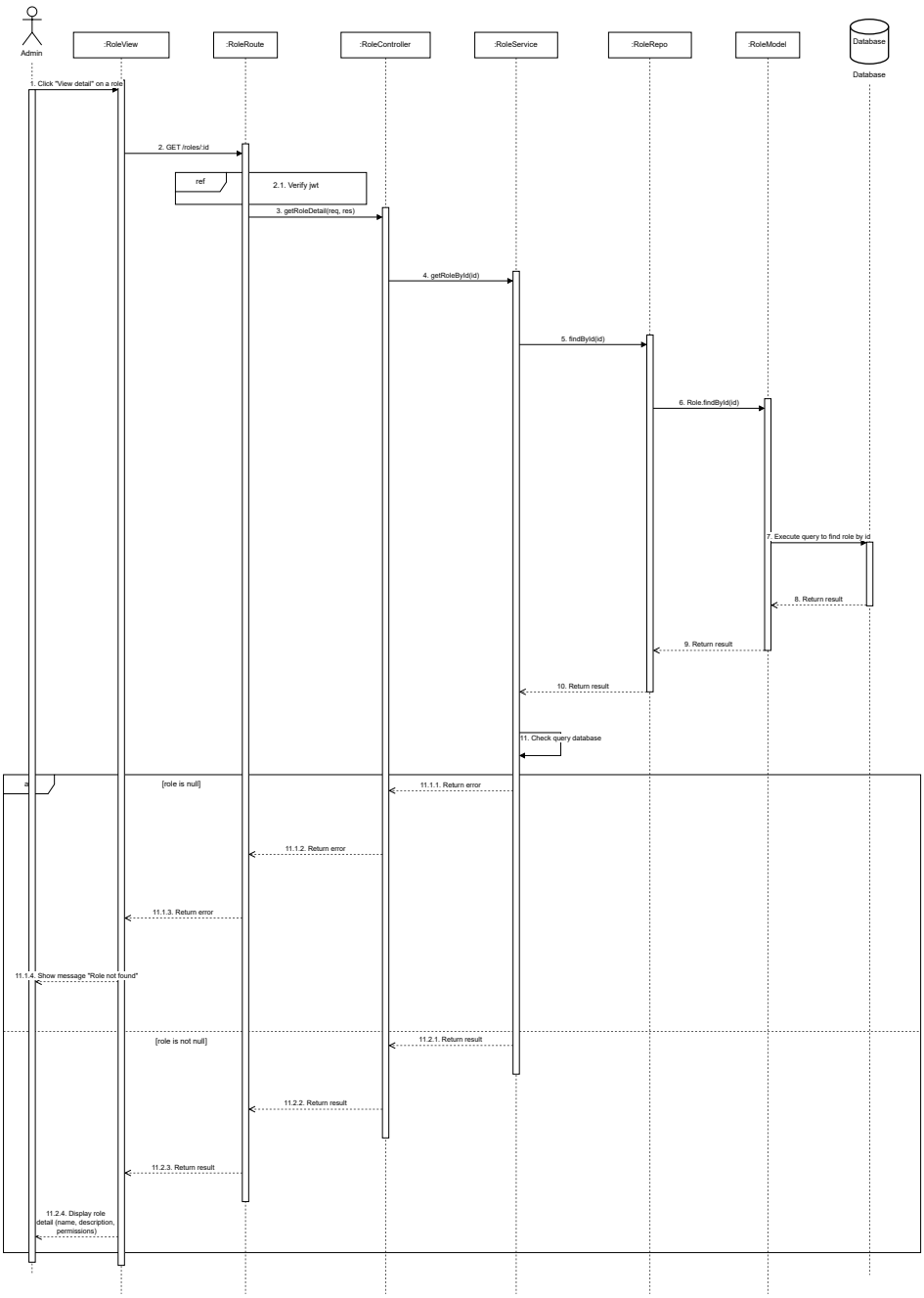

Figure 43: Sequence diagram of UC-06.02 View Role Detail

3.4.3 UC-06.03 – Create Role

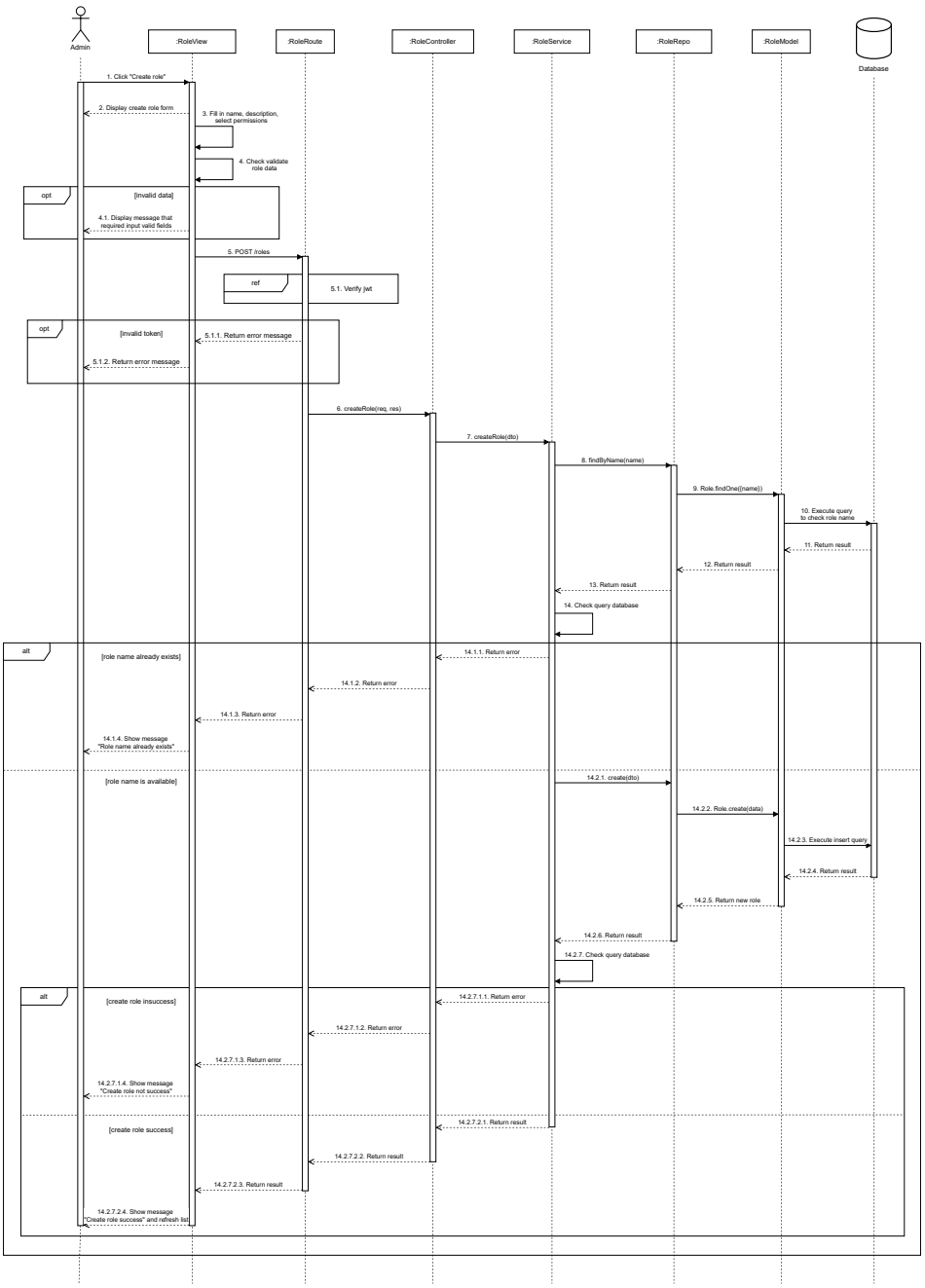

Figure 44: Sequence diagram of UC-06.03 Create Role

3.4.4 UC-06.04 – Update Role

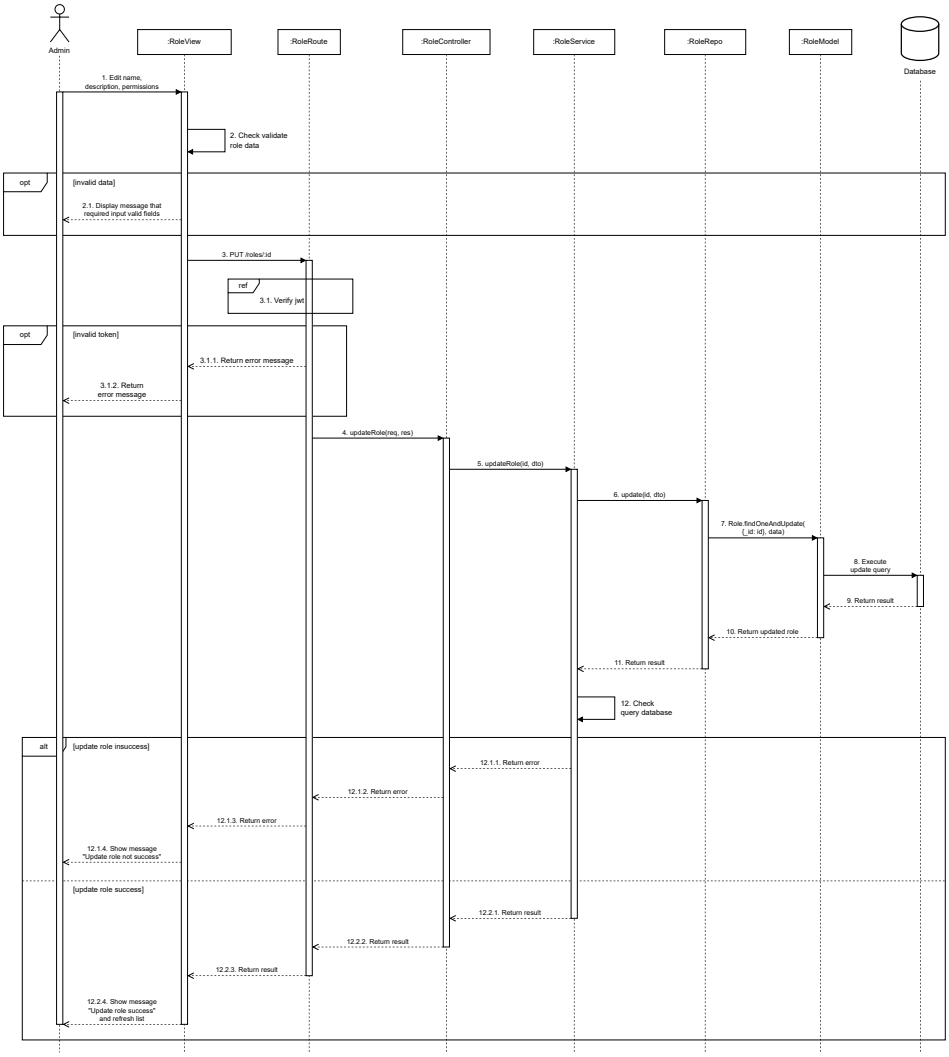

Figure 45: Sequence diagram of UC-06.04 Update Role

3.4.5 UC-06.05 – Delete Role

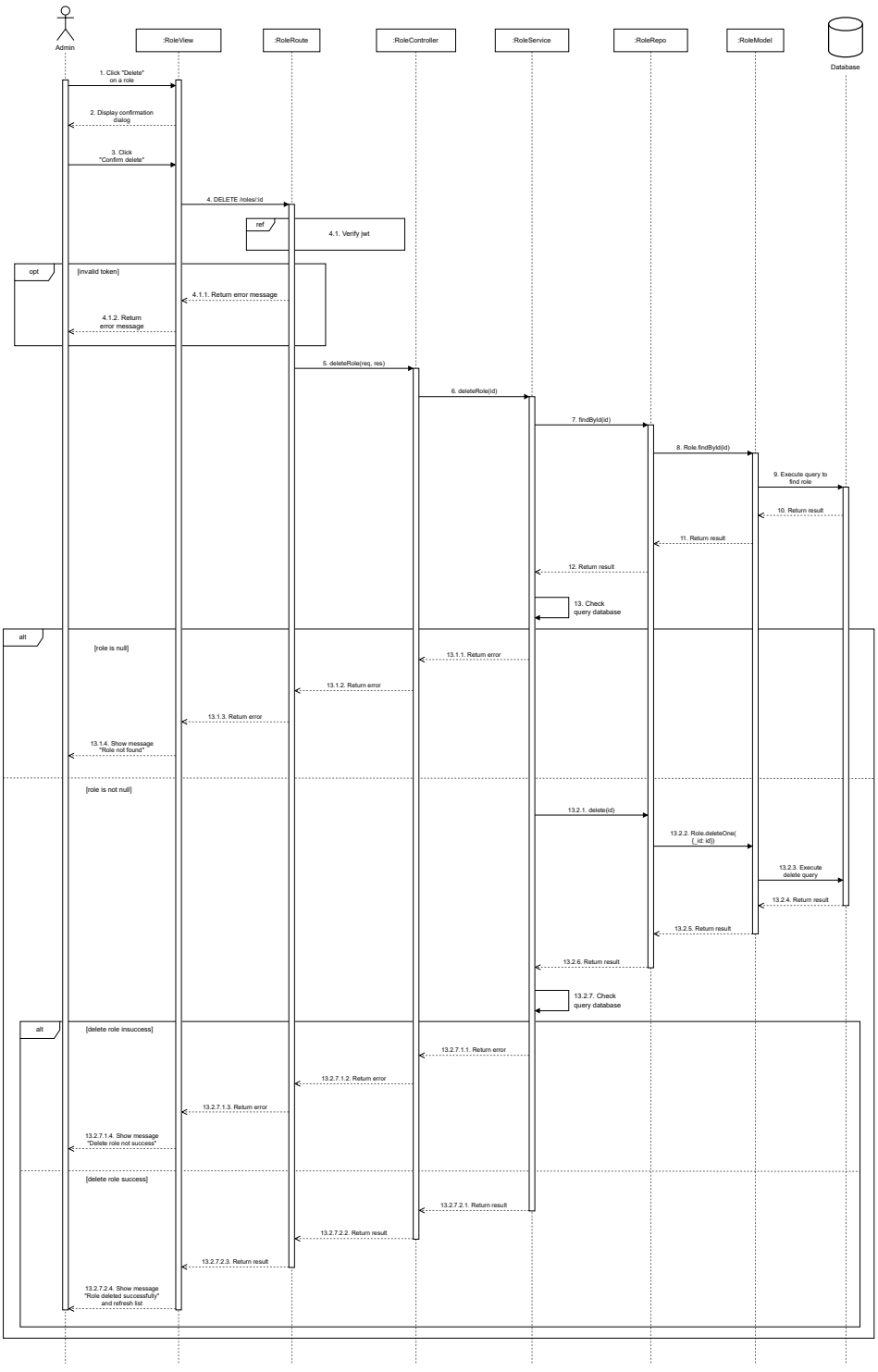

Figure 46: Sequence diagram of UC-06.05 Delete Role

3.4.6 UC-06.06 – Update Role Permissions

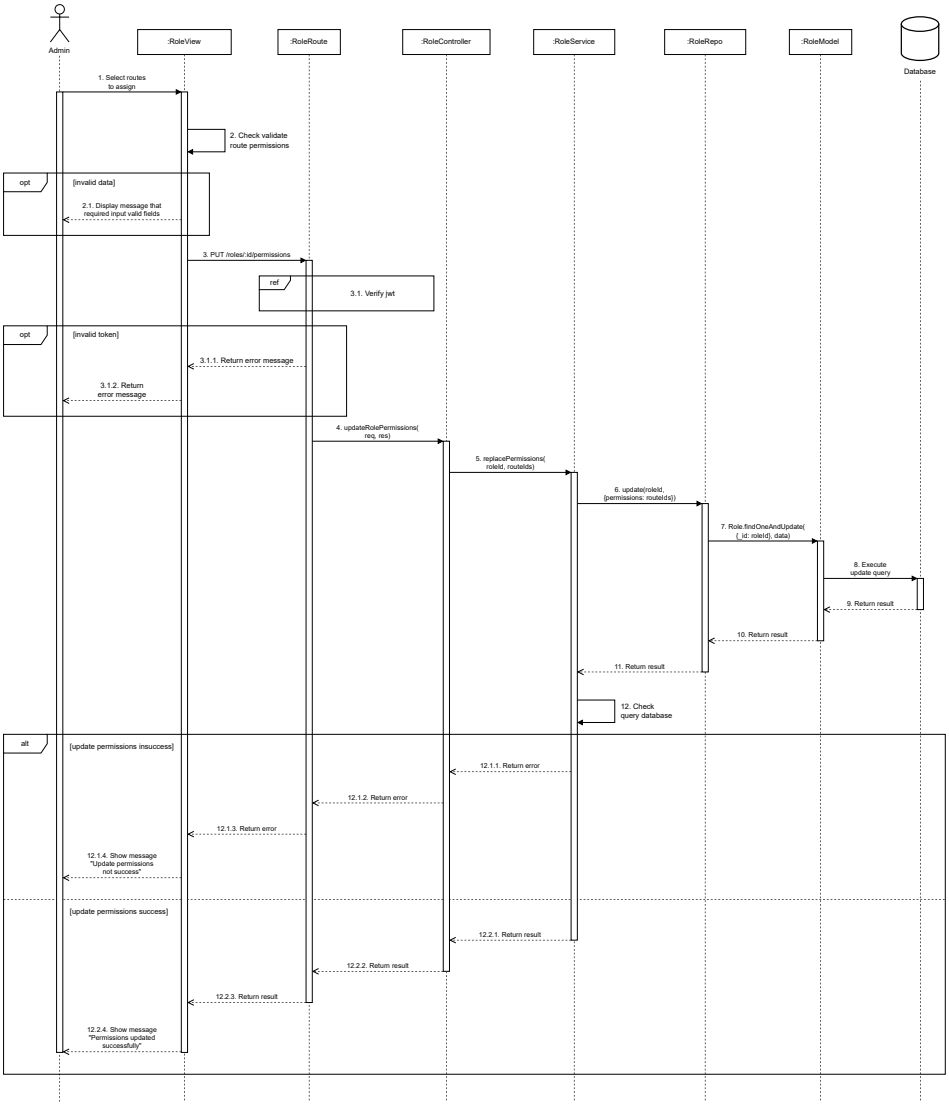

Figure 47: Sequence diagram of UC-06.06 Update Role Permissions

3.5 Manage Conversation

Manage Conversation covers the UC-27 private messaging module: listing and starting one-to-one conversations, opening a conversation, sending text messages and files, marking messages as read, recalling a message, and pinning or unpinning a conversation. Delivery is served both by the REST endpoints and by the realtime gateway in the same namespace.

a. Class Diagram

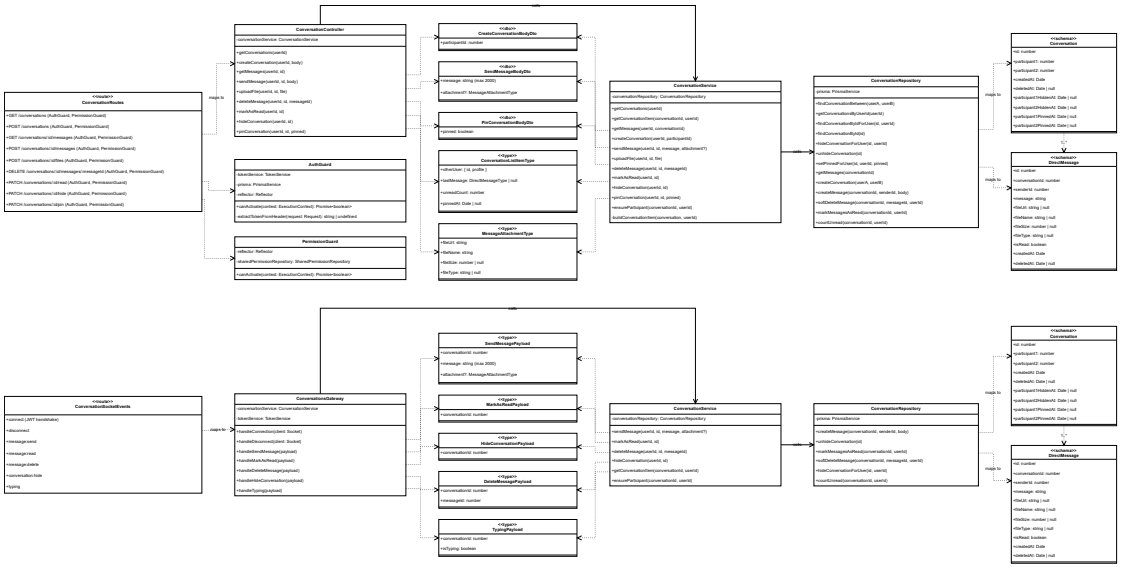

Figure 48: Class diagram of Manage Conversation

b. Sequence Diagram(s)

3.5.1 UC-27.01 – View Conversation List

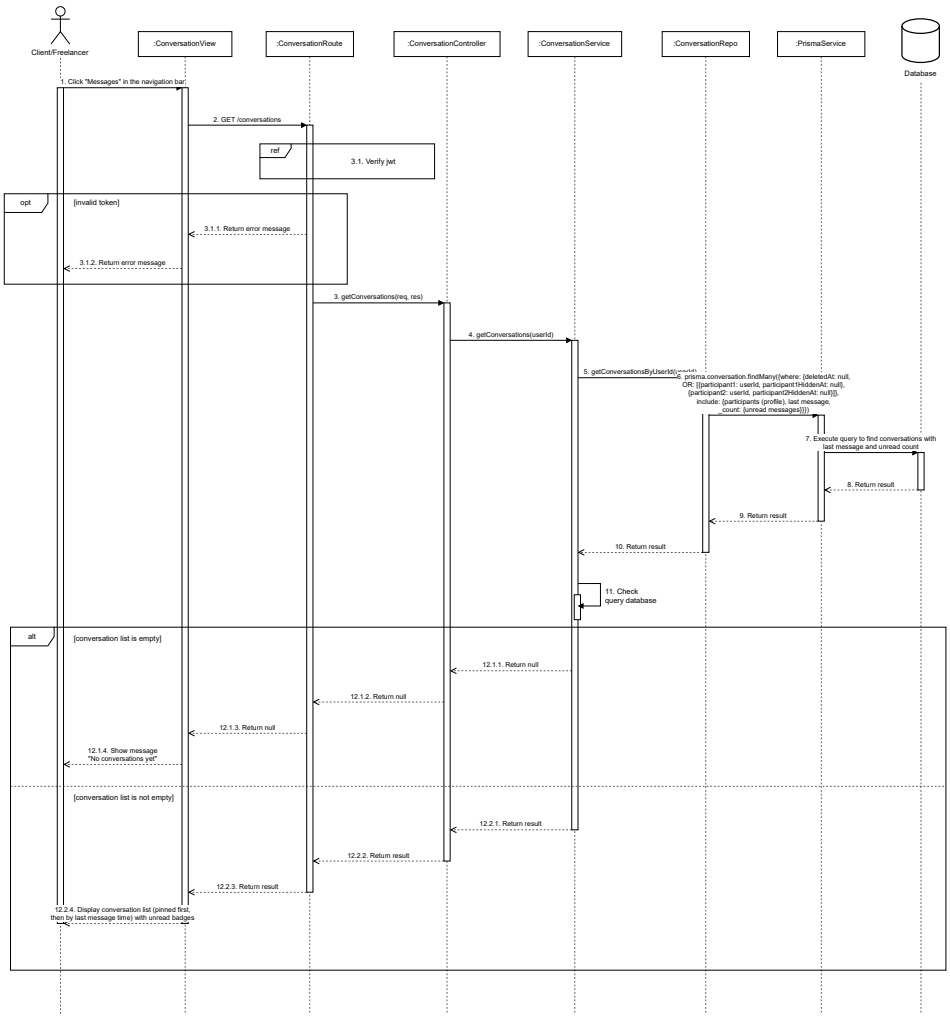

Figure 49: Sequence diagram of UC-27.01 View Conversation List

3.5.2 UC-27.02 – Start Conversation

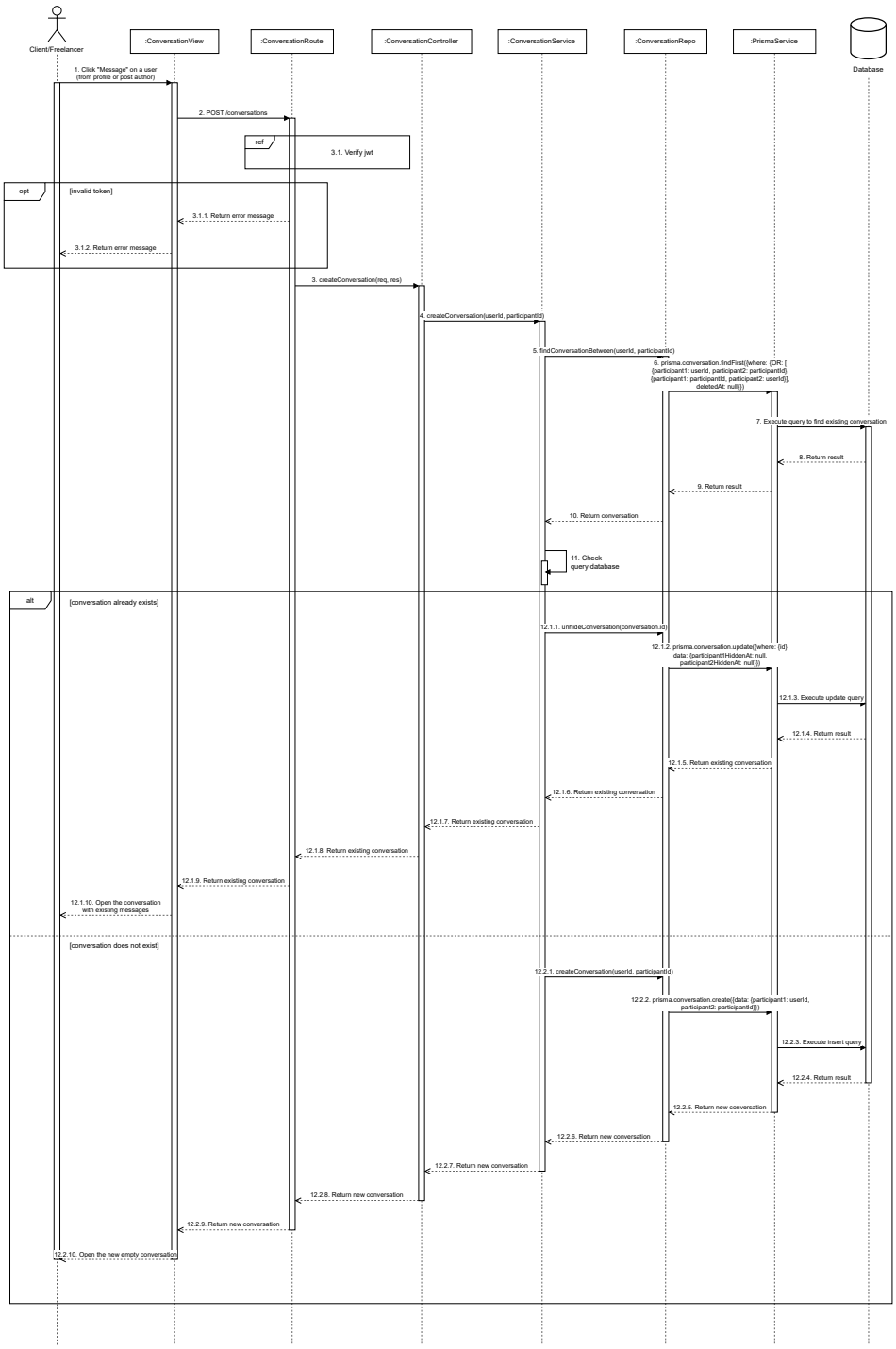

Figure 50: Sequence diagram of UC-27.02 Start Conversation

3.5.3 UC-27.03 – View Conversation Detail

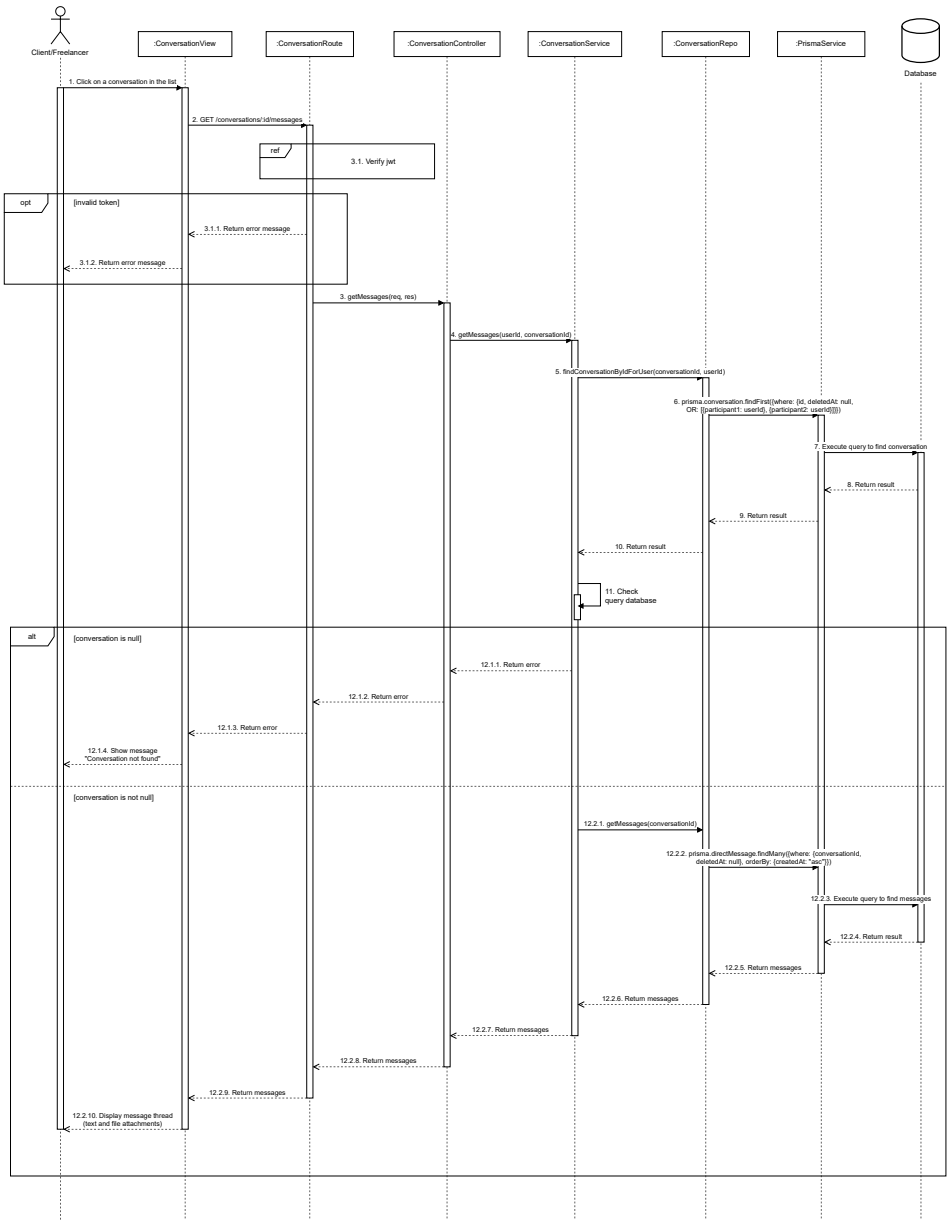

Figure 51: Sequence diagram of UC-27.03 View Conversation Detail

3.5.4 UC-27.04 – Send Message

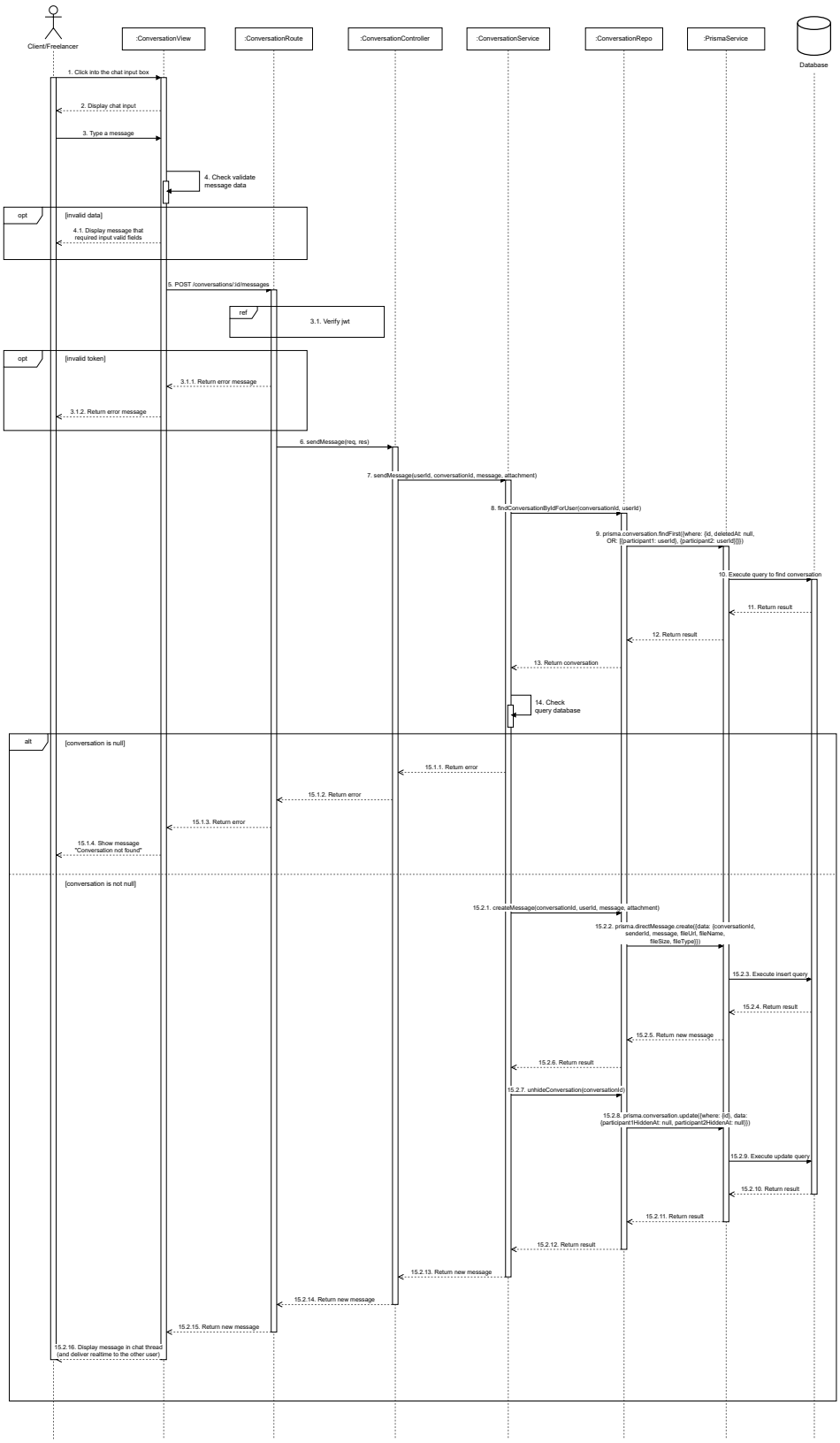

Figure 52: Sequence diagram of UC-27.04 Send Message

3.5.5 UC-27.05 – Mark Message as Read

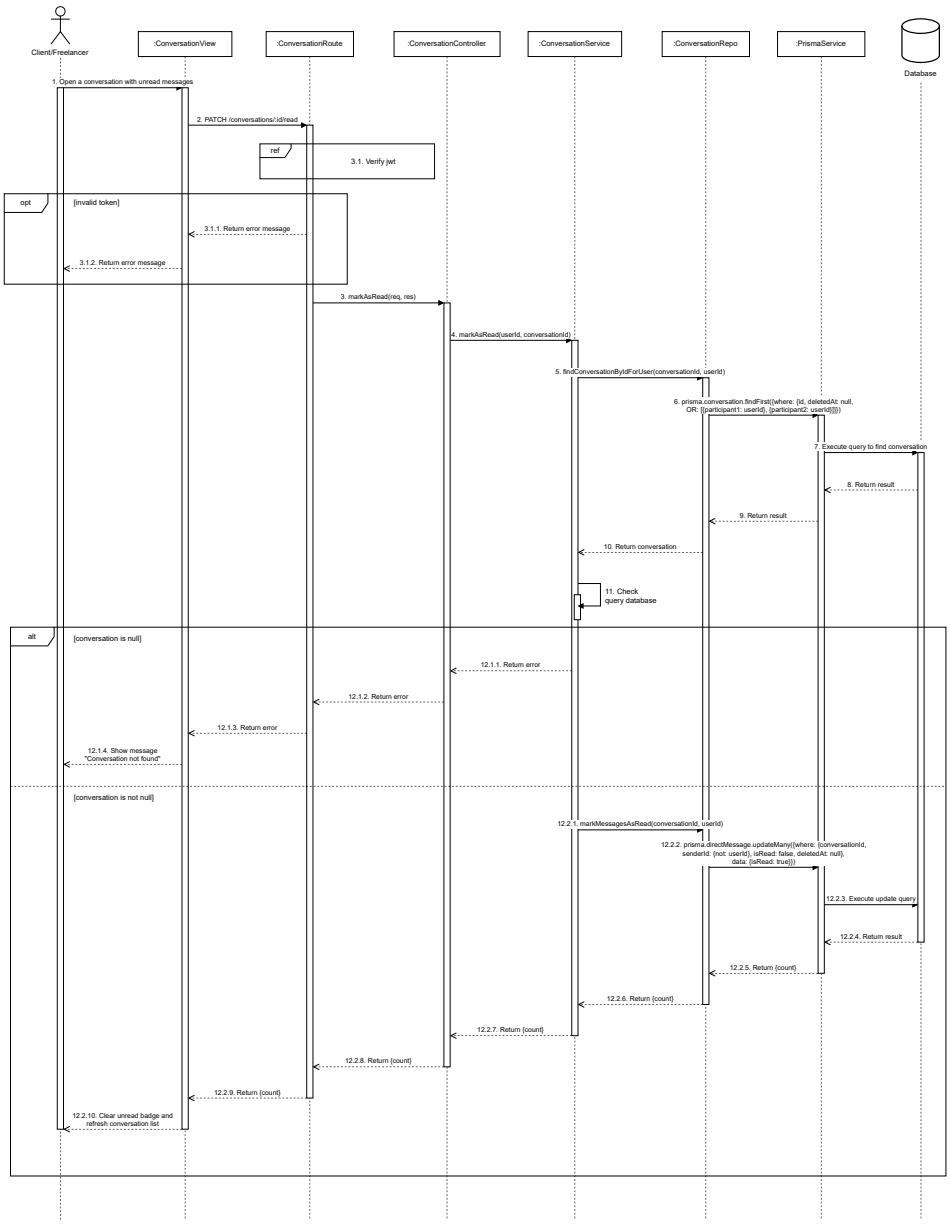

Figure 53: Sequence diagram of UC-27.05 Mark Message as Read

3.5.6 UC-27.06 – Delete/Recall Message

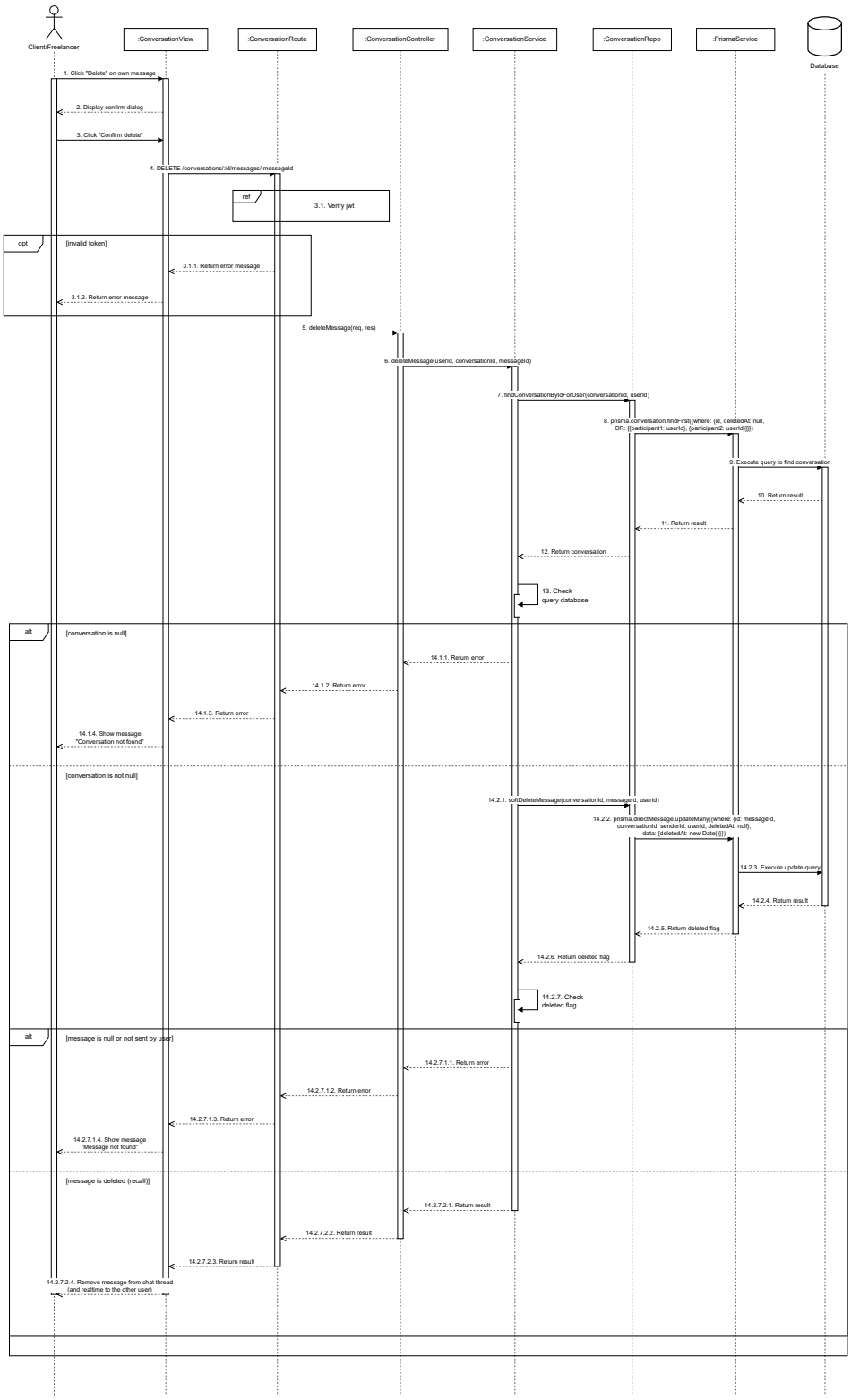

Figure 54: Sequence diagram of UC-27.06 Delete/Recall Message

3.5.7 UC-27.07 – Pin/Unpin Conversation

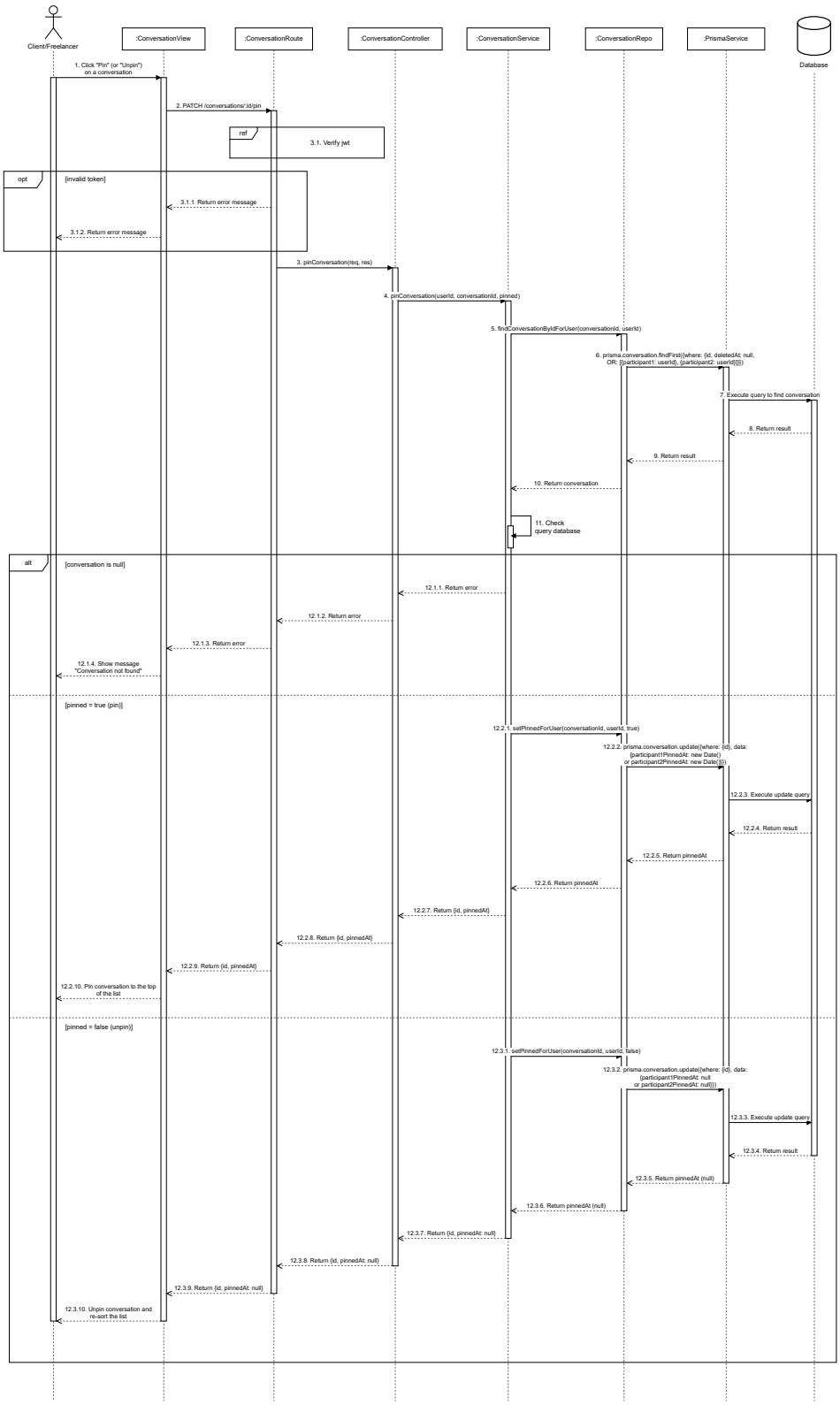

Figure 55: Sequence diagram of UC-27.07 Pin/Unpin Conversation

3.5.8 UC-27.08 – Send File

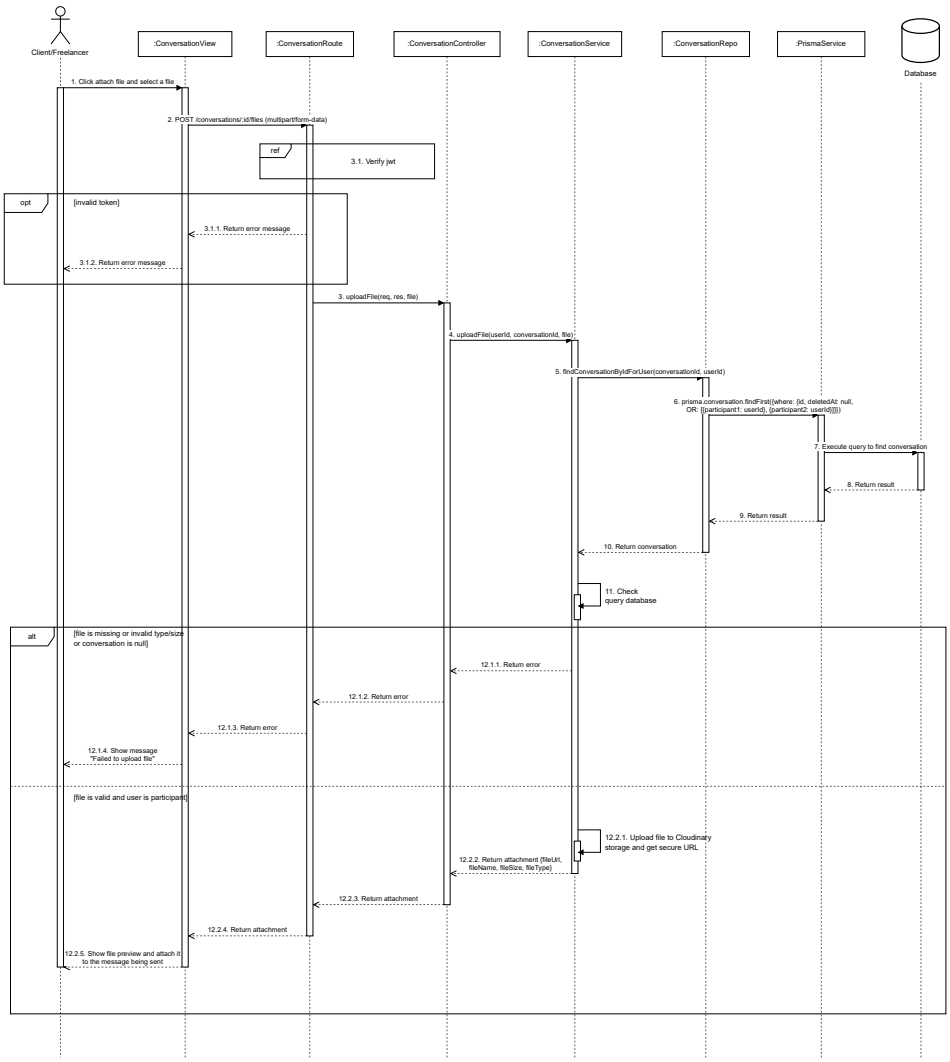

Figure 56: Sequence diagram of UC-27.08 Send File

3.6 Milestone Management

Milestone Management covers the UC-24 milestone lifecycle within an active contract: listing milestones with financial metrics, viewing milestone details, creating project milestones, modifying terms, deleting draft milestones, requesting deliverable revisions, approving deliverables, and updating execution progress.

a. Class Diagram

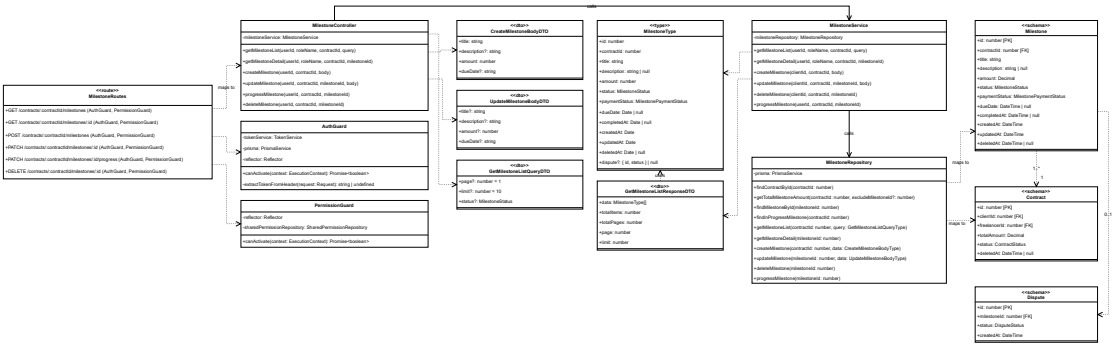

Figure 57: Class diagram of Milestone Management

b. Sequence Diagram(s)

3.6.1 UC-24.01 – View Milestone List

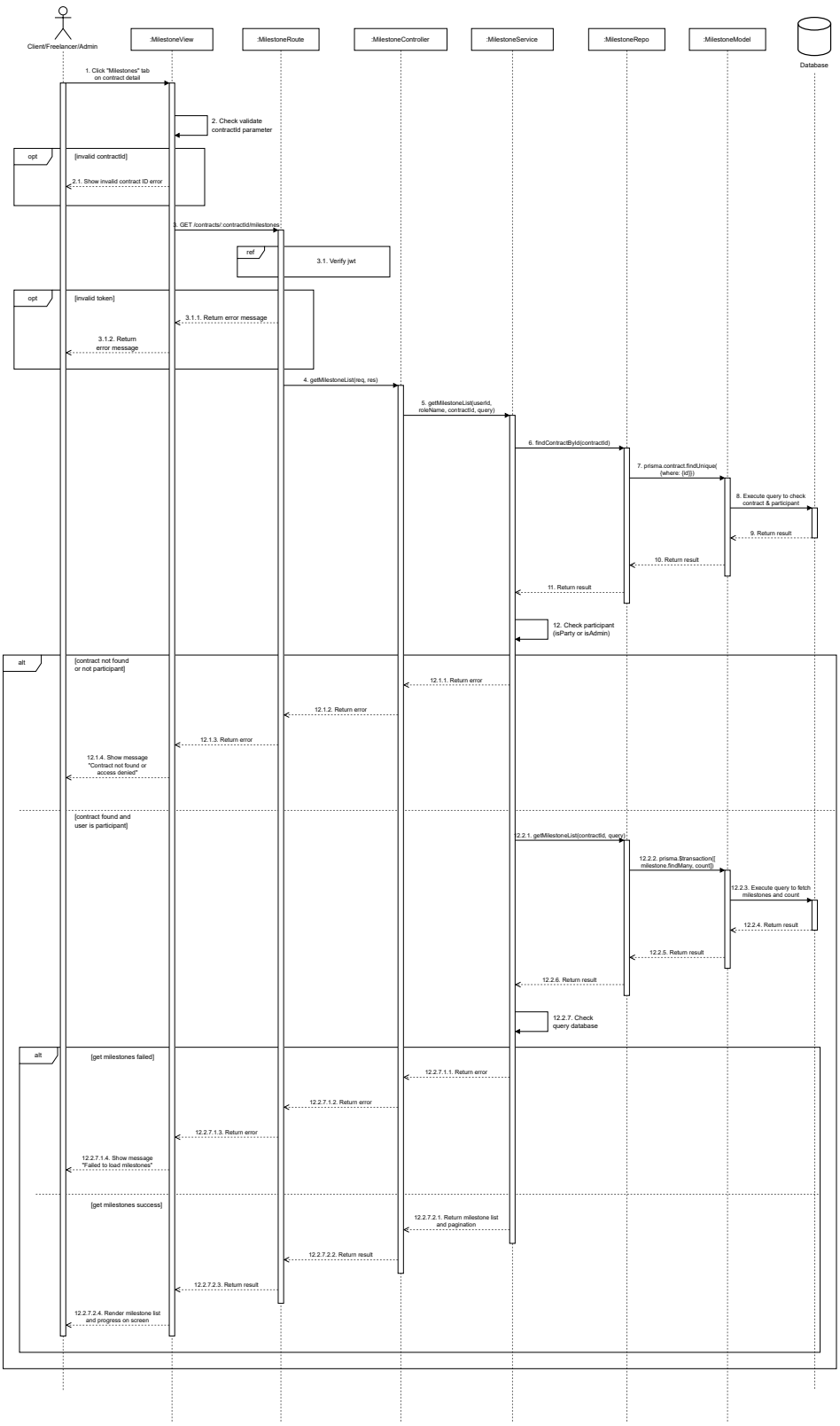

Figure 58: Sequence diagram of UC-24.01 View Milestone List

3.6.2 UC-24.02 – View Milestone Detail

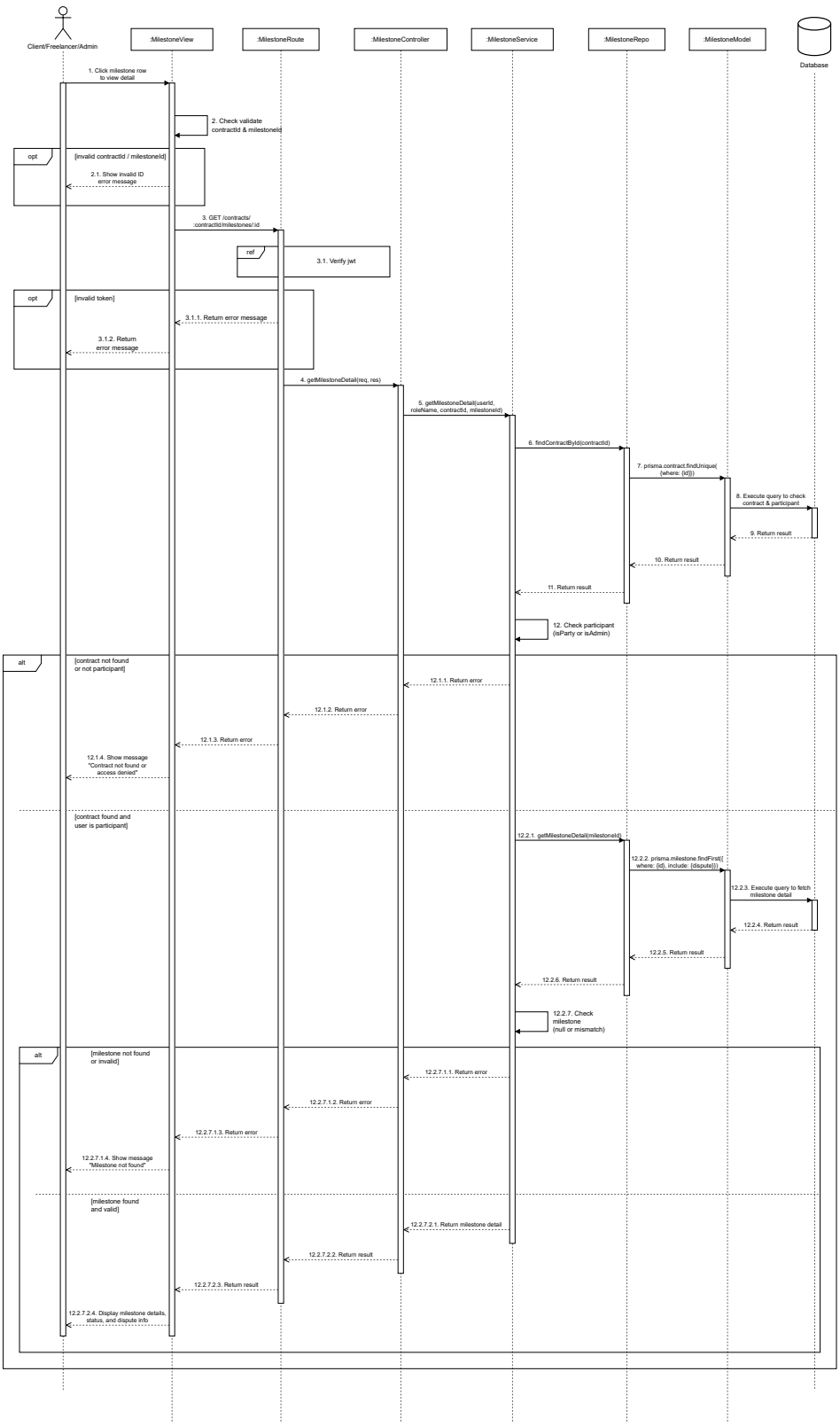

Figure 59: Sequence diagram of UC-24.02 View Milestone Detail

3.6.3 UC-24.03 – Create Milestone

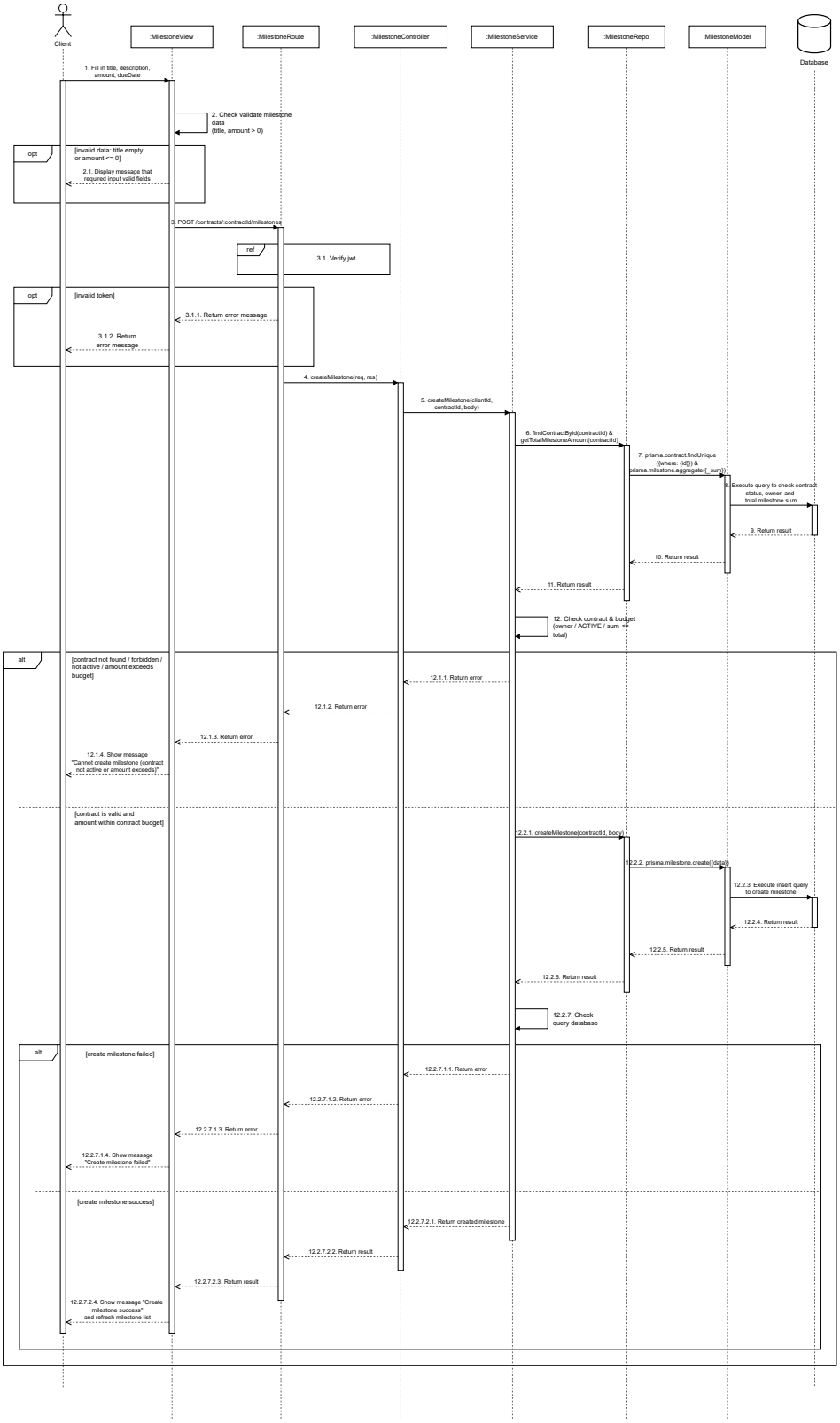

Figure 60: Sequence diagram of UC-24.03 Create Milestone

3.6.4 UC-24.04 – Update Milestone

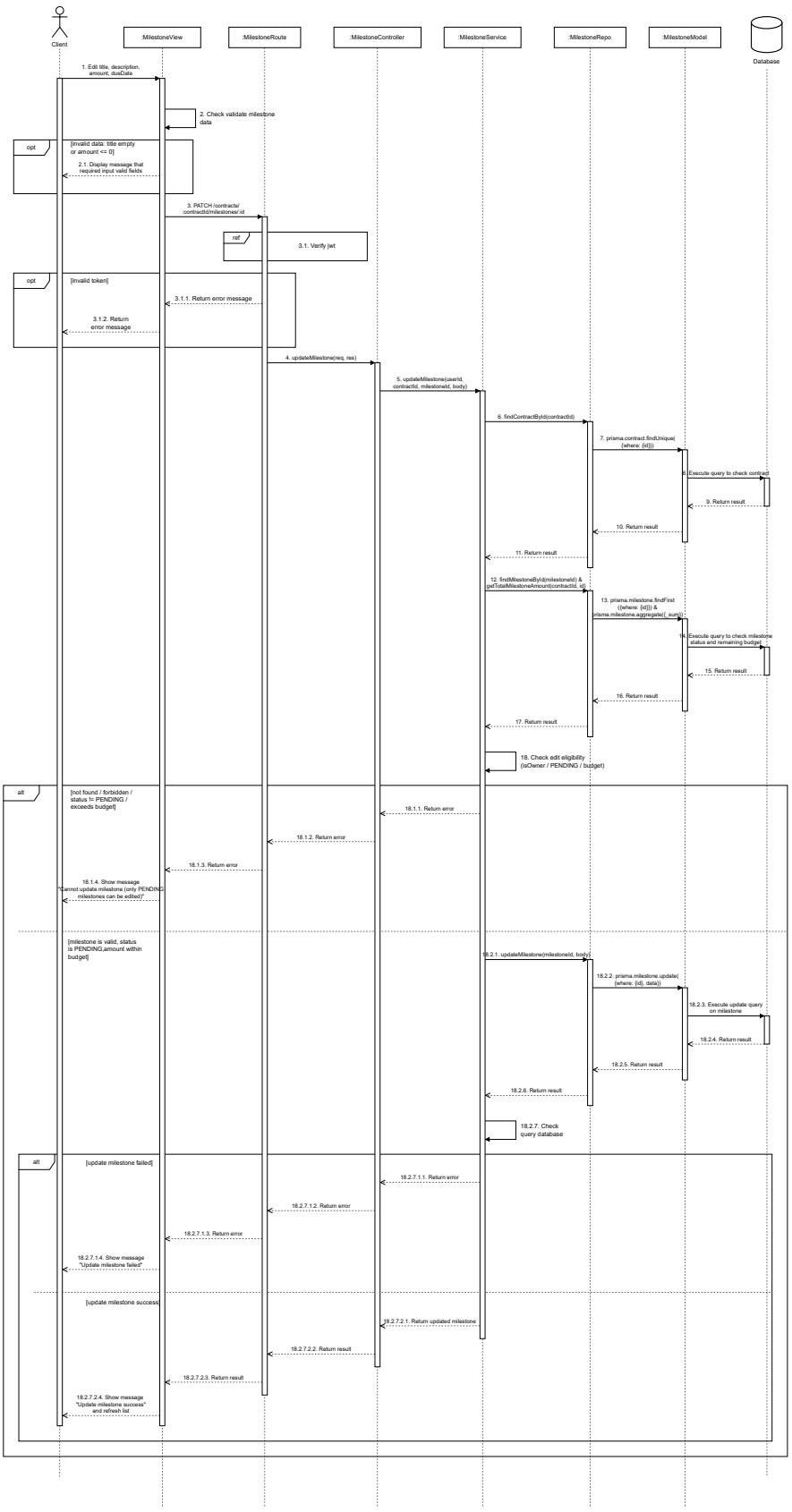

Figure 61: Sequence diagram of UC-24.04 Update Milestone

3.6.5 UC-24.05 – Delete Milestone

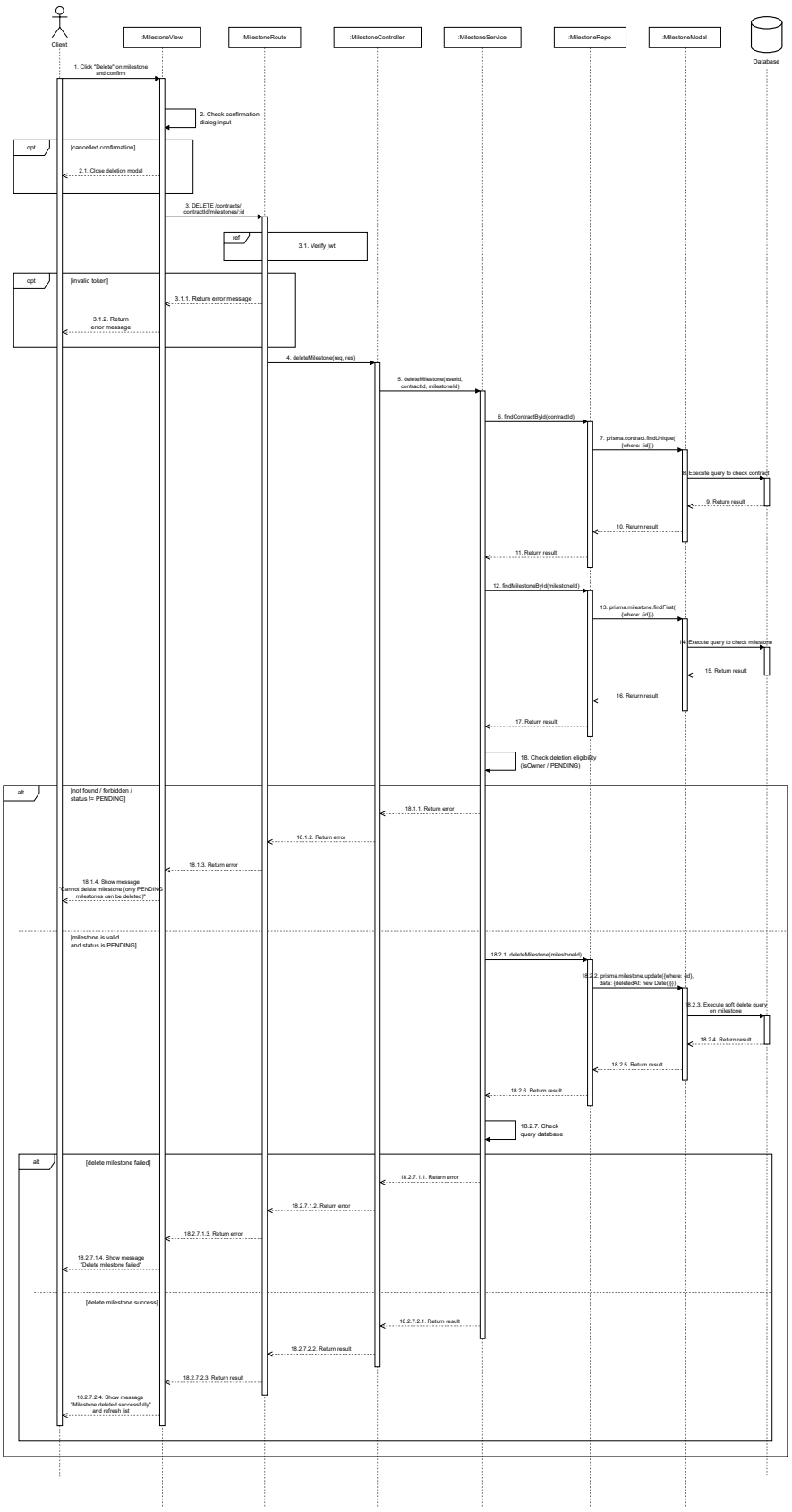

Figure 62: Sequence diagram of UC-24.05 Delete Milestone

3.6.6 UC-24.06 – Request Milestone Changes

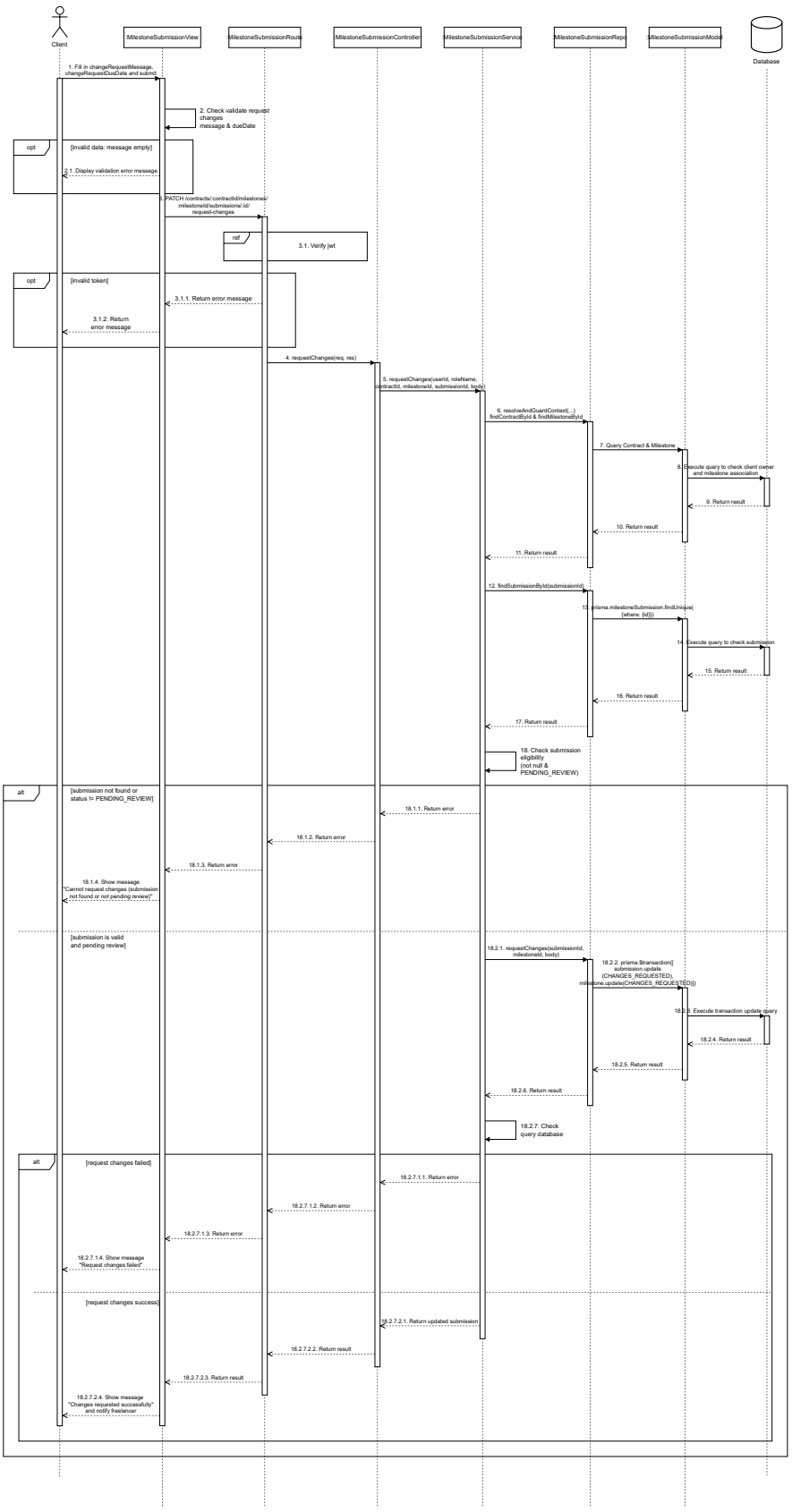

Figure 63: Sequence diagram of UC-24.06 Request Milestone Changes

3.6.7 UC-24.07 – Approve Milestone

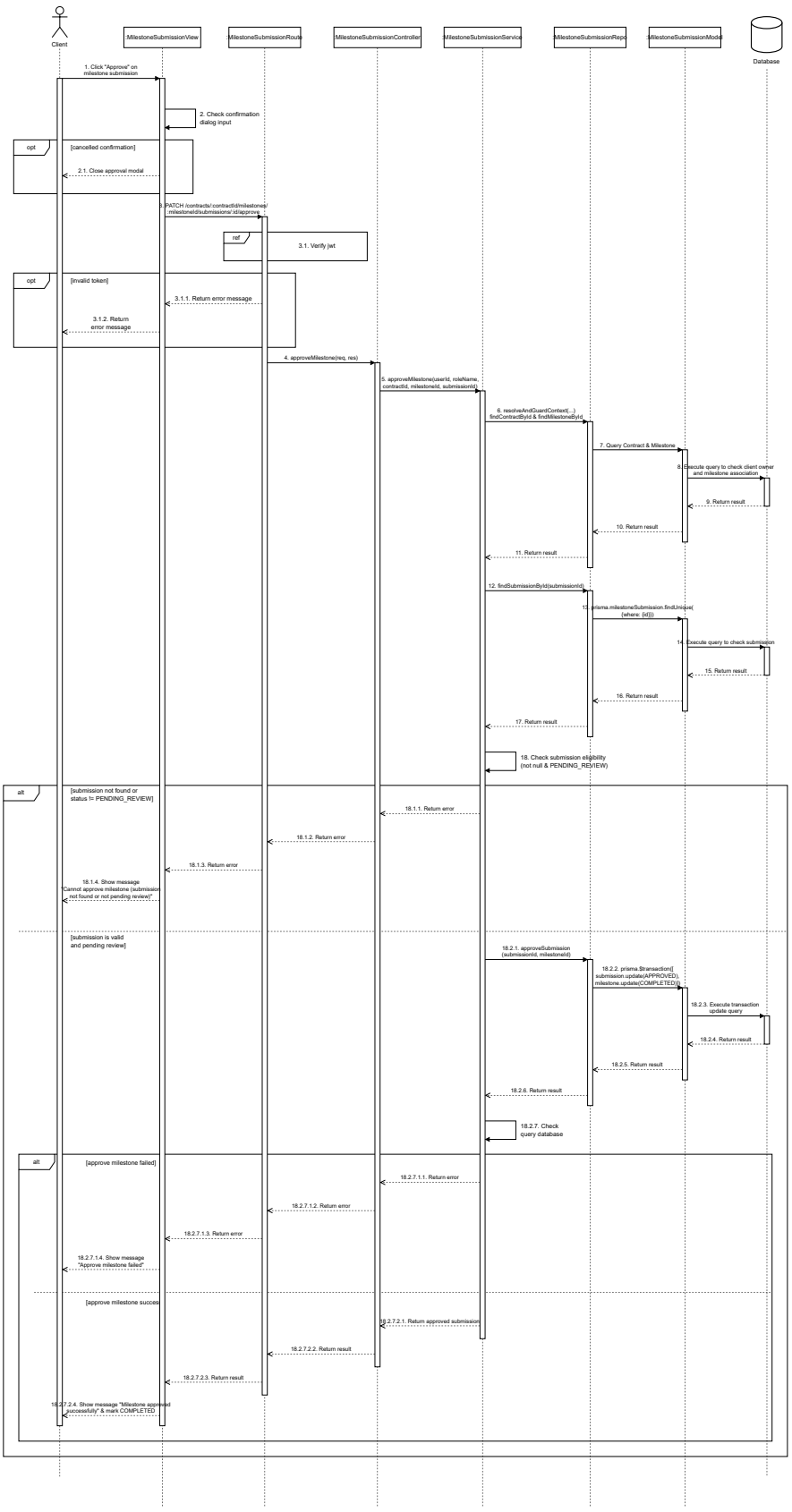

Figure 64: Sequence diagram of UC-24.07 Approve Milestone

3.6.8 UC-24.08 – Update Milestone Progress

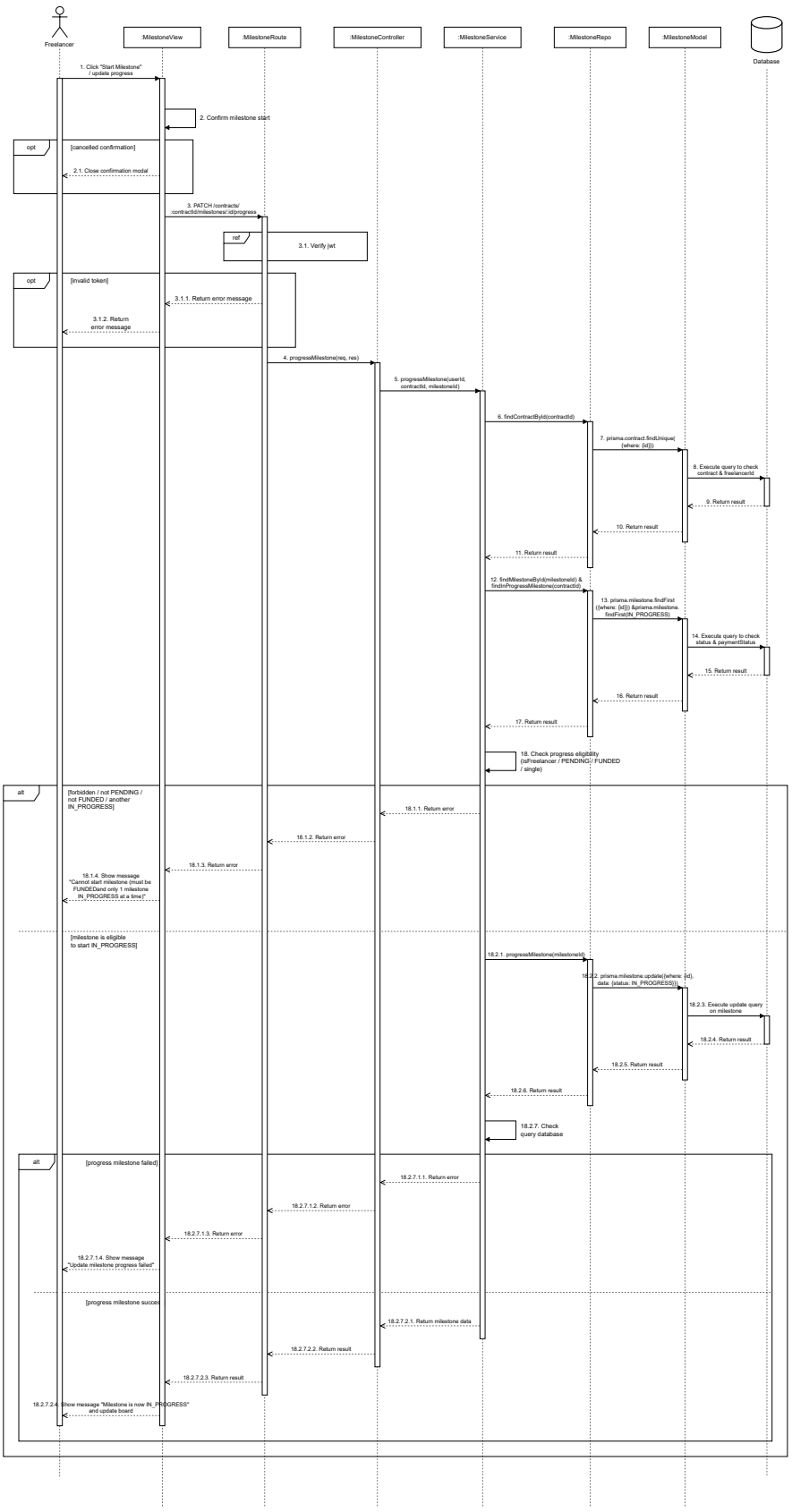

Figure 65: Sequence diagram of UC-24.08 Update Milestone Progress

3.7 Milestone File Management

Milestone File Management covers the UC-25 auxiliary attachment workflow: retrieving files associated with specific milestones, uploading requirement documents or project assets, and removing obsolete files.

a. Class Diagram

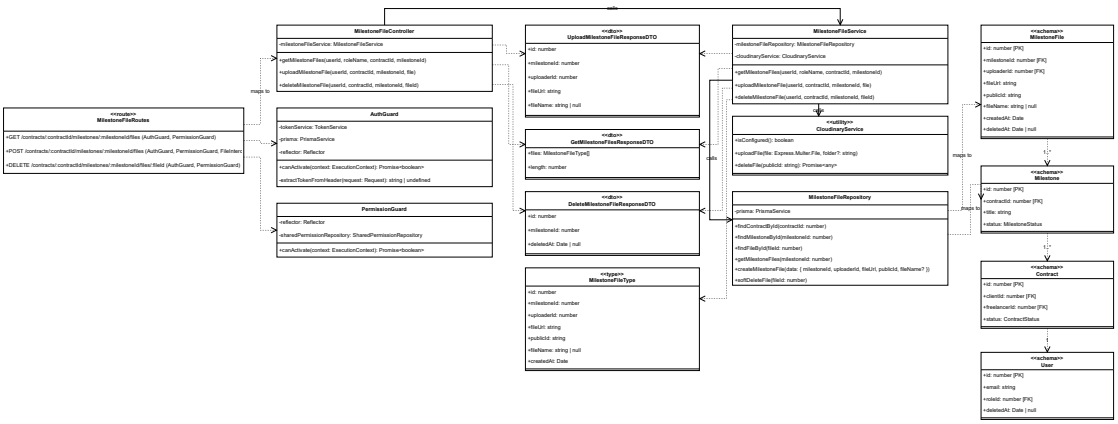

Figure 66: Class diagram of Milestone File Management

b. Sequence Diagram(s)

3.7.1 UC-25.01 – View Milestone Files

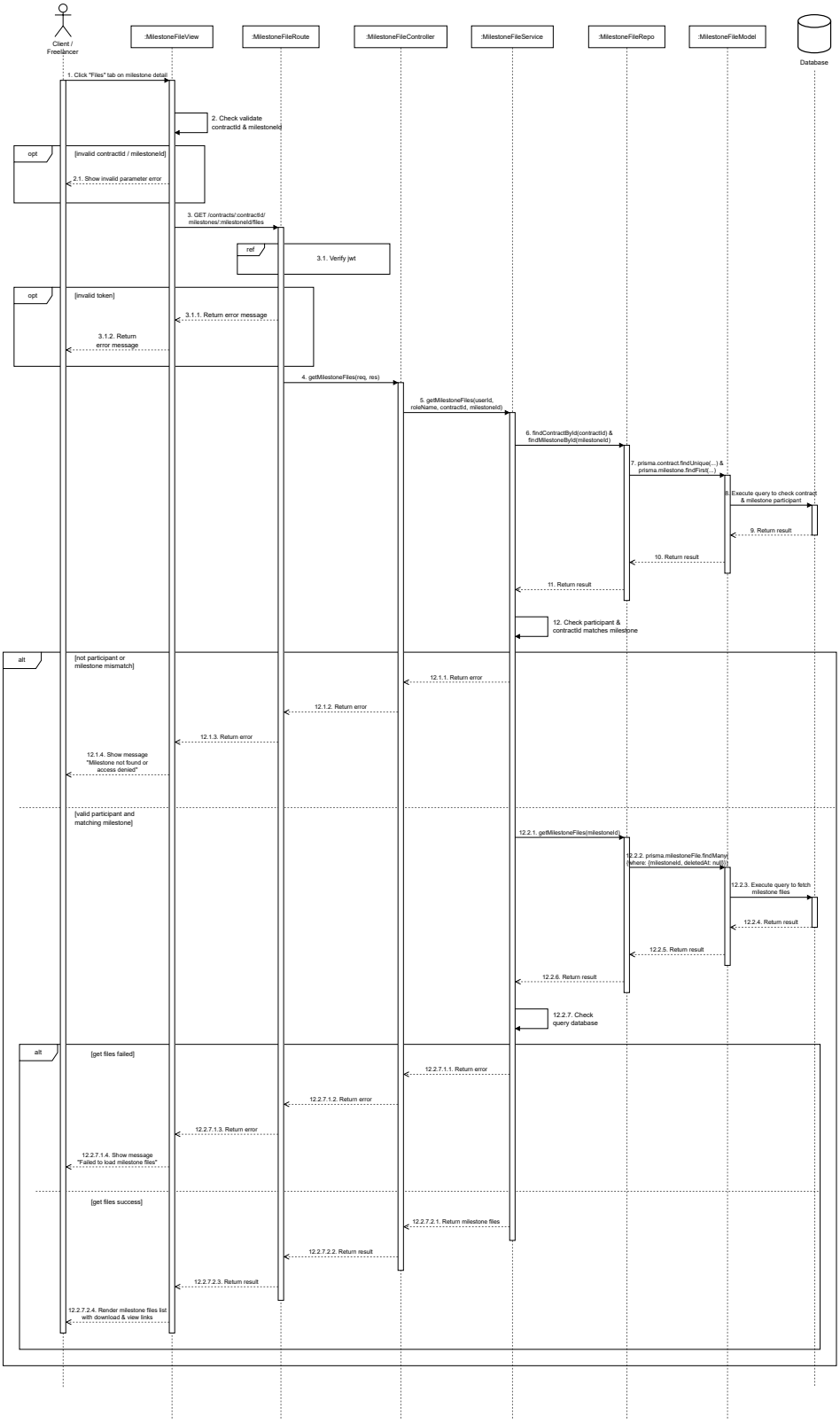

Figure 67: Sequence diagram of UC-25.01 View Milestone Files

3.7.2 UC-25.02 – Upload Milestone File

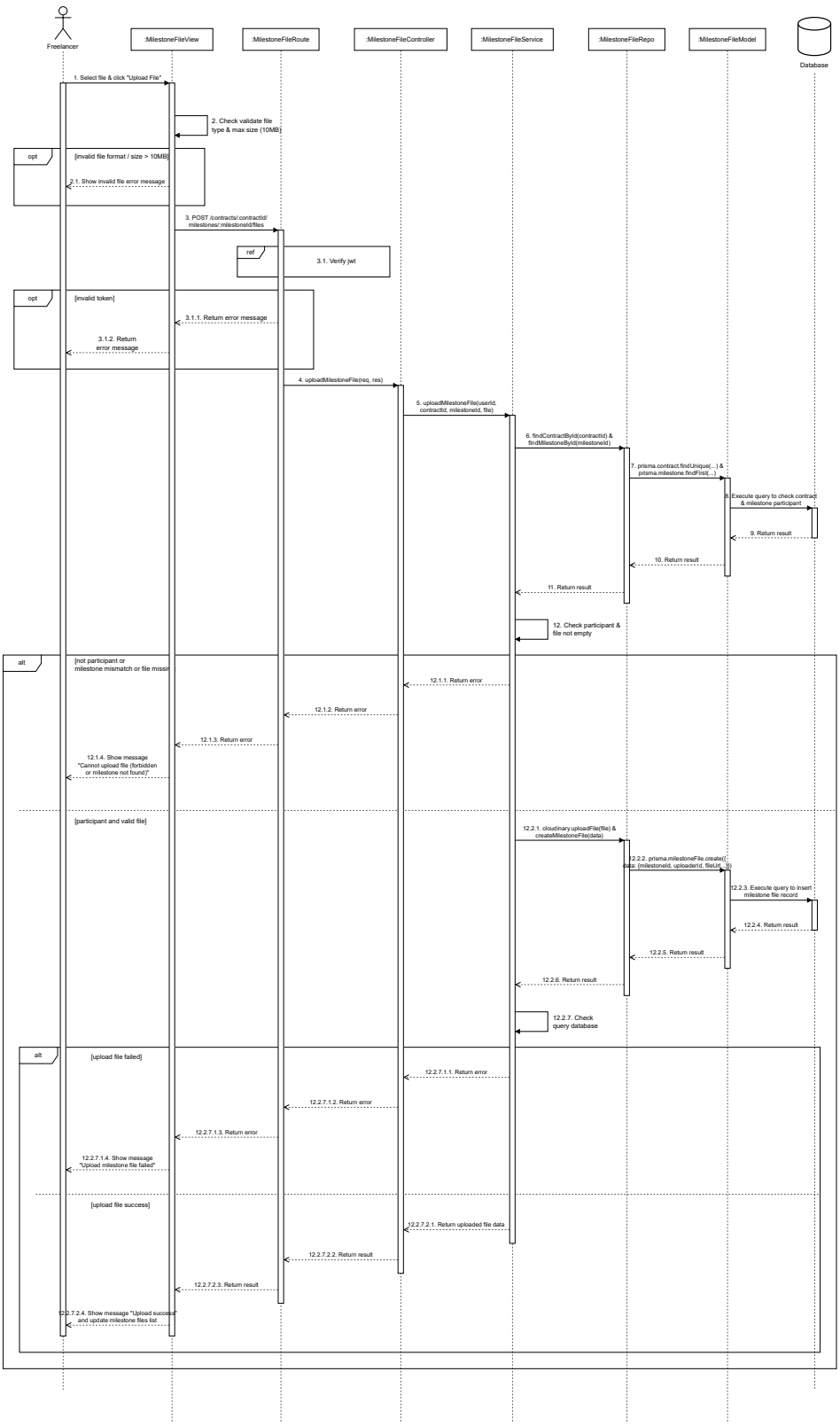

Figure 68: Sequence diagram of UC-25.02 Upload Milestone File

3.7.3 UC-25.03 – Delete Milestone File

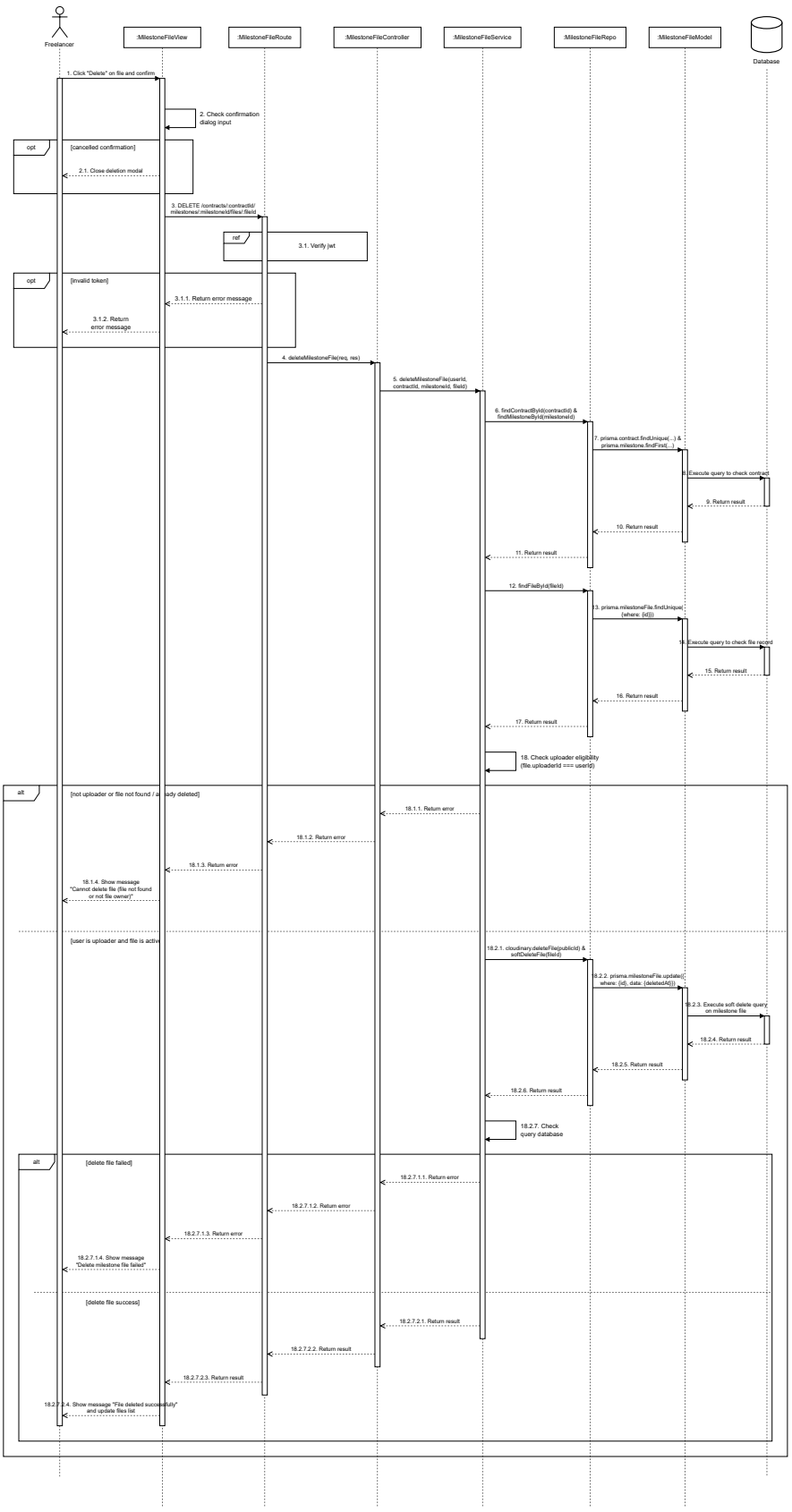

Figure 69: Sequence diagram of UC-25.03 Delete Milestone File

3.8 Milestone Submission Management

Milestone Submission Management covers the UC-26 deliverable submission lifecycle: submitting completed work packages with comments and attachments, withdrawing or deleting pending submissions, listing past submission packages, and inspecting submission package details.

a. Class Diagram

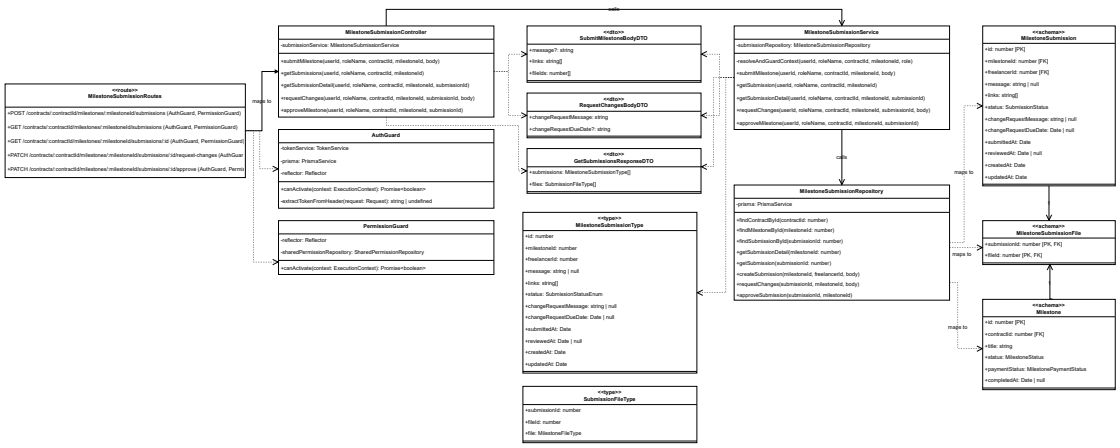

Figure 70: Class diagram of Milestone Submission Management

b. Sequence Diagram(s)

3.8.1 UC-26.01 – Submit Milestone

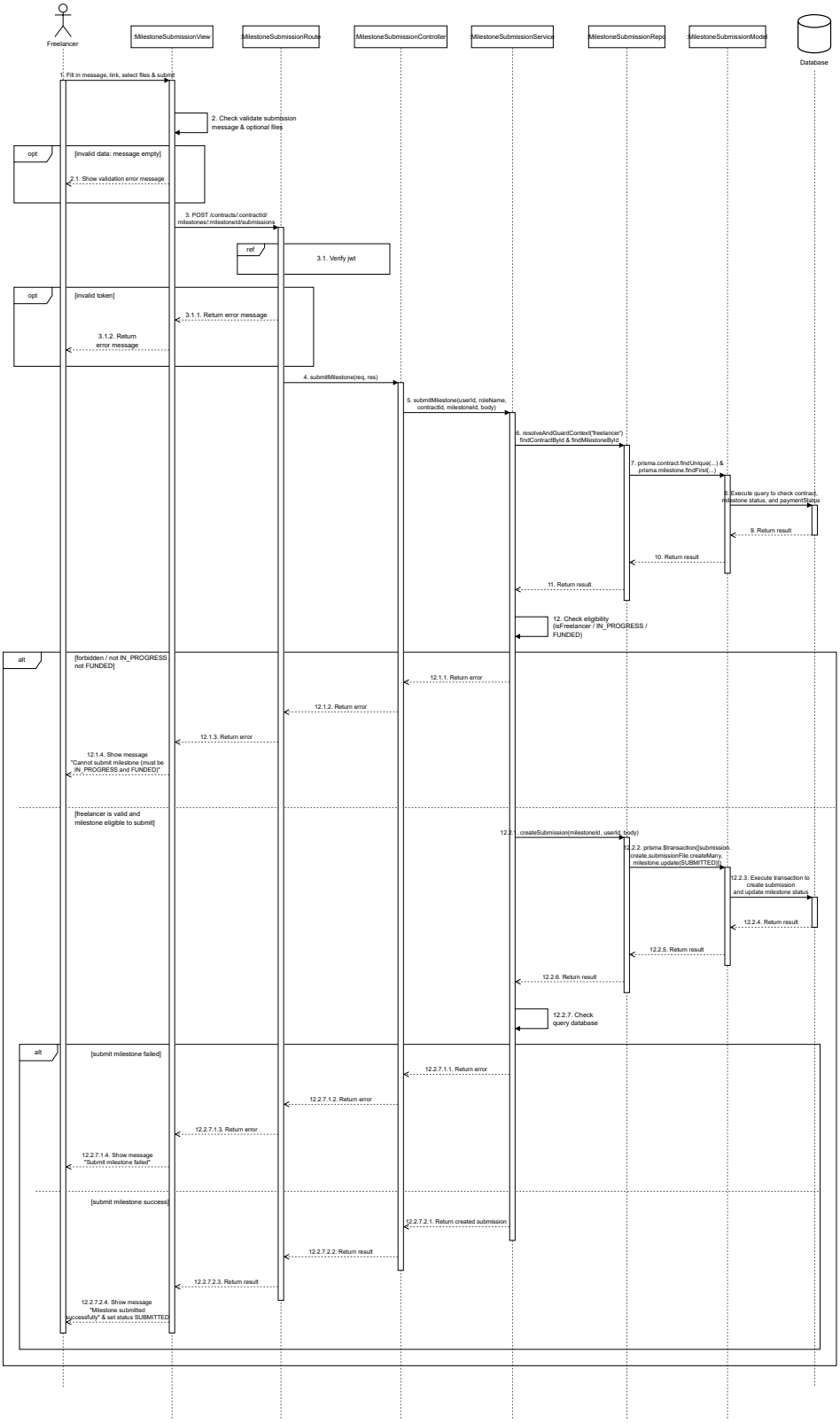

Figure 71: Sequence diagram of UC-26.01 Submit Milestone

3.8.2 UC-26.02 – Delete Milestone Submission

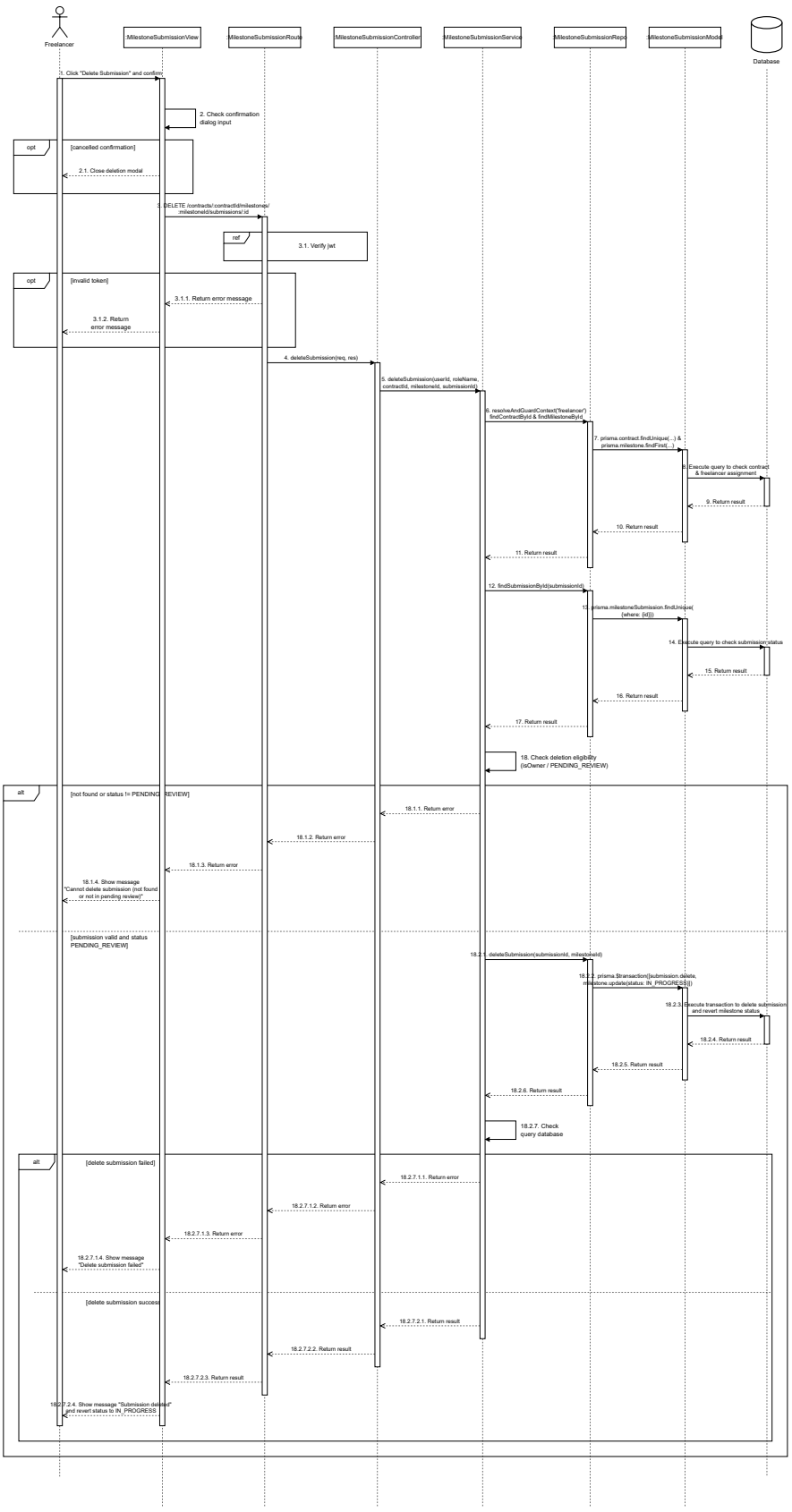

Figure 72: Sequence diagram of UC-26.02 Delete Milestone Submission

3.8.3 UC-26.03 – View Milestone Submission List

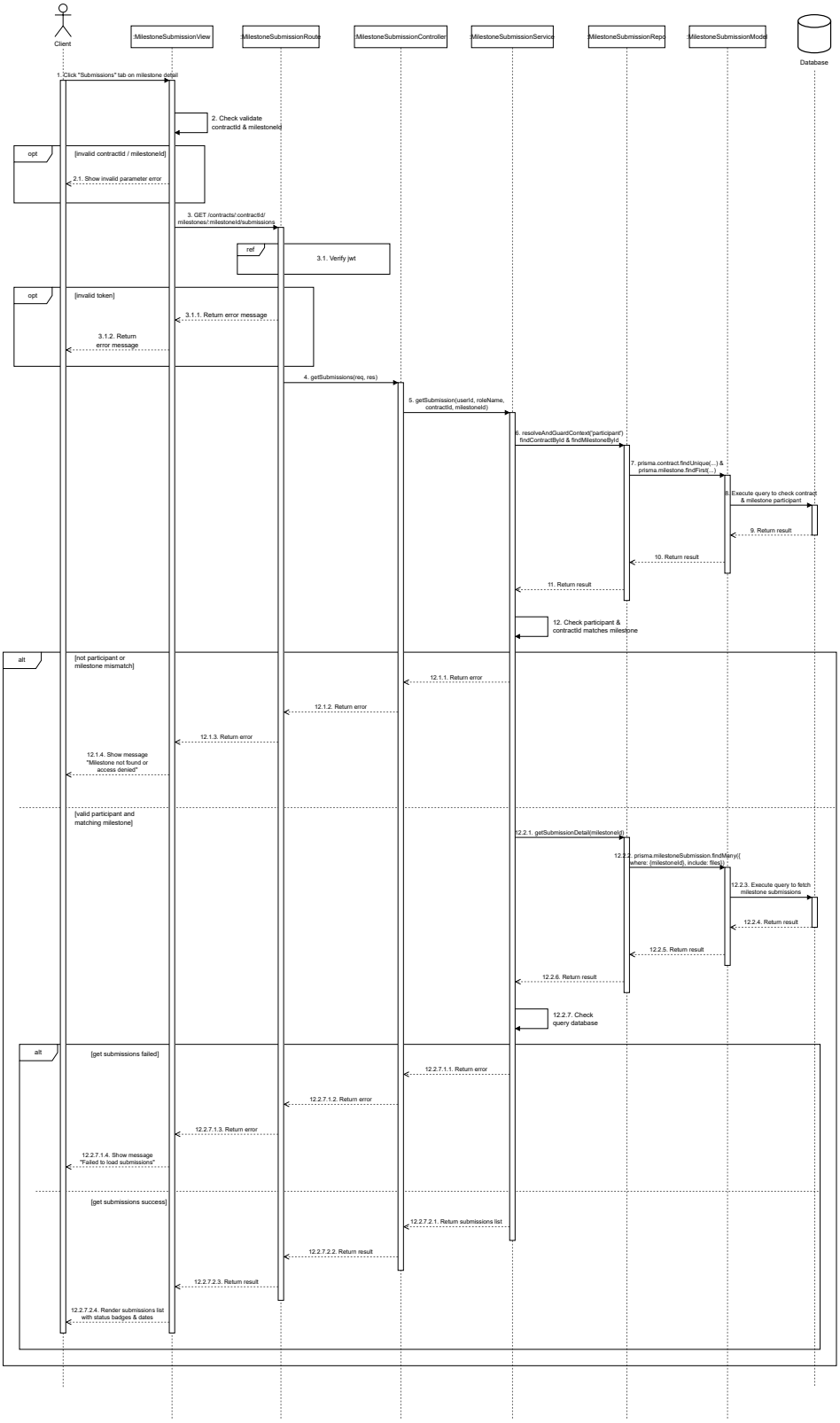

Figure 73: Sequence diagram of UC-26.03 View Milestone Submission List

3.8.4 UC-26.04 – View Milestone Submission Detail

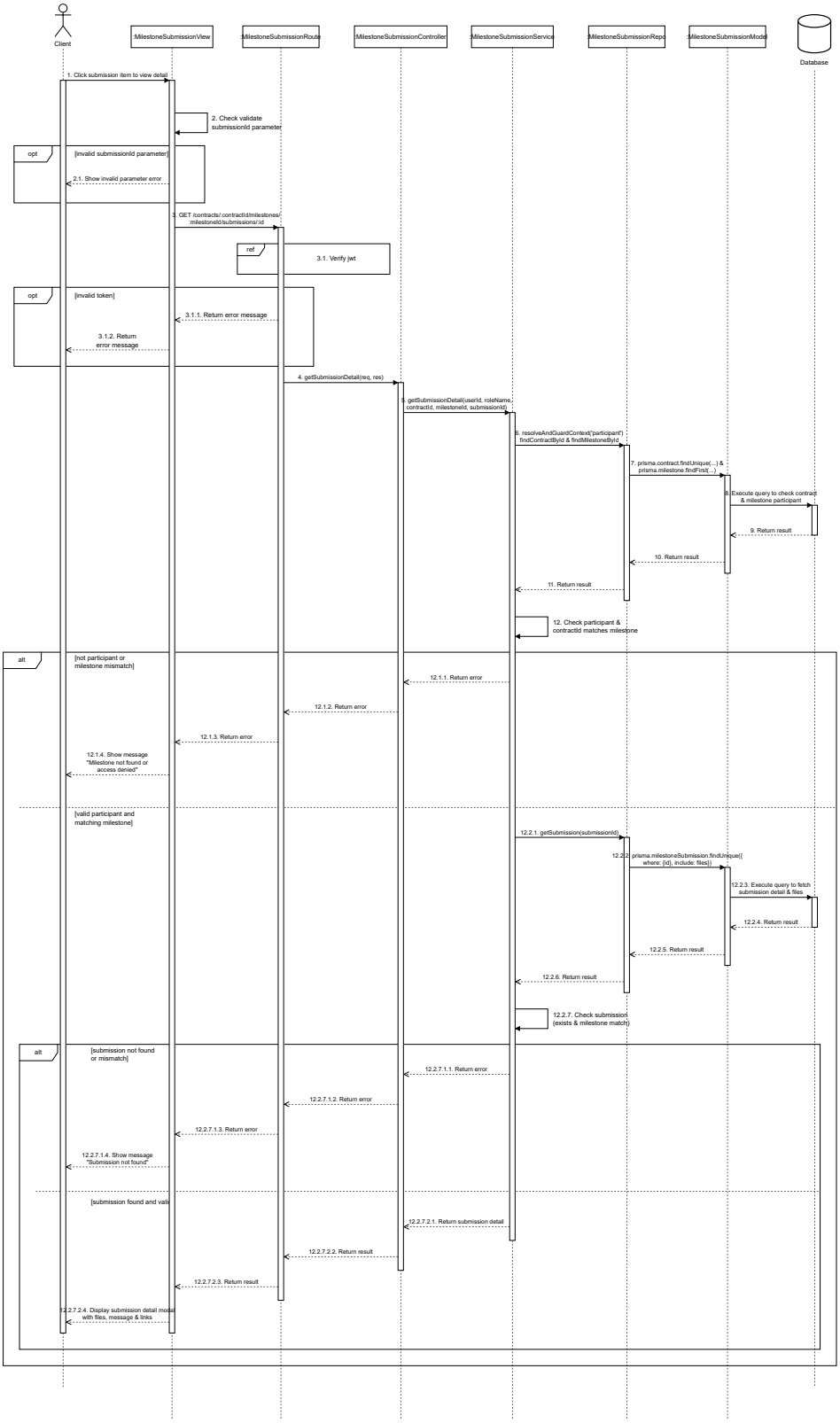

Figure 74: Sequence diagram of UC-26.04 View Milestone Submission Detail

3.9 Permission Management

Permission Management exposes the route-level permission catalogue so that an administrator can inspect the permissions available for role assignment.

a. Class Diagram

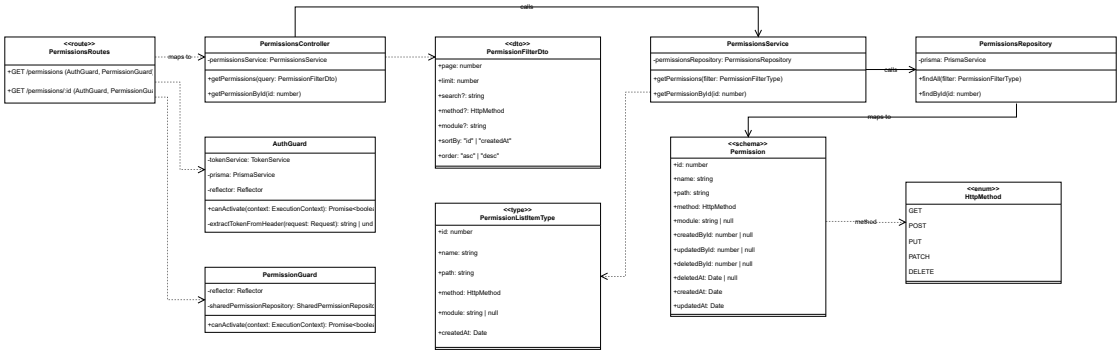

Figure 75: Class diagram of Permission Management

b. Sequence Diagram(s)

3.9.1 UC-15.01 – View Permission List

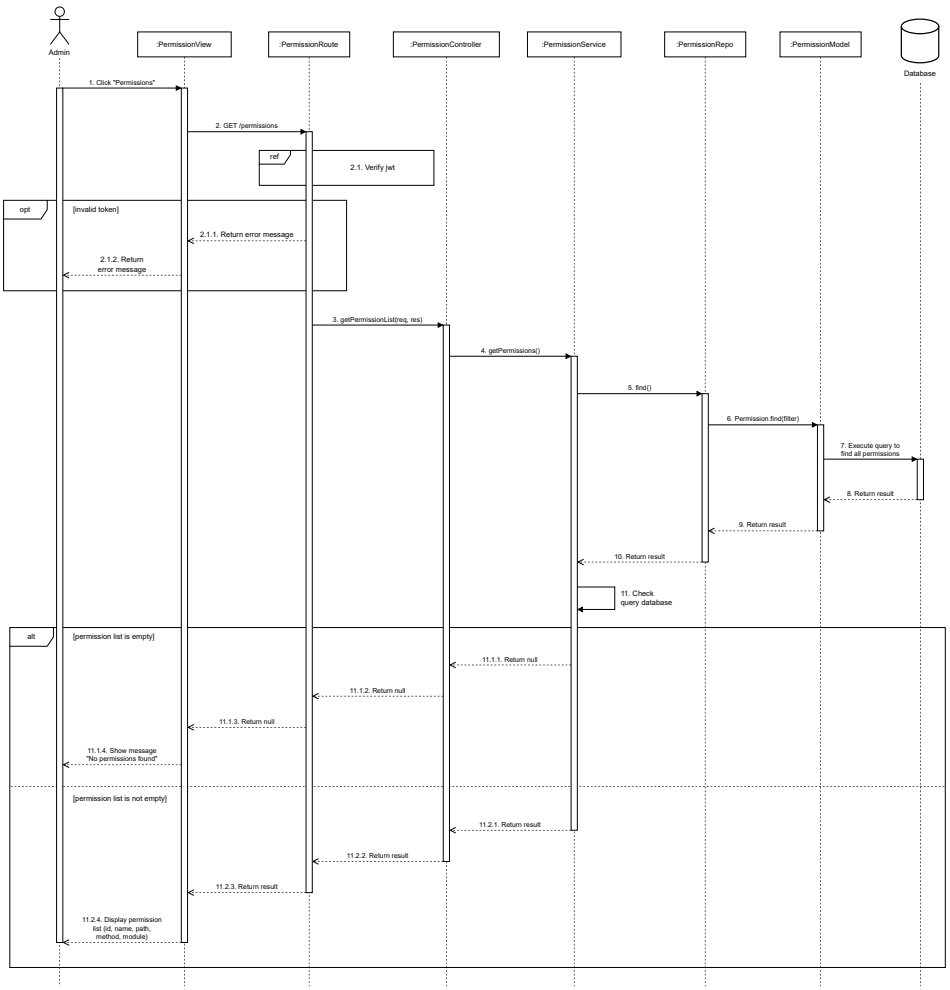

Figure 76: Sequence diagram of UC-15.01 View Permission List

3.9.2 UC-15.02 – View Permission Detail

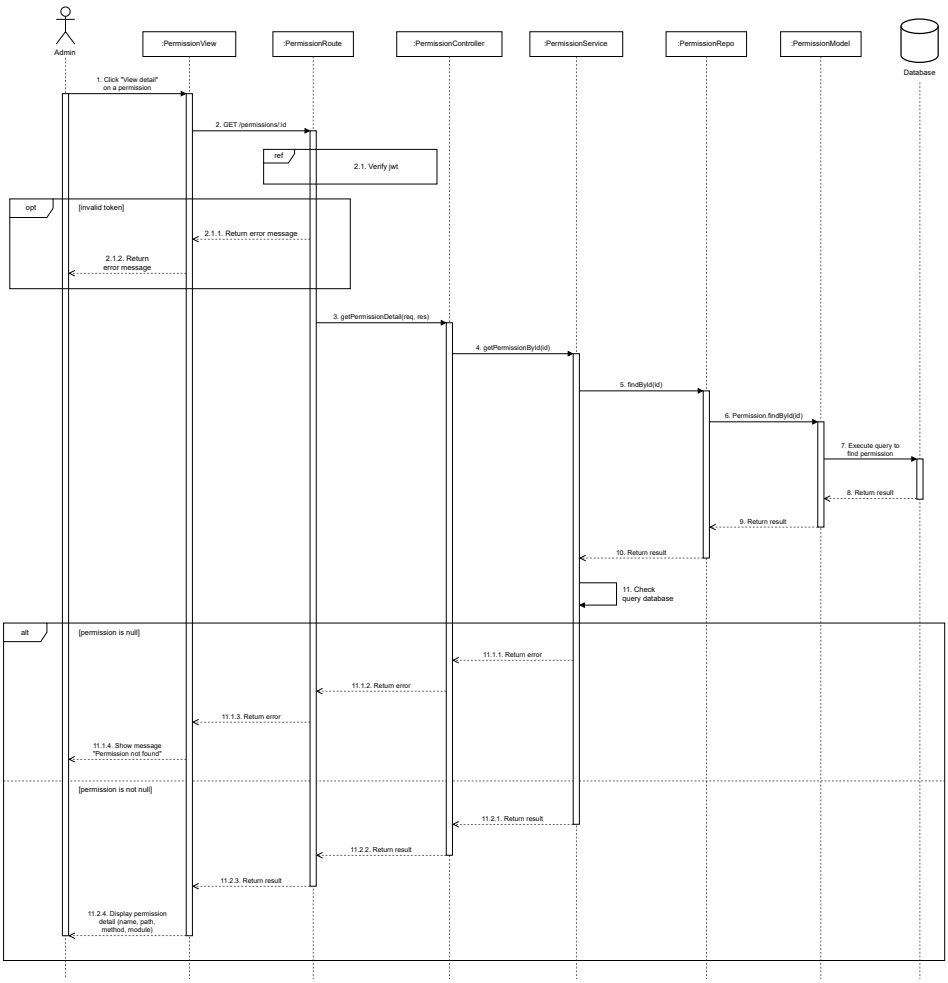

Figure 77: Sequence diagram of UC-15.02 View Permission Detail

3.10 Session Management

Session Management gives an authenticated user visibility over the devices that currently hold a refresh-token session and allows a session to be revoked remotely.

a. Class Diagram

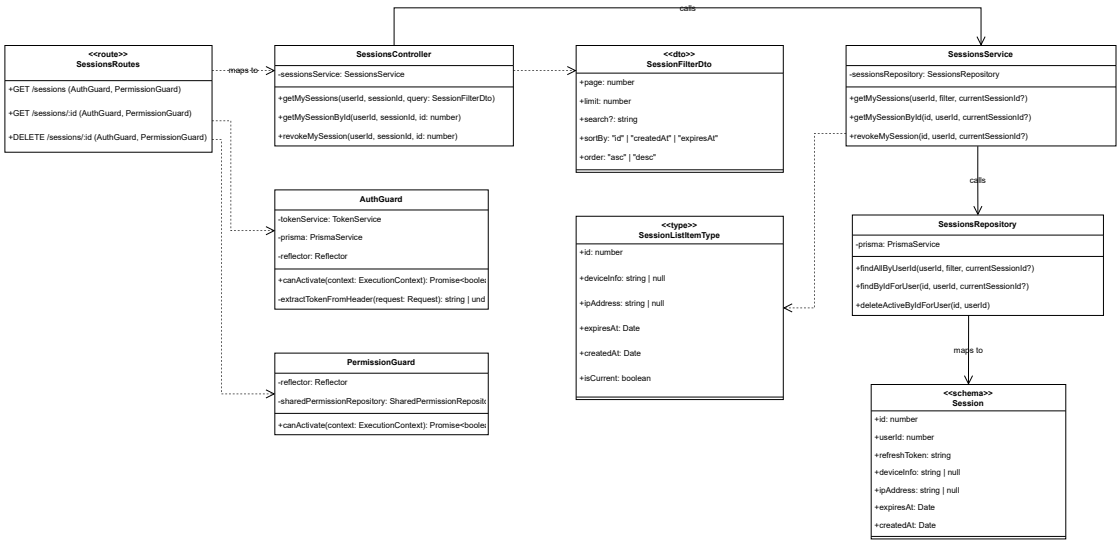

Figure 78: Class diagram of Session Management

b. Sequence Diagram(s)

3.10.1 UC-16.01 – View Session List

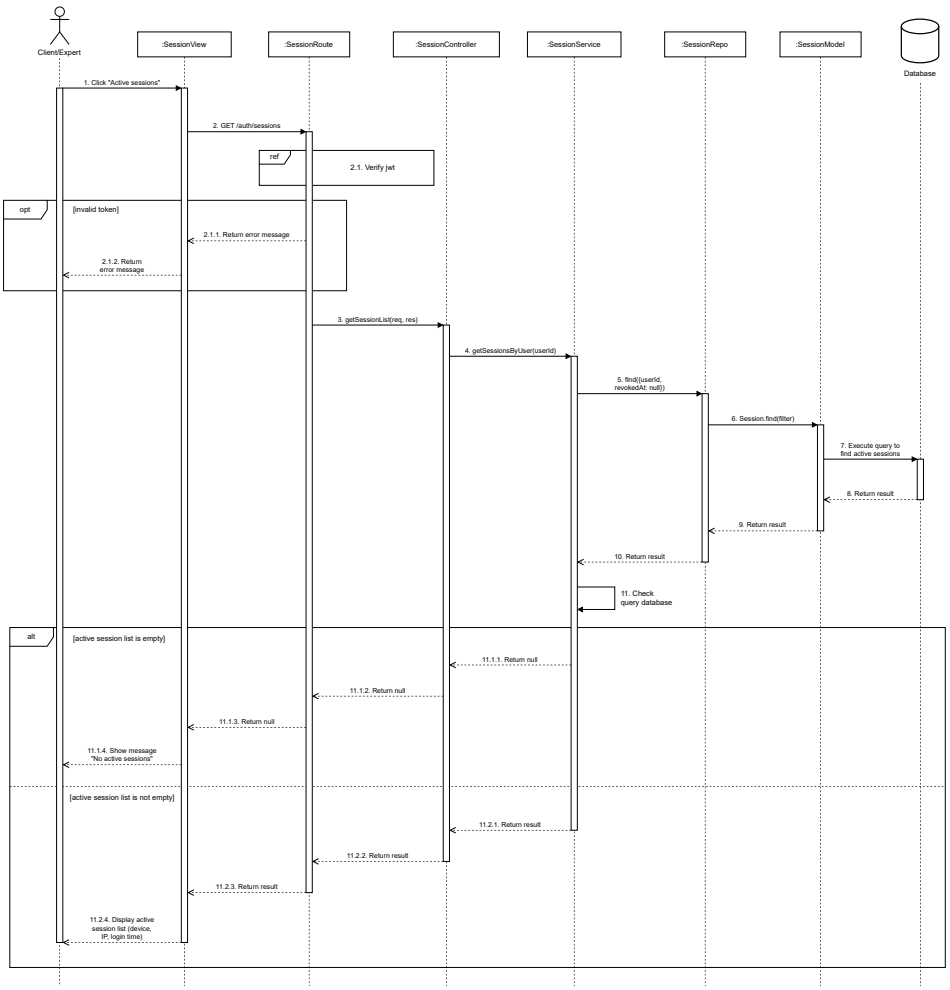

Figure 79: Sequence diagram of UC-16.01 View Session List

3.10.2 UC-16.02 – View Session Detail

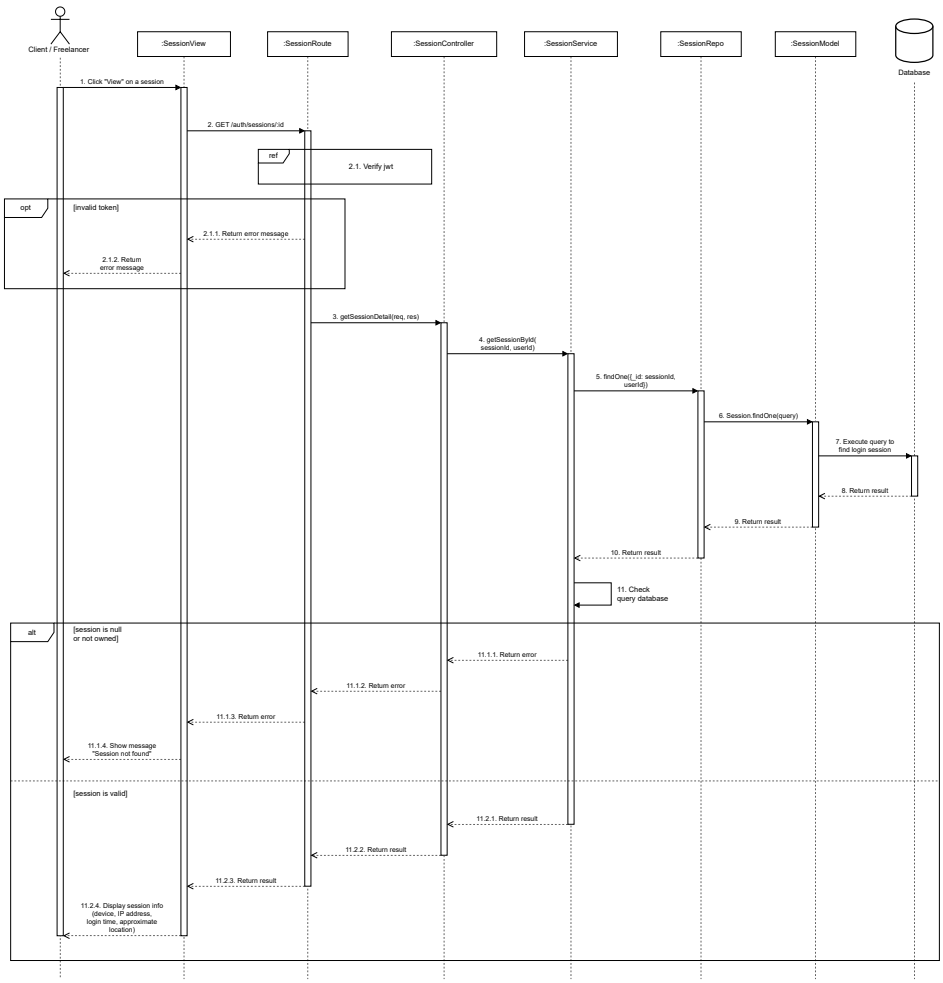

Figure 80: Sequence diagram of UC-16.02 View Session Detail

3.10.3 UC-16.03 – Revoke Session

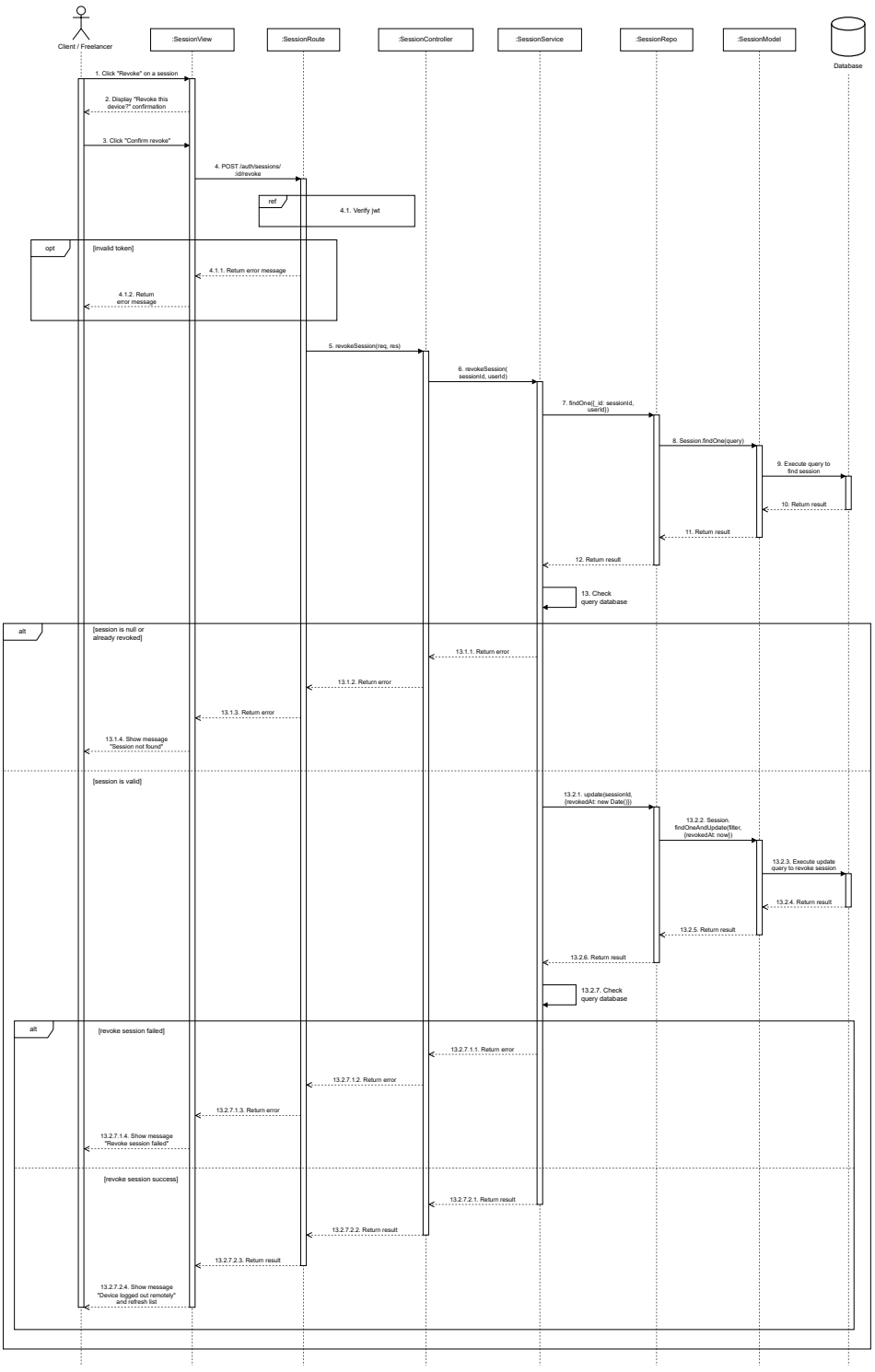

Figure 81: Sequence diagram of UC-16.03 Revoke Session

3.11 Forum Category Management

Forum Category Management covers the administrator-side category catalogue and the public category views used by the forum. Deletion is a soft delete, so a category can be restored later.

a. Class Diagram

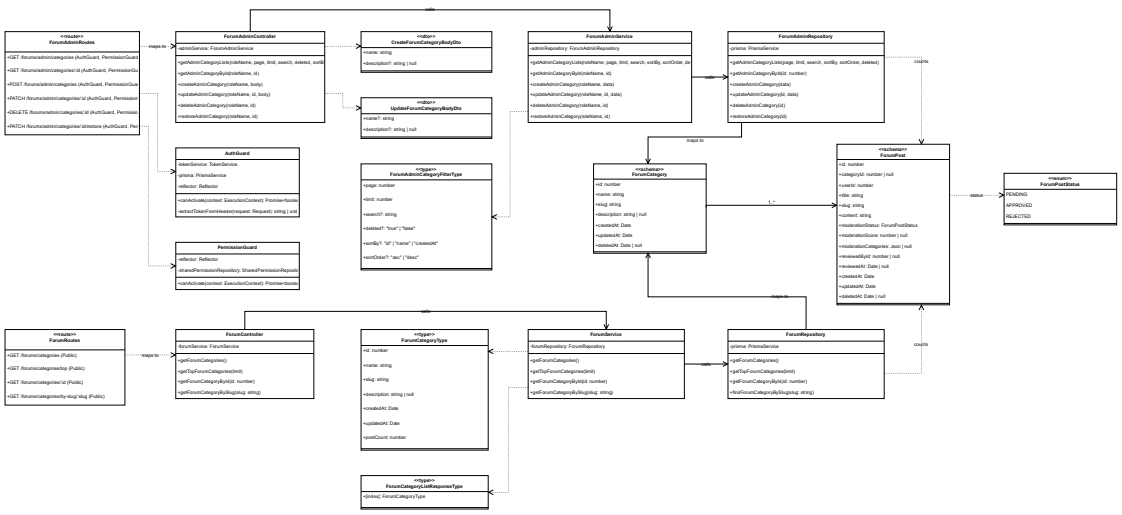

Figure 82: Class diagram of Forum Category Management

b. Sequence Diagram(s)

3.11.1 UC-17.01 – Create Forum Category

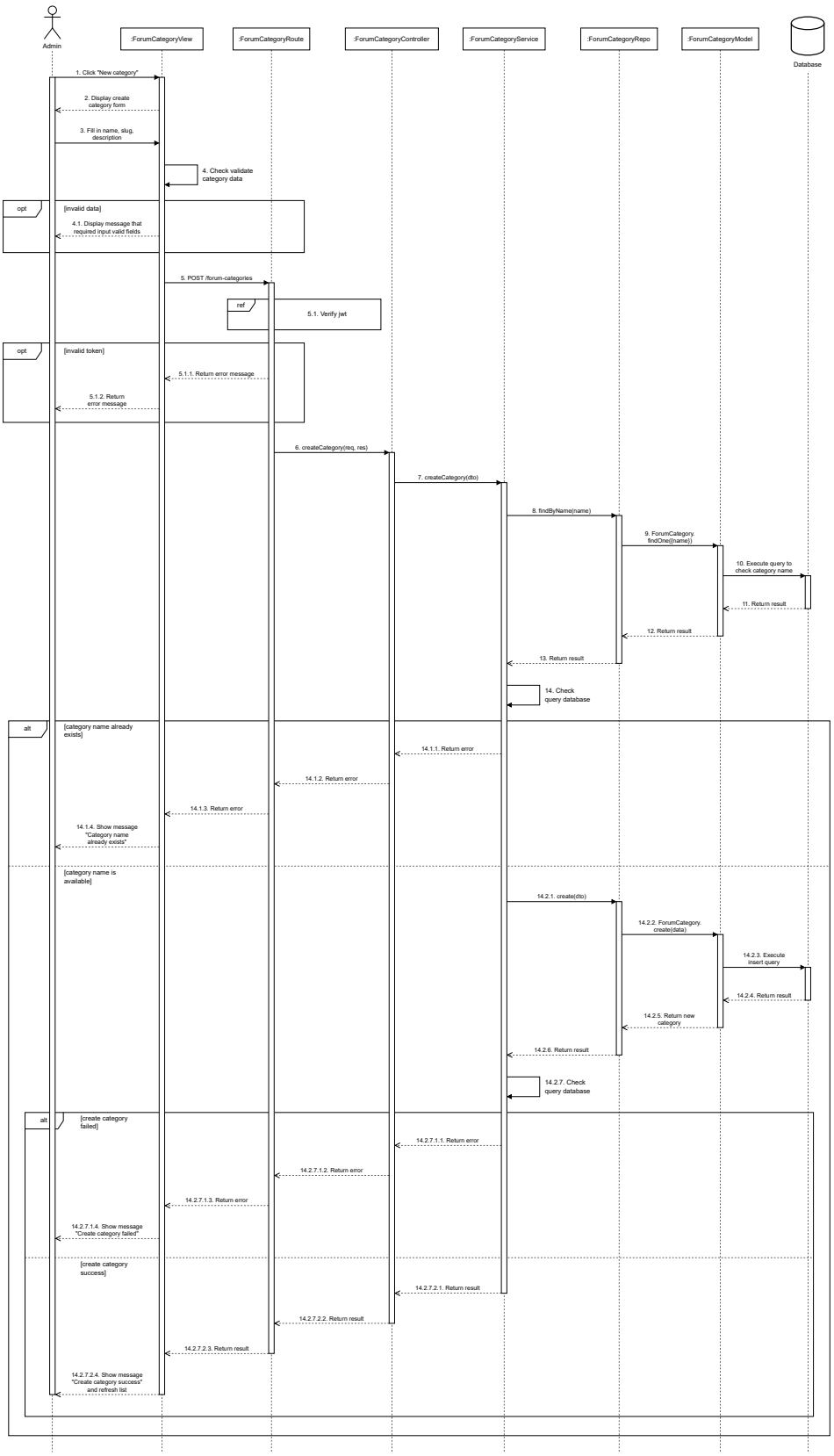

Figure 83: Sequence diagram of UC-17.01 Create Forum Category

3.11.2 UC-17.02 – Update Forum Category

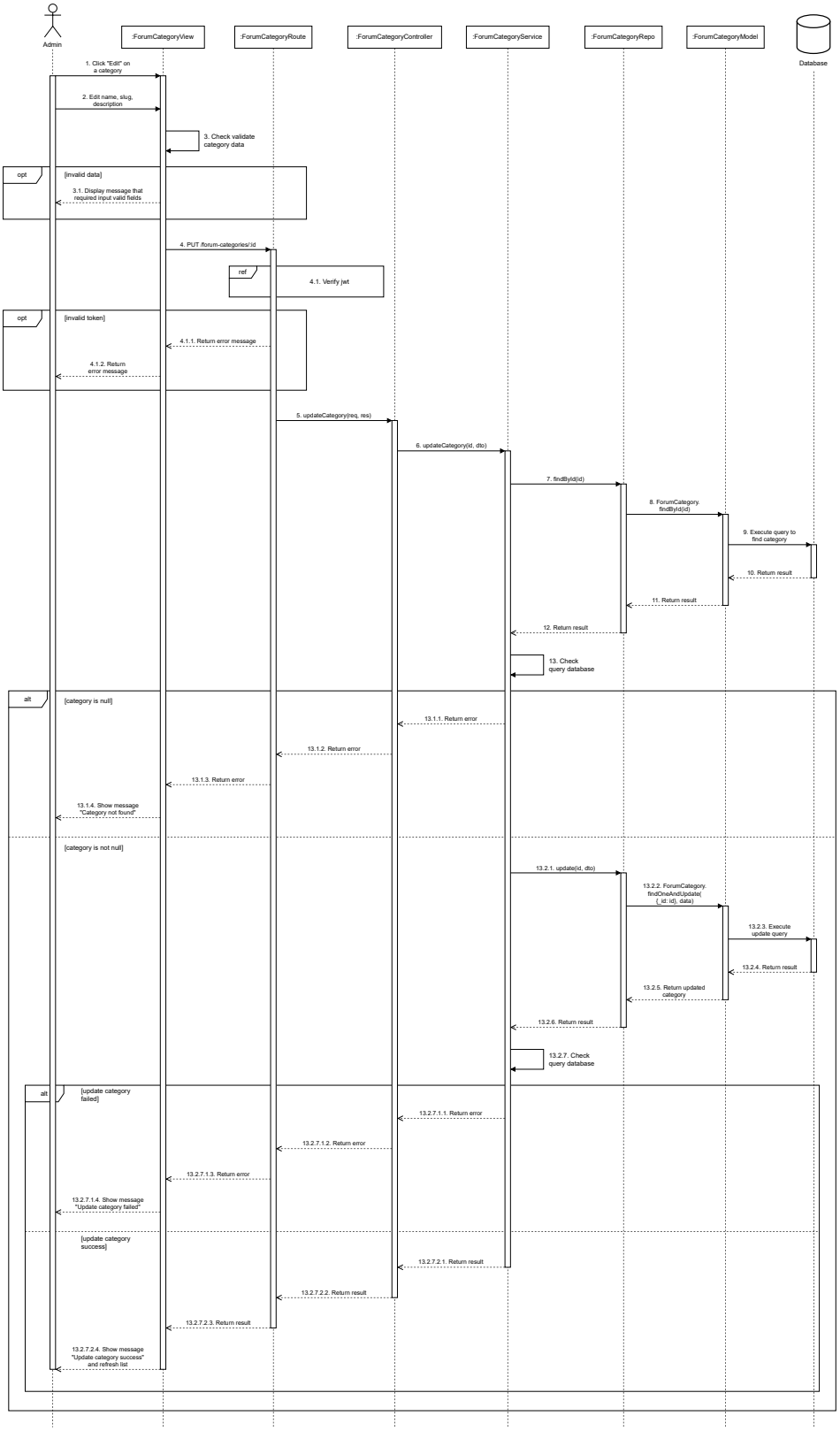

Figure 84: Sequence diagram of UC-17.02 Update Forum Category

3.11.3 UC-17.03 – Delete Forum Category

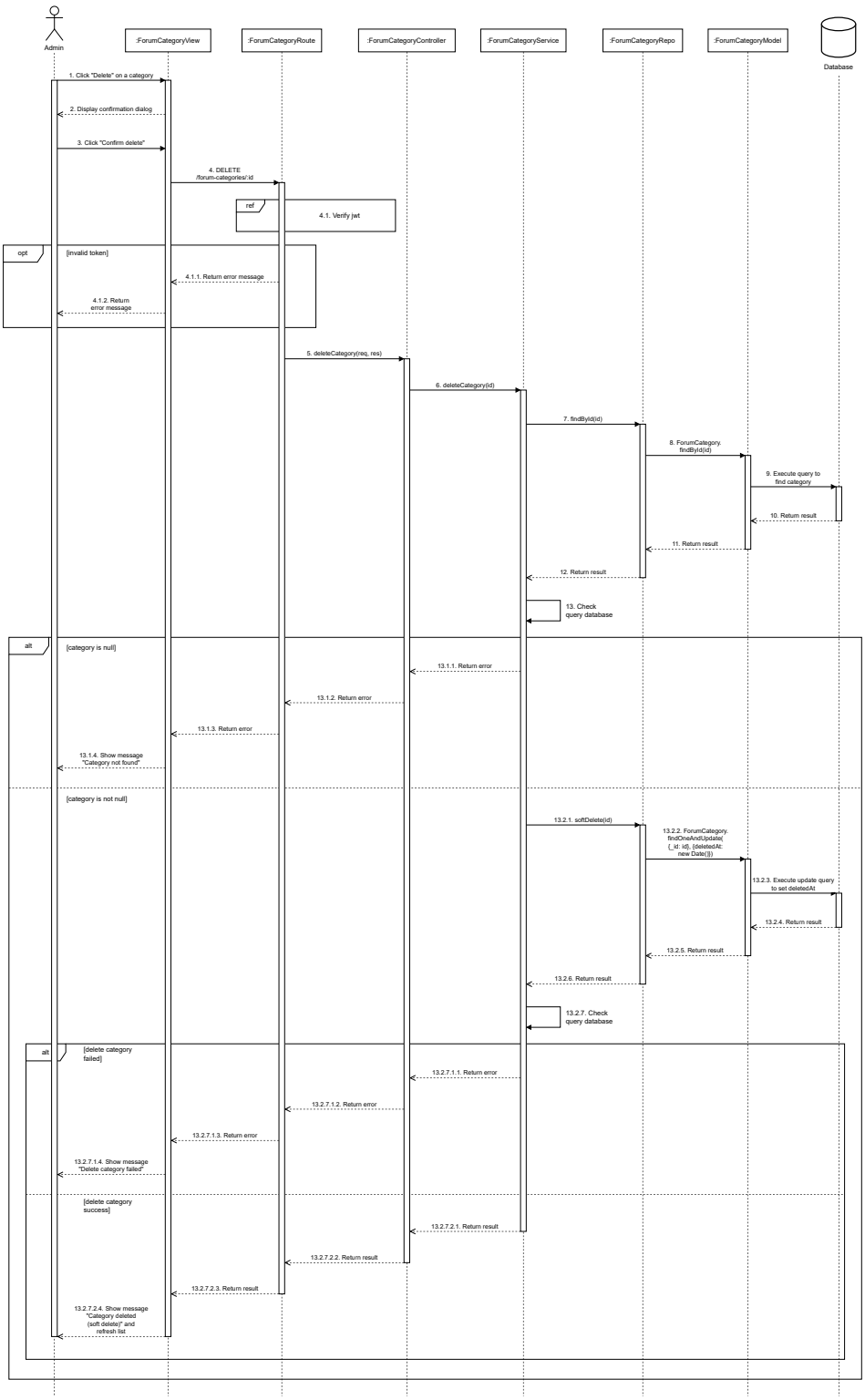

Figure 85: Sequence diagram of UC-17.03 Delete Forum Category

3.11.4 UC-17.06 – Restore Forum Category

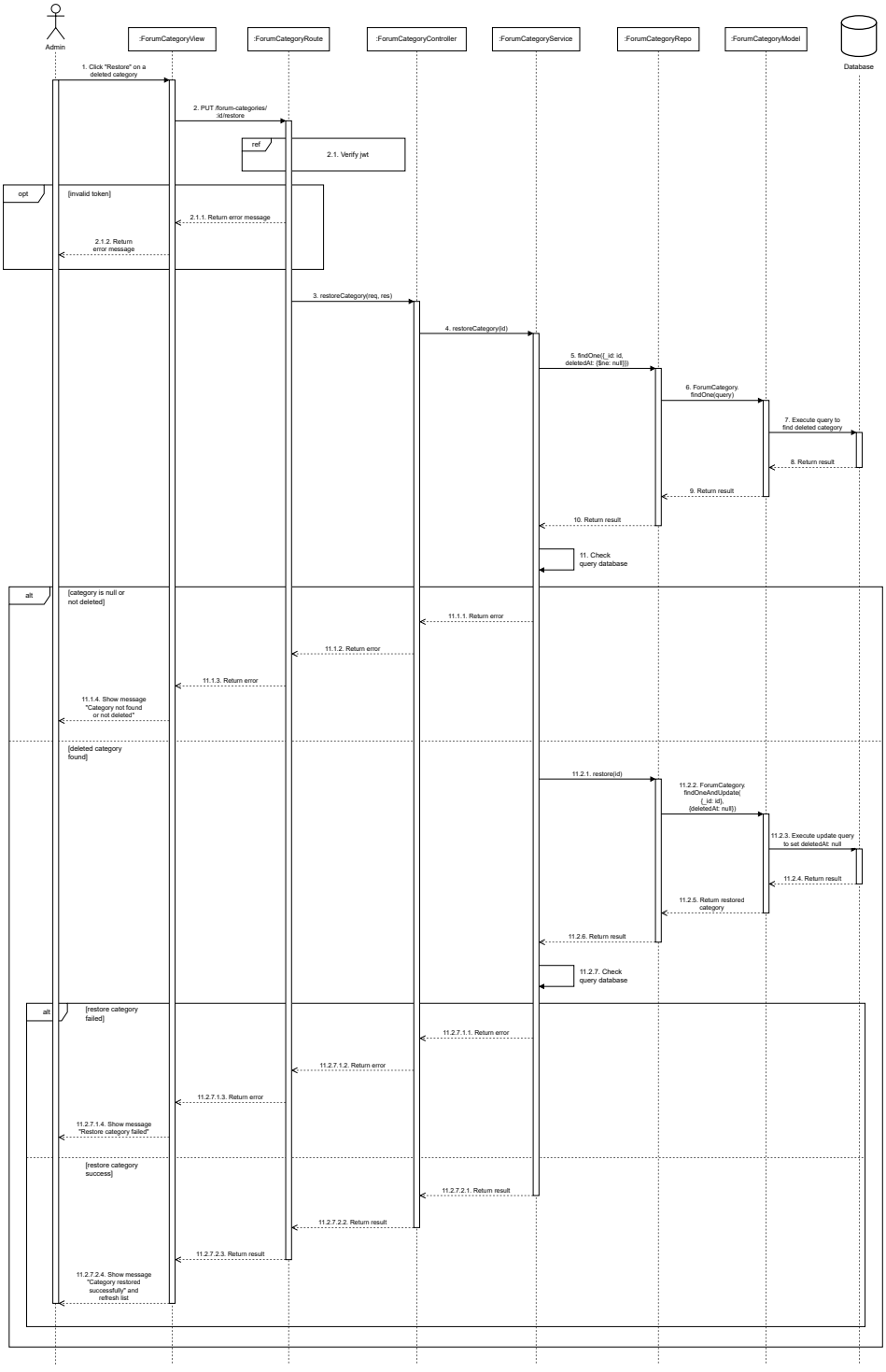

Figure 86: Sequence diagram of UC-17.06 Restore Forum Category

3.11.5 UC-14.01 – View Forum Category List

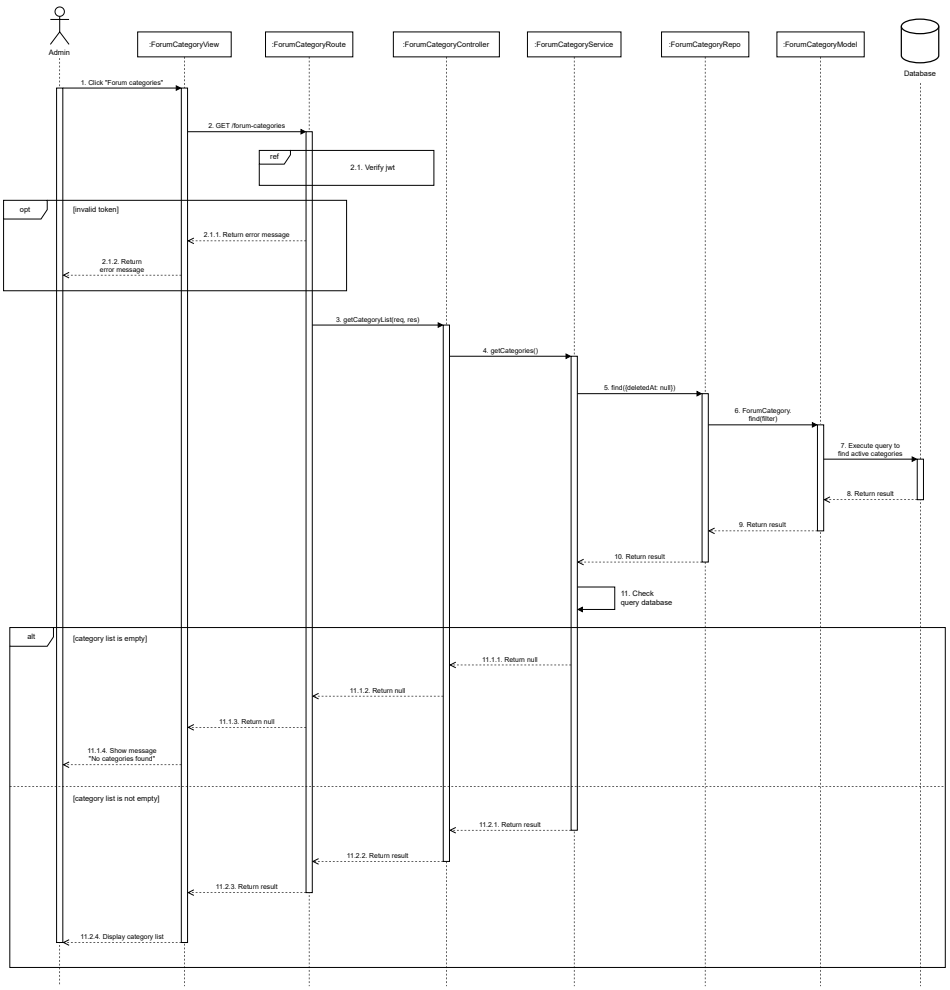

Figure 87: Sequence diagram of UC-14.01 View Forum Category List

3.11.6 UC-14.02 – View Forum Category Detail

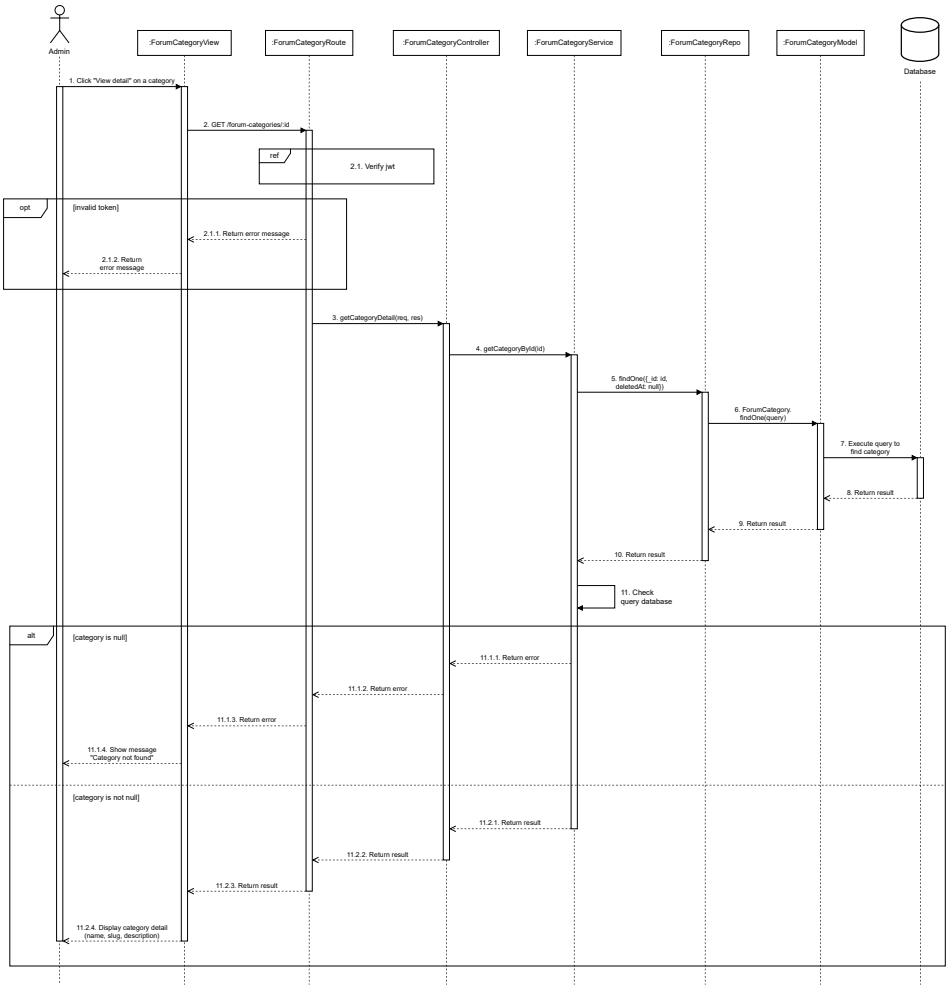

Figure 88: Sequence diagram of UC-14.02 View Forum Category Detail

3.12 Manage Moderation

Manage Moderation covers the UC-34 review workflow of the administration console: listing the posts waiting for review, opening a flagged post and approving or rejecting it. A review writes the moderation status, score and reviewer back to the forum post.

a. Class Diagram

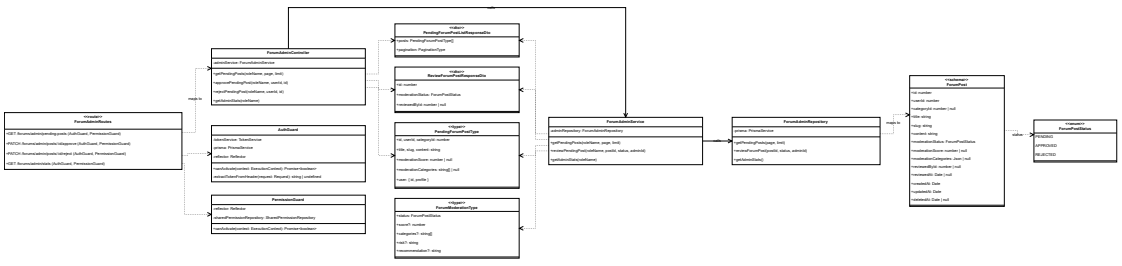

Figure 89: Class diagram of Manage Moderation

b. Sequence Diagram(s)

3.12.1 UC-34.01 – View Moderation Queue

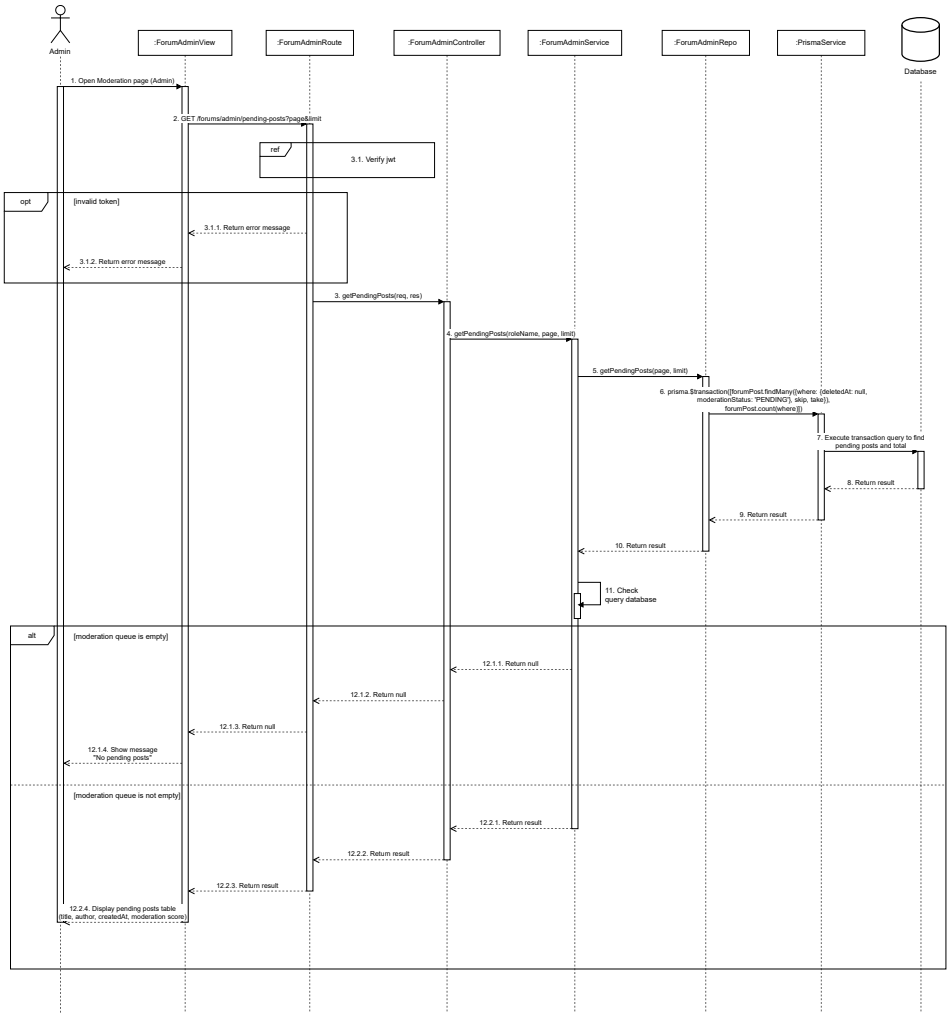

Figure 90: Sequence diagram of UC-34.01 View Moderation Queue

3.12.2 UC-34.02 – View Moderation Item Detail

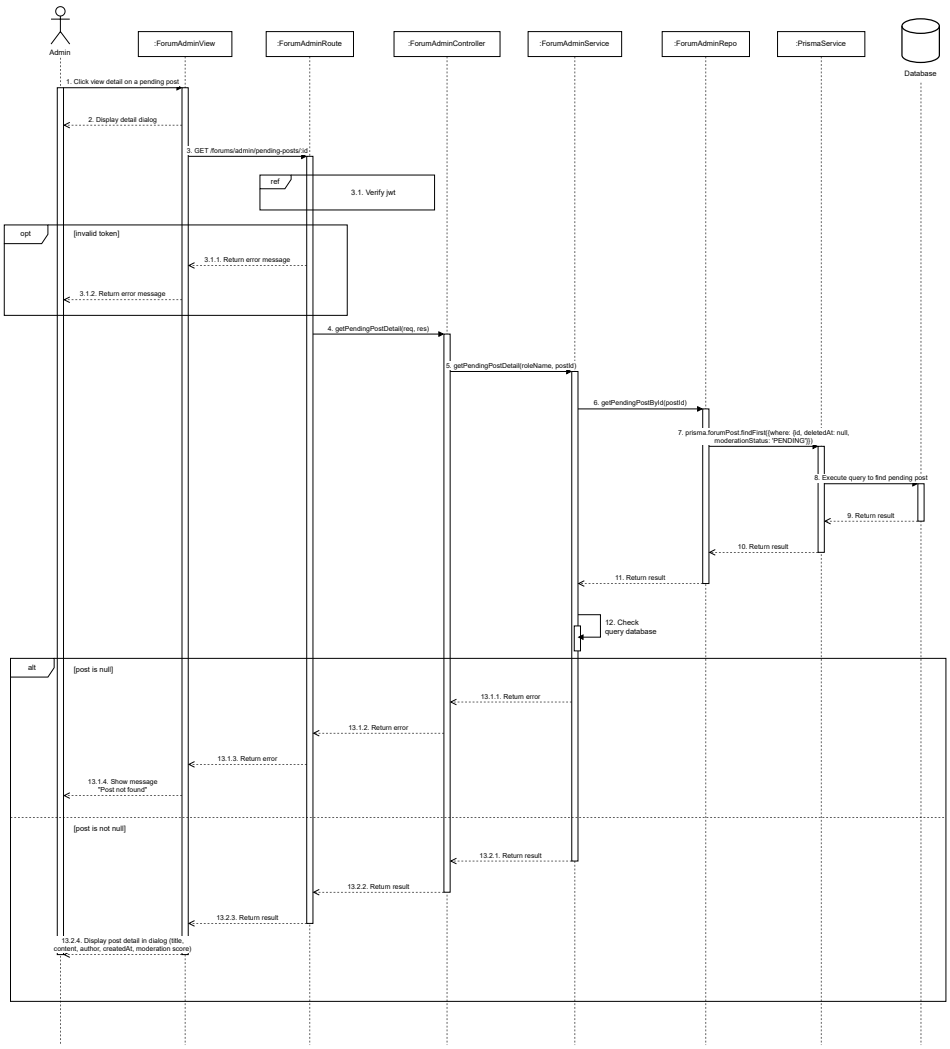

Figure 91: Sequence diagram of UC-34.02 View Moderation Item Detail

3.12.3 UC-34.03 – Resolve Moderation Item

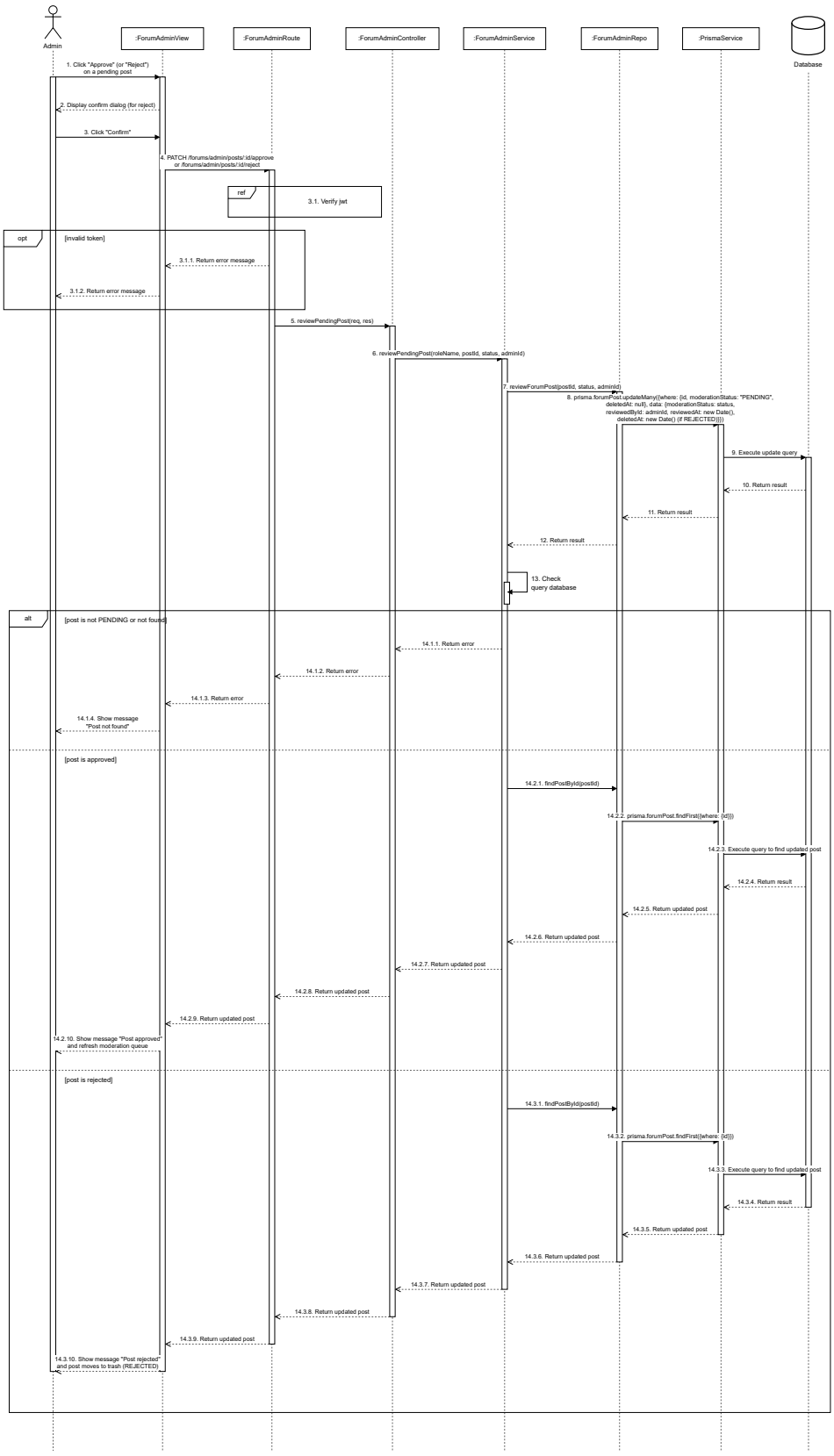

Figure 92: Sequence diagram of UC-34.03 Resolve Moderation Item

3.13 Manage ID Verification

Manage ID Verification covers the UC-35 identity review workflow: listing the submitted identity documents, opening a request with its uploaded file, and approving or rejecting the request. A decision updates the verification status and the reviewer of the identity document.

a. Class Diagram

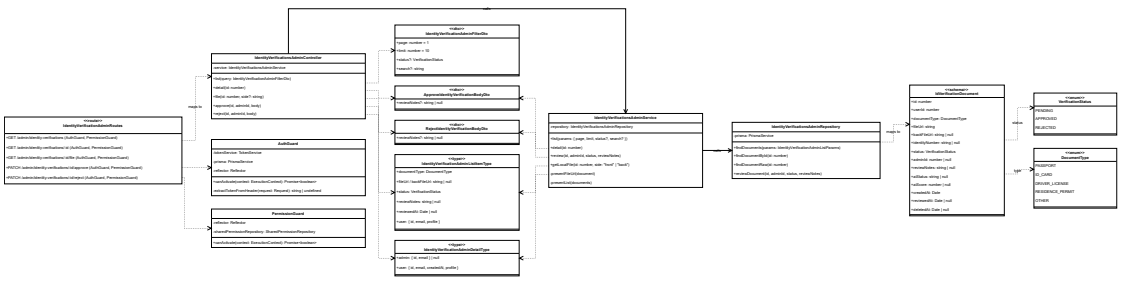

Figure 93: Class diagram of Manage ID Verification

b. Sequence Diagram(s)

3.13.1 UC-35.01 – View ID Verification Request List

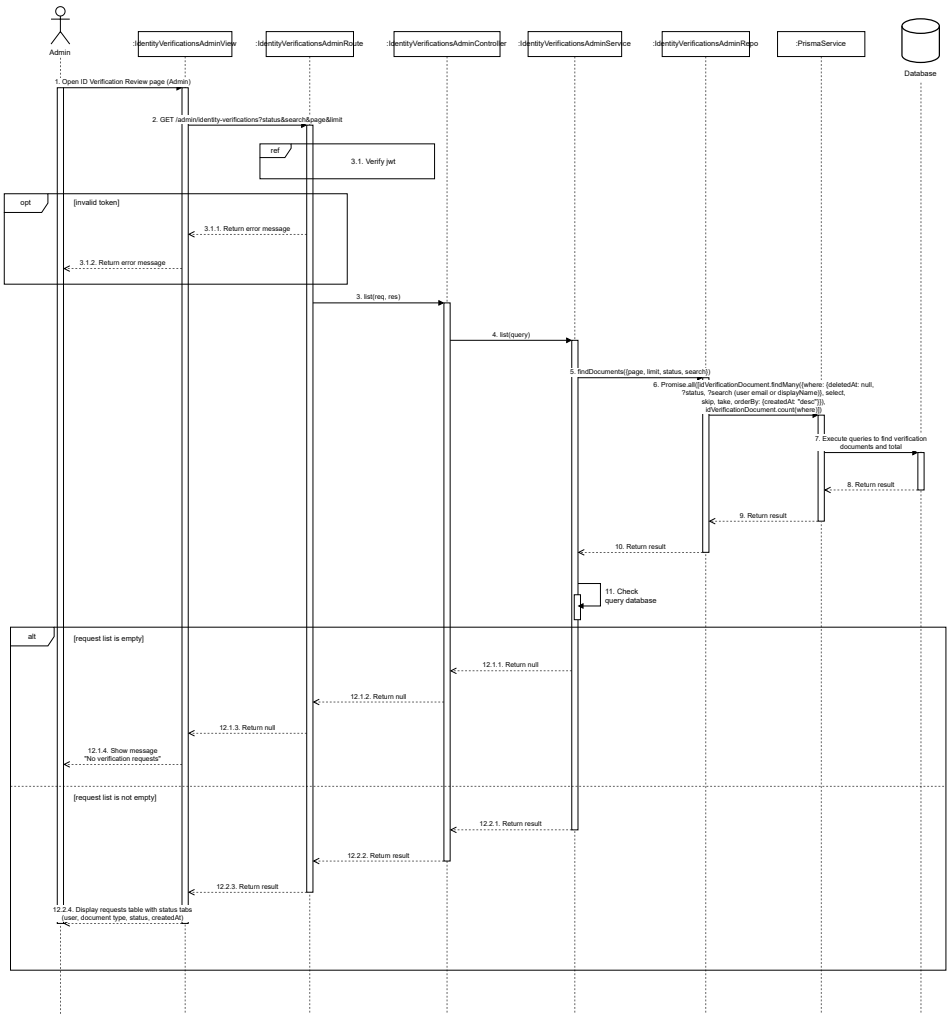

Figure 94: Sequence diagram of UC-35.01 View ID Verification Request List

3.13.2 UC-35.02 – View ID Verification Request Detail

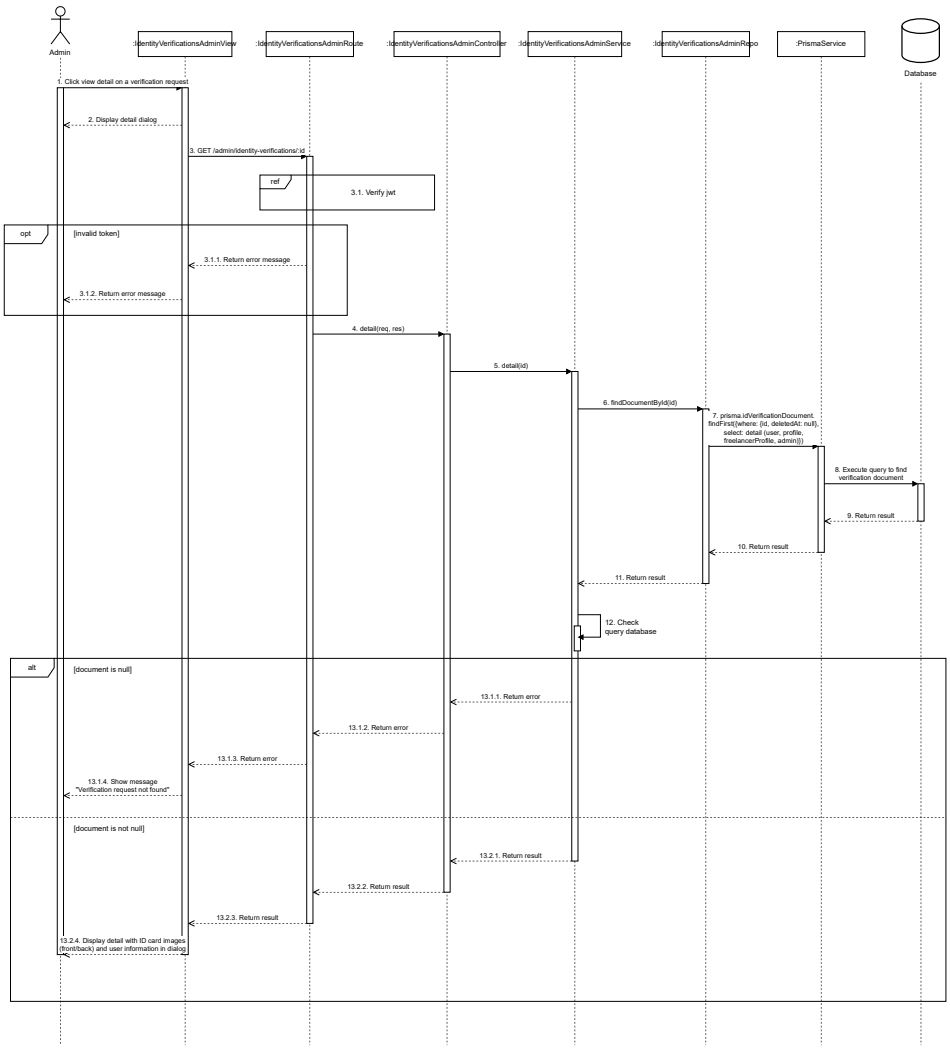

Figure 95: Sequence diagram of UC-35.02 View ID Verification Request Detail

3.13.3 UC-35.03 – Approve ID Verification

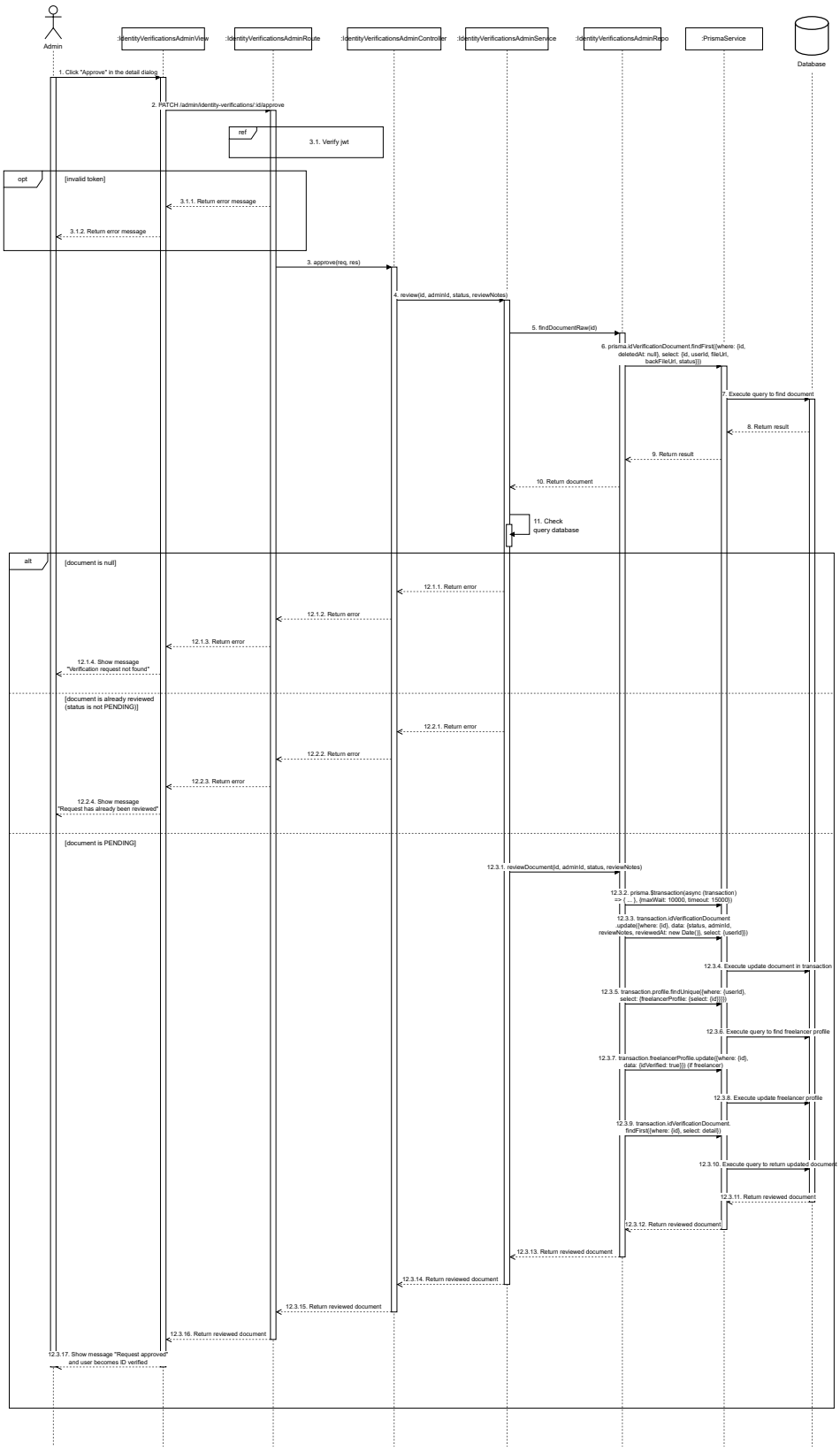

Figure 96: Sequence diagram of UC-35.03 Approve ID Verification

3.13.4 UC-35.04 – Reject ID Verification

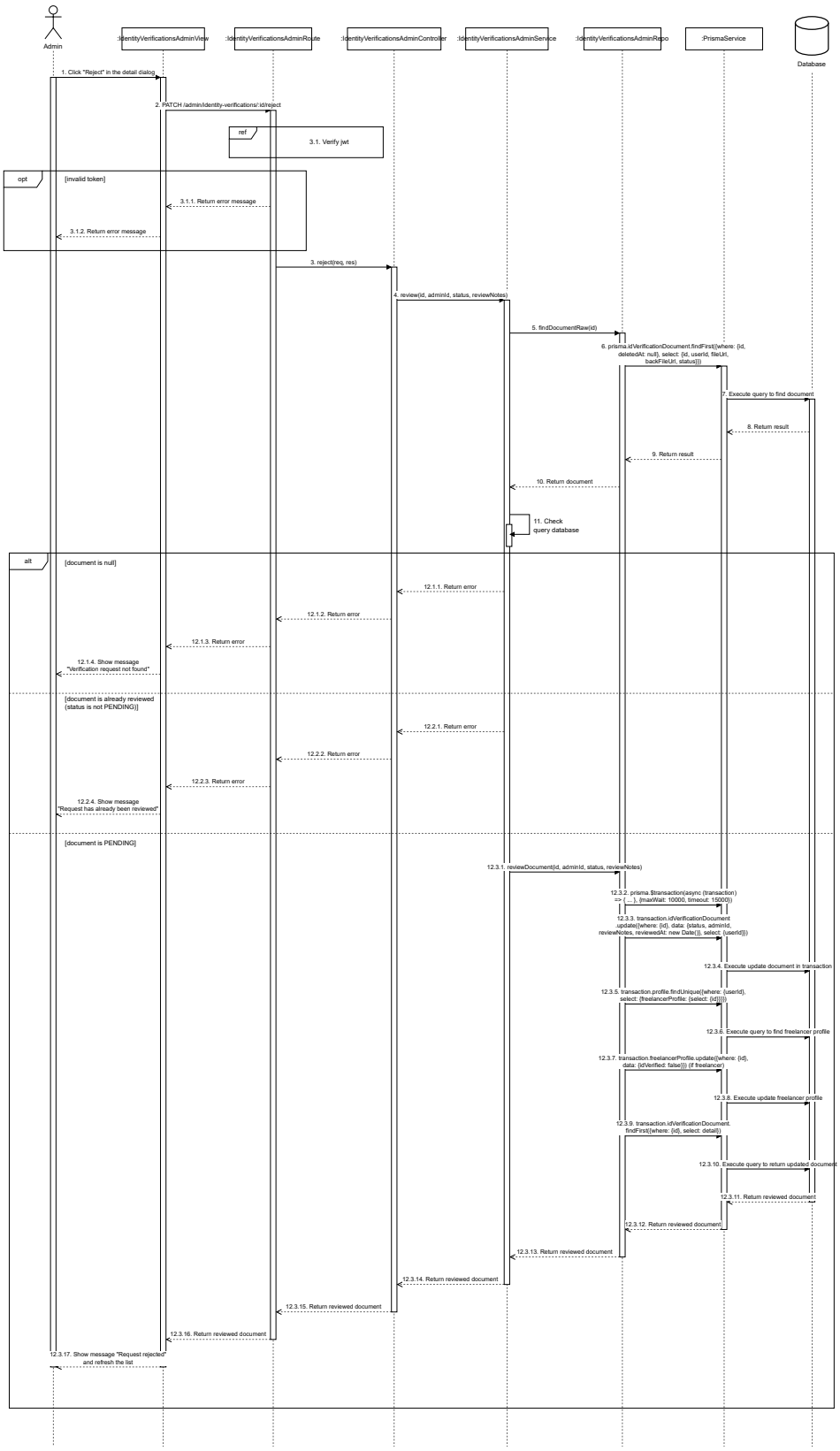

Figure 97: Sequence diagram of UC-35.04 Reject ID Verification

3.14 Manage Forum Trash

Manage Forum Trash covers the UC-36 recovery workflow: listing the soft-deleted posts and comments kept for review, opening one deleted item, and restoring it so that it becomes visible in the forum again.

a. Class Diagram

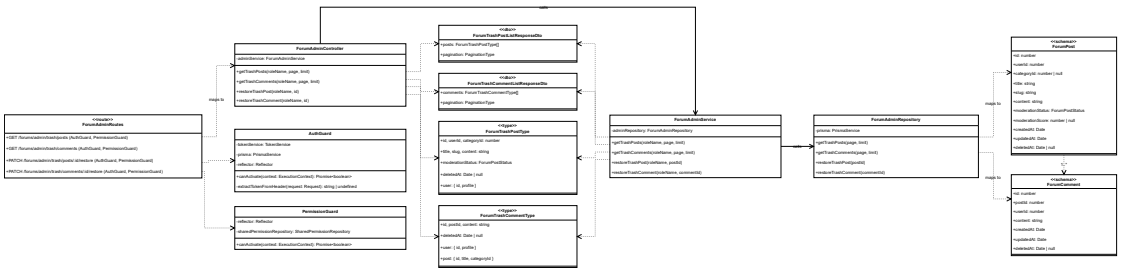

Figure 98: Class diagram of Manage Forum Trash

b. Sequence Diagram(s)

3.14.1 UC-36.01 – View Deleted Posts/Comments List

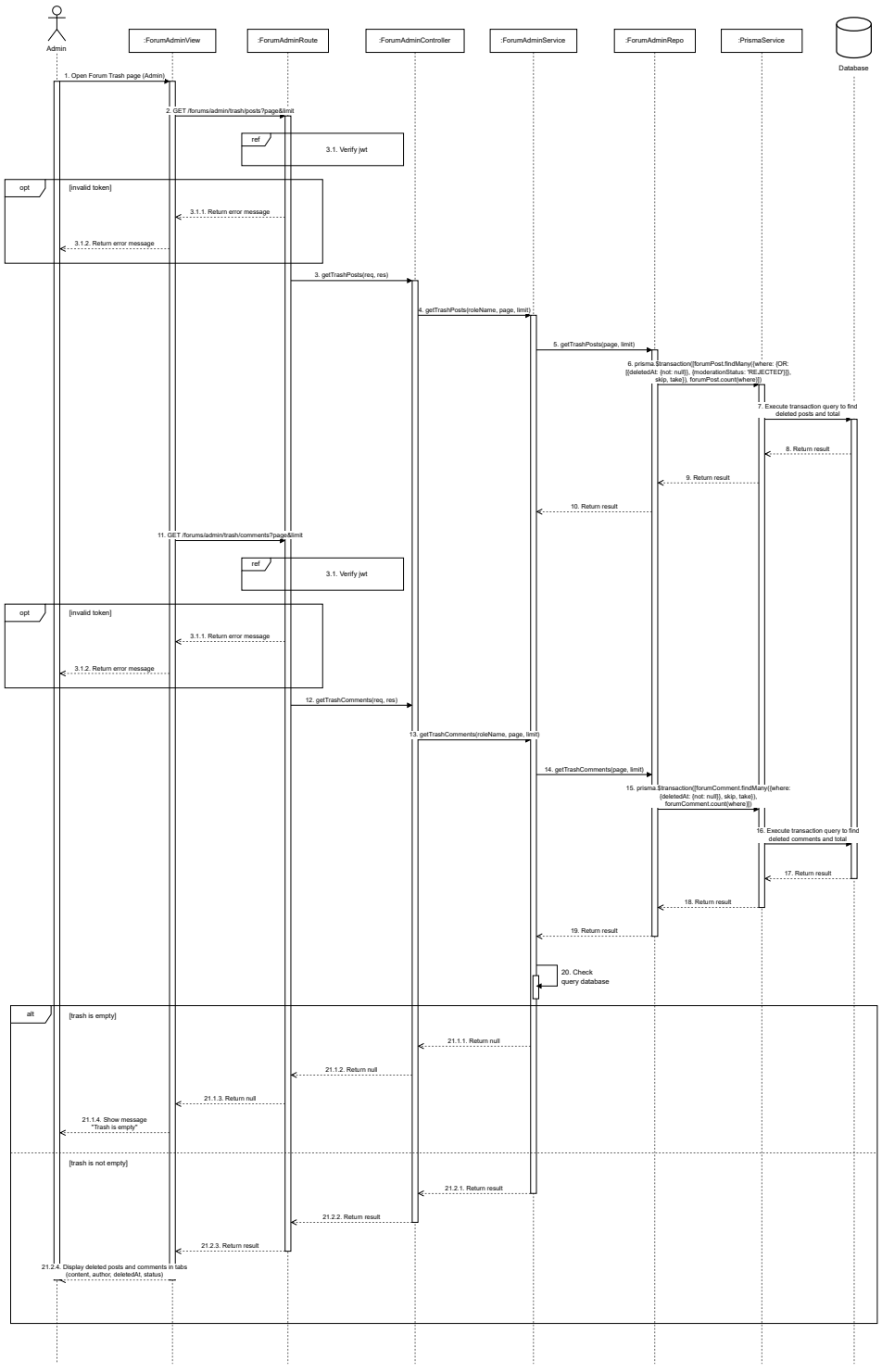

Figure 99: Sequence diagram of UC-36.01 View Deleted Posts/Comments List

3.14.2 UC-36.02 – View Deleted Posts/Comments Detail

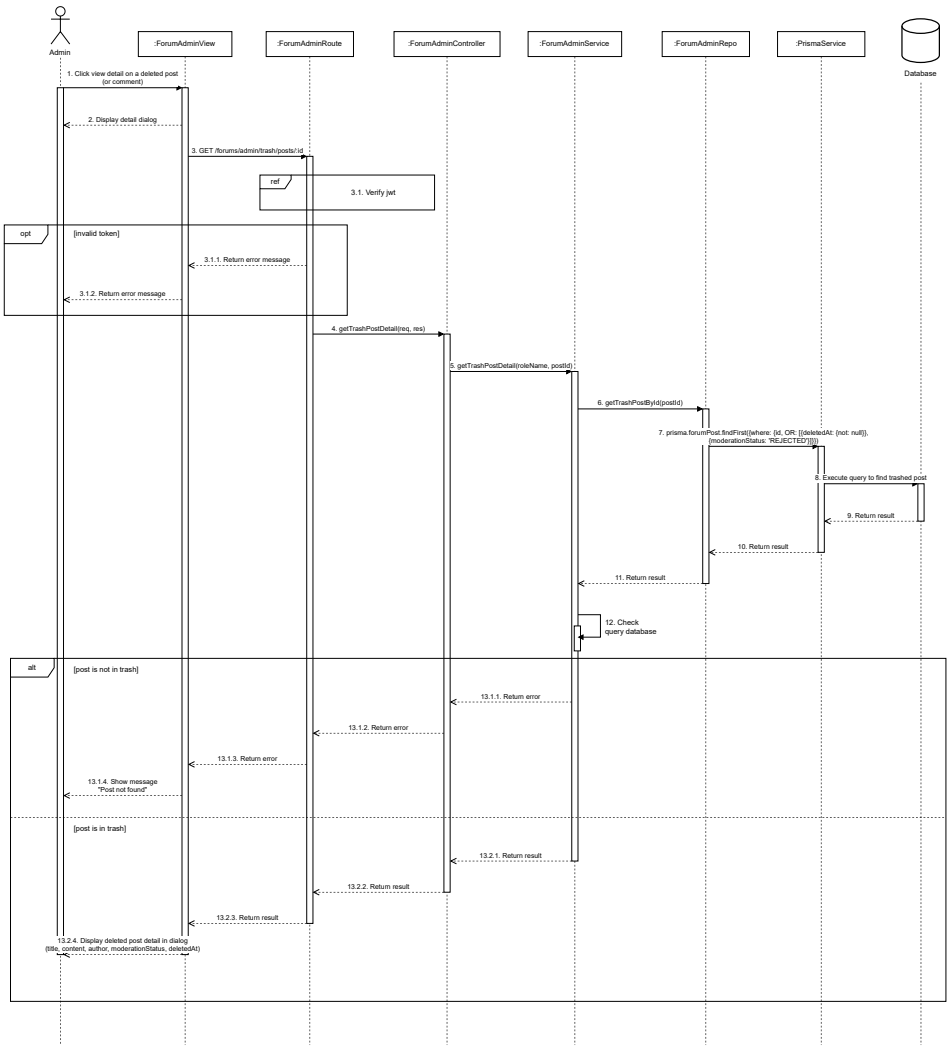

Figure 100: Sequence diagram of UC-36.02 View Deleted Posts/Comments Detail

3.14.3 UC-36.03 – Restore Deleted Posts/Comments

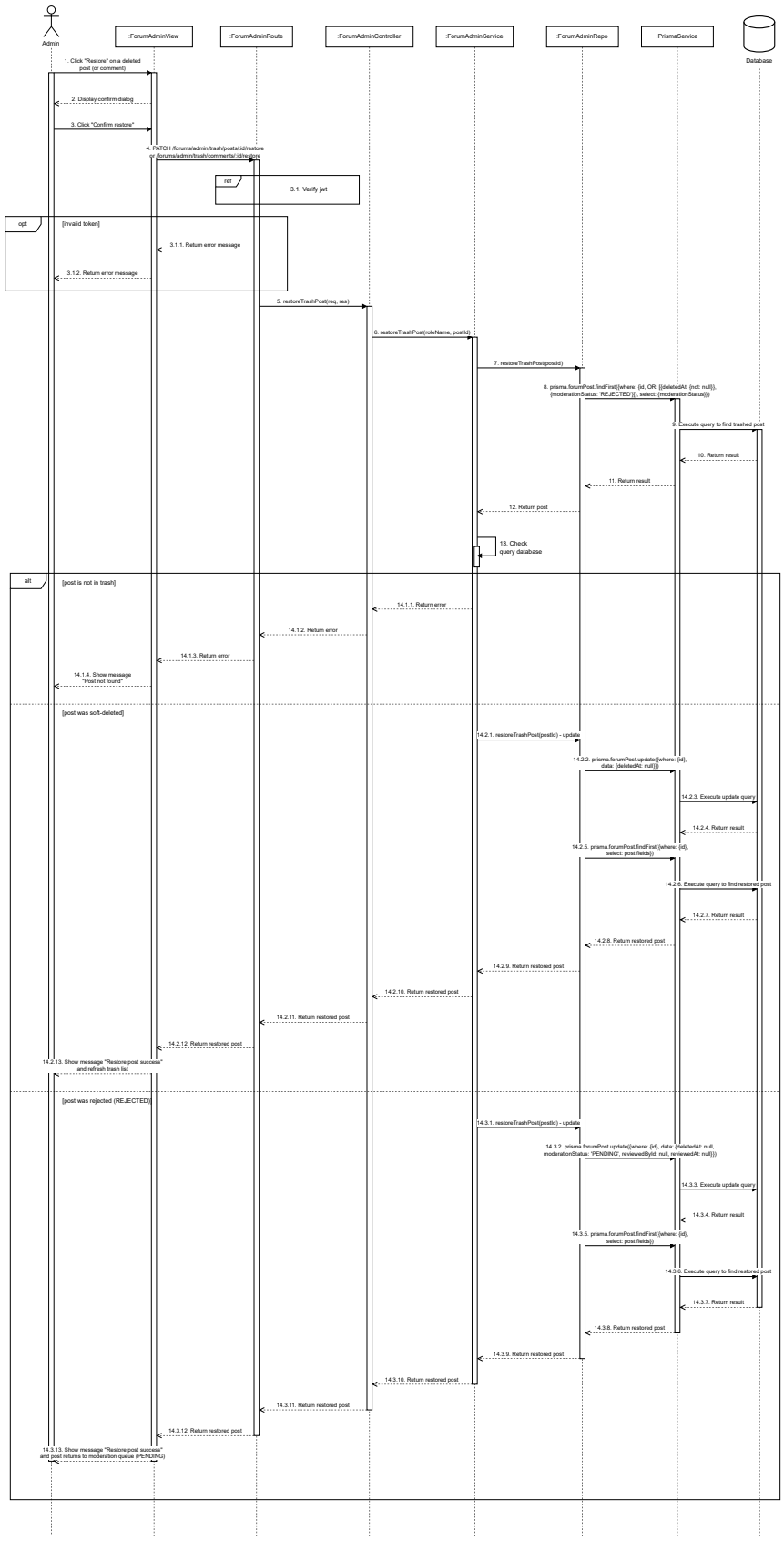

Figure 101: Sequence diagram of UC-36.03 Restore Deleted Posts/Comments

3.15 Dispute Management

Dispute Management covers the UC-33 formal arbitration workflow for funded milestones: opening a formal dispute case when conflicts arise, listing disputes with status and deadline badges, viewing comprehensive dispute dossiers with claims, evidence, and fee status, submitting counter-statements with rebuttal evidence, responding to arbitrator resolutions, and enabling administrators to evaluate evidence and propose binding resolution split decisions.

a. Class Diagram

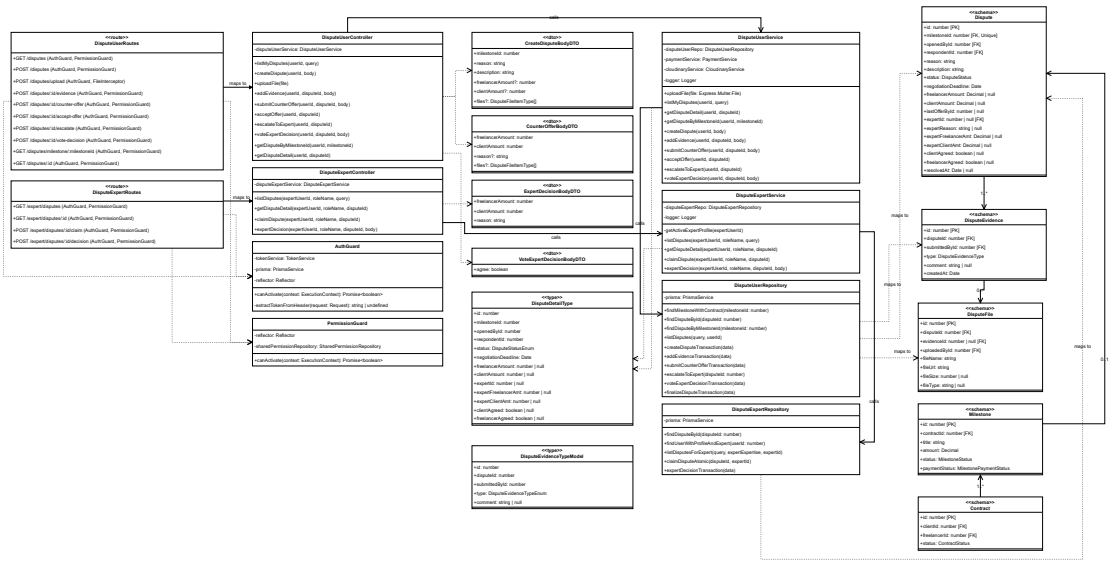

Figure 102: Class diagram of Dispute Management

b. Sequence Diagram(s)

3.15.1 UC-33.01 – Open Dispute

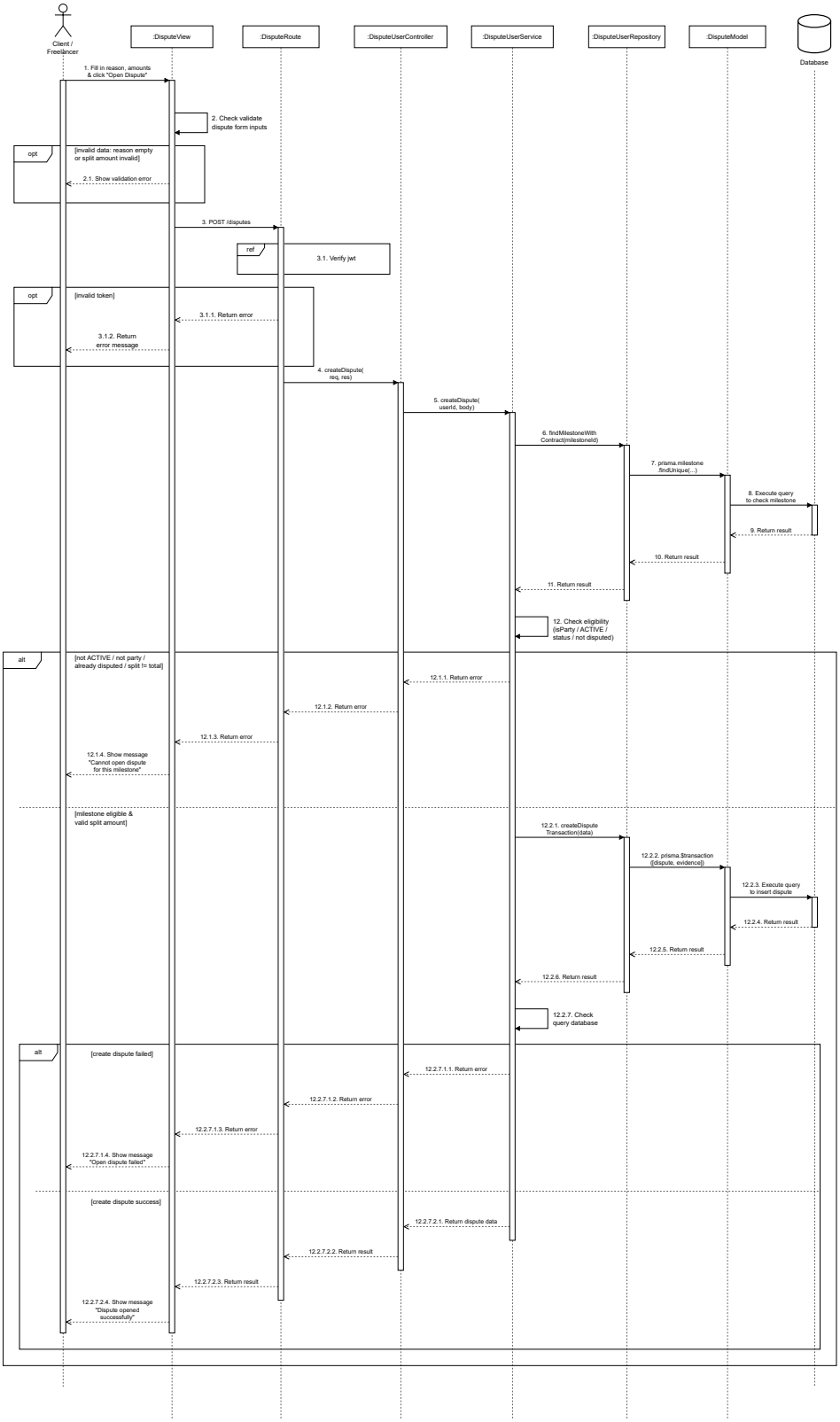

Figure 103: Sequence diagram of UC-33.01 Open Dispute

3.15.2 UC-33.02 – View Dispute List

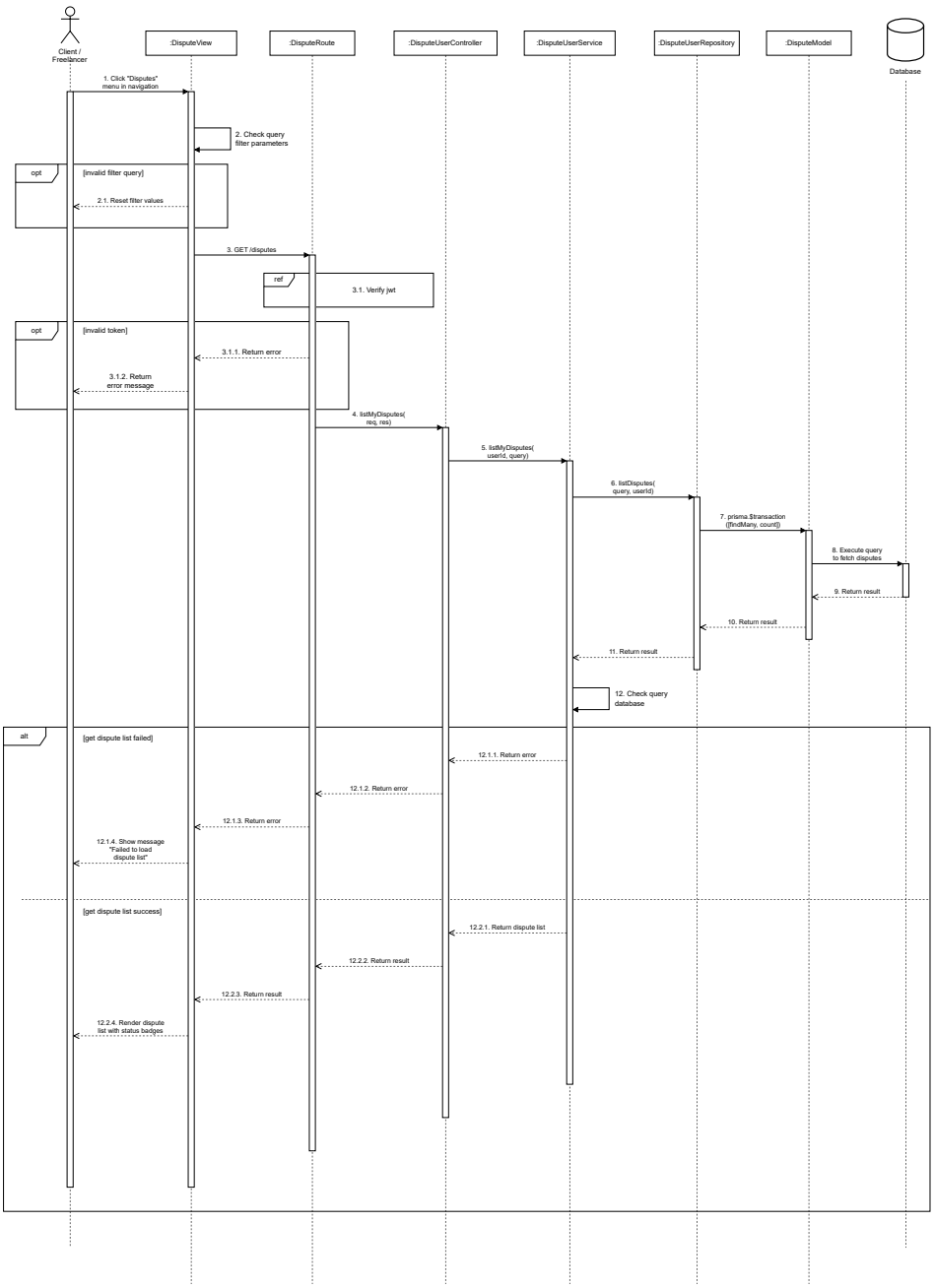

Figure 104: Sequence diagram of UC-33.02 View Dispute List

3.15.3 UC-33.03 – View Dispute Detail

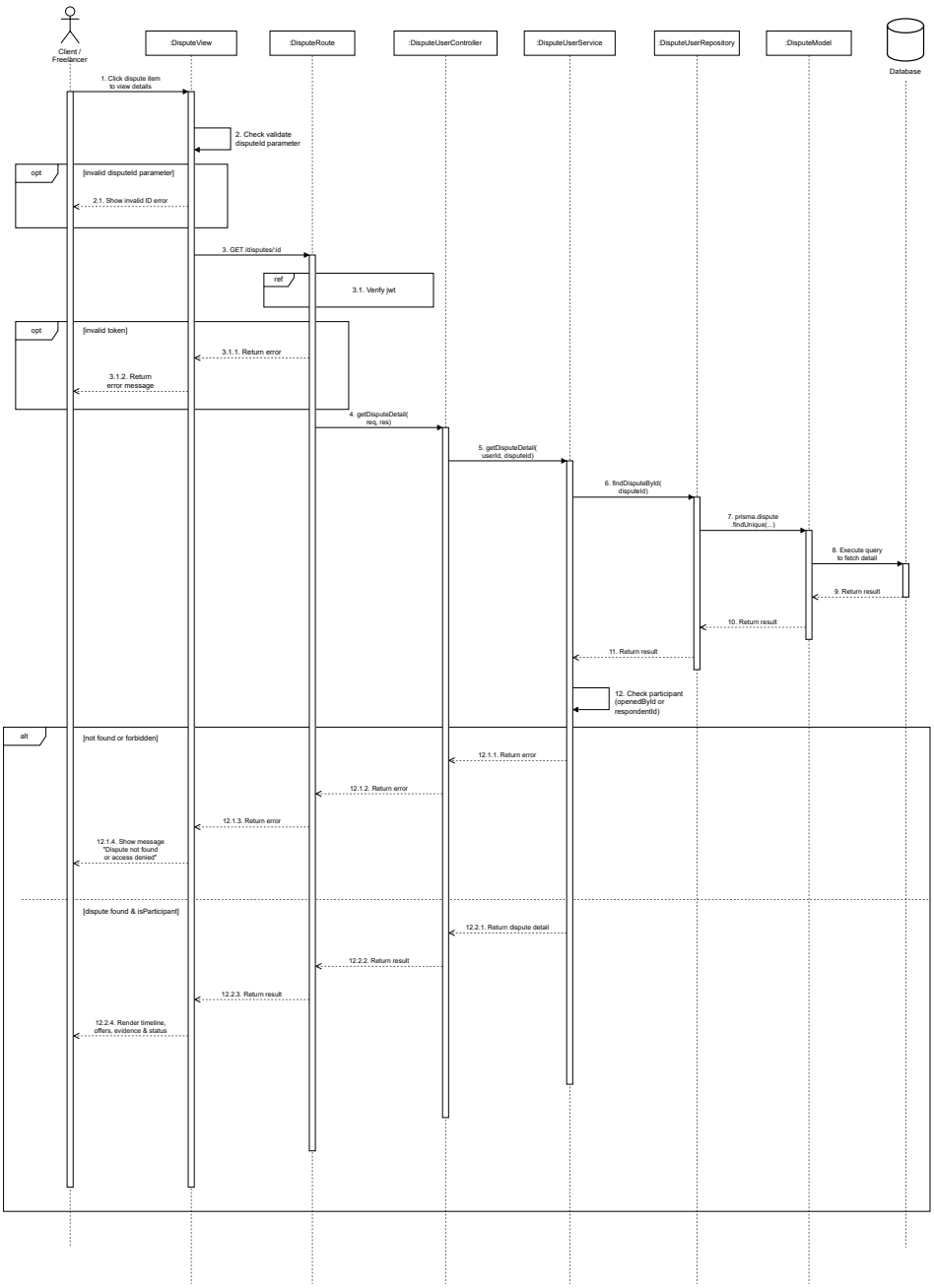

Figure 105: Sequence diagram of UC-33.03 View Dispute Detail

3.15.4 UC-33.04 – Respond to Dispute

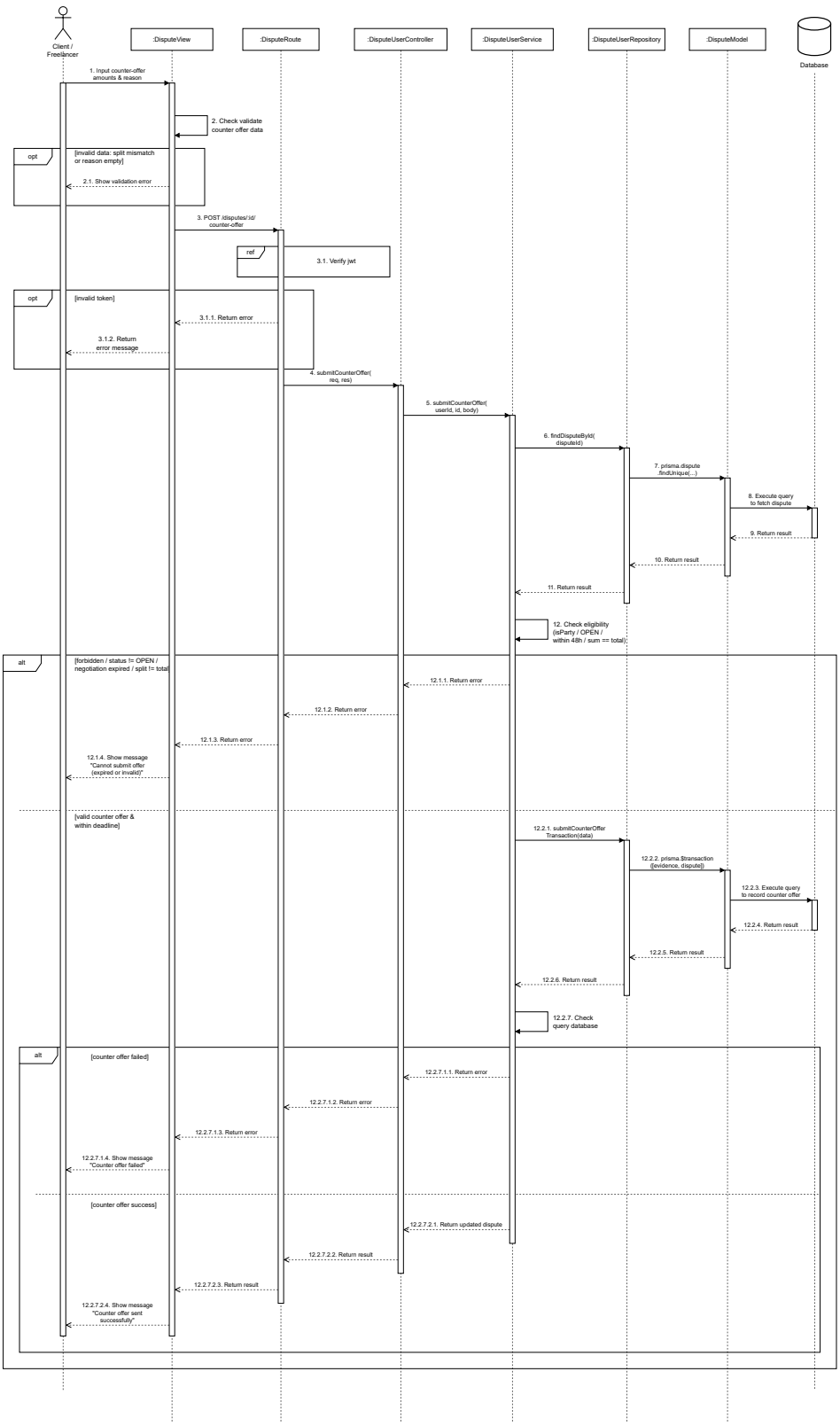

Figure 106: Sequence diagram of UC-33.04 Respond to Dispute

3.15.5 UC-33.05 – Respond to Dispute Resolution

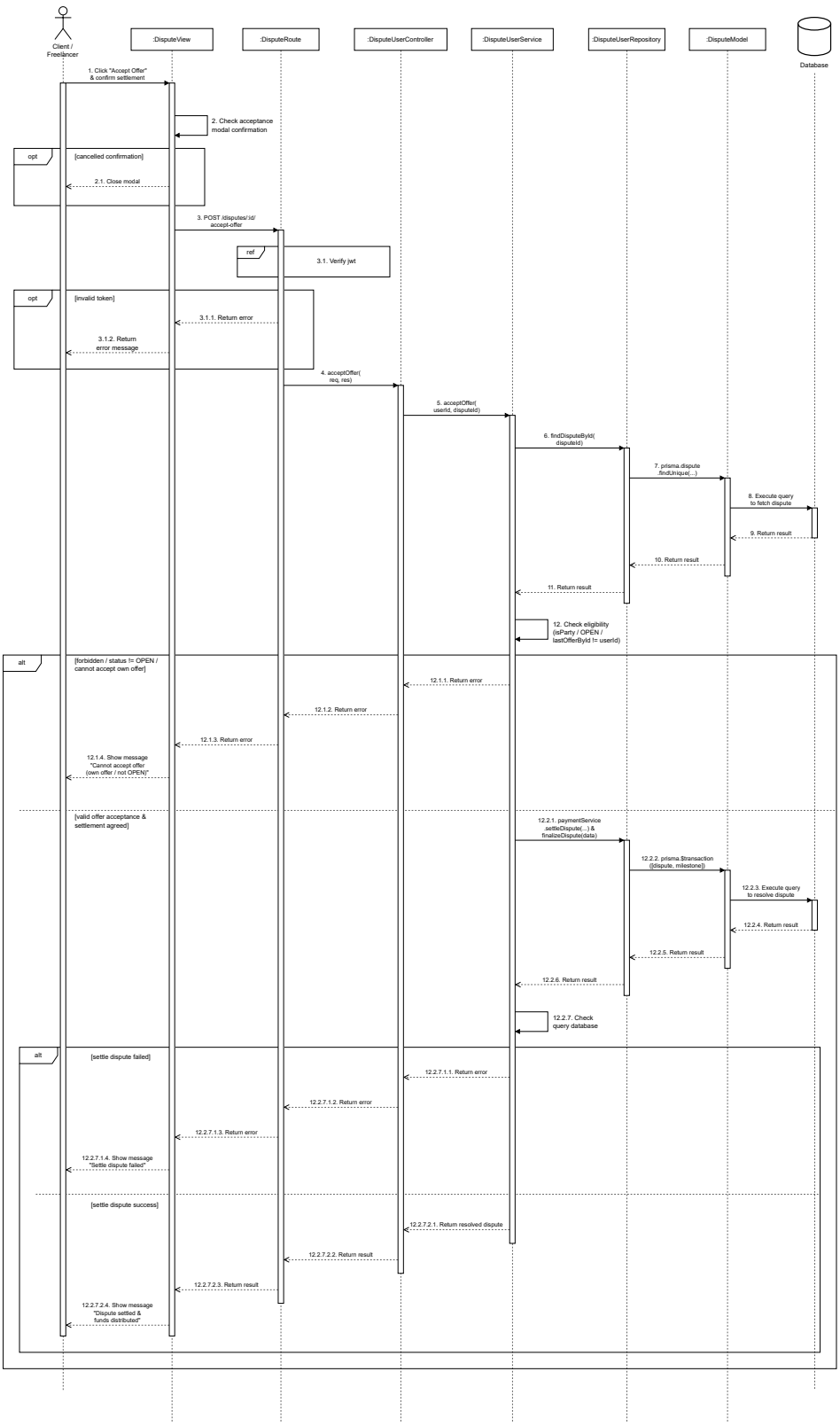

Figure 107: Sequence diagram of UC-33.05 Respond to Dispute Resolution

3.15.6 UC-33.06 – Propose Dispute Resolution

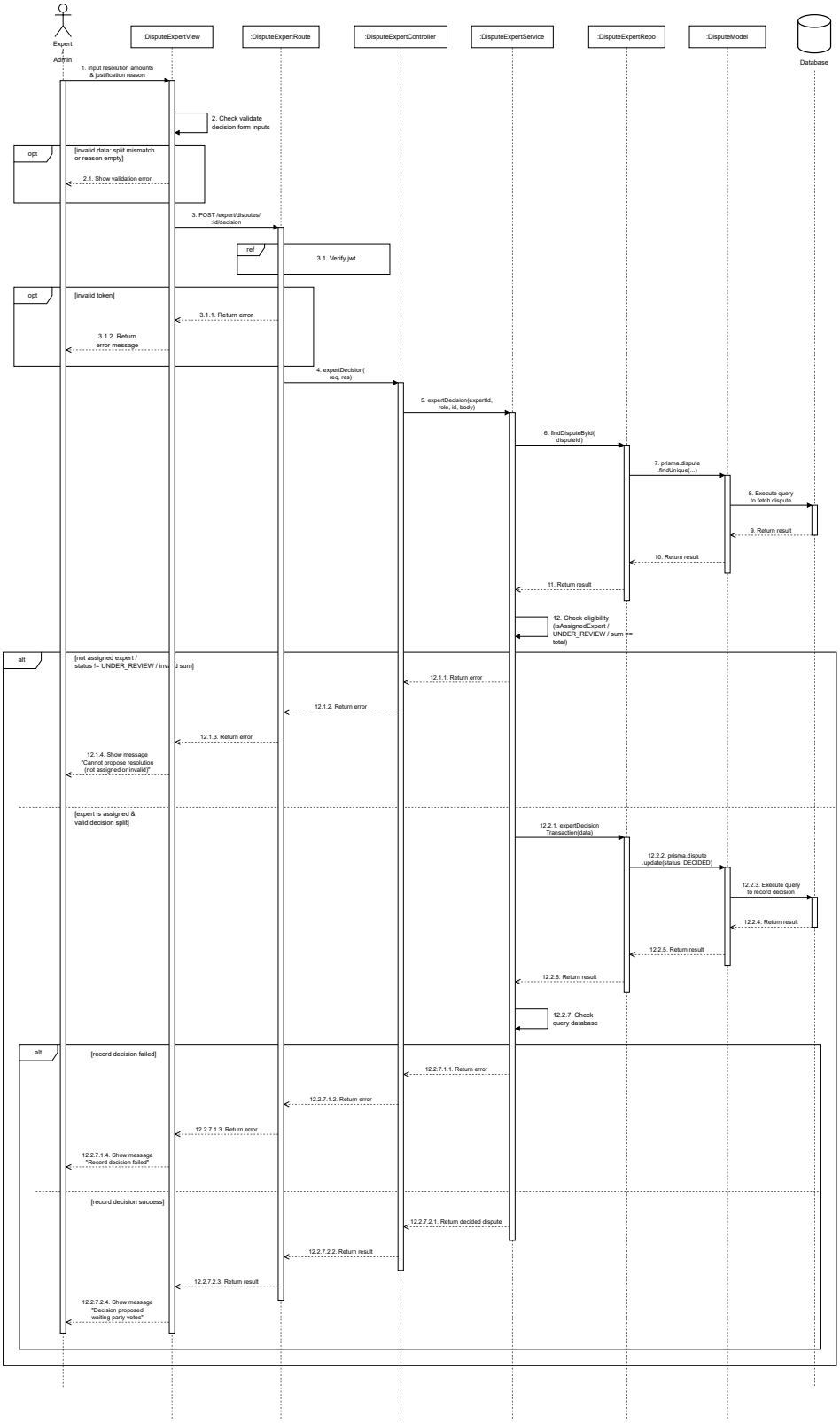

Figure 108: Sequence diagram of UC-33.06 Propose Dispute Resolution

3.16 User Management

User Management provides the administrator with a searchable user catalogue, a consolidated account detail view, and dedicated editors for the Client and Freelancer profiles attached to an account.

a. Class Diagram

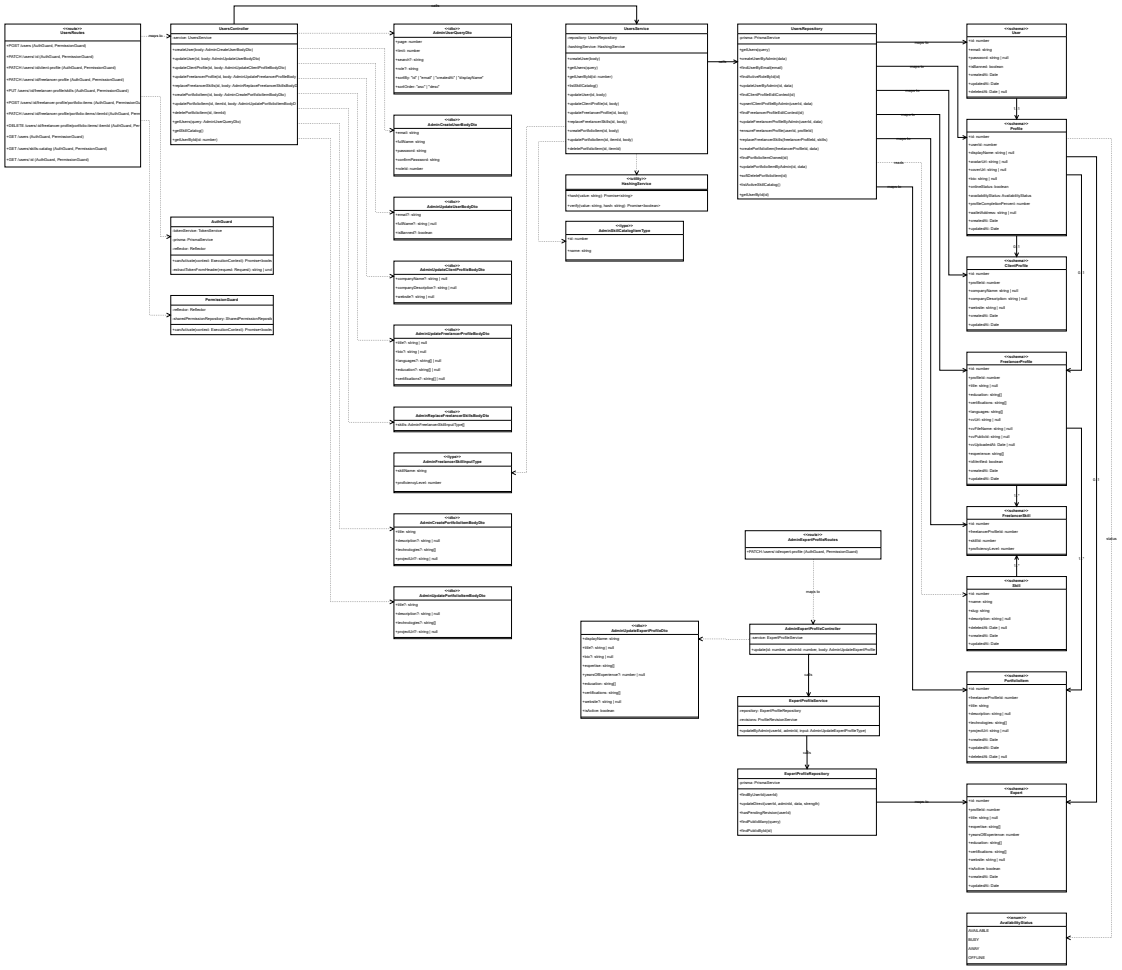

Figure 109: Class diagram of User Management

b. Sequence Diagram(s)

3.16.1 UC-28.01 – View User List

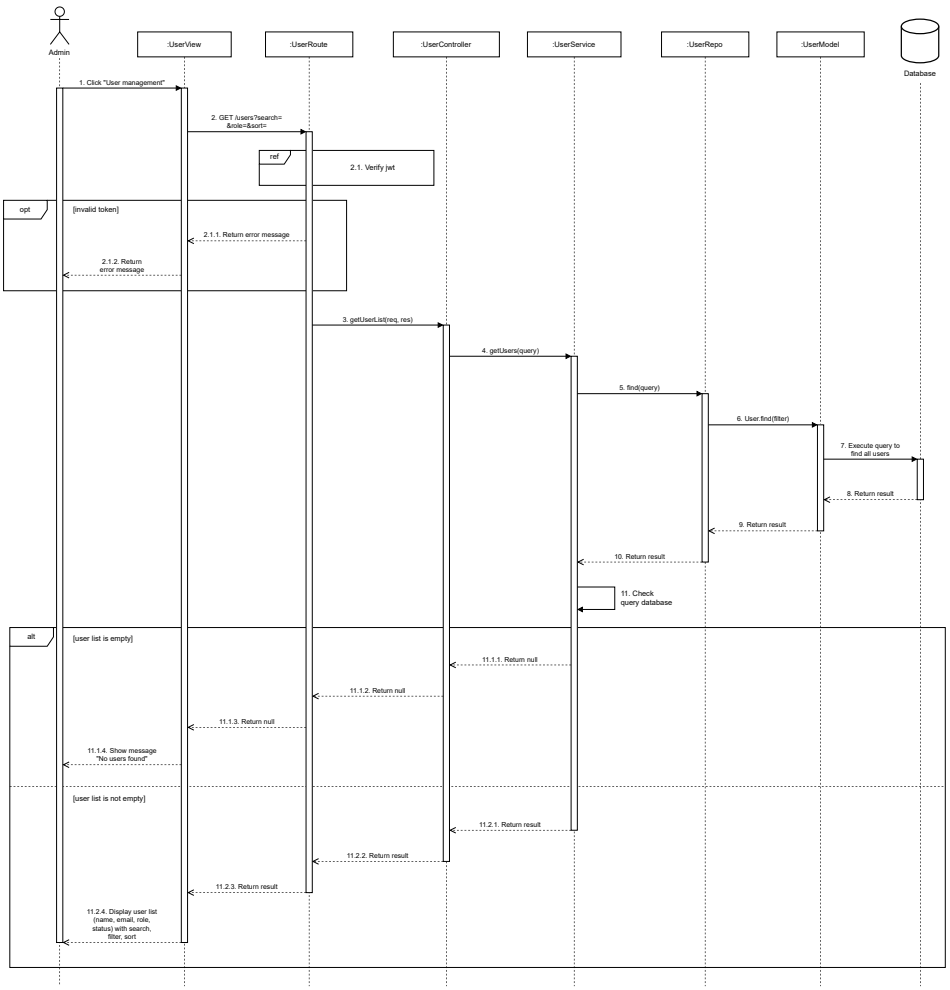

Figure 110: Sequence diagram of UC-28.01 View User List

3.16.2 UC-28.02 – View User Detail

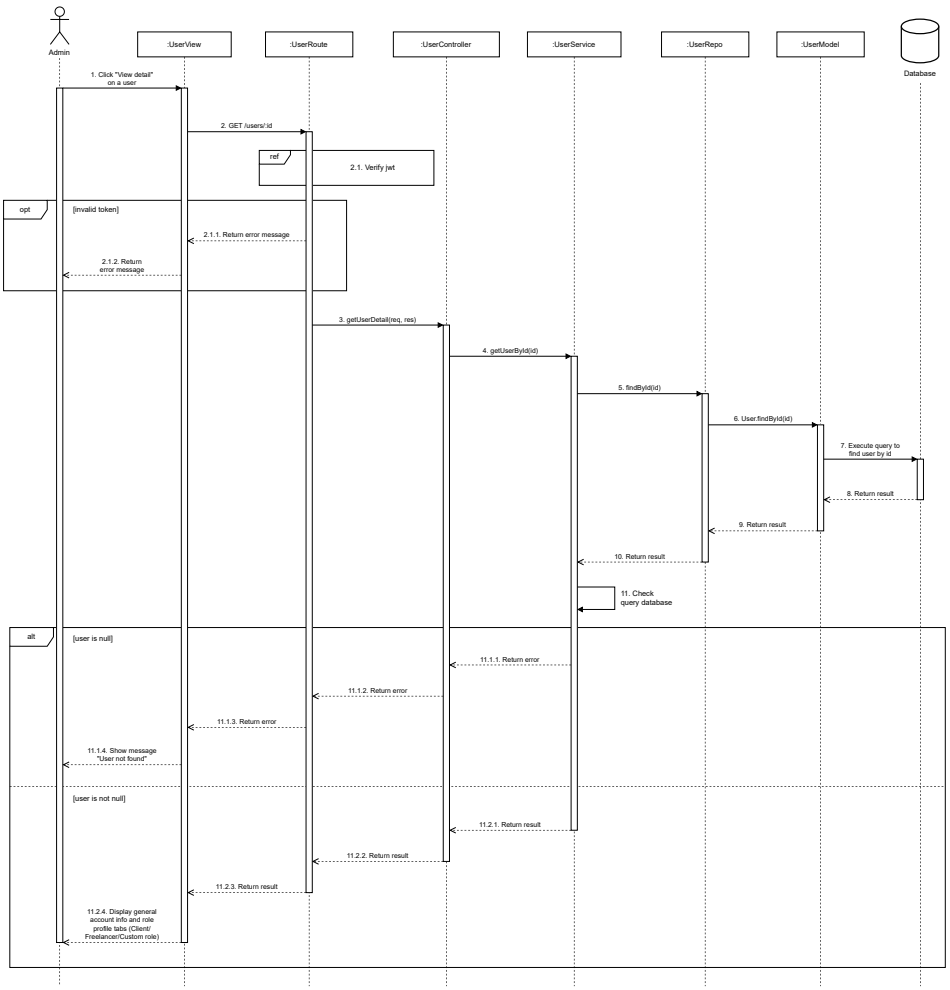

Figure 111: Sequence diagram of UC-28.02 View User Detail

3.16.3 UC-28.03 – Create User

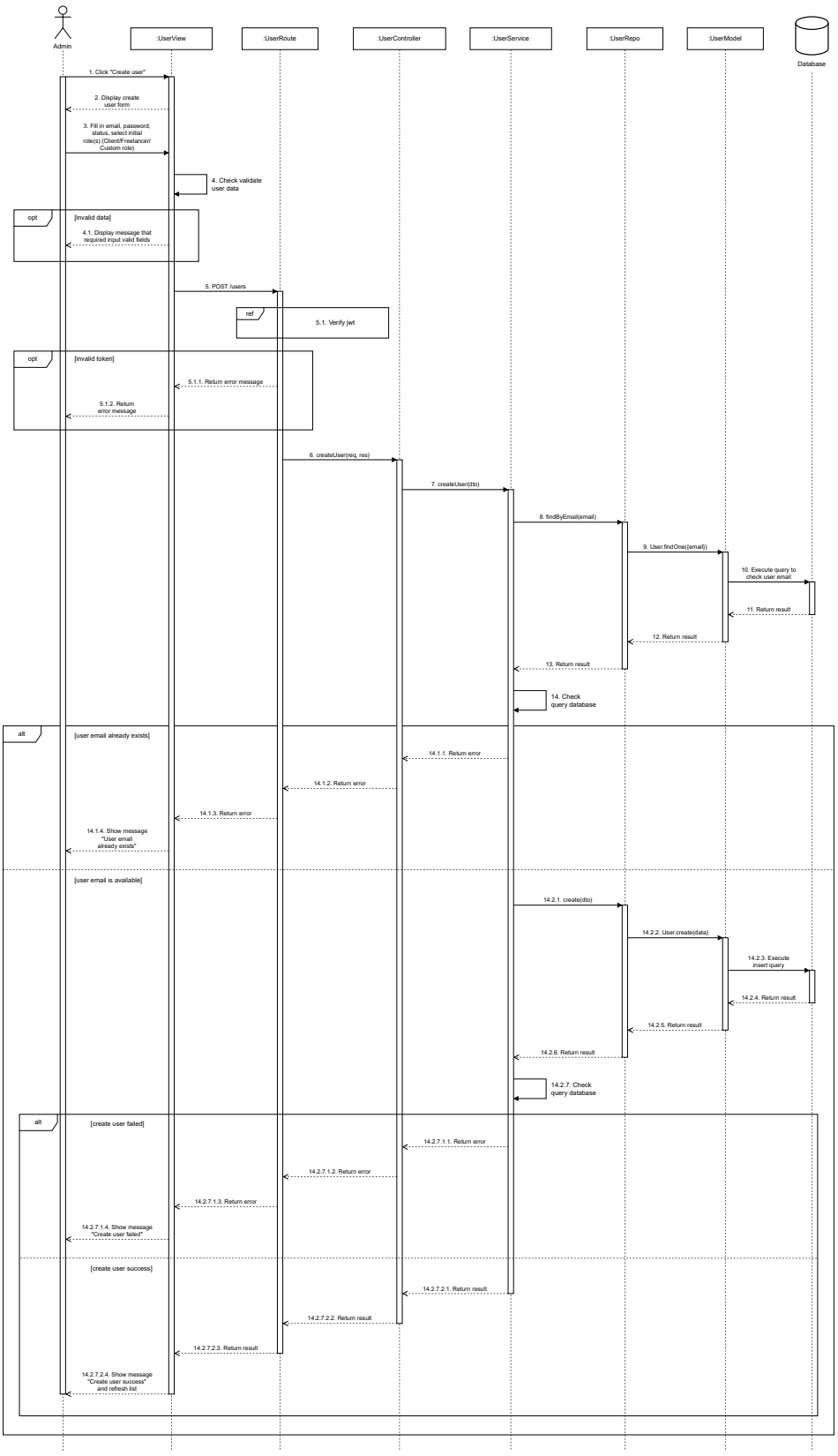

Figure 112: Sequence diagram of UC-28.03 Create User

3.16.4 UC-28.04 – Update User

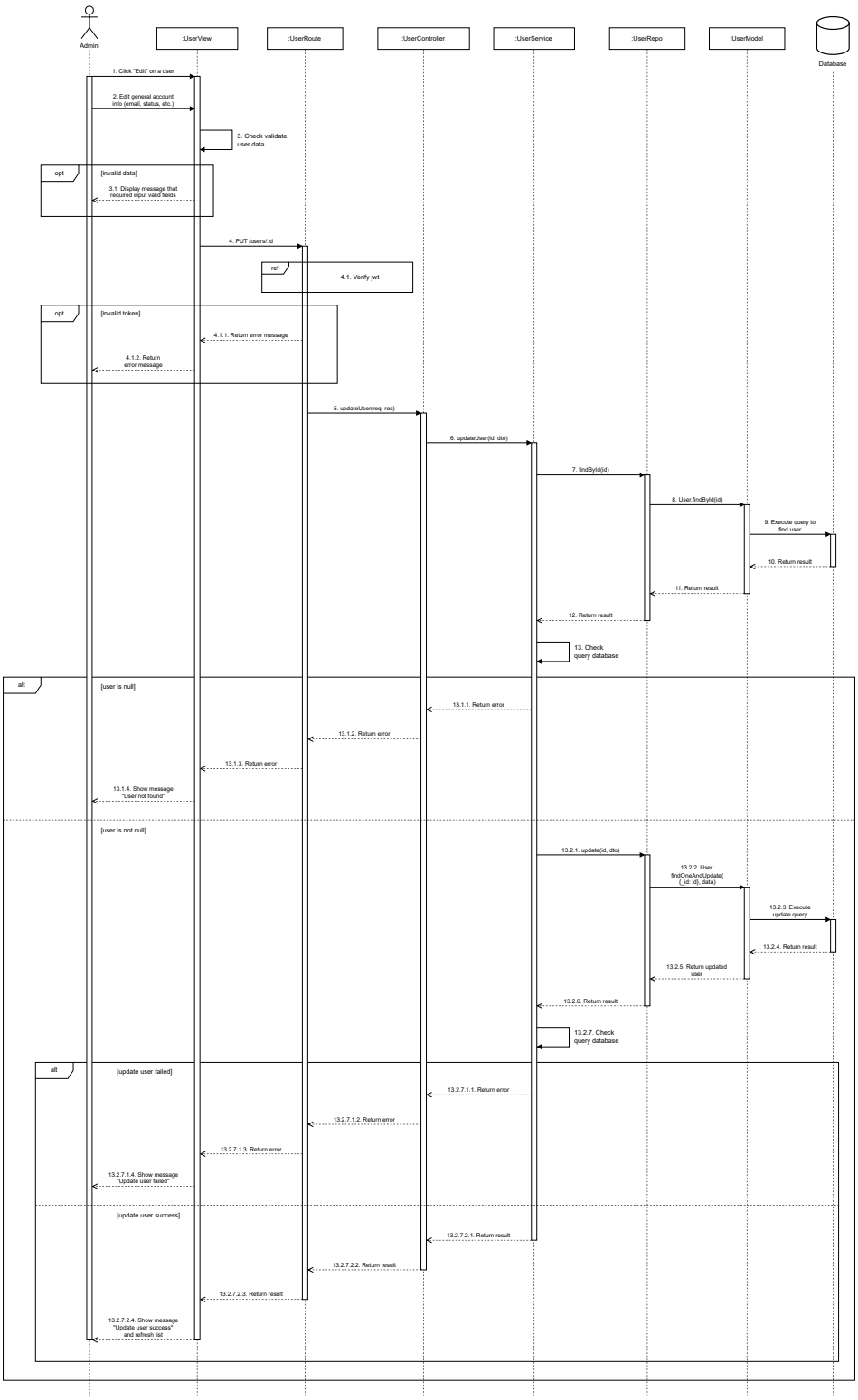

Figure 113: Sequence diagram of UC-28.04 Update User

3.16.5 UC-28.05 – Update Client Profile

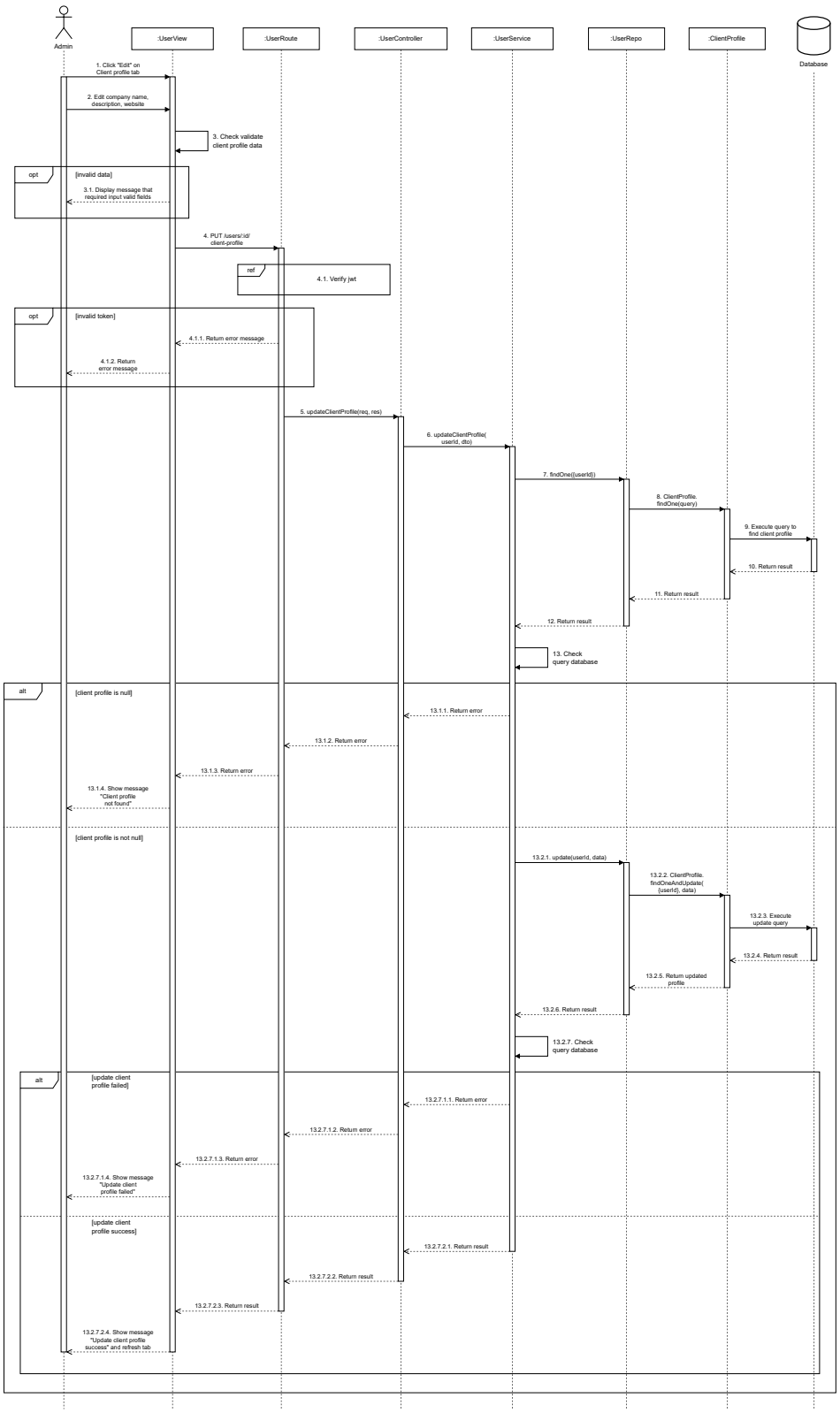

Figure 114: Sequence diagram of UC-28.05 Update Client Profile

3.16.6 UC-28.06 – Update Freelancer Basic Info

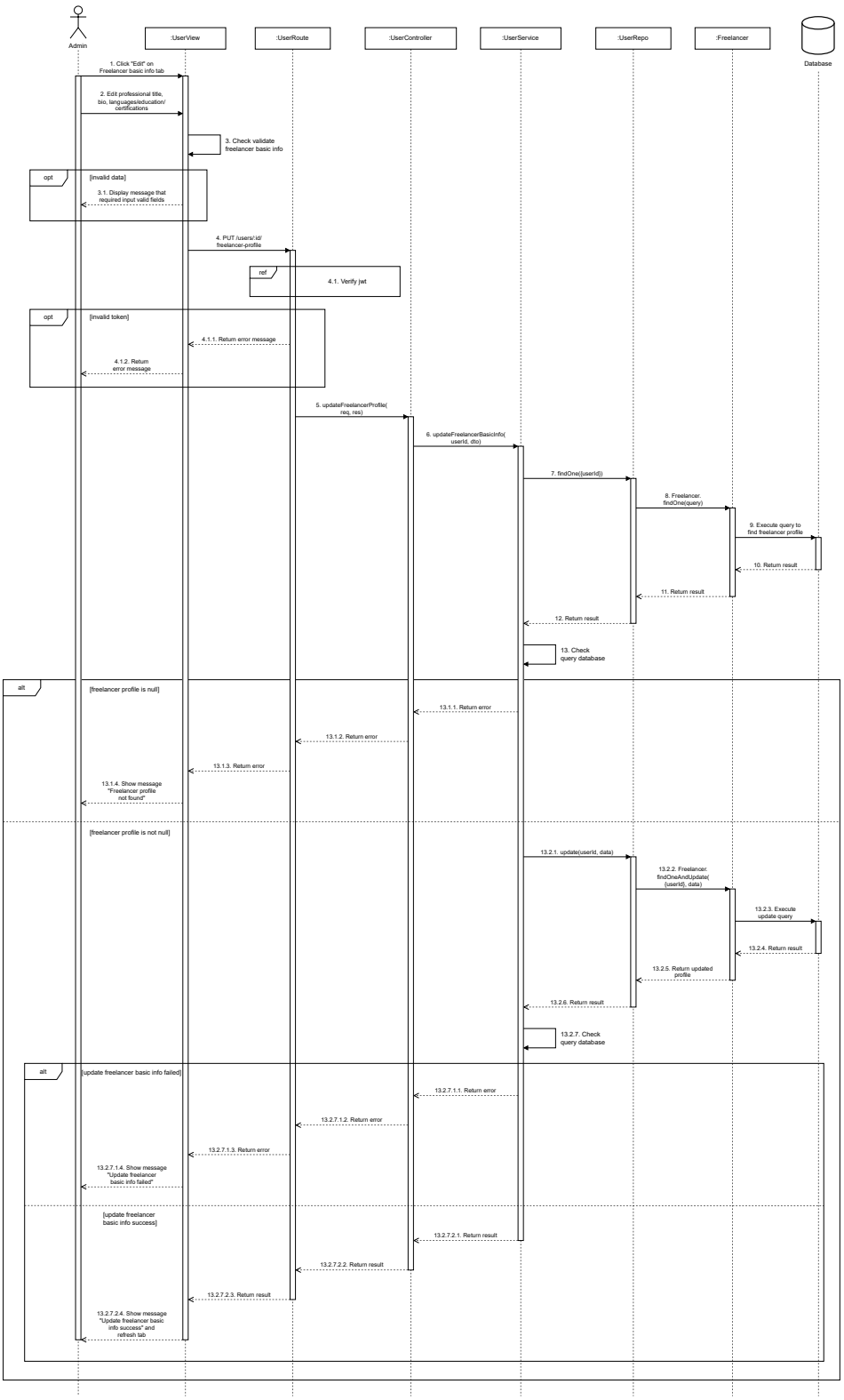

Figure 115: Sequence diagram of UC-28.06 Update Freelancer Basic Info

3.16.7 UC-28.07 – Update Freelancer Skills

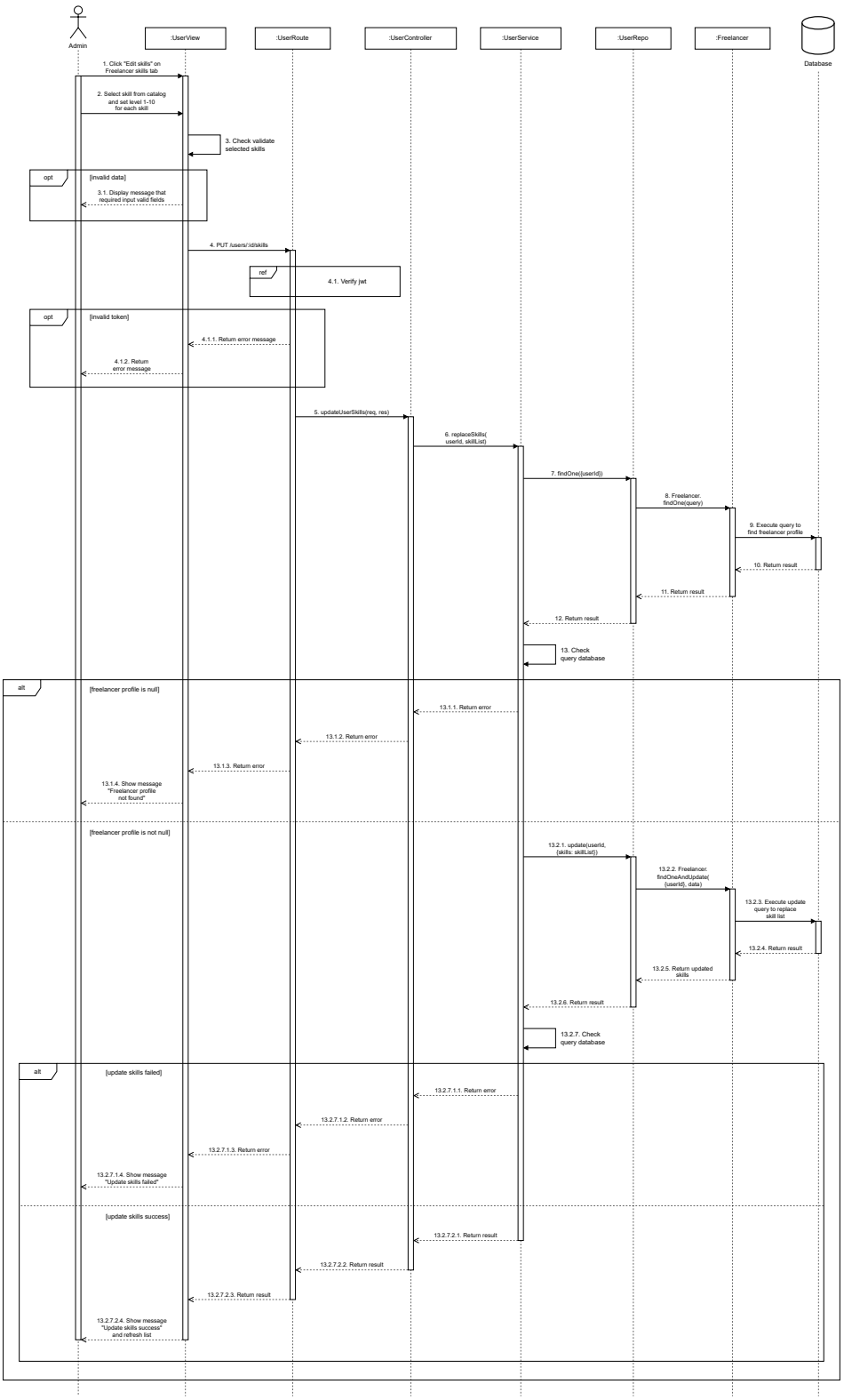

Figure 116: Sequence diagram of UC-28.07 Update Freelancer Skills

3.16.8 UC-28.08 – Update Freelancer Portfolio

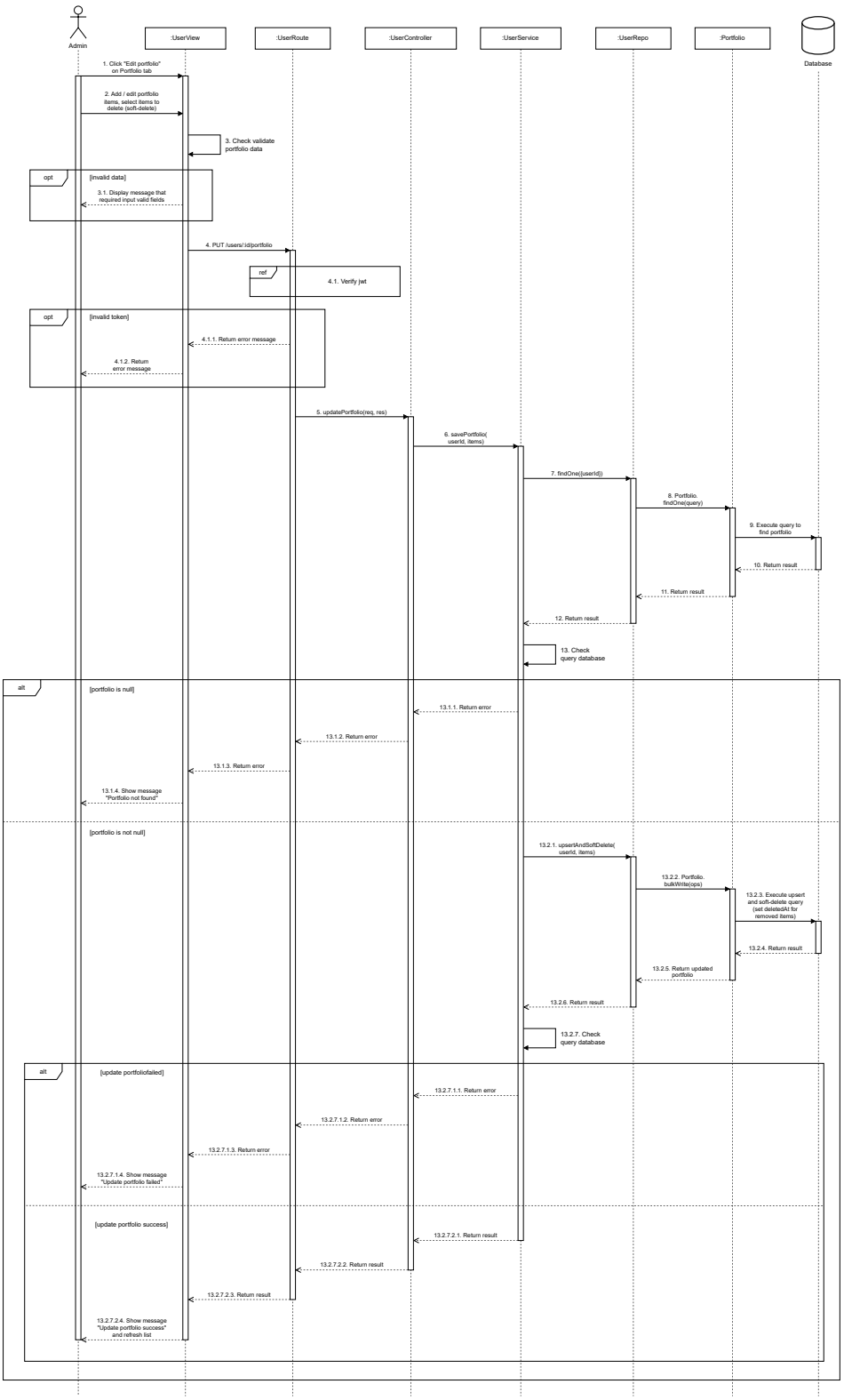

Figure 117: Sequence diagram of UC-28.08 Update Freelancer Portfolio

3.17 Payment Management

Payment Management covers the UC-43 financial transaction and payment infrastructure integrated with Stripe: connecting freelancer payout bank accounts via Stripe Connect Express, inspecting payout transfer histories and breakdowns, funding milestone escrow deposits, issuing milestone refunds, processing contract platform fees, authorizing milestone releases to freelancers, paying dispute arbitration fees, inspecting comprehensive account payment history and receipts, executing multi-party dispute settlement payouts, processing asynchronous Stripe webhook events, and synchronizing payment intent statuses with the payment gateway.

a. Class Diagram

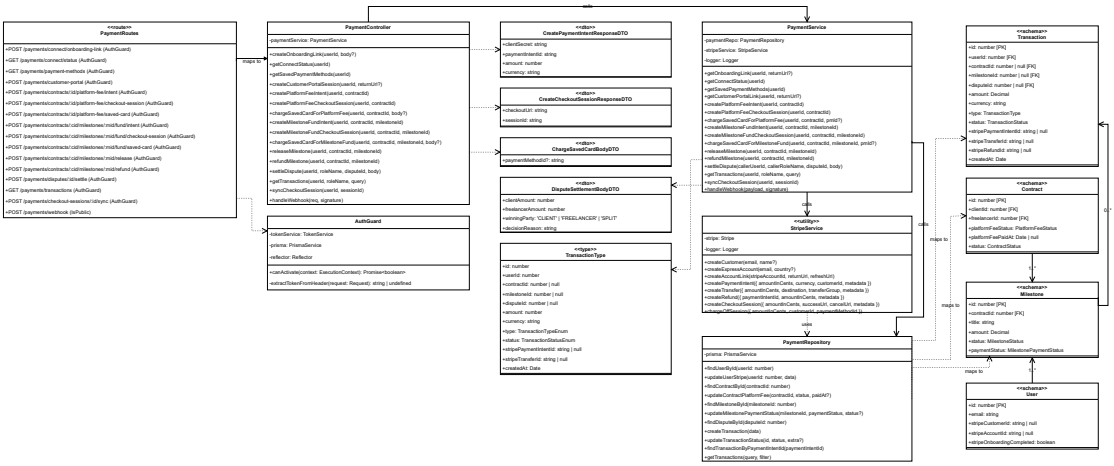

Figure 118: Class diagram of Payment Management

b. Sequence Diagram(s)

3.17.1 UC-43.01 – Connect Stripe Account

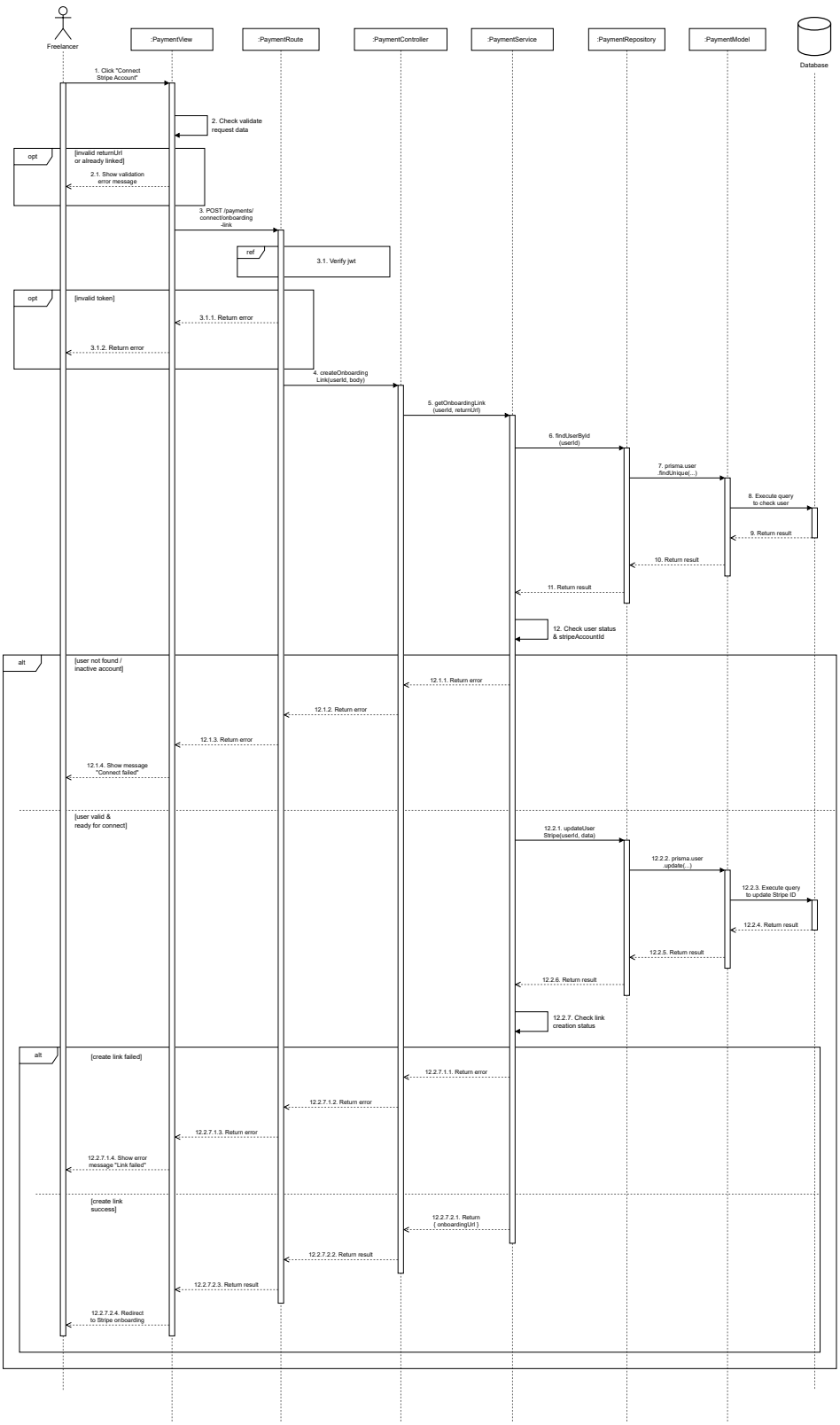

Figure 119: Sequence diagram of UC-43.01 Connect Stripe Account

3.17.2 UC-43.02 – View Payout History

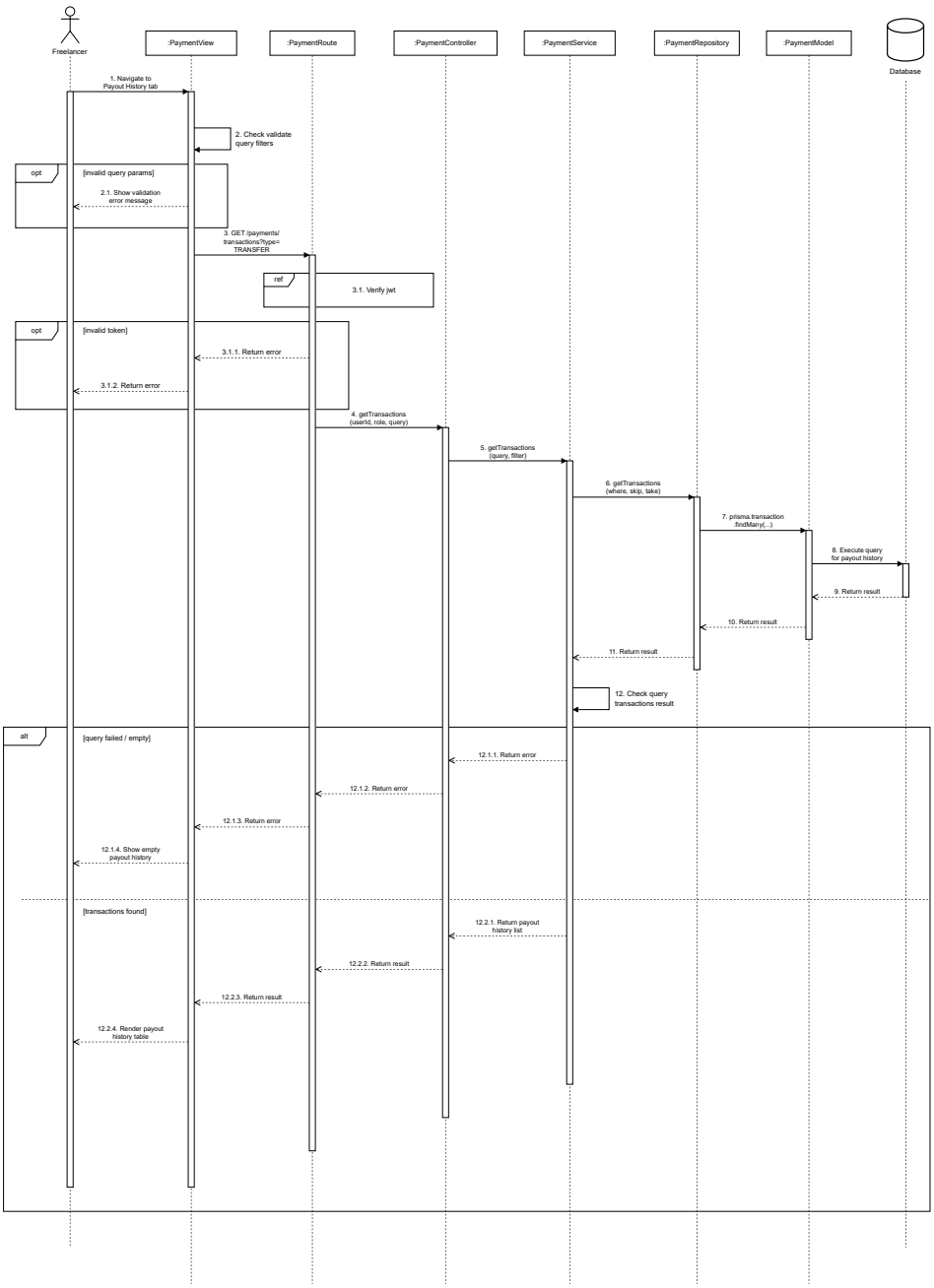

Figure 120: Sequence diagram of UC-43.02 View Payout History

3.17.3 UC-43.03 – View Payout Detail

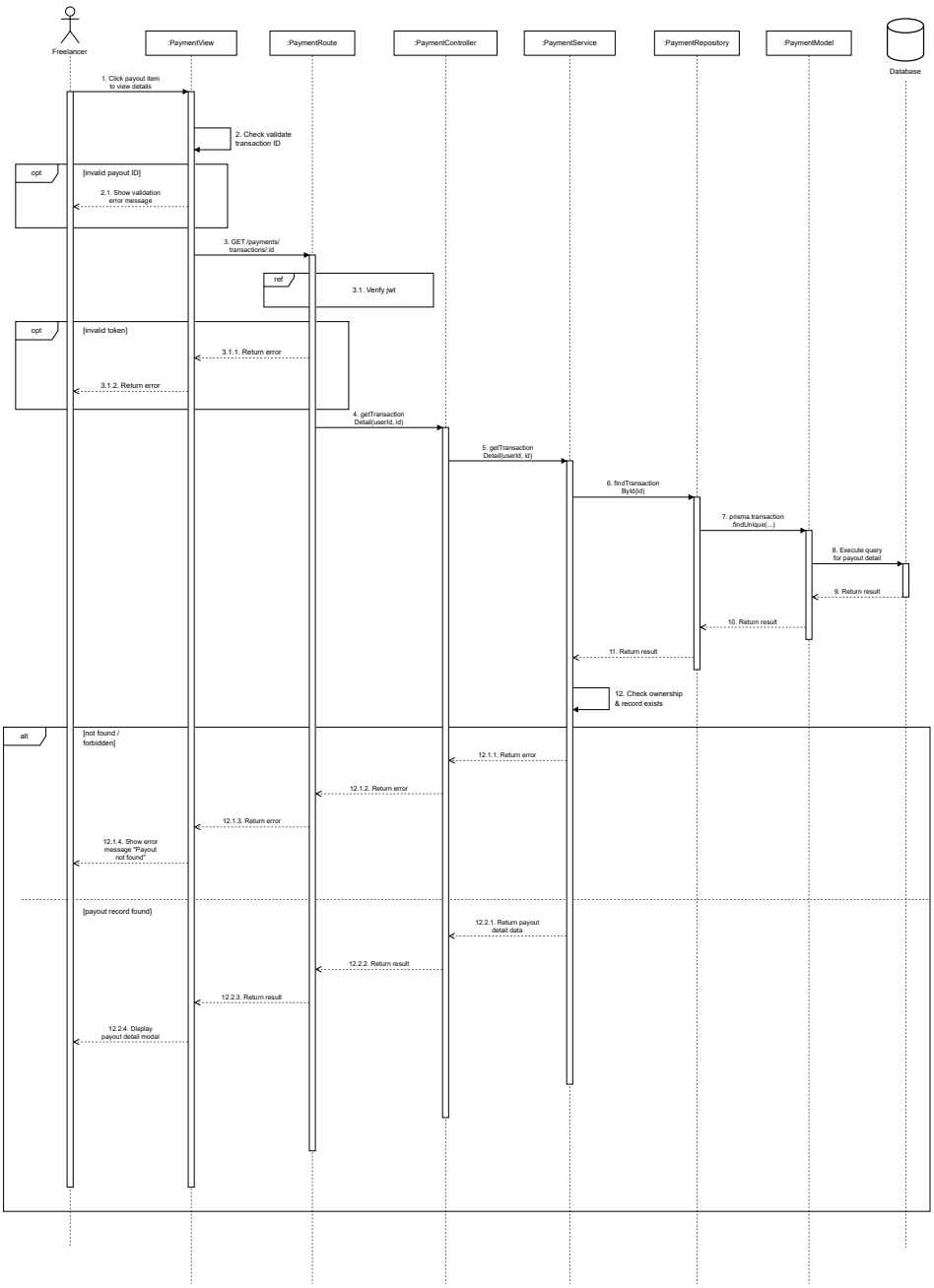

Figure 121: Sequence diagram of UC-43.03 View Payout Detail

3.17.4 UC-43.04 – Fund Milestone

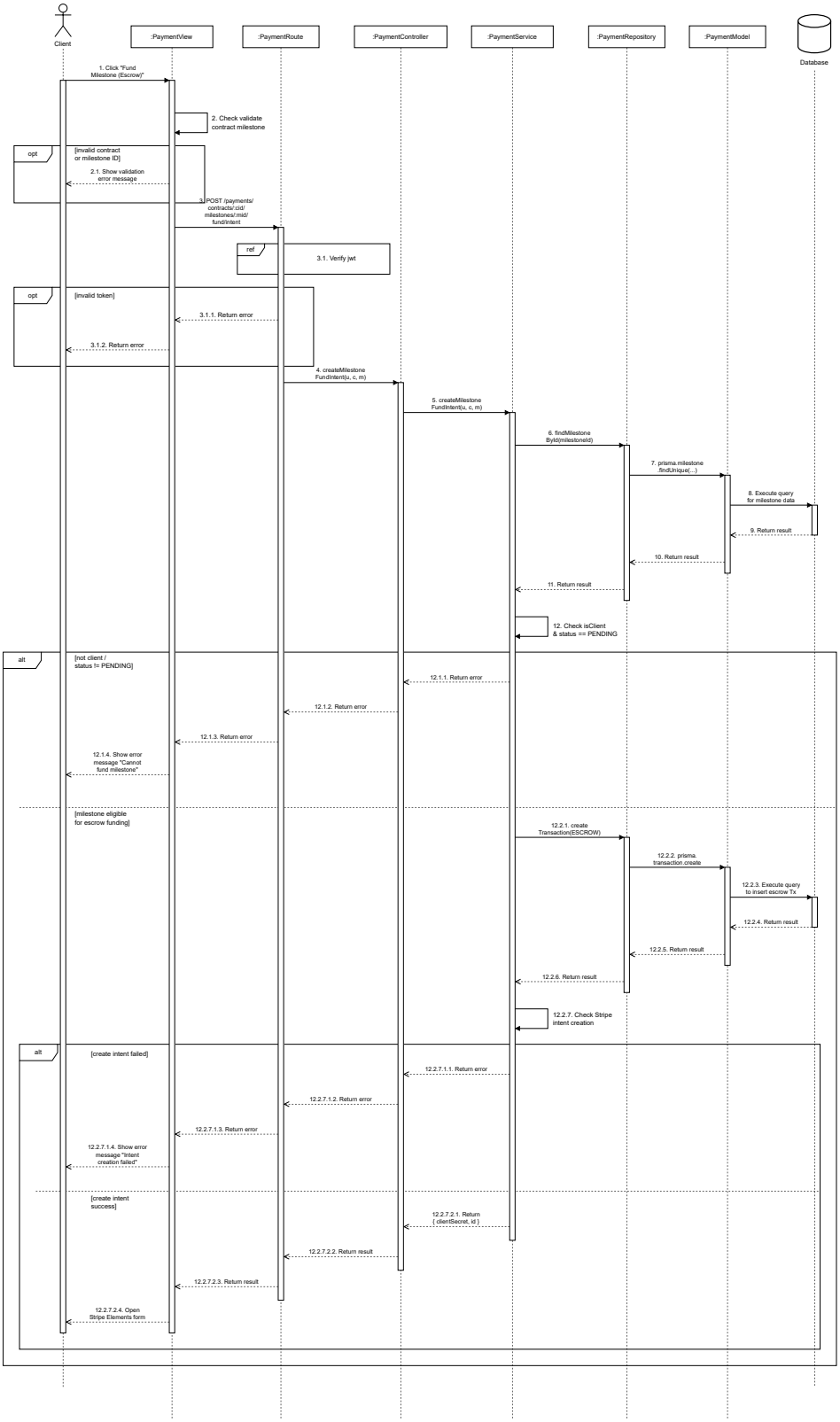

Figure 122: Sequence diagram of UC-43.04 Fund Milestone

3.17.5 UC-43.05 – Refund Payment

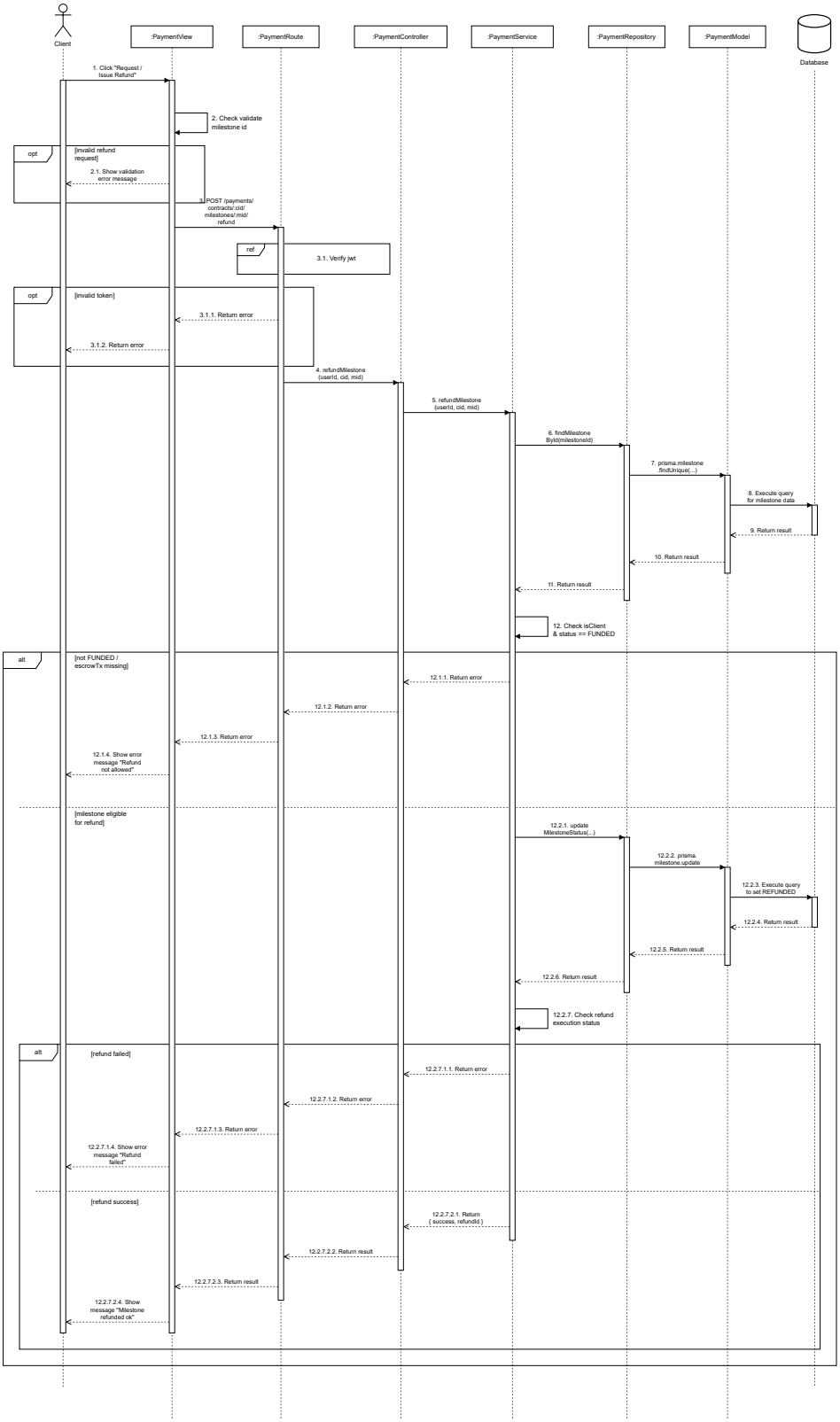

Figure 123: Sequence diagram of UC-43.05 Refund Payment

3.17.6 UC-43.06 – Pay Platform Fee

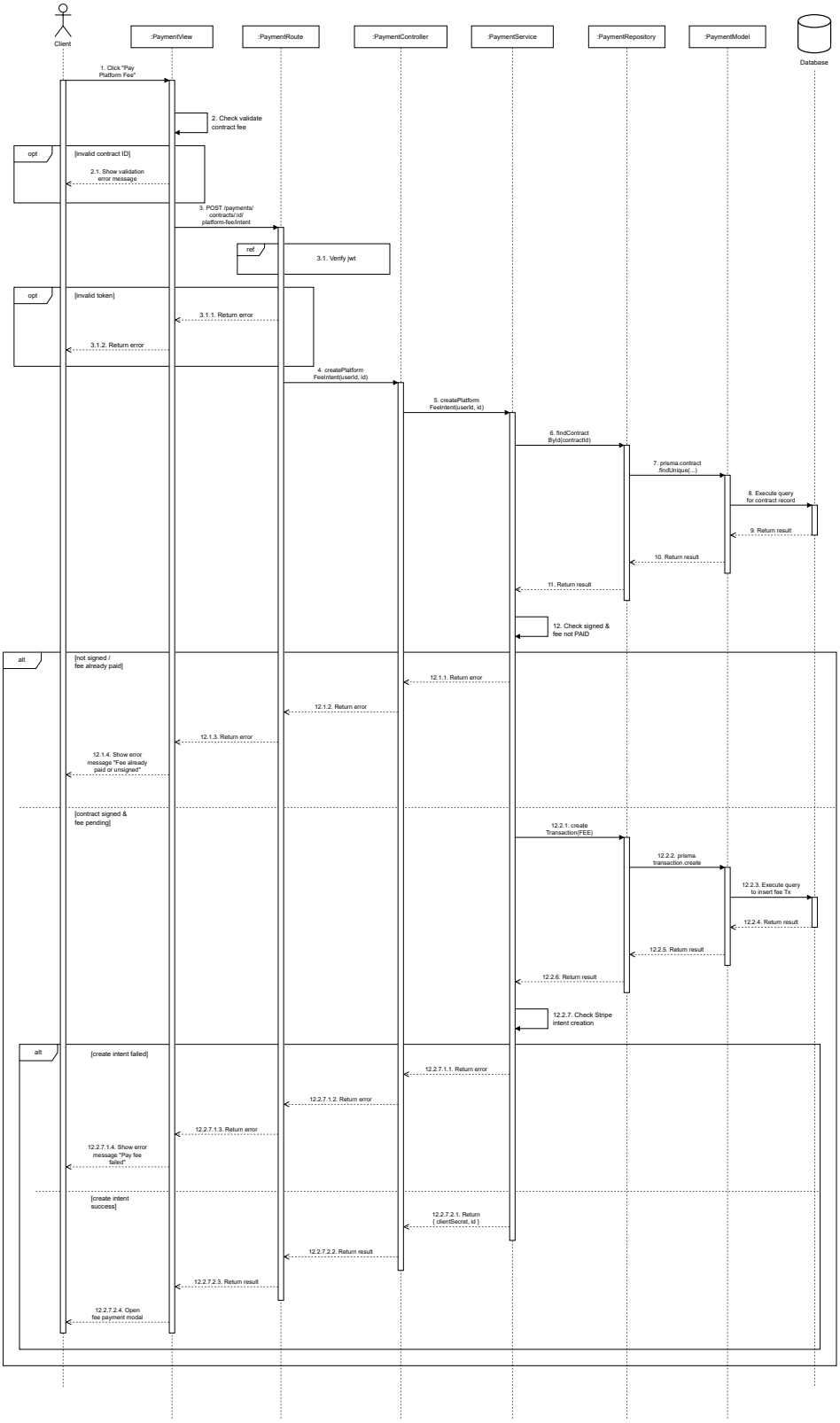

Figure 124: Sequence diagram of UC-43.06 Pay Platform Fee

3.17.7 UC-43.07 – Release Payment

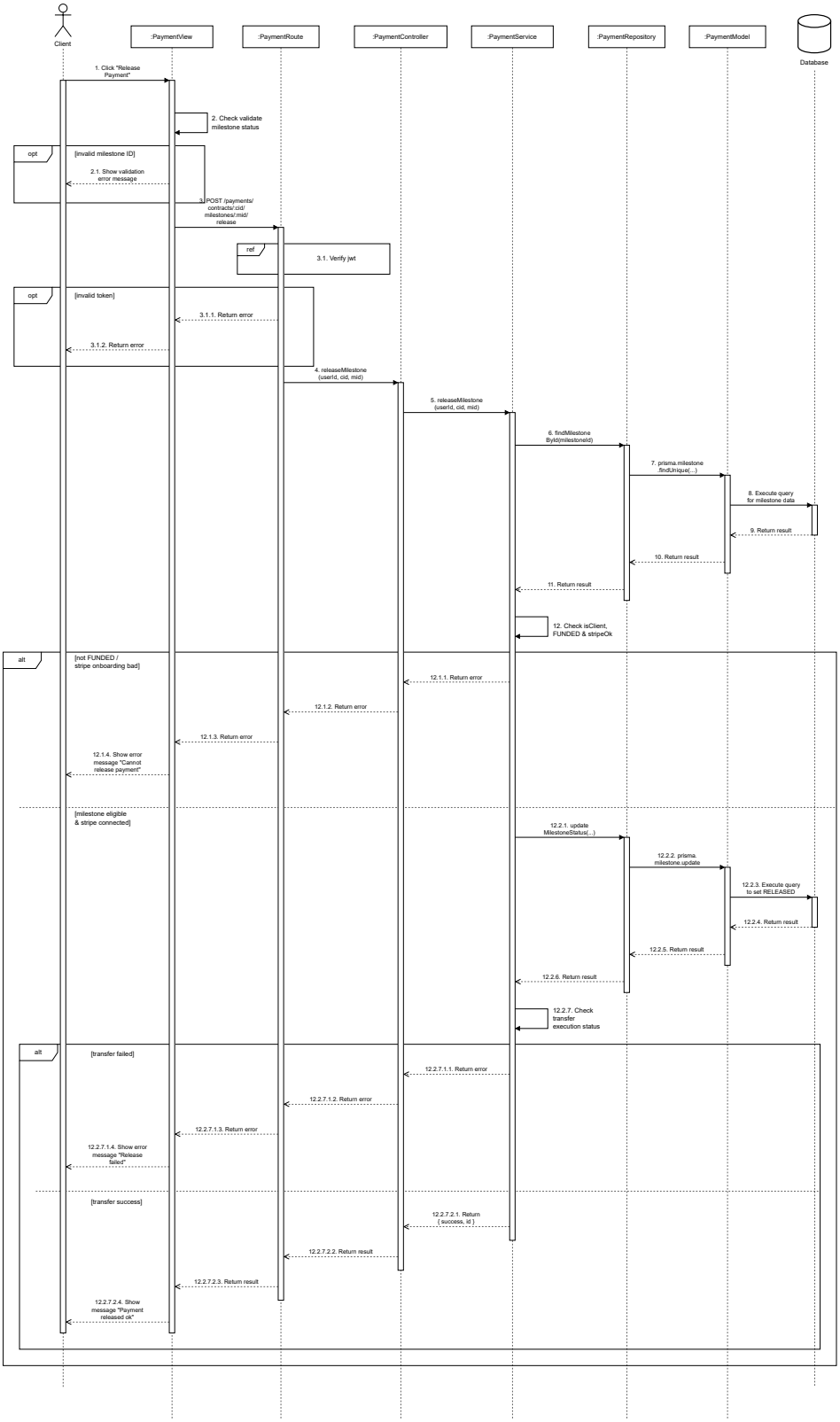

Figure 125: Sequence diagram of UC-43.07 Release Payment

3.17.8 UC-43.08 – Pay Dispute Fee

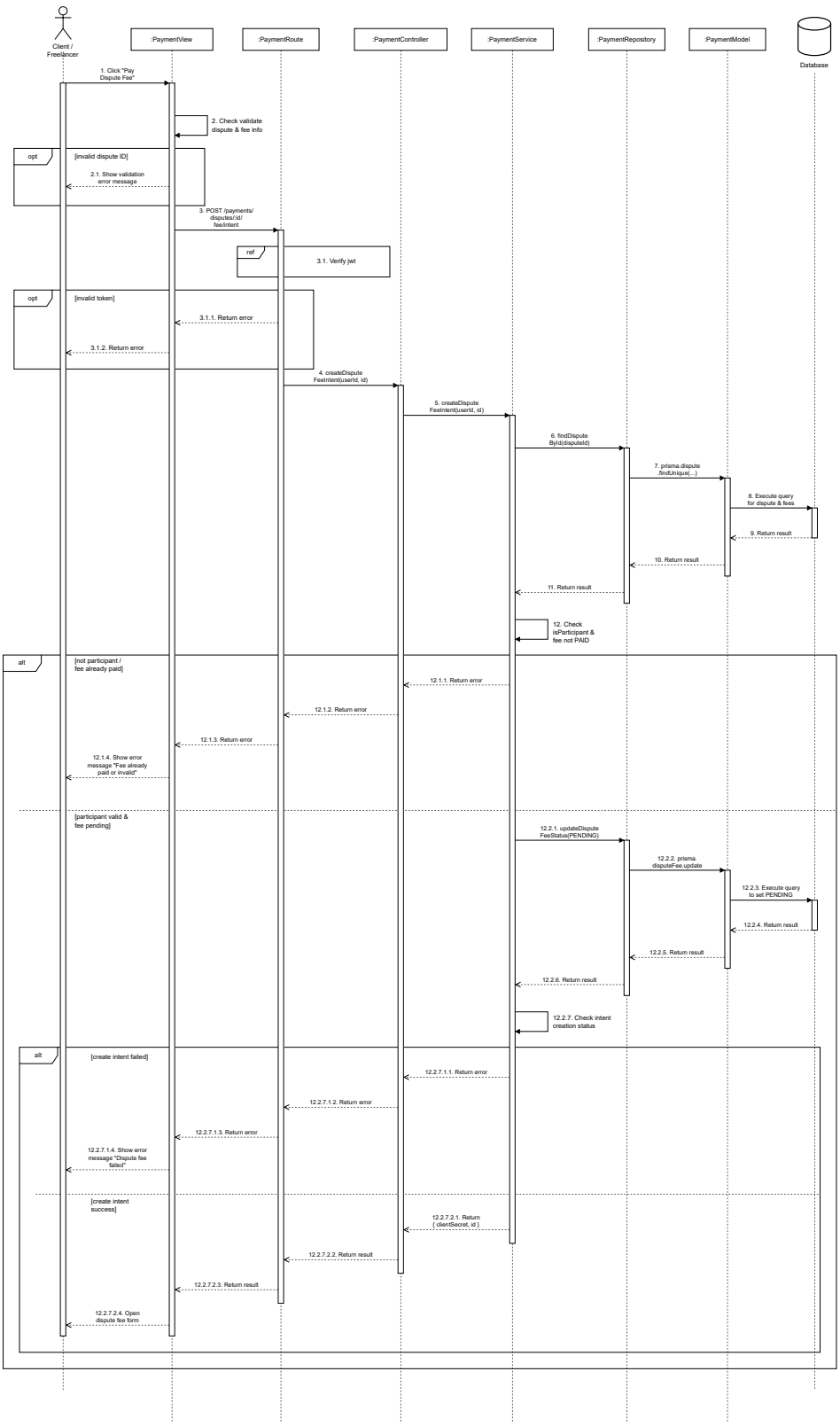

Figure 126: Sequence diagram of UC-43.08 Pay Dispute Fee

3.17.9 UC-43.09 – View Payment History

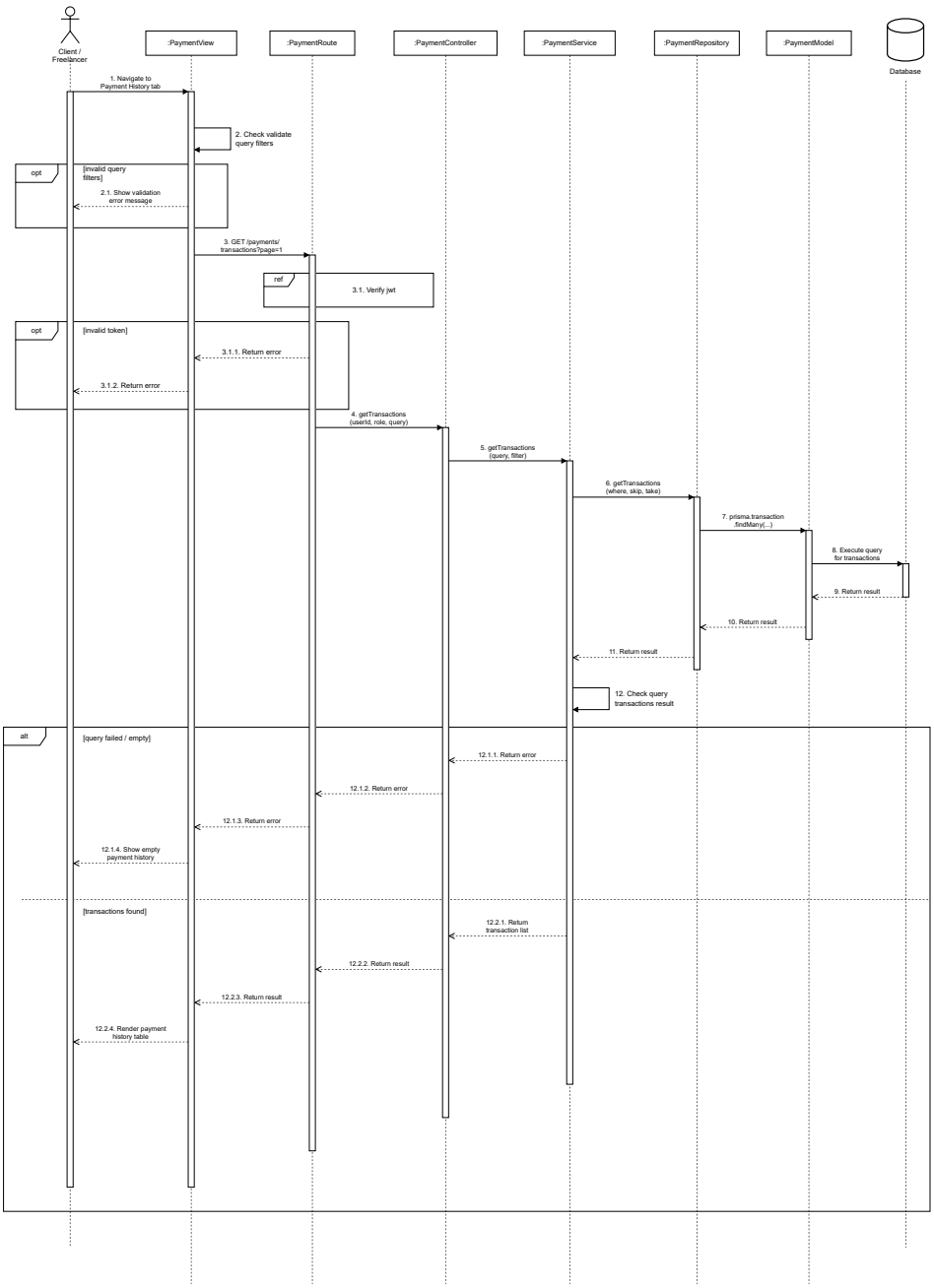

Figure 127: Sequence diagram of UC-43.09 View Payment History

3.17.10 UC-43.10 – View Payment Detail

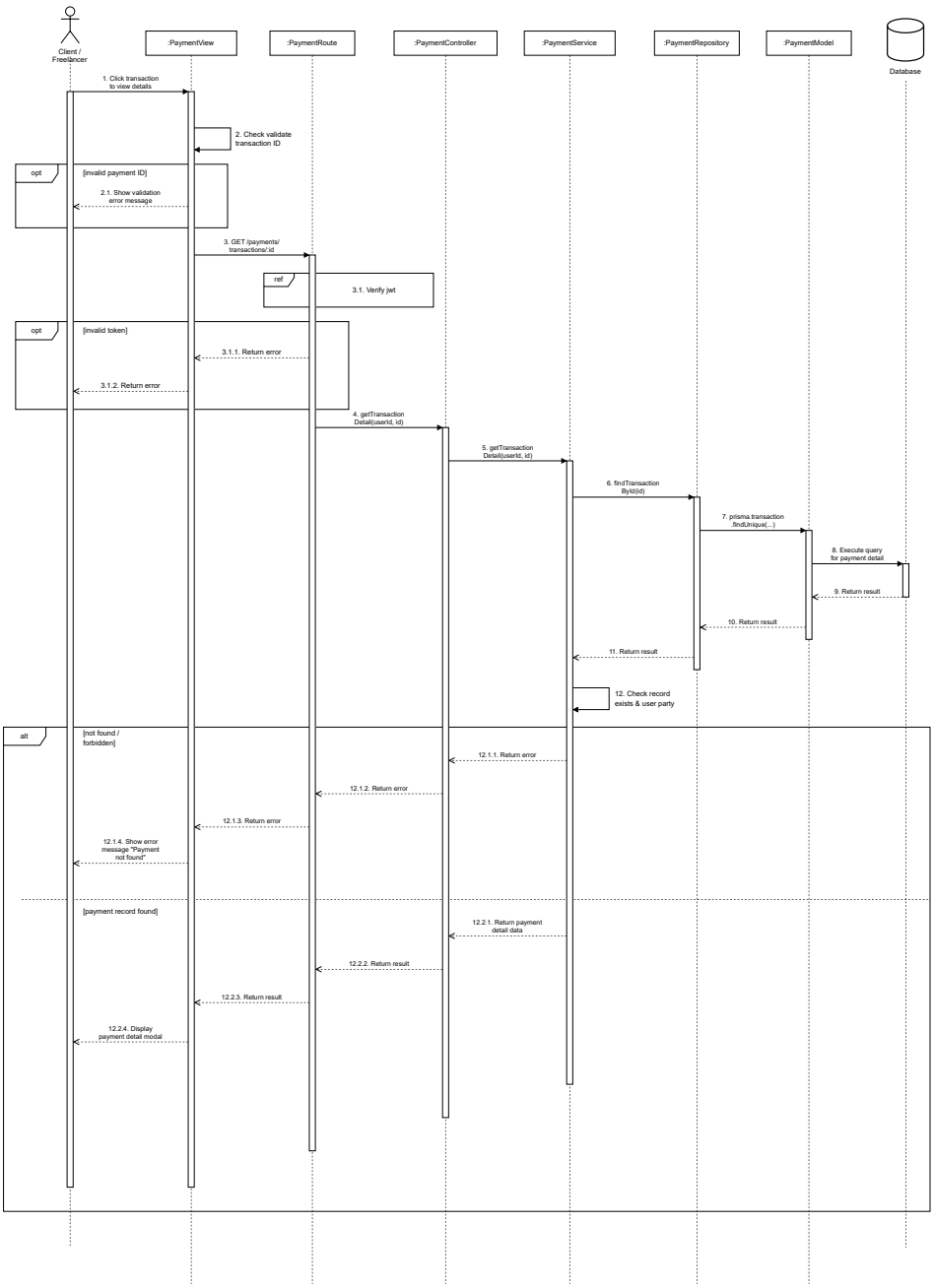

Figure 128: Sequence diagram of UC-43.10 View Payment Detail

3.17.11 UC-43.11 – Execute Dispute Resolution Payment

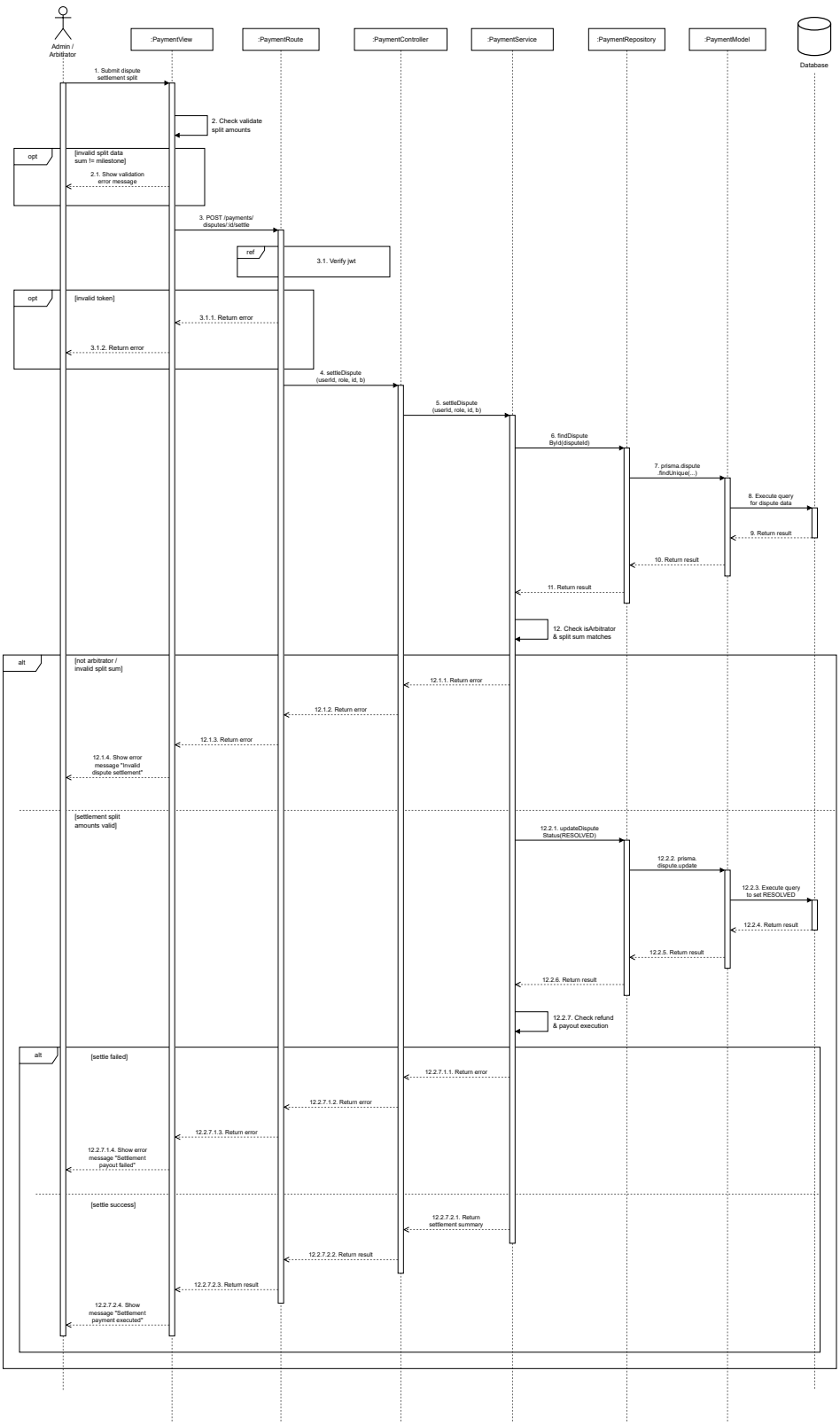

Figure 129: Sequence diagram of UC-43.11 Execute Dispute Resolution Payment

3.17.12 UC-43.12 – Handle Stripe Webhook Event

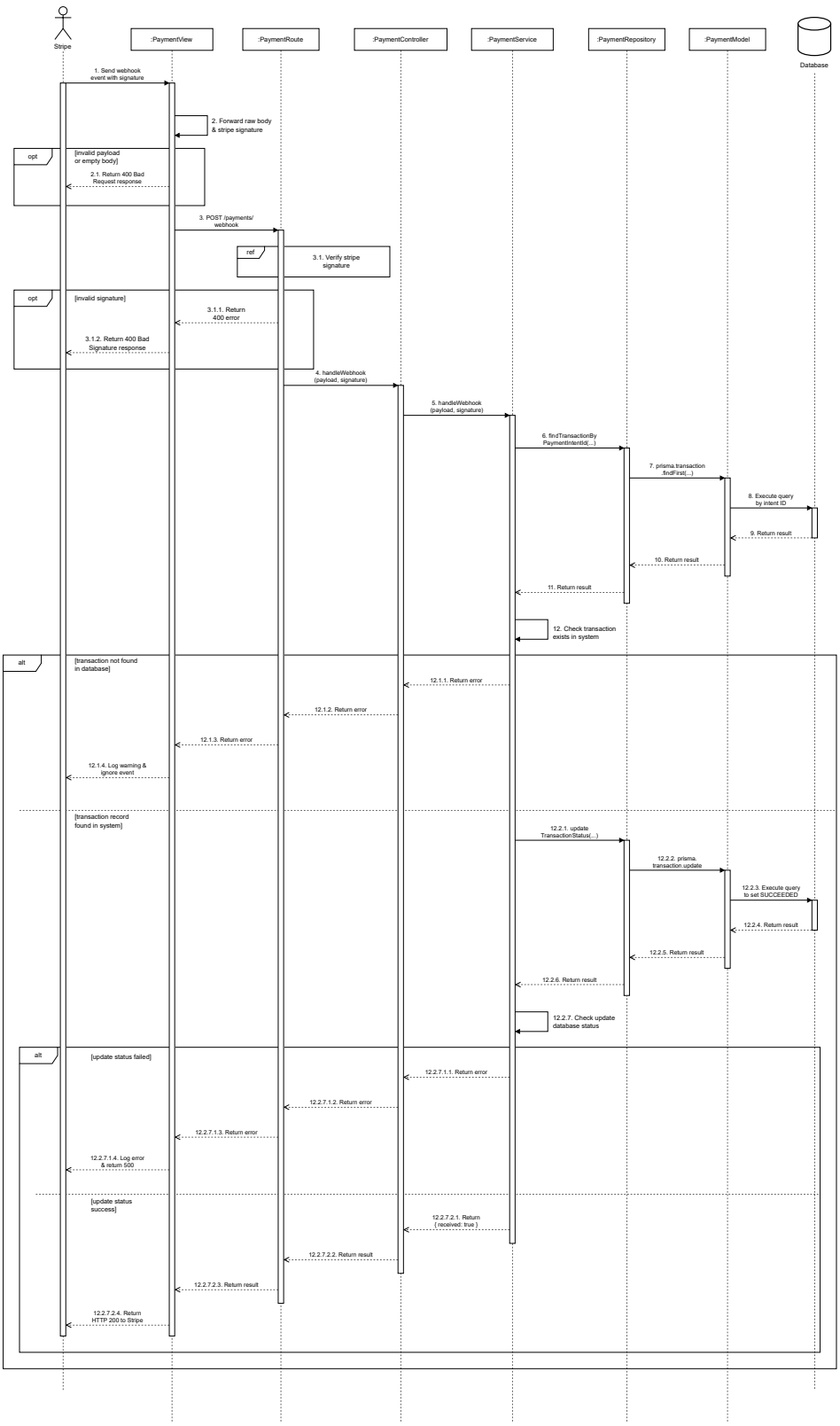

Figure 130: Sequence diagram of UC-43.12 Handle Stripe Webhook Event

3.17.13 UC-43.13 – Sync Payment Intent Status

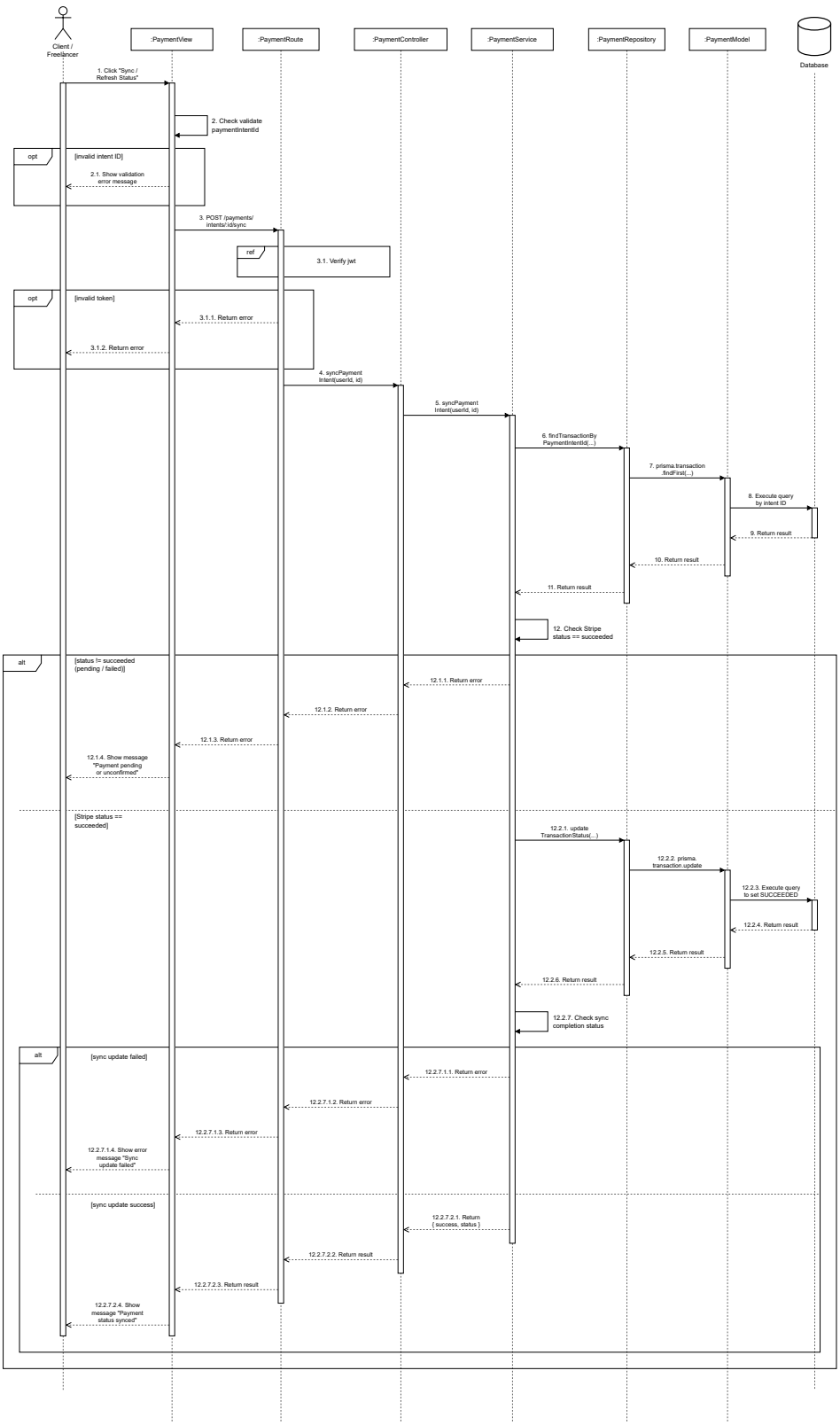

Figure 131: Sequence diagram of UC-43.13 Sync Payment Intent Status

3.18 Banner Management

Banner Management maintains the advertisement banners shown in configured page positions. The module supports image upload, lifecycle dates, activation state, and hard deletion.

a. Class Diagram

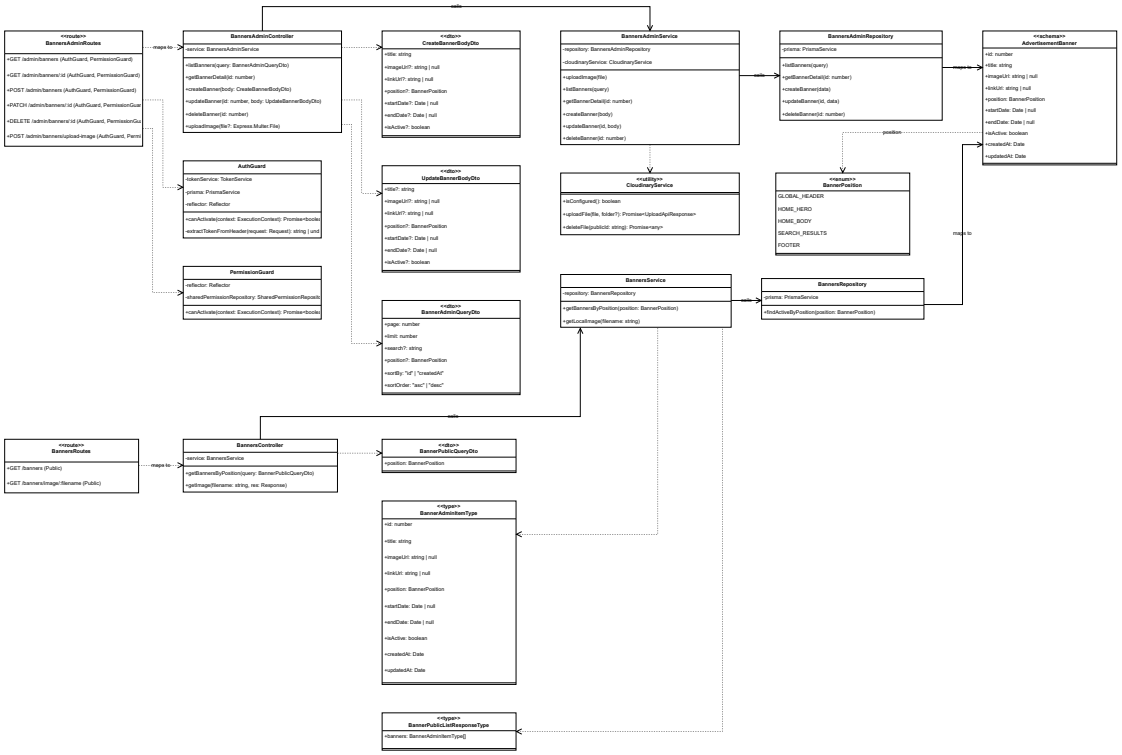

Figure 132: Class diagram of Banner Management

b. Sequence Diagram(s)

3.18.1 UC-37.01 – View Banner List

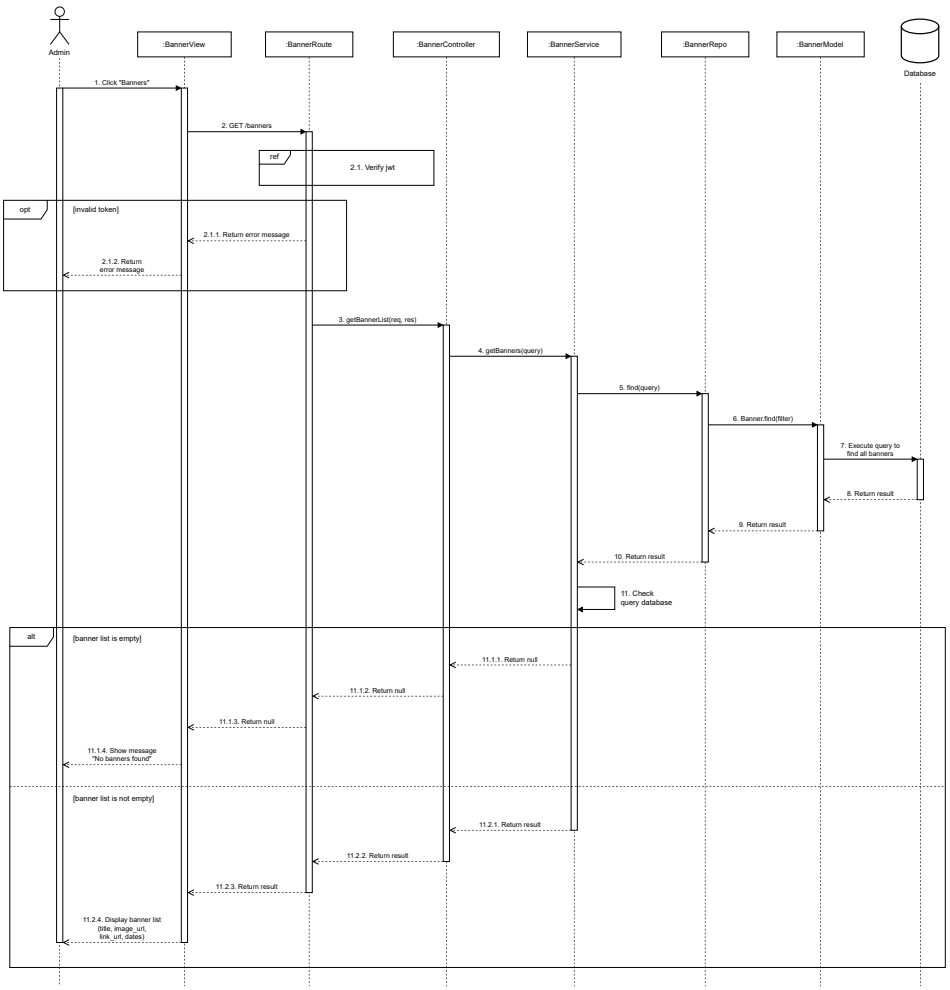

Figure 133: Sequence diagram of UC-37.01 View Banner List

3.18.2 UC-37.02 – View Banner Detail

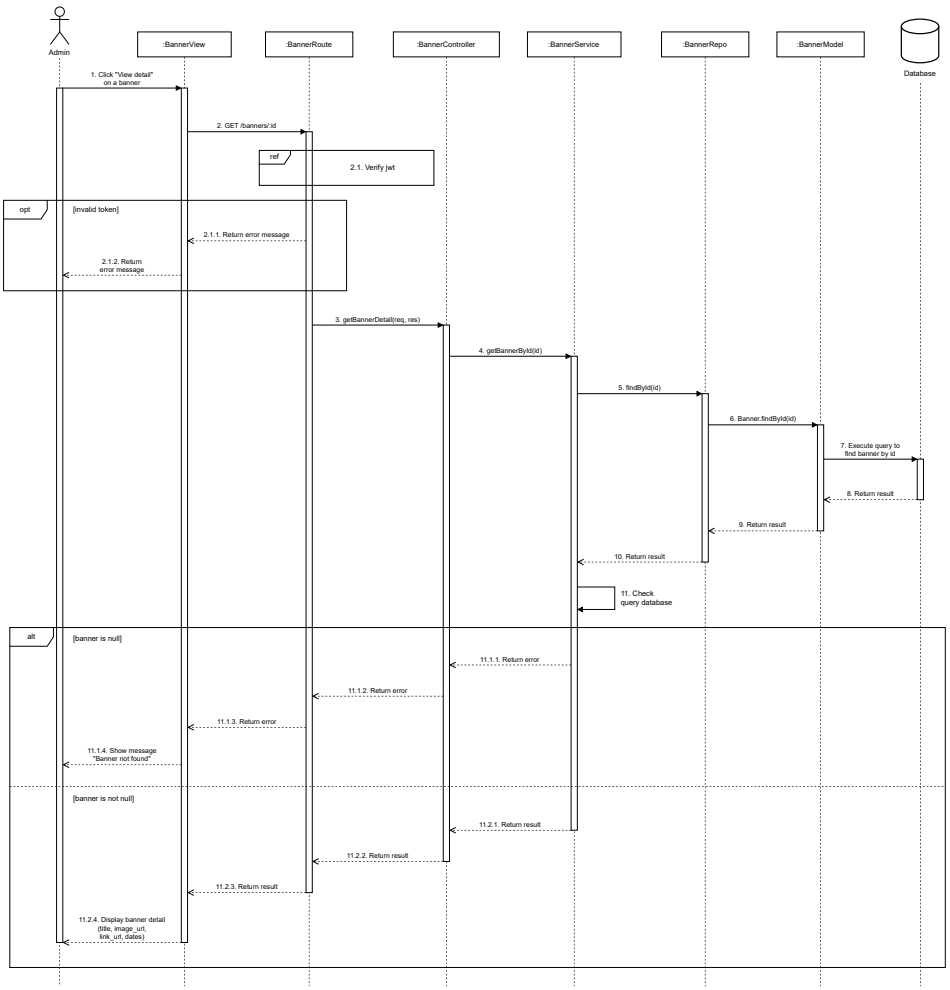

Figure 134: Sequence diagram of UC-37.02 View Banner Detail

3.18.3 UC-37.03 – Create Banner

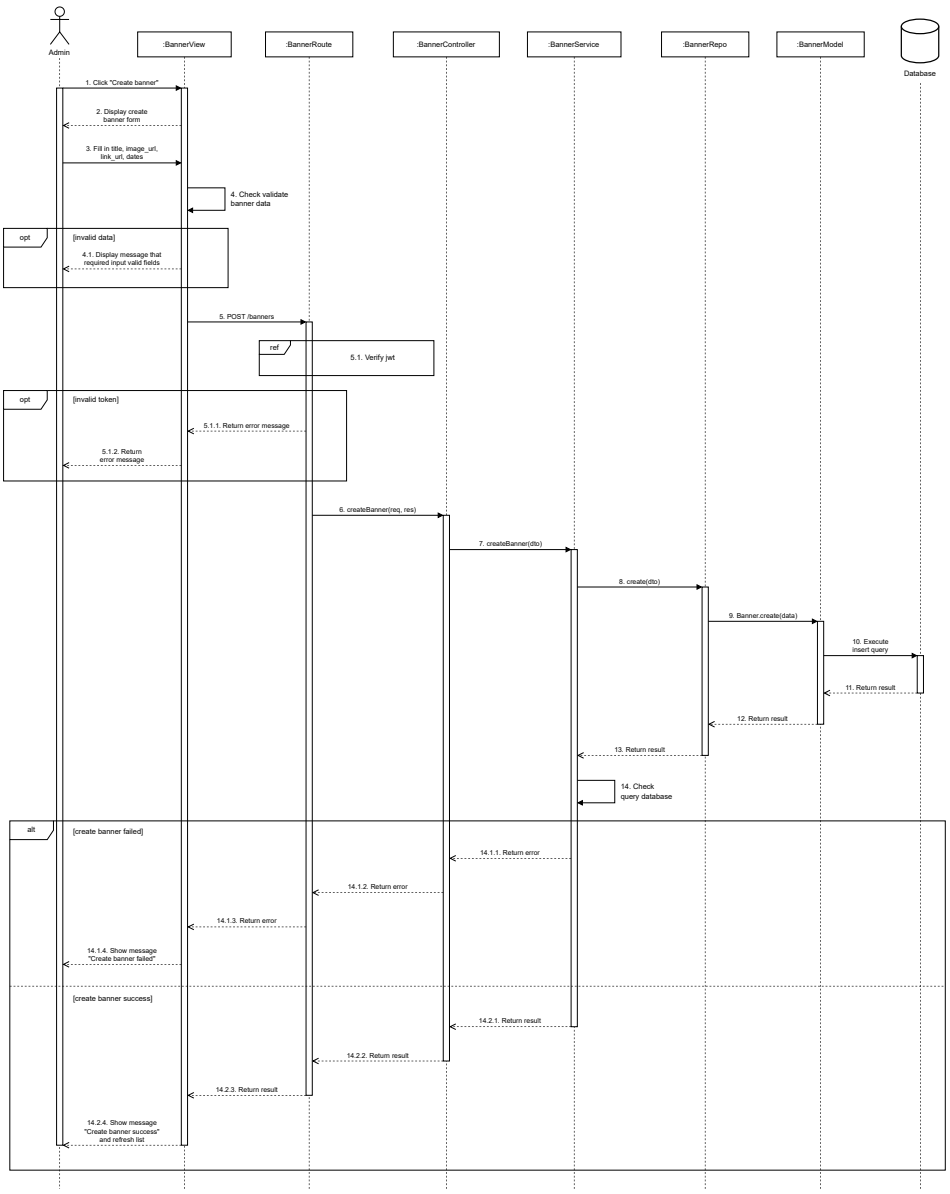

Figure 135: Sequence diagram of UC-37.03 Create Banner

3.18.4 UC-37.04 – Update Banner

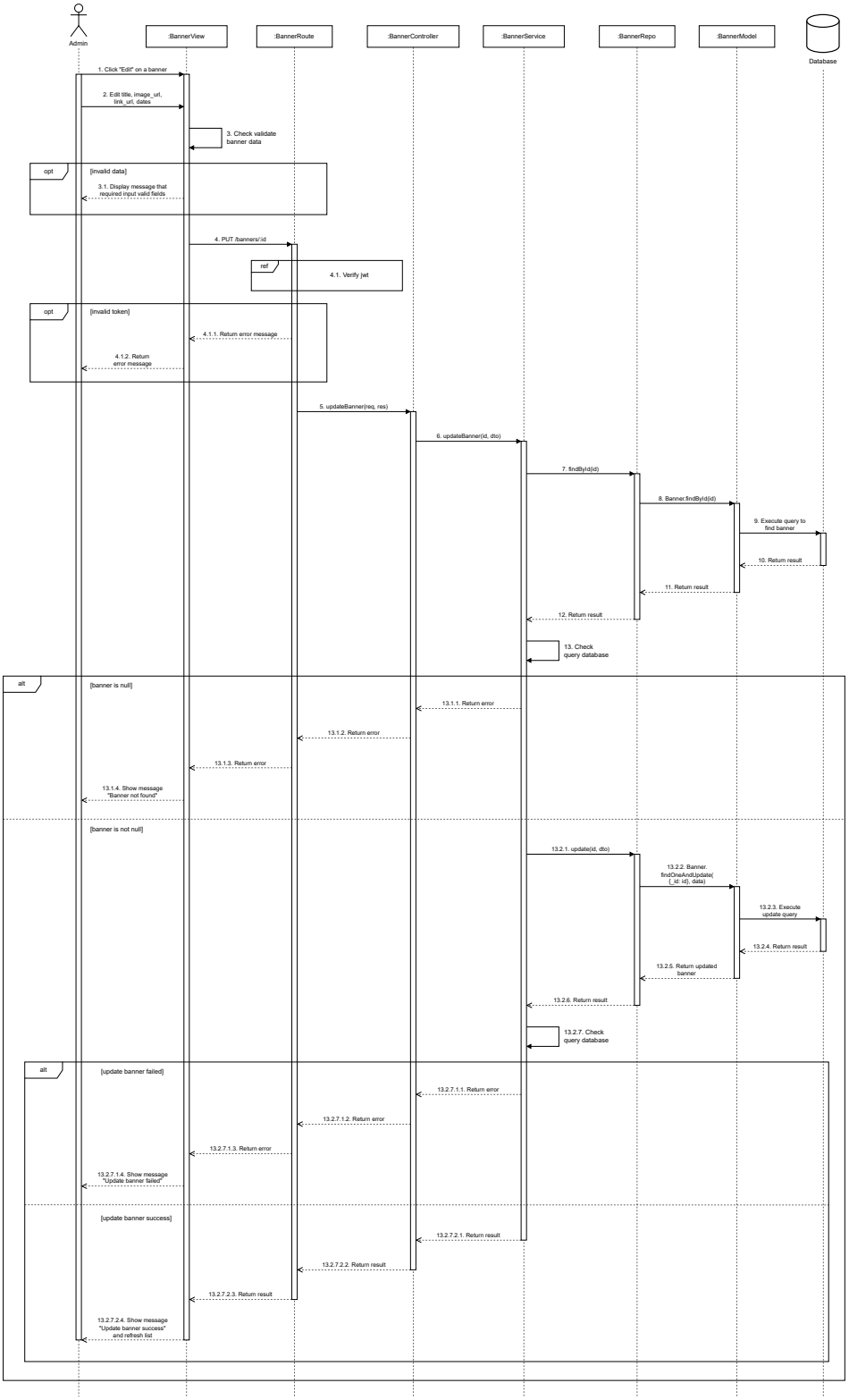

Figure 136: Sequence diagram of UC-37.04 Update Banner

3.18.5 UC-37.05 – Delete Banner

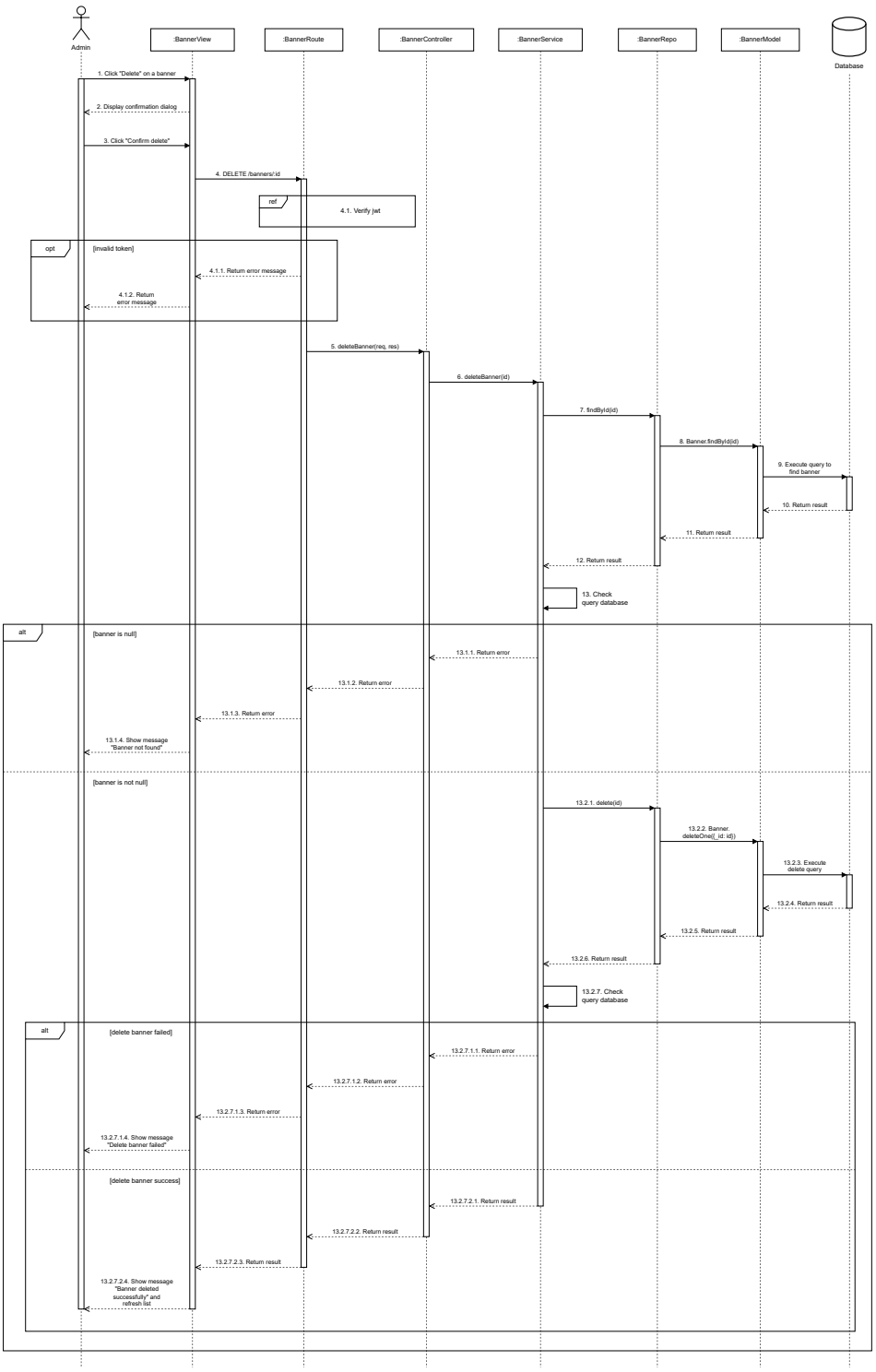

Figure 137: Sequence diagram of UC-37.05 Delete Banner

3.19 Skill Management

Skill Management owns the master skill catalogue used by freelancer profiles and job postings. Skills are soft-deleted so that existing references remain valid, and a deleted skill can be restored.

a. Class Diagram

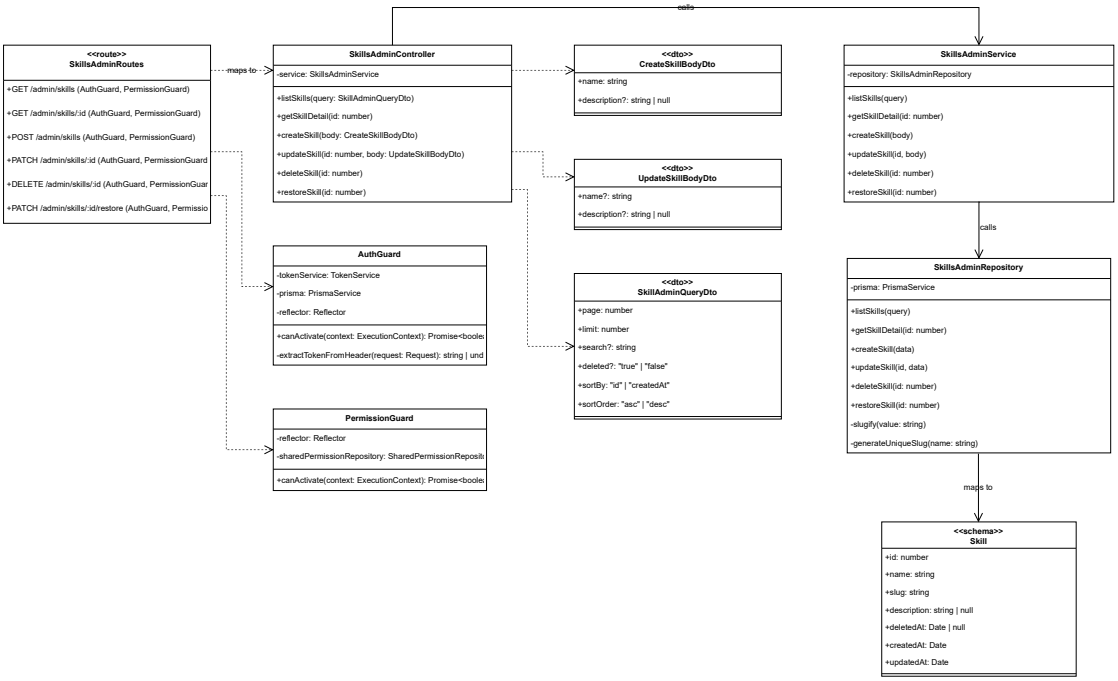

Figure 138: Class diagram of Skill Management

b. Sequence Diagram(s)

3.19.1 UC-38.01 – View Skill List

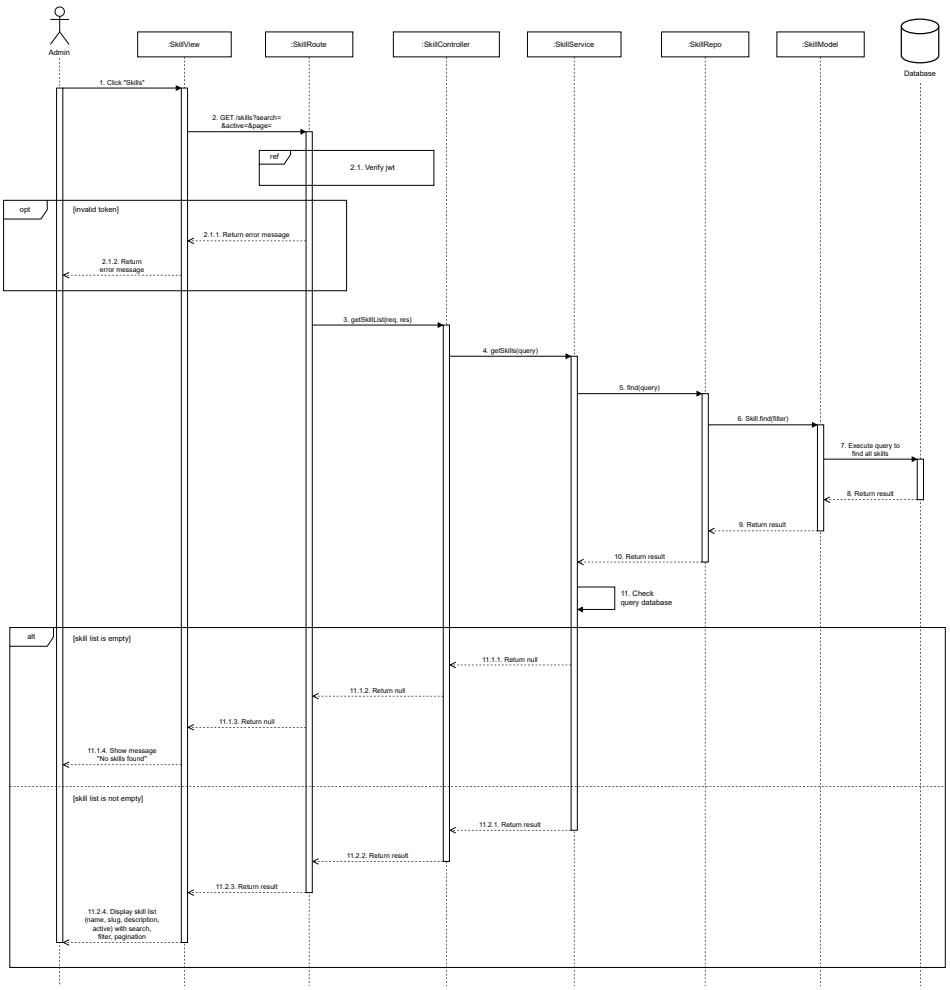

Figure 139: Sequence diagram of UC-38.01 View Skill List

3.19.2 UC-38.02 – View Skill Detail

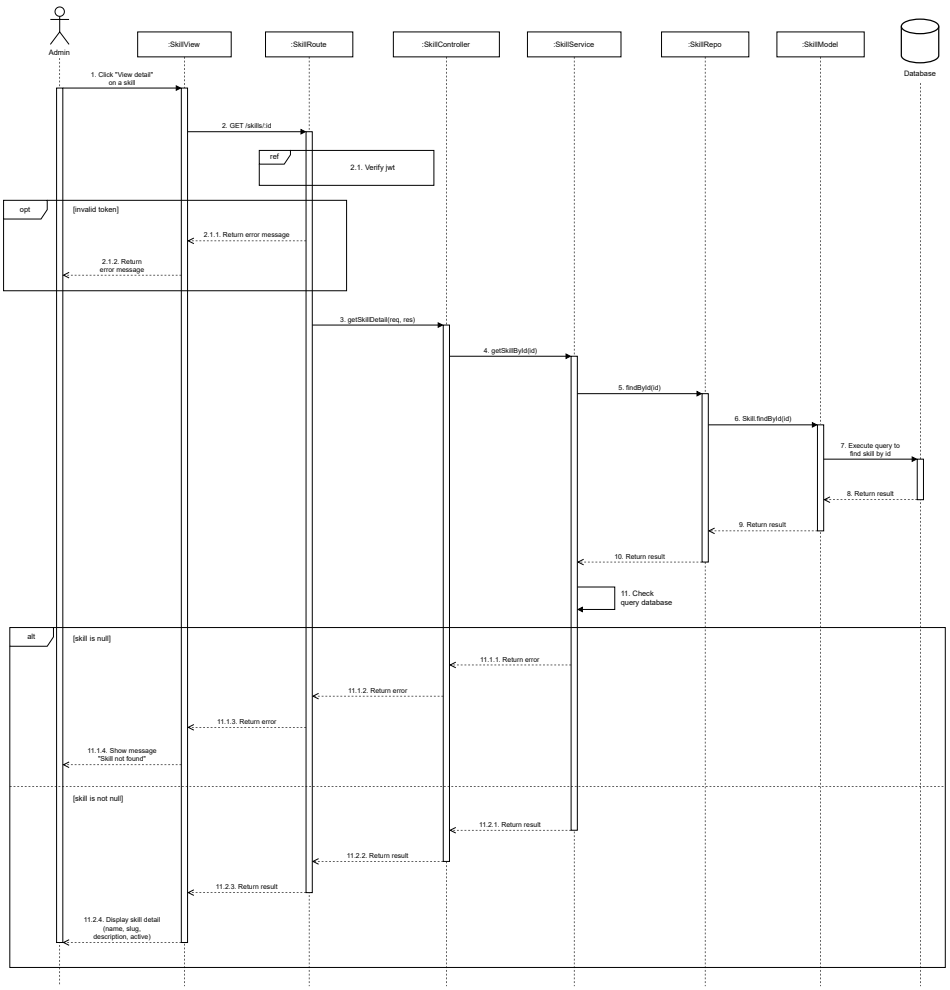

Figure 140: Sequence diagram of UC-38.02 View Skill Detail

3.19.3 UC-38.03 – Create Skill

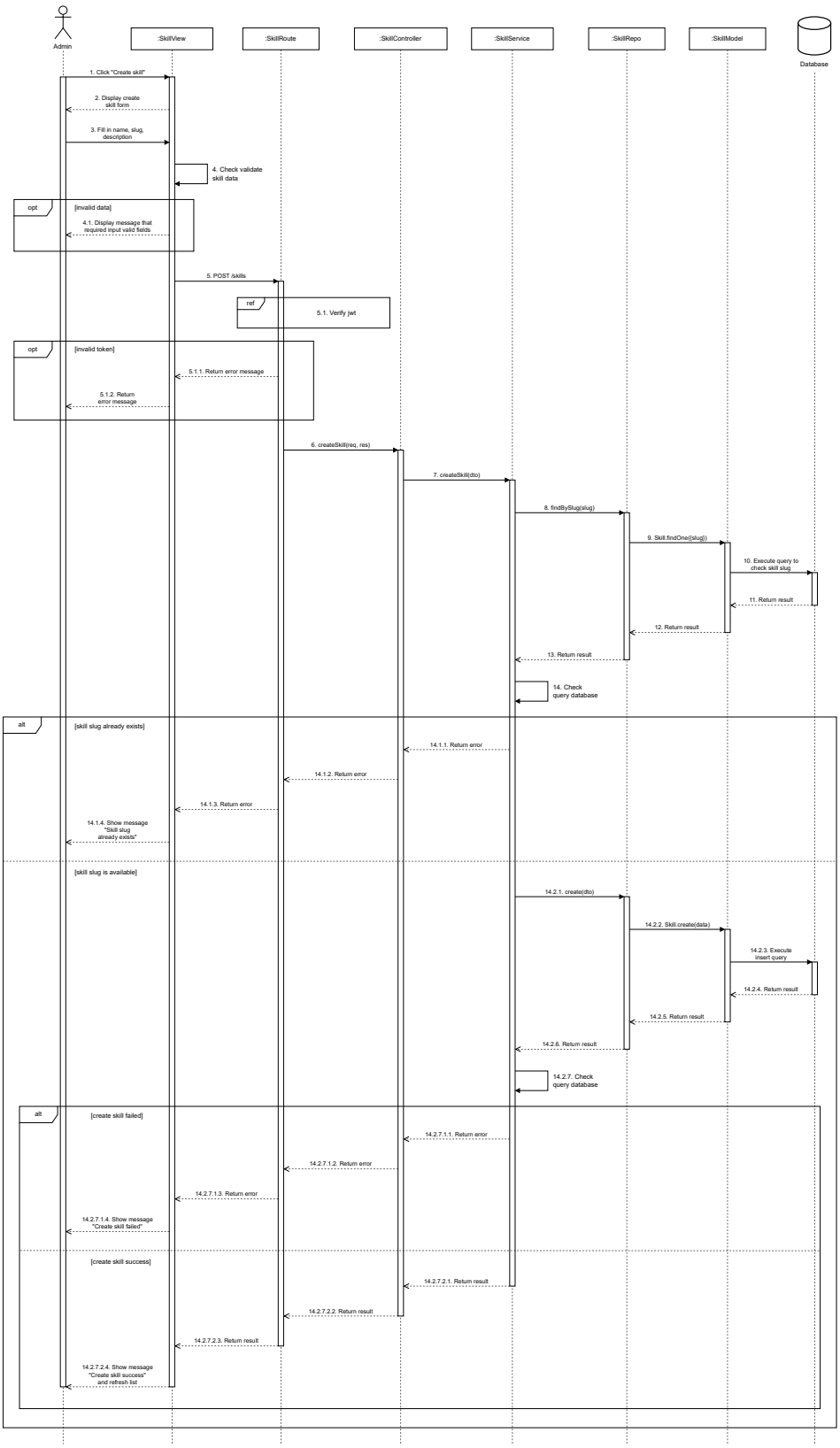

Figure 141: Sequence diagram of UC-38.03 Create Skill

3.19.4 UC-38.04 – Update Skill

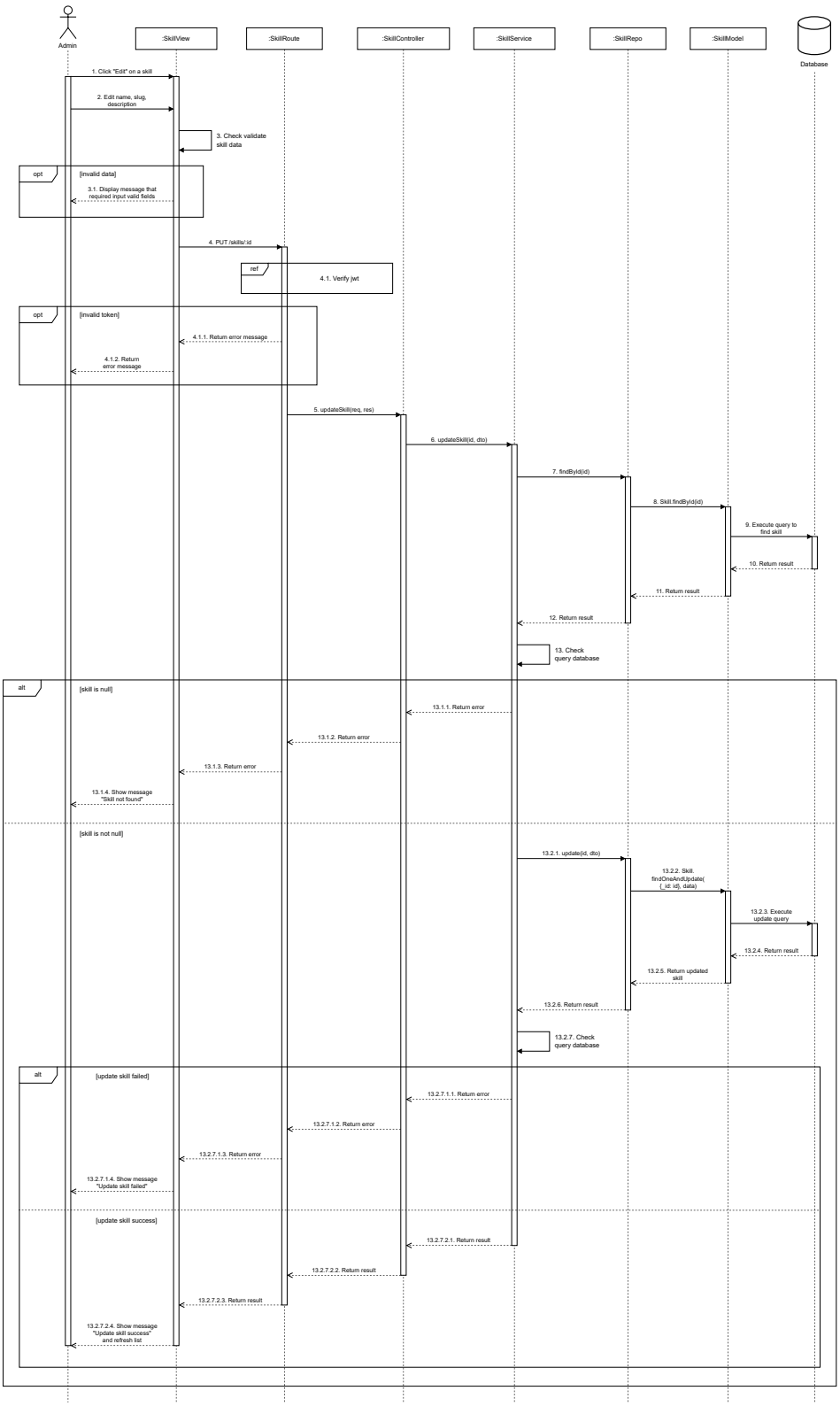

Figure 142: Sequence diagram of UC-38.04 Update Skill

3.19.5 UC-38.05 – Delete Skill

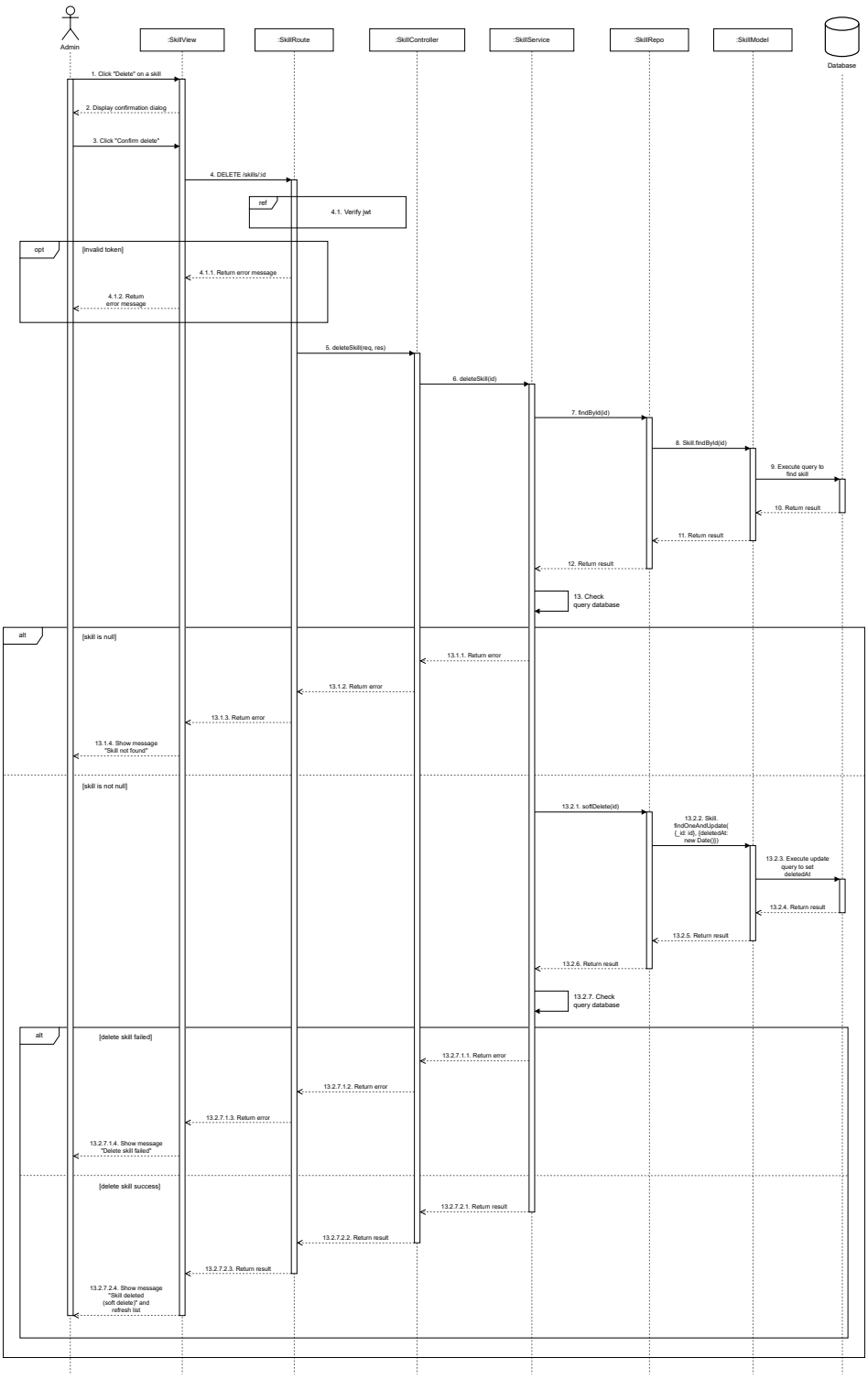

Figure 143: Sequence diagram of UC-38.05 Delete Skill

3.19.6 UC-38.06 – Restore Skill

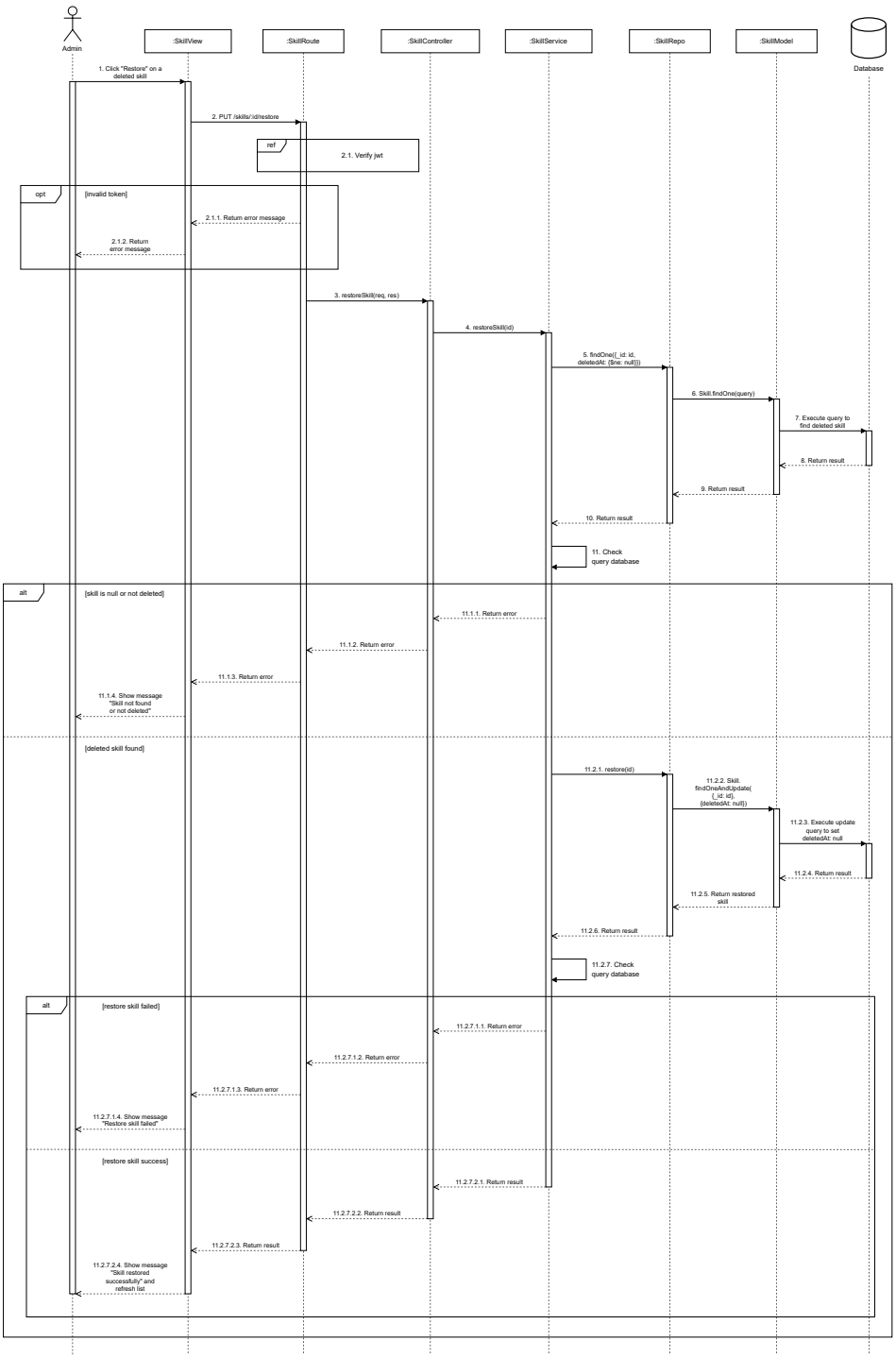

Figure 144: Sequence diagram of UC-38.06 Restore Skill

3.20 Expert Profile Management

The administrator-side Expert Profile editor updates an expert account directly. Any pending self-service revision is superseded, and the account owner is notified of the administra-

tive change.

a. Class Diagram

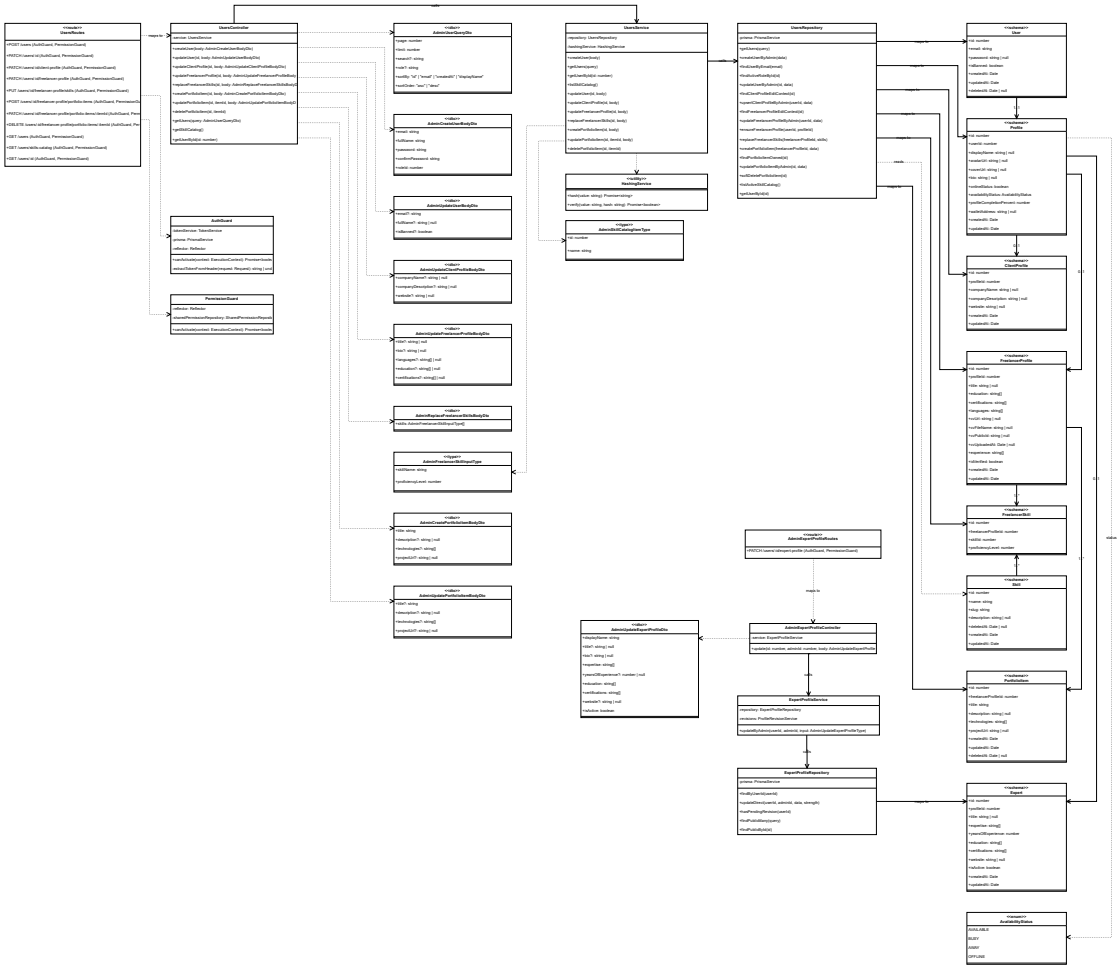

Figure 145: Class diagram of Expert Profile Management

b. Sequence Diagram(s)

3.20.1 UC-28.09 – Update Expert Profile

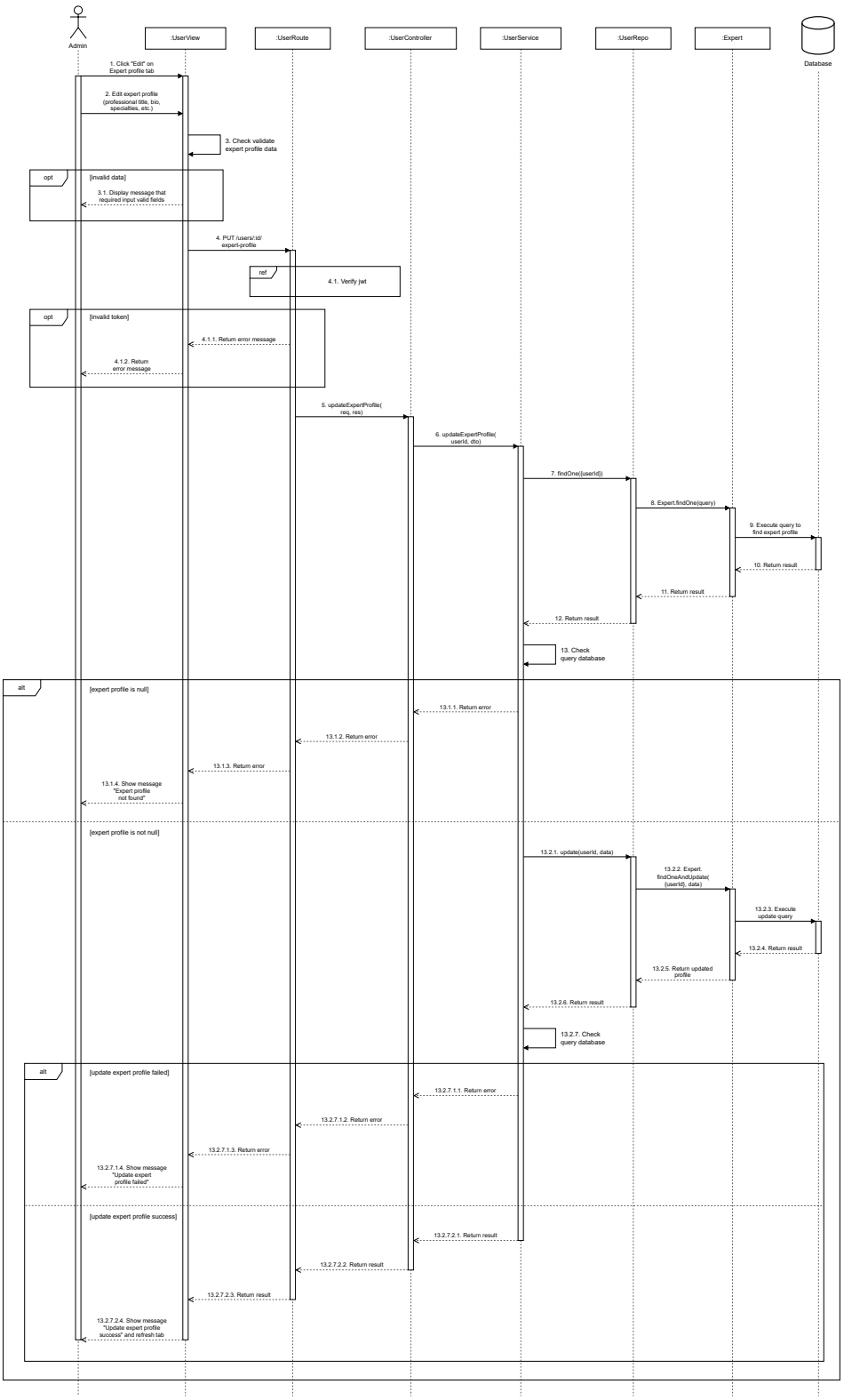

Figure 146: Sequence diagram of UC-28.09 Update Expert Profile

3.21 Job Category Management

Job Category Management maintains the taxonomy used by job postings. A category can be created, updated, soft-deleted, and restored; deletion is blocked while active jobs still reference the category.

a. Class Diagram

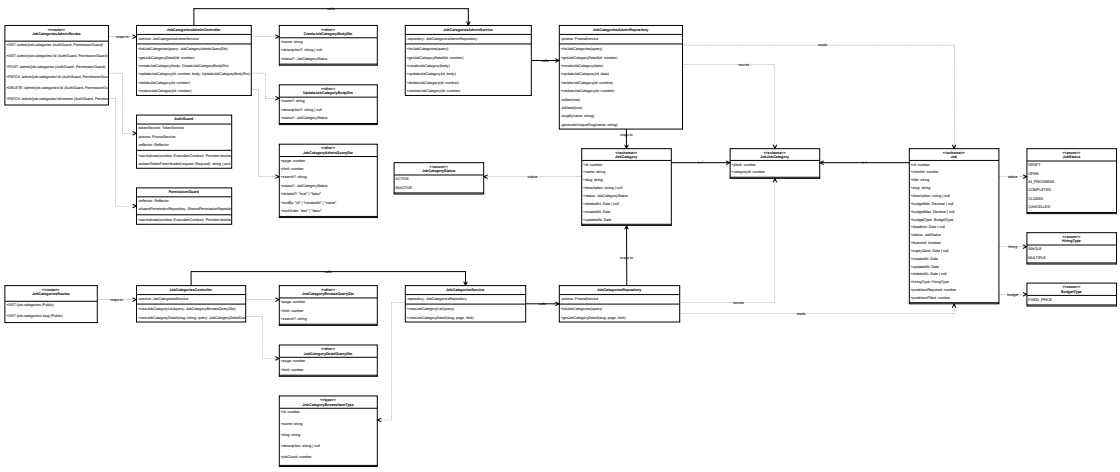

Figure 147: Class diagram of Job Category Management

b. Sequence Diagram(s)

3.21.1 UC-46.01 – View Job Category List

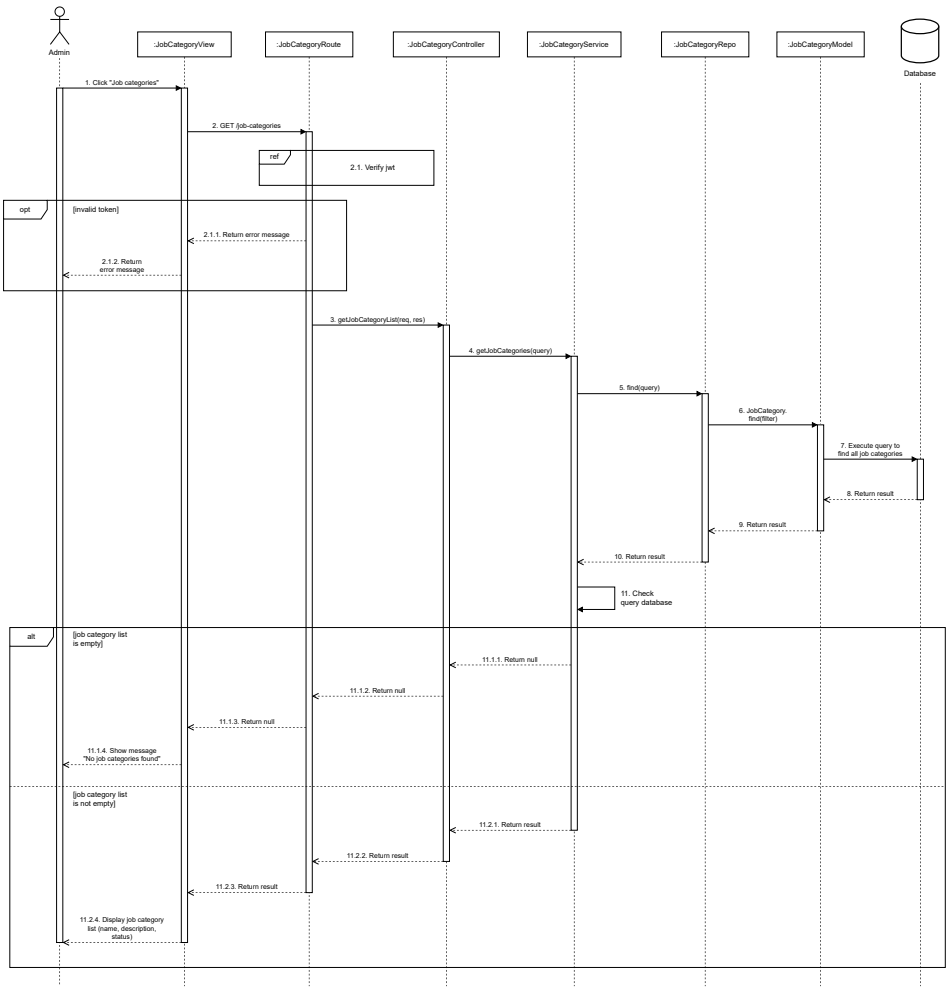

Figure 148: Sequence diagram of UC-46.01 View Job Category List

3.21.2 UC-46.02 – View Job Category Detail

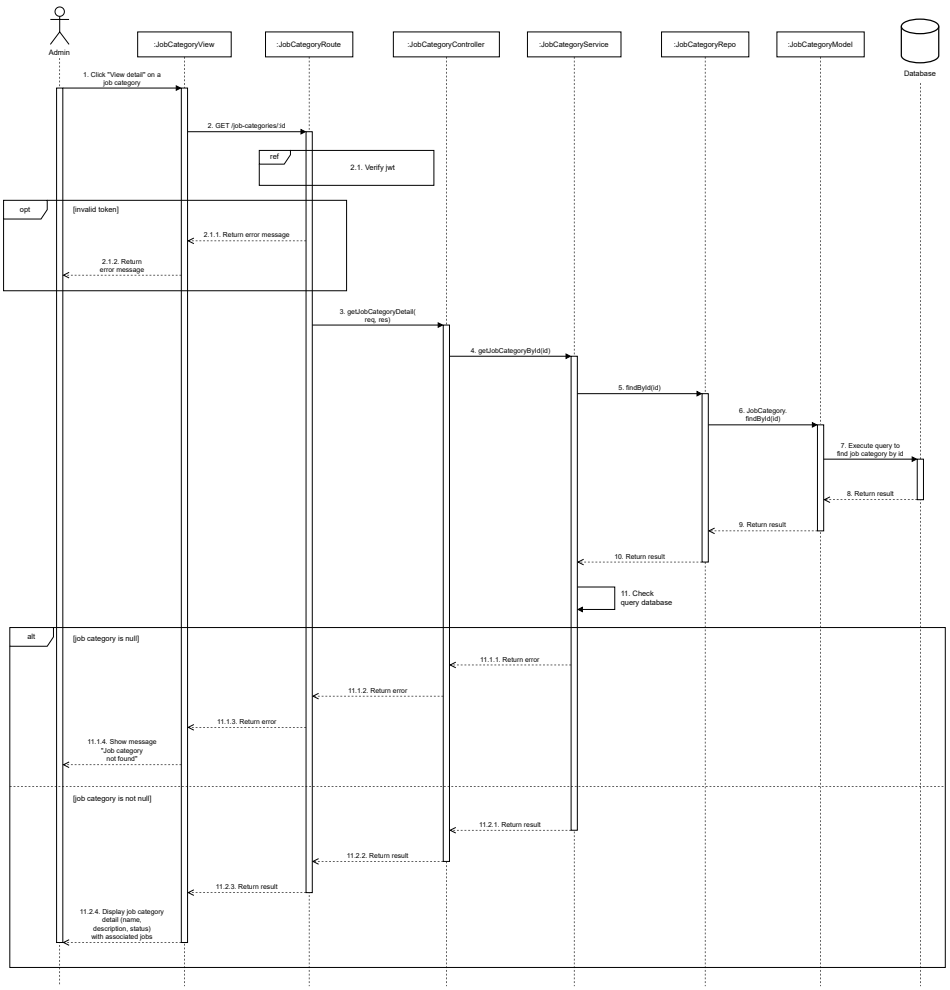

Figure 149: Sequence diagram of UC-46.02 View Job Category Detail

3.21.3 UC-46.03 – Create Job Category

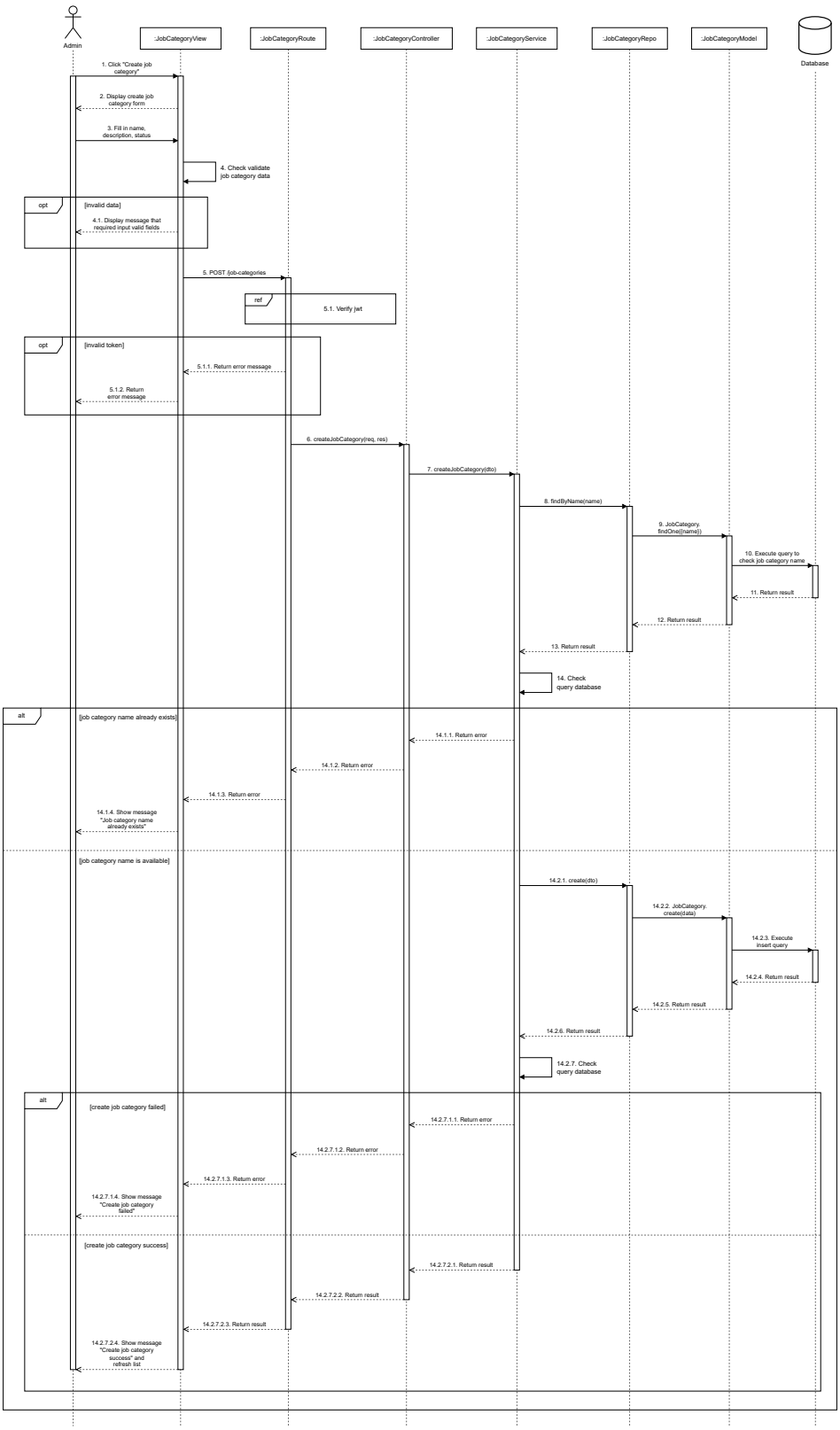

Figure 150: Sequence diagram of UC-46.03 Create Job Category

3.21.4 UC-46.04 – Update Job Category

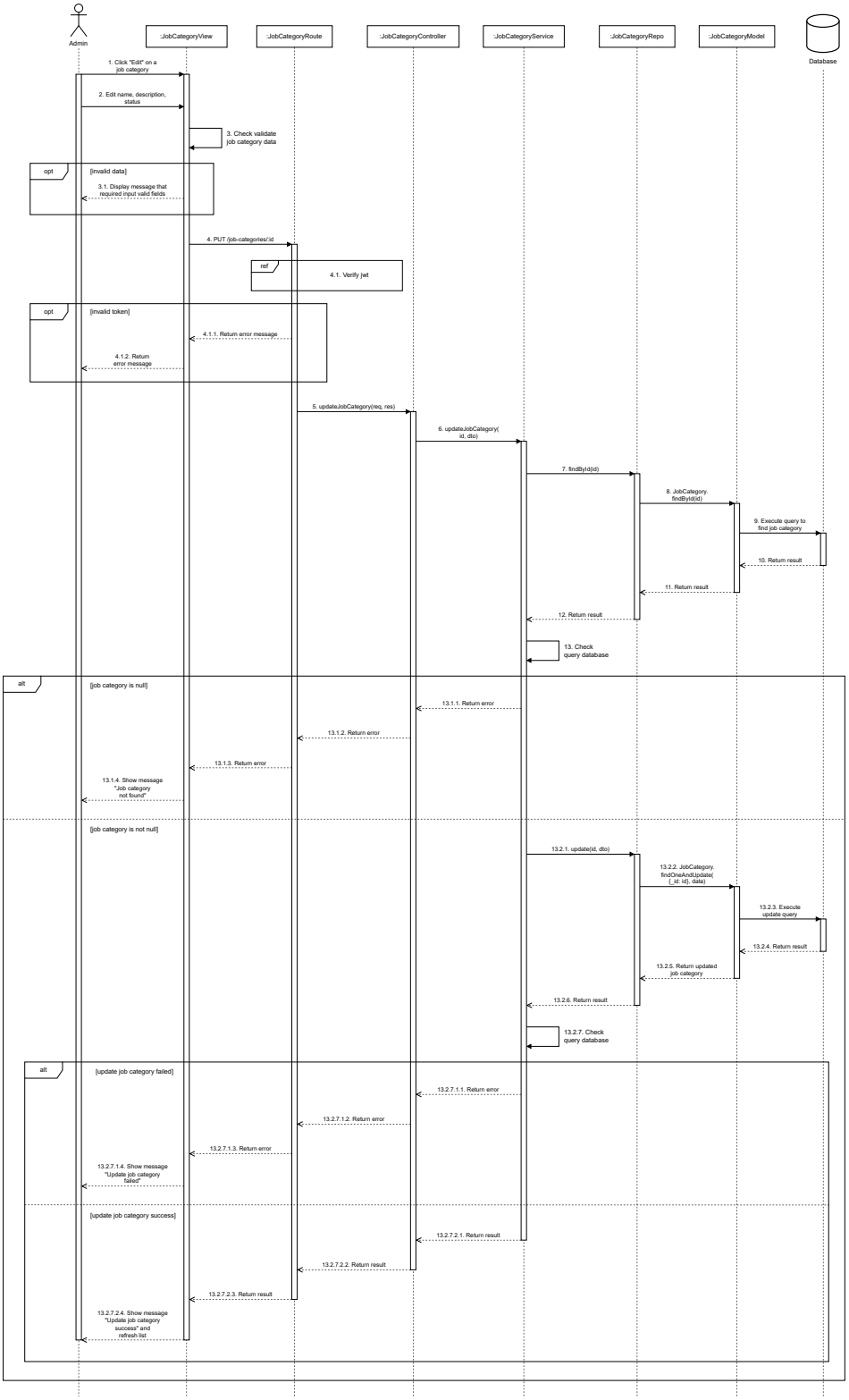

Figure 151: Sequence diagram of UC-46.04 Update Job Category

3.21.5 UC-46.05 – Delete Job Category

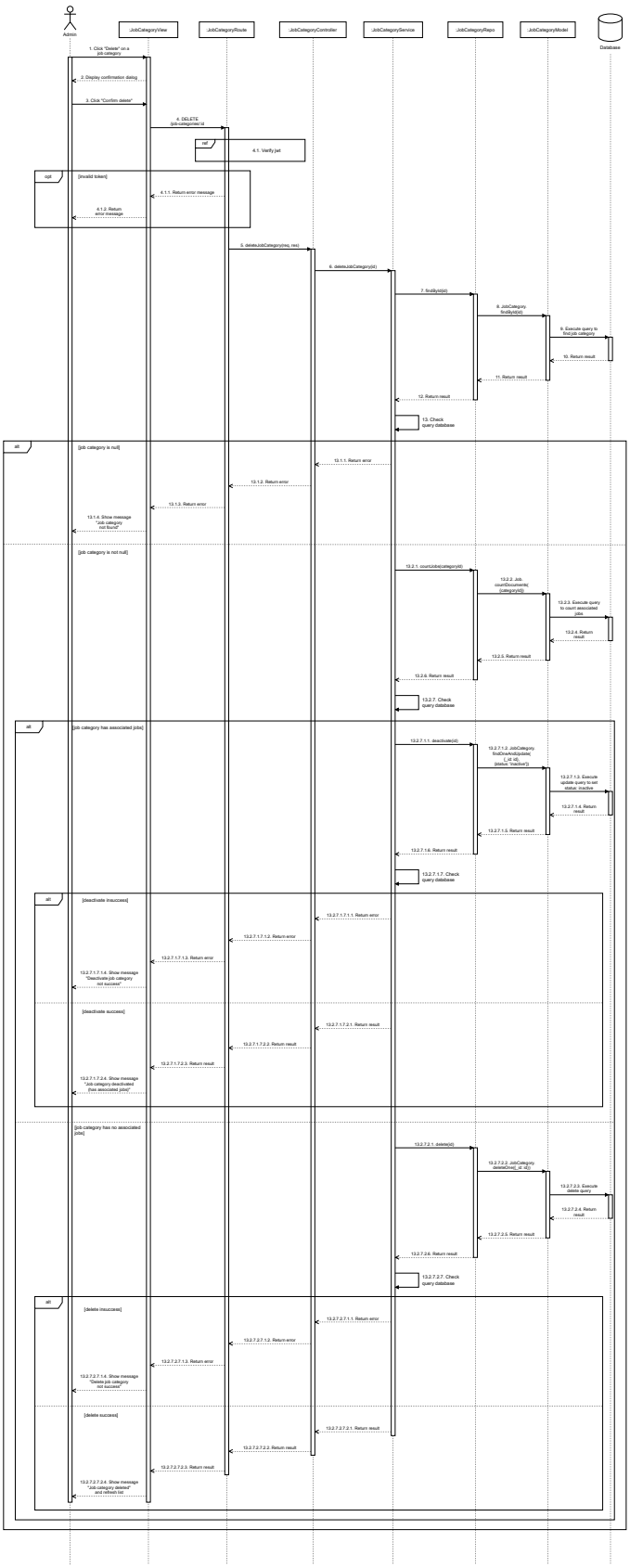

Figure 152: Sequence diagram of UC-46.05 Delete Job Category

4 Class Specification

4.1 ForumController Class

4.1.1 ClassMethod

No	Method	Description
01	`ForumController(forumService: ForumService)`	A method to initialize a new instance of the controller with the forum service. – `forumService: ForumService` – the service that owns category and post use cases. – *Returns:* `void` – constructor, no return value.
02	`getForumCategories()`	A method to retrieve all active forum categories. – *Returns:* `Promise<ForumCategoryListResponseType>` – the category list produced by the service. – Route: `GET /forums/categories`; public endpoint.
03	`getTopForumCategories({ query: { limit: number } })`	A method to retrieve the categories with the largest number of public posts. – `query.limit: number` – maximum number of categories, validated by `TopLimitSchema` with default 3 and range 1–10. – *Returns:* `Promise<ForumCategoryTopListResponseType>` – ranked category list. – Route: `GET /forums/categories/top`; public endpoint.
04	`getTopActiveUsers({ query: { limit: number } })`	A method to retrieve the users with the most forum activity. – `query.limit: number` – maximum number of users, validated by `TopLimitSchema` with default 3 and range 1–10. – *Returns:* `Promise<ForumTopActiveUserListResponseType>` – users with post and comment counts. – Route: `GET /forums/users/top`; public endpoint.

No	Method	Description
05	`getForumCategoryById(id: number)`	A method to retrieve one forum category by its identifier. – `id: number` – category identifier parsed by `ParseIntPipe`. – *Returns:* `Promise<ForumCategoryDetailResponseType>` – category detail with post count. – Route: `GET /forums/categories/:id`; public endpoint.
06	`getForumCategoryBySlug(slug: string)`	A method to retrieve one forum category by its slug. – `slug: string` – unique category slug. – *Returns:* `Promise<ForumCategoryDetailResponseType>` – category detail with post count. – Route: `GET /forums/categories/by-slug/:slug`; public endpoint.
07	`getForumPostLists(filter: ForumPostFilterType)`	A method to retrieve a paginated, filterable list of public posts. – `filter: ForumPostFilterType` – page, limit, optional categoryId, optional userId, and optional search text, validated by `ForumPostFilterDto`. – *Returns:* `Promise<ForumPostListResponseType>` – posts plus pagination metadata. – Route: `GET /forums/posts`; public endpoint.
08	`getTopInteractedPosts(query: { limit: number; categoryId?: number })`	A method to retrieve the most interacted posts from the previous seven days. – `query.limit: number` – maximum number of posts, default 3. – `query.categoryId: number | undefined` – optional category filter. – *Returns:* `Promise<ForumTopPostListResponseType>` – ranked posts with like and comment counts. – Route: `GET /forums/posts/top`; public endpoint.
09	`viewForumPostDetail(id: number)`	A method to retrieve the public detail of one post. – `id: number` – post identifier. – *Returns:* `Promise<ViewForumPostDetailResponseType>` – post, author summary, and category summary. – Route: `GET /forums/posts/:id`; public endpoint.

No	Method	Description
10	`getForumPostBySlug(slug: string)`	A method to retrieve the public detail of one post by slug. – `slug: string` – unique post slug. – *Returns:* `Promise<ViewForumPostDetailResponseType>` – post detail. – Route: `GET /forums/posts/by-slug/:slug`; public endpoint.
11	`createForumPost( userId: number, body: CreateForumPostType)`	A method to create a forum post for the authenticated user. – `userId: number` – author identifier injected by `@UserActive('userId')`. – `body: CreateForumPostType` – categoryId, title, content, and optional moderation result, validated by `CreateForumPostDto`. – *Returns:* `Promise<ForumPostType>` – the created post. – Route: `POST /forums/posts`; authenticated endpoint.
12	`deleteForumPost( userId: number, roleName: string, id: number)`	A method to soft-delete a post when the caller is its owner or an administrator. – `userId: number` – current user identifier. – `roleName: string` – current role name, used for the administrator override. – `id: number` – post identifier. – *Returns:* `Promise<ForumPostType>` – the soft-deleted post. – Route: `DELETE /forums/posts/:id`; authenticated endpoint.

4.1.2 ClassAttribute

No	Attribute	Type
01	`forumService`	`ForumService` – private readonly service dependency.

4.2 ForumService Class

4.2.1 ClassMethod

No	Method	Description
01	`ForumService(forumRepository: ForumRepository)`	A method to initialize the service with the forum repository. – `forumRepository: ForumRepository` – persistence dependency. – *Returns:* `void` – constructor, no return value.
02	`getForumCategories()`	A method to load all active categories. – *Returns:* `Promise<ForumCategoryListResponseType>` – category list with public post counts. – Failure is mapped to `FailedToLoadForumCategoriesException`.
03	`getTopForumCategories(limit: number = 3)`	A method to load the categories with the most public posts. – `limit: number` – maximum number of categories; default 3. – *Returns:* `Promise<ForumCategoryTopListResponseType>` – ranked categories. – Failure is mapped to `FailedToLoadForumCategoriesException`.
04	`getTopActiveUsers(limit: number = 5)`	A method to load the most active users by post and comment counts. – `limit: number` – maximum number of users; default 5. – *Returns:* `Promise<ForumTopActiveUserListResponseType>` – user activity summaries. – The current implementation reuses `FailedToLoadForumCategoriesException` for this failure path.
05	`getForumCategoryById(id: number)`	A method to load one category by identifier. – `id: number` – category identifier. – *Returns:* `Promise<ForumCategoryDetailResponseType>` – category detail. – Prisma known errors are mapped to `FailedToLoadForumCategoriesException`.
06	`getForumCategoryBySlug(slug: string)`	A method to resolve a category slug and return its detail. – `slug: string` – category slug. – *Returns:* `Promise<ForumCategoryDetailResponseType>` – category detail. – Throws `ForumCategoryNotFoundException` when the slug does not exist.

No	Method	Description
07	`getForumPostBySlug(slug: string)`	A method to resolve a post slug and return its public detail. – `slug: string` – post slug. – *Returns:* `Promise<ViewForumPostDetailResponseType>` – post detail. – Throws `ForumPostNotFoundException` when the slug does not exist.
08	`getForumPostLists(filter: ForumPostFilterType)`	A method to build a paginated post list from the repository result. – `filter: ForumPostFilterType` – page, limit, category, author, and search filters. – *Returns:* `Promise<ForumPostListResponseType>` – posts and pagination metadata, including `totalPages`. – Prisma known errors are mapped to `FailedToLoadForumPostsException`.
09	`createForumPost(userId: number, body: CreateForumPostType)`	A method to validate the optional category and create a post. – `userId: number` – author identifier. – `body: CreateForumPostType` – post data and optional AI moderation result. – *Returns:* `Promise<ForumPostType>` – the created post. – Throws `ForumCategoryNotFoundException` for an unknown category and `FailedToCreateForumPostException` for a persistence failure.
10	`viewForumPostDetail(id: number)`	A method to load the public detail of a post. – `id: number` – post identifier. – *Returns:* `Promise<ViewForumPostDetailResponseType>` – post, author, and category summary. – Prisma known errors are mapped to `FailedToViewForumPostException`.

No	Method	Description
11	`updateForumPost( id: number, userId: number, body: UpdateForumPostType)`	A method to update a post after checking existence and ownership. – `id: number` – post identifier. – `userId: number` – current user identifier. – `body: UpdateForumPostType` – updated category, title, and content. – *Returns:* `Promise<ForumPostType>` – the updated post. – Throws `ForumPostNotFoundException` or `ForumPostNotOwnedException`. The controller route is currently commented out.
12	`deleteForumPost( id: number, userId: number, roleName: string)`	A method to authorize and soft-delete a post. – `id: number` – post identifier. – `userId: number` – current user identifier. – `roleName: string` – current role name; `ADMIN` bypasses the owner check. – *Returns:* `Promise<ForumPostType>` – the soft-deleted post. – Throws `ForumPostNotFoundException`, `ForumPostNotOwnedException`, or `FailedToDeleteForumPostException`.
13	`getTopInteractedPosts( limit: number = 3, categoryId?: number)`	A method to return the most interacted posts from the last seven days. – `limit: number` – maximum number of posts; default 3. – `categoryId: number | undefined` – optional category restriction. – *Returns:* `Promise<ForumTopPostListResponseType>` – ranked posts. – Failure is mapped to `FailedToLoadForumPostsException`.

4.2.2 ClassAttribute

No	Attribute	Type
01	`forumRepository`	`ForumRepository` – private readonly persistence dependency.

4.3 ForumRepository Class

4.3.1 ClassMethod

No	Method	Description
01	`ForumRepository(prisma: PrismaService)`	A method to initialize the repository with the shared Prisma service. – `prisma`: `PrismaService` – pooled database client. – *Returns:* `void` – constructor, no return value.
02	`getForumCategories()`	A method to query active categories ordered by name. – *Returns:* `Promise<Array<Pick<ForumCategory Type, 'id' | 'name' | 'slug' | 'description' | 'createdAt' | 'updatedAt' | 'postCount'>>>` – category rows with public post counts. – Only non-deleted posts with moderation status `APPROVE D` are counted.
03	`getTopForumCategories( limit: number = 3)`	A method to query active categories ordered by public post count. – `limit`: `number` – maximum number of rows; default 3. – *Returns:* the same category projection as `getForum Categories()`, ranked by `posts._count` descending.
04	`getTopActiveUsers(limit: number = 5)`	A method to query non-deleted, non-banned users ordered by post count. – `limit`: `number` – maximum number of rows; default 5. – *Returns:* `Promise<ForumTopActiveUserType[]>` – user id, display name, avatar, post count, and comment count. – Comment counts include only non-deleted comments.
05	`findForumCategoryById(id: number)`	A method to check whether an active category exists. – `id`: `number` – category identifier. – *Returns:* `Promise<Pick<ForumCategoryType, 'id' > | null>` – the id when found, otherwise `null`.
06	`getForumCategoryById(id: number)`	A method to load one active category with its post count. – `id`: `number` – category identifier. – *Returns:* category projection with `postCount`. – Throws `ForumCategoryNotFoundException` when no active row exists.

No	Method	Description
07	`findForumCategoryBySlug( slug: string)`	A method to resolve an active category by slug. – `slug: string` – category slug. – *Returns:* `Promise<Pick<ForumCategoryType, 'id' > | null>` – the id when found, otherwise `null`.
08	`getForumPostLists(filter: ForumPostFilterType)`	A method to query a page of public posts and the matching total in one transaction. – `filter: ForumPostFilterType` – pagination, category, author, and case-insensitive title search. – *Returns:* `Promise<{ posts: ForumPostWithUser Type[]; total: number }>` – mapped post rows and total count. – Visibility requires `deletedAt = null` and moderation status `APPROVED`.
09	`createForumPost( categoryId: number | null, userId: number, title: string, content: string, moderation?: Forum ModerationType)`	A method to insert a post and generate its slug. – `categoryId: number | null` – optional category. – `userId: number` – author identifier. – `title, content: string` – post text. – `moderation: ForumModerationType | undefined` – optional AI result; status defaults to `PENDING`. – *Returns:* `Promise<ForumPostType>` – the inserted post.
10	`viewForumPostDetail(id: number)`	A method to query one public post with author and category summaries. – `id: number` – post identifier. – *Returns:* `Promise<ViewForumPostDetailResponse Type>` – post detail. – Throws `ForumPostNotFoundException` when the post is missing, deleted, or not approved.
11	`findForumPostById(id: number)`	A method to load a non-deleted post for ownership checks. – `id: number` – post identifier. – *Returns:* `Promise<ForumPostType | null>` – the post or `null`.
12	`findForumPostBySlug(slug: string)`	A method to resolve a public post slug. – `slug: string` – post slug. – *Returns:* `Promise<Pick<ForumPostType, 'id'> | null>` – the id when found, otherwise `null`.

No	Method	Description
13	`updateForumPost( id: number, data: UpdateForumPostType)`	A method to update a non-deleted post and regenerate its slug from the title. – `id: number` – post identifier. – `data: UpdateForumPostType` – category, title, and content. – *Returns:* `Promise<ForumPostType>` – the updated post.
14	`softDeleteForumPost(id: number)`	A method to set `deletedAt` on a non-deleted post. – `id: number` – post identifier. – *Returns:* `Promise<ForumPostType>` – the soft-deleted post.
15	`getTopInteractedPosts( limit: number = 3, categoryId?: number)`	A method to query posts created during the previous seven days and rank them by interactions. – `limit: number` – maximum number of rows; default 3. – `categoryId: number | undefined` – optional category restriction. – *Returns:* `Promise<ForumTopPostType[]>` – posts with like count, comment count, and `interactionScore`. – Ordering is by likes, then comments, then creation time, all descending.

4.3.2 ClassAttribute

No	Attribute	Type
01	`prisma`	`PrismaService` – private readonly database client.

Supporting DTO classes for category and post management

Class / Element	Attributes or Signature	Responsibility
`ForumCategoryList ResponseDto`	Array of `ForumCategoryType`: `id`, `name`, `slug`, `description`, `createdAt`, `updated At`, `postCount`.	Response of `GET / forums/categories`.

Class / Element	Attributes or Signature	Responsibility
ForumCategoryDetail ResponseDto	id: number, name: string, slug: string, description: string | null, createdAt: Date, updatedAt: Date, postCount: number.	Detail response for category id and slug routes.
ForumCategoryTopList ResponseDto	Array of ForumCategoryType.	Response of the top-category route.
ForumTopActiveUser ListResponseDto	Array of ForumTopActiveUserType: id, displayName, avatarUrl, postCount, commentCount.	Response of the top-active-user route.
ForumPostList ResponseDto	posts: ForumPostWithUserType[], pagination: PaginationType.	Paginated post-list response.
ForumPostFilterDto	page: number, limit: number, categoryId?: number, userId?: number, search?: string.	Query validation for the post list.
CreateForumPostDto	categoryId?: number, title: string, content: string, moderation?: Forum ModerationType.	Request body for post creation.
CreateForumPost ResponseDto	ForumPostType: id, categoryId, userId, title, slug, content, moderationStatus, moderationScore, moderation Categories, createdAt, updatedAt.	Response of post creation.
ViewForumPostDetail ResponseDto	ForumPostType fields plus category: { id, name, slug } | null and user: { id, profile }.	Public post-detail response.
UpdateForumPostDto	categoryId?: number, title: string, content: string.	Request body for the implemented update-post method.
UpdateForumPost ResponseDto	ForumPostType.	Response of the implemented update-post method.
ForumTopPostList ResponseDto	Array of ForumTopPostType: post fields plus likeCount, commentCount, interaction Score, user, and category.	Response of the top-post route.

4.4 ForumLikeController Class

4.4.1 ClassMethod

No	Method	Description
01	`ForumLikeController(forumLikeService: ForumLikeService)`	A method to initialize the controller with the post-like service. – `forumLikeService`: `ForumLikeService` – service dependency. – *Returns:* `void` – constructor, no return value.
02	`toggleLikePost(userId: number, id: number)`	A method to toggle the current user's like on a post. – `userId`: `number` – current user identifier. – `id`: `number` – post identifier. – *Returns:* `Promise<ToggleLikeResponseType>` – `liked: true` after a like, `liked: false` after an unlike. – Route: `POST /forums/posts/:id/like`; authenticated endpoint.
03	`viewForumLikeDetail(postId: number)`	A method to list the users who liked a post. – `postId`: `number` – post identifier. – *Returns:* `Promise<ForumLikeDetailResponseType>` – like rows with user summaries. – Route: `GET /forums/posts/:id/like`; public endpoint.

4.4.2 ClassAttribute

No	Attribute	Type
01	`forumLikeService`	`ForumLikeService` – private readonly service dependency.

4.5 ForumLikeService Class

4.5.1 ClassMethod

No	Method	Description
01	`ForumLikeService(forumLikeRepository: ForumLikeRepository)`	A method to initialize the service with the post-like repository. – `forumLikeRepository`: `ForumLikeRepository` – persistence dependency. – *Returns:* `void` – constructor, no return value.
02	`toggleLikePost(userId: number, postId: number)`	A method to decide whether a like must be created or removed. – `userId`: `number` – current user identifier. – `postId`: `number` – post identifier. – *Returns:* `Promise<ToggleLikeResponseType>` – the new like state. – Throws `ForumPostNotFoundException` when the post is missing; other failures become `FailedToLikeForumPostException`.
03	`viewForumLikeDetail(postId: number)`	A method to load the like rows of a post. – `postId`: `number` – post identifier. – *Returns:* `Promise<ForumLikeDetailResponseType>` – like rows with user summaries. – Failure is mapped to `FailedToViewForumLikeDetailException`.

4.5.2 ClassAttribute

No	Attribute	Type
01	`forumLikeRepository`	`ForumLikeRepository` – private readonly persistence dependency.

4.6 ForumLikeRepository Class

4.6.1 ClassMethod

No	Method	Description
01	`ForumLikeRepository( prisma: PrismaService)`	A method to initialize the repository with the shared Prisma service. – `prisma`: `PrismaService` – database client. – *Returns:* `void` – constructor, no return value.
02	`findPostById(postId: number)`	A method to check that a non-deleted post exists. – `postId`: `number` – post identifier. – *Returns:* `Promise<{ id: number } | null>` – the id when found, otherwise `null`.
03	`findLikeByUserAndPost( userId: number, postId: number)`	A method to find a user's reaction row for a post. – `userId`: `number` – user identifier. – `postId`: `number` – post identifier. – *Returns:* `Promise<{ id: number; commentId: number | null } | null>` – reaction id and optional comment link.
04	`findCommentOnlyLike(user Id: number)`	A method to find a reaction row that currently links only to a comment. – `userId`: `number` – user identifier. – *Returns:* `Promise<{ id: number } | null>` – the reusable reaction row, otherwise `null`.
05	`createPostLike(userId: number, postId: number)`	A method to create a post reaction, reusing a comment-only row when possible. – `userId`: `number` – user identifier. – `postId`: `number` – post identifier. – *Returns:* the created or updated `ForumLike` row.
06	`deletePostLike(userId: number, postId: number)`	A method to remove a post reaction while preserving a comment reaction on the same row. – `userId`: `number` – user identifier. – `postId`: `number` – post identifier. – *Returns:* the updated or deleted `ForumLike` row, or `undefined` when no row exists.
07	`viewForumLikeDetail(post Id: number)`	A method to query all reaction rows for a post. – `postId`: `number` – post identifier. – *Returns:* `Promise<ForumLikeDetailResponseType>` – reaction rows with user id, profile display name, and avatar.

4.6.2 ClassAttribute

No	Attribute	Type
01	`prisma`	`PrismaService` – private readonly database client.

Supporting DTO classes for post reactions

Class / Element	Attributes or Signature	Responsibility
`ToggleLikeResponseDto`	`liked: boolean`.	Response of the toggle-like route.
`ViewForumLikeDetail ResponseDto`	Array of `ForumLikeType` with `user`: `id`, `user Id`, `postId`, `commentId`, `reactionType`, `createdAt`, and user profile summary.	Response of the post-like detail route.

4.7 ForumCommentController Class

4.7.1 ClassMethod

No	Method	Description
01	`ForumCommentController( forumCommentService: ForumCommentService)`	A method to initialize the controller with the comment service. – `forumCommentService`: `ForumCommentService` – service dependency. – *Returns:* `void` – constructor, no return value.
02	`getForumCommentLists( postId: number, filter: ForumComment FilterType, userId: number)`	A method to list the comments of a post with pagination. – `postId`: `number` – parent post identifier. – `filter`: `ForumCommentFilterType` – page and limit. – `userId`: `number` – current user, used to compute `liked ByMe`. – *Returns:* `Promise<ForumCommentListResponse Type>` – comments and pagination. – Route: `GET /forums/posts/:postId/comments`; authenticated endpoint.

No	Method	Description
03	`createForumComment(userId: number, postId: number, body: CreateForumCommentType)`	A method to create a comment on a post. – `userId: number` – author identifier. – `postId: number` – parent post identifier. – `body: CreateForumCommentType` – comment content. – *Returns:* `Promise<ForumCommentType>` – the created comment. – Route: `POST /forums/posts/:postId/comments`; authenticated endpoint.
04	`updateForumComment(userId: number, postId: number, id: number, body: EditForumCommentType)`	A method to edit a comment owned by the current user. – `userId: number` – current user identifier. – `postId: number` – parent post identifier. – `id: number` – comment identifier. – `body: EditForumCommentType` – new content. – *Returns:* `Promise<ForumCommentType>` – the updated comment. – Route: `PATCH /forums/posts/:postId/comments/:id`; authenticated endpoint.
05	`deleteForumComment(userId: number, roleName: string, postId: number, id: number)`	A method to soft-delete a comment owned by the user or by an administrator. – `userId: number` – current user identifier. – `roleName: string` – current role name. – `postId: number` – parent post identifier. – `id: number` – comment identifier. – *Returns:* `Promise<ForumCommentType>` – the soft-deleted comment. – Route: `DELETE /forums/posts/:postId/comments/:id`; authenticated endpoint.
06	`toggleLikeComment(userId: number, postId: number, commentId: number)`	A method to toggle the current user's like on a comment. – `userId: number` – current user identifier. – `postId: number` – parent post identifier. – `commentId: number` – comment identifier. – *Returns:* `Promise<ToggleLikeCommentType>` – the new like state. – Route: `POST /forums/posts/:postId/comments/:commentId/like`; authenticated endpoint.

4.7.2 ClassAttribute

No	Attribute	Type
01	`forumCommentService`	`ForumCommentService` – private readonly service dependency.

4.8 ForumCommentService Class

4.8.1 ClassMethod

No	Method	Description
01	`ForumCommentService( forumCommentRepository: ForumCommentRepository)`	A method to initialize the service with the comment repository. – `forumCommentRepository: ForumComment Repository` – persistence dependency. – *Returns:* `void` – constructor, no return value.
02	`createForumComment( postId: number, userId: number, body: CreateForumComment Type)`	A method to verify the parent post and create a comment. – `postId: number` – parent post identifier. – `userId: number` – author identifier. – `body: CreateForumCommentType` – comment content. – *Returns:* `Promise<ForumCommentType>` – the created comment. – Throws `ForumPostNotFoundException` or `FailedTo CreateForumCommentException`.
03	`updateForumComment( postId: number, id: number, userId: number, body: EditForumCommentType)`	A method to verify comment existence, parent post, and ownership before editing. – `postId: number` – parent post identifier. – `id: number` – comment identifier. – `userId: number` – current user identifier. – `body: EditForumCommentType` – new content. – *Returns:* `Promise<ForumCommentType>` – the updated comment. – Throws `ForumCommentNotFoundException` or `Forum CommentNotOwnedException`.

No	Method	Description
04	deleteForumComment(postId: number, id: number, userId: number, roleName: string)	A method to authorize and soft-delete a comment. – postId: number – parent post identifier. – id: number – comment identifier. – userId: number – current user identifier. – roleName: string – ADMIN bypasses the owner check. – *Returns:* Promise<ForumCommentType> – the soft-deleted comment. – Throws ForumCommentNotFoundException, ForumCommentNotOwnedException, or FailedToDeleteForumCommentException.
05	toggleLikeComment(userId: number, postId: number, commentId: number)	A method to create or remove a comment reaction. – userId: number – current user identifier. – postId: number – parent post identifier. – commentId: number – comment identifier. – *Returns:* Promise<ToggleLikeCommentType> – the new like state. – Throws ForumCommentNotFoundException or FailedToLikeForumCommentException.
06	getForumCommentLists(filter: ForumCommentFilterType)	A method to build a paginated comment list. – filter: ForumCommentFilterType – page, limit, postId, and optional userId. – *Returns:* Promise<ForumCommentListResponseType> – comments and pagination metadata. – Prisma known errors are mapped to FailedToLoadForumCommentsException.

4.8.2 ClassAttribute

No	Attribute	Type
01	forumCommentRepository	ForumCommentRepository – private readonly persistence dependency.

4.9 ForumCommentRepository Class

4.9.1 ClassMethod

No	Method	Description
01	`ForumCommentRepository( prisma: PrismaService)`	A method to initialize the repository with the shared Prisma service. – `prisma`: `PrismaService` – database client. – *Returns:* `void` – constructor, no return value.
02	`findPostById(postId: number)`	A method to check that a non-deleted post exists. – `postId`: `number` – post identifier. – *Returns:* `Promise<{ id: number } | null>` – the id when found, otherwise `null`.
03	`findCommentById(id: number)`	A method to load a non-deleted comment with its parent post and author summary. – `id`: `number` – comment identifier. – *Returns:* the comment projection or `null`.
04	`createForumComment( postId: number, userId: number, content: string)`	A method to insert a comment and return the response shape. – `postId`: `number` – parent post identifier. – `userId`: `number` – author identifier. – `content`: `string` – comment text. – *Returns:* `Promise<ForumCommentType>` – the comment with `likeCount = 0` and `likedByMe = false`.
05	`updateForumComment( id: number, data: EditForumCommentType)`	A method to update the content of a non-deleted comment. – `id`: `number` – comment identifier. – `data`: `EditForumCommentType` – new content. – *Returns:* `Promise<ForumCommentType>` – the updated comment.
06	`softDeleteForumComment( id: number)`	A method to set `deletedAt` on a non-deleted comment. – `id`: `number` – comment identifier. – *Returns:* `Promise<ForumCommentType>` – the soft-deleted comment.
07	`getForumCommentLists( filter: ForumCommentFilterType)`	A method to query a page of comments and the matching total in one transaction. – `filter`: `ForumCommentFilterType` – page, limit, postId, and optional current user. – *Returns:* `Promise<{ comments: ForumCommentType[]; total: number }>` – mapped comments and total. – `likedByMe` is computed from the optional user reaction.

No	Method	Description
08	`findLikeByUserAndComment( userId: number, commentId: number)`	A method to find a user's comment reaction. – `userId: number` – user identifier. – `commentId: number` – comment identifier. – *Returns:* `Promise<{ id: number; postId: number | null } | null>` – reaction id and optional post link.
09	`findLikeByUserAndPost( userId: number, postId: number)`	A method to find a user's post reaction for reuse by the comment-like flow. – `userId: number` – user identifier. – `postId: number` – post identifier. – *Returns:* `Promise<{ id: number; commentId: number | null } | null>` – reaction id and optional comment link.
10	`createCommentLike( userId: number, postId: number, commentId: number)`	A method to create a comment reaction, reusing a post-only row when possible. – `userId: number` – user identifier. – `postId: number` – parent post identifier. – `commentId: number` – comment identifier. – *Returns:* the created or updated `ForumLike` row.
11	`deleteCommentLike( userId: number, commentId: number)`	A method to remove a comment reaction while preserving a post reaction on the same row. – `userId: number` – user identifier. – `commentId: number` – comment identifier. – *Returns:* the updated or deleted `ForumLike` row, or `undefined` when no row exists.

4.9.2 ClassAttribute

No	Attribute	Type
01	`prisma`	`PrismaService` – private readonly database client.

Supporting DTO classes for comments

Class / Element	Attributes or Signature	Responsibility
ForumCommentList ResponseDto	comments: ForumCommentType[], pagination: PaginationType.	Paginated comment-list response.
ForumCommentFilterDto	page: number, limit: number.	Query validation for comment listing.
CreateForumCommentDto	content: string.	Comment-creation body.
CreateForumComment ResponseDto	ForumCommentType: id, postId, user, content, createdAt, updatedAt, like Count, likedByMe.	Comment-creation response.
EditForumCommentDto	content: string.	Comment-update body.
EditForumComment ResponseDto	ForumCommentType.	Comment-update response.
DeleteForumComment ResponseDto	ForumCommentType.	Soft-delete response.
ToggleLikeComment ResponseDto	liked: boolean.	Comment-like toggle response.

4.10 ForumReportController Class

4.10.1 ClassMethod

No	Method	Description
01	ForumReportController(forumReportService: ForumReportService)	A method to initialize the controller with the report service. – forumReportService: ForumReportService – service dependency. – *Returns:* void – constructor, no return value.

No	Method	Description
02	`createPostReport( userId: number, postId: number, body: CreateForumReportType)`	A method to report a post. – `userId: number` – reporter identifier. – `postId: number` – reported post identifier. – `body: CreateForumReportType` – report reason. – *Returns:* `Promise<CreateForumReportResponseType>` – the created report. – Route: `POST /forums/posts/:postId/reports`; authenticated endpoint.
03	`createCommentReport( userId: number, postId: number, commentId: number, body: CreateForumReportType)`	A method to report a comment. – `userId: number` – reporter identifier. – `postId: number` – parent post identifier. – `commentId: number` – reported comment identifier. – `body: CreateForumReportType` – report reason. – *Returns:* `Promise<CreateForumReportResponseType>` – the created report. – Route: `POST /forums/posts/:postId/comments/:commentId/reports`; authenticated endpoint.
04	`checkPostReported( userId: number, postId: number)`	A method to check whether the current user has reported a post. – `userId: number` – current user identifier. – `postId: number` – post identifier. – *Returns:* `Promise<{ reported: boolean }>` – true when a matching report exists. – Route: `GET /forums/posts/:postId/is-reported`; authenticated endpoint.
05	`checkCommentReported( userId: number, postId: number, commentId: number)`	A method to check whether the current user has reported a comment. – `userId: number` – current user identifier. – `postId: number` – parent post identifier. – `commentId: number` – comment identifier. – *Returns:* `Promise<{ reported: boolean }>` – true when a matching report exists. – Route: `GET /forums/posts/:postId/comments/:commentId/is-reported`; authenticated endpoint.

No	Method	Description
06	`getForumReportLists( userId: number, roleName: string, page: number, limit: number, status: string, search: string)`	A method to list the report queue for administrators. – `userId: number` – current user identifier. – `roleName: string` – role checked by the service. – `page, limit: number` – pagination, defaulted to 1 and 10. – `status, search: string` – optional status and reason filters. – *Returns:* `Promise<ForumReportListResponseType>` – reports and pagination. – Route: `GET /forums/reports`; administrator endpoint.
07	`updateReportStatus( userId: number, roleName: string, reportId: number, body: UpdateReportStatusType)`	A method to change the status of a report. – `userId: number` – administrator identifier recorded on the report. – `roleName: string` – role checked by the service. – `reportId: number` – report identifier. – `body: UpdateReportStatusType` – target status. – *Returns:* `Promise<UpdateReportStatusResponseType>` – the updated report. – Route: `PATCH /forums/reports/:reportId/status`; administrator endpoint.

4.10.2 ClassAttribute

No	Attribute	Type
01	`forumReportService`	`ForumReportService` – private readonly service dependency.

4.11 ForumReportService Class

4.11.1 ClassMethod

No	Method	Description
01	`ForumReportService( forumReportRepository: ForumReportRepository)`	A method to initialize the service with the report repository. – `forumReportRepository: ForumReportRepository` – persistence dependency. – *Returns:* `void` – constructor, no return value.
02	`createPostReport( reporterId: number, postId: number, reason: string)`	A method to validate a post and create its first report by the user. – `reporterId: number` – reporter identifier. – `postId: number` – reported post identifier. – `reason: string` – report reason. – *Returns:* `Promise<CreateForumReportResponseType>` – the created report. – Throws `ForumPostNotFoundException`, `ForumReportAlreadyExistsException`, or `FailedToCreateForumReportException`.
03	`createCommentReport( reporterId: number, postId: number, commentId: number, reason: string)`	A method to validate a comment and create its first report by the user. – `reporterId: number` – reporter identifier. – `postId: number` – parent post identifier. – `commentId: number` – reported comment identifier. – `reason: string` – report reason. – *Returns:* `Promise<CreateForumReportResponseType>` – the created report. – Throws `ForumCommentNotFoundException`, `ForumReportAlreadyExistsException`, or `FailedToCreateForumReportException`.
04	`checkReportedByUser( reporterId: number, postId: number | null, commentId: number | null)`	A method to find an existing report for the current user and target. – `reporterId: number` – reporter identifier. – `postId: number | null` – post target, or `null`. – `commentId: number | null` – comment target, or `null`. – *Returns:* `Promise<{ id: number } | null>` – the report id when one exists.

No	Method	Description
05	`getForumReportLists( roleName: string, page: number, limit: number, status?: string, search?: string)`	A method to authorize an administrator and return the report queue. – `roleName: string` – must equal `ADMIN`. – `page, limit: number` – pagination. – `status, search: string | undefined` – optional filters. – *Returns:* `Promise<ForumReportListResponseType>` – reports and pagination. – Throws `ForumReportForbiddenException` or `FailedToLoadForumReportsException`.
06	`updateReportStatus( reportId: number, status: string, adminId: number, roleName: string)`	A method to authorize an administrator and update a post report status. – `reportId: number` – report identifier. – `status: string` – target status. – `adminId: number` – administrator identifier. – `roleName: string` – must equal `ADMIN`. – *Returns:* `Promise<UpdateReportStatusResponseType>` – the updated report. – Throws `ForumReportForbiddenException`, `ForumReportNotFoundException`, or `FailedToUpdateReportStatusException`.

4.11.2 ClassAttribute

No	Attribute	Type
01	`forumReportRepository`	`ForumReportRepository` – private readonly persistence dependency.

4.12 ForumReportRepository Class

4.12.1 ClassMethod

No	Method	Description
01	`ForumReportRepository( prisma: PrismaService)`	A method to initialize the repository with the shared Prisma service. – `prisma`: `PrismaService` – database client. – *Returns:* `void` – constructor, no return value.
02	`findPostById(postId: number)`	A method to check that a non-deleted post exists. – `postId`: `number` – post identifier. – *Returns:* `Promise<{ id: number } | null>` – the id when found, otherwise `null`.
03	`findCommentById(commentId: number)`	A method to check that a non-deleted comment exists and belongs to a post. – `commentId`: `number` – comment identifier. – *Returns:* `Promise<{ id: number; postId: number } | null>` – comment id and parent post id.
04	`findExistingReport( reporterId: number, postId: number | null, commentId: number | null)`	A method to find a duplicate report by the same user and target. – `reporterId`: `number` – reporter identifier. – `postId`: `number | null` – post target, or `null`. – `commentId`: `number | null` – comment target, or `null`. – *Returns:* `Promise<{ id: number } | null>` – the report id when one exists.
05	`createReport( reporterId: number, postId: number | null, commentId: number | null, reason: string)`	A method to insert a report. – `reporterId`: `number` – reporter identifier. – `postId`, `commentId`: `number | null` – exactly one target is populated by the service. – `reason`: `string` – report reason. – *Returns:* `Promise<ForumReportType>` – the created report.
06	`findReportById(id: number)`	A method to load a report by identifier. – `id`: `number` – report identifier. – *Returns:* `Promise<ForumReportType | null>` – the report or `null`.

No	Method	Description
07	`getReportList( page: number, limit: number, status?: string, search?: string)`	A method to query a page of reports and the matching total in one transaction. – `page`, `limit`: `number` – pagination. – `status`: `string | undefined` – optional status filter. – `search`: `string | undefined` – optional case-insensitive reason search. – *Returns:* `Promise<{ reports: ForumReportListResponseType['reports']; total: number }>` – report rows and total.
08	`updateReportStatus( id: number, status: string, adminId: number | null)`	A method to update a report status and record the reviewing administrator. – `id`: `number` – report identifier. – `status`: `string` – target status, converted to upper case. – `adminId`: `number | null` – reviewing administrator. – *Returns:* `Promise<ForumReportType>` – the updated report.

4.12.2 ClassAttribute

No	Attribute	Type
01	`prisma`	`PrismaService` – private readonly database client.

Supporting DTO classes for reports

Class / Element	Attributes or Signature	Responsibility
`CreateForumReportDto`	`reason: string.`	Report-creation body.
`CreateForumReportResponseDto`	`ForumReportType`: `id`, `reporterId`, `postId`, `commentId`, `reason`, `status`, `adminId`, `createdAt`.	Report-creation response.
`UpdateReportStatusDto`	`status: PENDING | REVIEWED | RESOLVED | DISMISSED.`	Report-status body.
`UpdateReportStatusResponseDto`	`ForumReportType`.	Report-status response.

Class / Element	Attributes or Signature	Responsibility
ForumReportList ResponseDto	reports: ForumReportType[] with reporter, post, and comment summaries, plus pagination.	Administrator report queue.

4.13 ContractController Class

4.13.1 ClassMethod

No	Method	Description
01	ContractController(contractService: ContractService)	A method to initialize the controller with the contract service dependency. – contractService: ContractService – service that encapsulates contract lifecycle use cases. – *Returns:* void – constructor, no return value.
02	getContractList(userId: number, role Name: string, query: GetContractList QueryDTO)	A method to retrieve a paginated and filterable list of contracts for the current user. – userId: number – caller user identifier injected by @ UserActive('userId'). – roleName: string – active role of the caller for role-scoped visibility. – query: GetContractListQueryDTO – search keyword, status, date bounds, page, and limit. – *Returns:* Promise<GetContractListResponseDTO> – list of contracts and pagination metadata. – Route: GET /contracts; authenticated endpoint.
03	getContractDetail(userId: number, role Name: string, contractId: number)	A method to fetch comprehensive contract details including participants, job info, and financial metrics. – userId: number – caller user identifier. – roleName: string – caller role name. – contractId: number – contract identifier parsed by ParseIntPipe. – *Returns:* Promise<GetContractDetailResponseDTO> – complete contract details. – Route: GET /contracts/:id; authenticated endpoint.

No	Method	Description
04	createContract(userId: number, body: CreateContractBodyDTO)	A method to create a new contract from an accepted (hired) proposal. – userId: number – client identifier injected by @UserActive('userId'). – body: CreateContractBodyDTO – proposalId, contract title, description, and terms. – *Returns:* Promise<CreateContractResponseDTO> – the newly created contract record. – Route: POST /contracts; authenticated endpoint.
05	updateContract(userId: number, contractId: number, body: UpdateContractBodyDTO)	A method to update terms of an unfinalized contract prior to mutual signing. – userId: number – client identifier. – contractId: number – contract identifier. – body: UpdateContractBodyDTO – revised title, description, budget, or deadline. – *Returns:* Promise<UpdateContractResponseDTO> – the updated contract record. – Route: PATCH /contracts/:id; authenticated endpoint.
06	signContract(userId: number, contractId: number)	A method to record a formal digital signature from the client or freelancer on the contract. – userId: number – user identifier signing the contract. – contractId: number – target contract identifier. – *Returns:* Promise<SignContractResponseDTO> – updated signing flags and active status. – Route: PATCH /contracts/:id/sign; authenticated endpoint.
07	completeContract(userId: number, contractId: number)	A method for the client to mark an active contract as successfully completed once all deliverables are accepted. – userId: number – client identifier. – contractId: number – contract identifier. – *Returns:* Promise<CompleteContractResponseDTO> – contract marked as completed. – Route: PATCH /contracts/:id/complete; authenticated endpoint.

No	Method	Description
08	`cancelContract( userId: number, contractId: number)`	A method to terminate a contract when mutual agreement or project circumstances require cancellation. – `userId: number` – caller user identifier. – `contractId: number` – contract identifier. – *Returns:* `Promise<CancelContractResponseDTO>` – cancelled contract result. – Route: `PATCH /contracts/:id/cancel`; authenticated endpoint.

4.13.2 ClassAttribute

No	Attribute	Type
01	`contractService`	`ContractService` – private readonly service dependency.

4.14 ContractService Class

4.14.1 ClassMethod

No	Method	Description
01	`ContractService( contractRepository: ContractRepository)`	A method to initialize the service with the contract repository dependency. – `contractRepository: ContractRepository` – persistence access layer for contracts. – *Returns:* `void` – constructor, no return value.
02	`createContract( clientId: number, body: CreateContractBodyType)`	A method to validate proposal status, verify client ownership, ensure no duplicate contract exists, and persist the contract. – `clientId: number` – client identifier. – `body: CreateContractBodyType` – proposalId and contract details. – *Returns:* `Promise<ContractType>` – created contract.

No	Method	Description
03	`updateContract( clientId: number, contractId: number, body: UpdateContractBody Type)`	A method to verify contract ownership, ensure status is PENDING_SIGN, and update contract fields. – `clientId: number` – client identifier. – `contractId: number` – contract identifier. – `body: UpdateContractBodyType` – fields to update. – *Returns:* `Promise<ContractType>` – updated contract.
04	`signContract( userId: number, contract Id: number)`	A method to validate participant role, record signature, and activate the contract once both parties sign. – `userId: number` – signing participant identifier. – `contractId: number` – contract identifier. – *Returns:* `Promise<ContractType>` – signed contract.
05	`completeContract( clientId: number, contractId: number)`	A method to verify client ownership, confirm all milestones are resolved, and transition contract status to COMPLETED. – `clientId: number` – client identifier. – `contractId: number` – contract identifier. – *Returns:* `Promise<ContractType>` – completed contract.
06	`cancelContract( userId: number, contract Id: number)`	A method to verify participant authorization, check cancellation eligibility, and set status to CANCELLED. – `userId: number` – caller user identifier. – `contractId: number` – contract identifier. – *Returns:* `Promise<ContractType>` – cancelled contract.
07	`getContractList( userId: number, role Name: string, query: GetContractList QueryType)`	A method to construct role-based query filters and retrieve paginated contracts. – `userId: number` – user identifier. – `roleName: string` – role name. – `query: GetContractListQueryType` – pagination and filtering criteria. – *Returns:* `Promise<{ data: ContractListItem Type[]; total: number }>` – list and total count.
08	`getContractDetail( userId: number, role Name: string, contractId: number)`	A method to verify access permissions and load detailed contract specifications with participants. – `userId: number` – user identifier. – `roleName: string` – role name. – `contractId: number` – contract identifier. – *Returns:* `Promise<ContractDetailType>` – full contract dossier.

4.14.2 ClassAttribute

No	Attribute	Type
01	`contractRepository`	`ContractRepository` – private readonly persistence dependency.

4.15 ContractRepository Class

4.15.1 ClassMethod

No	Method	Description
01	`ContractRepository( prisma: PrismaService)`	A method to initialize the repository with the Prisma database client. – `prisma`: `PrismaService` – injectable database connection service. – *Returns:* `void` – constructor, no return value.
02	`findProposalWithJob( proposalId: number)`	A method to query a proposal record with its associated job and client information. – `proposalId`: `number` – proposal identifier. – *Returns:* `Promise<ProposalWithJob | null>` – proposal and parent job.
03	`findContractByProposalId( proposalId: number)`	A method to check if a contract already exists for a given proposal. – `proposalId`: `number` – proposal identifier. – *Returns:* `Promise<Contract | null>` – existing contract or null.
04	`findContractById(contract Id: number)`	A method to query a basic contract record by its primary key. – `contractId`: `number` – contract identifier. – *Returns:* `Promise<Contract | null>` – contract record or null.
05	`createContract(data: CreateContractData)`	A method to insert a new contract row into the database with initial PENDING_SIGN status. – `data`: `CreateContractData` – contract entity attributes. – *Returns:* `Promise<Contract>` – persisted contract record.

No	Method	Description
06	`updateContract( contractId: number, data: UpdateContractBody Type)`	A method to execute database updates on contract fields. – `contractId`: `number` – contract identifier. – `data`: `UpdateContractBodyType` – dictionary of updated attributes. – *Returns:* `Promise<Contract>` – updated contract row.
07	`signContract( contractId: number, role: 'client' | ' freelancer')`	A method to atomically set the client or freelancer signing flag and transition status to ACTIVE when both have signed. – `contractId`: `number` – contract identifier. – `role`: `'client' | 'freelancer'` – party role being recorded. – *Returns:* `Promise<Contract>` – contract row after signing.
08	`completeContract( contractId: number, job Id: number)`	A method to set contract status to COMPLETED and update the parent job status. – `contractId`: `number` – contract identifier. – `jobId`: `number` – associated job identifier. – *Returns:* `Promise<Contract>` – completed contract row.
09	`cancelContract(contract Id: number)`	A method to update contract status to CANCELLED in the database. – `contractId`: `number` – contract identifier. – *Returns:* `Promise<Contract>` – cancelled contract row.
10	`getContractList( where: Prisma.Contract WhereInput, skip: number, limit: number)`	A method to query paginated contract records matching role-specific search conditions. – `where`: `Prisma.ContractWhereInput` – Prisma query filters. – `skip`: `number` – pagination offset. – `limit`: `number` – maximum rows to return. – *Returns:* `Promise<{ data: Contract[]; total: number }>` – matched contracts and count.
11	`getContractDetail( contractId: number)`	A method to query detailed contract information including client, freelancer, milestones, and proposal relations. – `contractId`: `number` – contract identifier. – *Returns:* `Promise<ContractDetail | null>` – detailed contract graph.

4.15.2 ClassAttribute

No	Attribute	Type
01	`prisma`	`PrismaService` – private readonly Prisma client instance.

4.16 SharedFileController Class

4.16.1 ClassMethod

No	Method	Description
01	`SharedFileController( sharedFileService: SharedFileService)`	A method to initialize the controller with the shared file service. – `sharedFileService: SharedFileService` – service that handles contract attachments. – *Returns:* `void` – constructor, no return value.
02	`getSharedFiles( userId: number, role Name: string, contractId: number)`	A method to list all active files shared within a specific contract. – `userId: number` – current user identifier. – `roleName: string` – caller role name. – `contractId: number` – contract identifier parsed from route parameter. – *Returns:* `Promise<GetSharedFilesResponseDTO>` – list of shared file records. – Route: `GET /contracts/:contractId/files`; authenticated endpoint.
03	`uploadSharedFile( userId: number, role Name: string, contractId: number, file: Express.Multer. File)`	A method to upload an attachment to cloud storage and associate it with the contract. – `userId: number` – uploader identifier. – `roleName: string` – caller role name. – `contractId: number` – target contract identifier. – `file: Express.Multer.File` – validated multipart upload file. – *Returns:* `Promise<UploadSharedFileResponseDTO>` – metadata of the uploaded file. – Route: `POST /contracts/:contractId/files`; authenticated endpoint.

No	Method	Description
04	`deleteSharedFile( userId: number, roleName: string, contractId: number, fileId: number)`	A method to delete a shared file if requested by its original uploader or an administrator. – `userId: number` – caller user identifier. – `roleName: string` – caller role name. – `contractId: number` – contract identifier. – `fileId: number` – shared file identifier parsed by `ParseIntPipe`. – *Returns:* `Promise<DeleteSharedFileResponseDTO>` – deletion confirmation. – Route: `DELETE /contracts/:contractId/files/:fileId`; authenticated endpoint.

4.16.2 ClassAttribute

No	Attribute	Type
01	`sharedFileService`	`SharedFileService` – private readonly file management service.

4.17 SharedFileService Class

4.17.1 ClassMethod

No	Method	Description
01	`SharedFileService( sharedFileRepo: SharedFileRepository, cloudinaryService: CloudinaryService)`	A method to initialize the service with repository and cloud storage services. – `sharedFileRepo: SharedFileRepository` – persistence access layer for shared files. – `cloudinaryService: CloudinaryService` – cloud storage service for file uploads and deletions. – *Returns:* `void` – constructor, no return value.

No	Method	Description
02	`getSharedFiles( userId: number, roleName: string, contractId: number)`	A method to verify contract membership and retrieve active shared files. – `userId: number` – current user identifier. – `roleName: string` – caller role name. – `contractId: number` – contract identifier. – *Returns:* `Promise<SharedFileListType>` – array of shared file records.
03	`uploadSharedFile( userId: number, roleName: string, contractId: number, file: Express.Multer.File)`	A method to check participant authorization, stream file to Cloudinary, and save file metadata. – `userId: number` – uploader identifier. – `roleName: string` – caller role name. – `contractId: number` – contract identifier. – `file: Express.Multer.File` – uploaded binary payload. – *Returns:* `Promise<SharedFileType>` – stored file record.
04	`deleteSharedFile( userId: number, roleName: string, contractId: number, fileId: number)`	A method to verify file ownership or admin privilege, delete the asset from Cloudinary, and soft-delete the database record. – `userId: number` – caller user identifier. – `roleName: string` – caller role name. – `contractId: number` – contract identifier. – `fileId: number` – file identifier. – *Returns:* `Promise<{ success: boolean; id: number }>` – operation status.

4.17.2 ClassAttribute

No	Attribute	Type
01	`sharedFileRepo`	`SharedFileRepository` – private readonly repository dependency.
02	`cloudinaryService`	`CloudinaryService` – private readonly cloud storage client.

4.18 SharedFileRepository Class

4.18.1 ClassMethod

No	Method	Description
01	`SharedFileRepository( prisma: PrismaService)`	A method to initialize the repository with the Prisma client. – `prisma`: `PrismaService` – database access service. – *Returns:* `void` – constructor, no return value.
02	`create(data: CreateSharedFileData)`	A method to persist a shared file attachment record linked to a contract and user. – `data`: `CreateSharedFileData` – file name, URL, size, mimeType, and contractId. – *Returns:* `Promise<SharedFile>` – saved database record.
03	`findById(id: number)`	A method to retrieve a shared file record by its primary key. – `id`: `number` – file identifier. – *Returns:* `Promise<SharedFile | null>` – file record or null.
04	`findByContractId(contractId: number)`	A method to query all non-deleted files belonging to a specific contract. – `contractId`: `number` – contract identifier. – *Returns:* `Promise<SharedFile[]>` – list of file records.
05	`softDelete(id: number)`	A method to set the deletedAt timestamp on a shared file record. – `id`: `number` – file identifier. – *Returns:* `Promise<SharedFile>` – updated file record.
06	`findContractById(contractId: number)`	A method to load contract participant IDs to verify access permissions. – `contractId`: `number` – contract identifier. – *Returns:* `Promise<Contract | null>` – contract record.

4.18.2 ClassAttribute

No	Attribute	Type
01	prisma	PrismaService – private readonly Prisma connection instance.

Supporting DTO classes for contract and shared file management

Class / Element	Attributes or Signature	Responsibility
CreateContractBodyDTO	proposalId: number, title: string, terms?: string	Validates contract creation input payload.
UpdateContractBodyDTO	title?: string, description?: string, terms?: string	Validates mutable contract terms before mutual signature.
GetContractListQueryDTO	page: number, limit: number, status?: string, q?: string	Validates contract list filter and pagination parameters.
UploadSharedFile ResponseDTO	id: number, fileName: string, fileUrl: string, size: number	Serializes uploaded contract attachment response data.

4.19 AuthController Class

4.19.1 ClassMethod

No	Method	Description
01	AuthController(auth Service: AuthService)	A method to initialize the controller with the authentication service. – authService: AuthService – service that owns the authentication use cases. – *Returns:* void – constructor, no return value.
02	register(body: Register Dto)	A method to create an email/password account after OTP verification. – body: RegisterDto – email, OTP code, password, full name, and initial role. – *Returns:* Promise<RegisterResponseType> – the created account without password fields. – Route: POST /auth/register; public endpoint.

No	Method	Description
03	`sendOTP(body: SendOTPBodyDTO)`	A method to generate and email an OTP for registration or password reset. – `body`: `SendOTPBodyDTO` – email and verification-code type. – *Returns:* `Promise<MessageResType>` – a success message. – Route: `POST /auth/otp`; public endpoint.
04	`login(body: LoginBodyDTO, userAgent: string, ipAddress: string)`	A method to authenticate an email/password account and create a session. – `body`: `LoginBodyDTO` – email and password. – `userAgent`: `string` – device information used as session metadata. – `ipAddress`: `string` – client IP stored with the session. – *Returns:* `Promise<LoginResponseType>` – access and refresh tokens. – Route: `POST /auth/login`; public endpoint.
05	`refreshToken(body: RefreshTokenBodyDTO, userAgent: string, ipAddress: string)`	A method to rotate a valid refresh token into a new token pair. – `body.refreshToken`: `string` – the current refresh token. – `userAgent`: `string` – device information for the replacement session. – `ipAddress`: `string` – client IP for the replacement session. – *Returns:* `Promise<LoginResponseType>` – the new access and refresh tokens. – Route: `POST /auth/refresh-token`; public endpoint.
06	`logout(body: LogoutBodyDTO)`	A method to terminate the session identified by a refresh token. – `body.refreshToken`: `string` – refresh token of the session to remove. – *Returns:* `Promise<MessageResType>` – a success message. – Route: `POST /auth/logout`; public endpoint.

No	Method	Description
07	`forgotPassword(body: ForgotPasswordBodyDTO)`	A method to reset a password after validating a password-reset OTP. – `body`: `ForgotPasswordBodyDTO` – email, OTP code, and new password. – *Returns:* `Promise<MessageResType>` – a success message. – Route: `POST /auth/forgot-password`; public endpoint.
08	`getGoogleLink( userAgent: string, ip: string)`	A method to build the Google OAuth consent URL. – `userAgent`: `string` – encoded in the OAuth state. – `ip`: `string` – encoded in the OAuth state. – *Returns:* `GetAuthorizationUrlResType` – the authorization URL. – Route: `GET /auth/google-link`; public endpoint.
09	`googleCallback( code: string, state: string, res: Response)`	A method to exchange the Google authorization code and redirect the browser. – `code`: `string` – authorization code returned by Google. – `state`: `string` – base64url payload with device and IP information. – `res`: `Response` – Express response used for the redirect. – *Returns:* `Promise<void>` – redirects to the web application with tokens or an error flag. – Route: `GET /auth/google/callback`; public endpoint.
10	`getMe(userId: number)`	A method to return the authenticated account and its roles. – `userId`: `number` – current user injected by `@User Active('userId')`. – *Returns:* `Promise<GetMeResType>` – account, profile summary, and roles. – Route: `GET /auth/me`; authenticated endpoint.
11	`switchRole( userId: number, session Id: number, body: SwitchRoleBodyDto)`	A method to make an already assigned role the primary role. – `userId`: `number` – current user. – `sessionId`: `number` – current session identifier. – `body.role`: role name to activate. – *Returns:* `Promise<SwitchRoleResponseType>` – a new access token for that role. – Route: `POST /auth/switch-role`; authenticated endpoint.

No	Method	Description
12	`joinRole( userId: number, sessionId: number, body: JoinRoleBodyDto)`	A method to assign an additional Client or Freelancer role. – `userId`: `number` – current user. – `sessionId`: `number` – current session identifier. – `body.role`: role name to join. – *Returns:* `Promise<JoinRoleResponseType>` – a new access token for the joined role. – Route: `POST /auth/join-role`; authenticated endpoint.

4.19.2 ClassAttribute

No	Attribute	Type
01	`authService`	`AuthService` – private readonly authentication service.
02	`logger`	`Logger` – private readonly NestJS logger for OAuth callback errors.

4.20 AuthService Class

4.20.1 ClassMethod

No	Method	Description
01	`AuthService( hashingService: HashingService, emailService: EmailService, sharedRoleRepository: SharedRoleRepository, authRepository: AuthRepository, tokenService: TokenService)`	A method to initialize the service and its Google OAuth client. – `hashingService`: `HashingService` – password hashing and verification. – `emailService`: `EmailService` – OTP delivery. – `sharedRoleRepository`: `SharedRoleRepository` – resolves system role identifiers. – `authRepository`: `AuthRepository` – account, OTP, session, and OAuth persistence. – `tokenService`: `TokenService` – JWT signing and verification. – *Returns:* `void` – constructor, no return value.

No	Method	Description
02	`register(body: RegisterBodyType)`	A method to validate registration data and create an account. – `body: RegisterBodyType` – email, OTP, password, full name, and role. – *Returns:* `Promise<UserType>` – the created account without sensitive fields. – Throws `EmailAlreadyExistsException`, `UserBannedException`, `InvalidVerificationCodeException`, or `OTPExpiredException`.
03	`sendOTP(body: SendOTPBodyType)`	A method to issue an OTP while enforcing the attempt window. – `body: SendOTPBodyType` – email and verification-code type. – *Returns:* `Promise<MessageResType>` – a success message. – Throws `EmailAlreadyExistsException`, `EmailNotFoundException`, or `TooManyAttemptsException`.
04	`login(body: LoginBodyType and session metadata)`	A method to verify credentials, ban state, and the primary role before issuing tokens. – `body: LoginBodyType` – email, password, user agent, and IP address. – *Returns:* `Promise<{ accessToken: string; refreshToken: string }>` – the token pair. – Throws `IncorrectEmailException`, `IncorrectPasswordException`, `UserBannedException`, or `RoleNotFoundException`.
05	`generateAccessAndRefreshTokens(payload)`	A private method to sign a refresh token, persist a session, and sign the access token. – `payload`: object – user id, role id, role name, user agent, and IP address. – *Returns:* `Promise<{ accessToken: string; refreshToken: string }>` – the new token pair. – The access token carries the persisted session id.

No	Method	Description
06	`refreshToken(payload)`	A method to validate and rotate a refresh token. – `payload`: object – refresh token, user agent, and IP address. – *Returns:* `Promise<{ accessToken: string; refreshToken: string }>` – the replacement token pair. – Throws `RefreshTokenRevokedException` or `UserBannedException`.
07	`logout(refreshToken: string)`	A method to remove the session that owns a refresh token. – `refreshToken`: `string` – token of the session to terminate. – *Returns:* `Promise<MessageResType>` – a success message. – Throws `RefreshTokenRevokedException` when the session is absent.
08	`forgotPassword(payload: ForgotPasswordBodyType)`	A method to validate a reset OTP, hash the new password, and remove the OTP. – `payload`: `ForgotPasswordBodyType` – email, OTP, and new password. – *Returns:* `Promise<MessageResType>` – a success message. – Throws `EmailNotFoundException`, `UserBannedException`, or OTP validation errors.
09	`validateVerificationCode(uniqueValue)`	A method to load an OTP and reject missing or expired codes. – `uniqueValue`: object – email, code, and verification type. – *Returns:* `Promise<EmailVerificationType>` – the matching OTP record. – Throws `InvalidVerificationCodeException` or `OTPExpiredException`.
10	`getMe(userId: number)`	A method to return the current account, profile summary, and roles. – `userId`: `number` – current account identifier. – *Returns:* `Promise<GetMeResType>` – the authenticated account view. – Throws `NotFoundException` when absent or `UserBannedException` when banned.

No	Method	Description
11	`switchRole(userId, sessionId, body)`	A method to change the primary role and issue a new access token. – `userId`: `number` – current account. – `sessionId`: `number` – current session. – `body`: `SwitchRoleBodyType` – target role. – *Returns:* `Promise<{ accessToken: string }>` – the role-scoped access token. – Throws `RoleNotFoundException`.
12	`joinRole(userId, sessionId, body)`	A method to assign an additional role and issue a new access token. – `userId`: `number` – current account. – `sessionId`: `number` – current session. – `body`: `JoinRoleBodyType` – role to join. – *Returns:* `Promise<{ accessToken: string }>` – the role-scoped access token. – Throws `RoleNotFoundException` or `RoleAlreadyAssignedException`.
13	`getAuthorizationUrl(payload)`	A method to build the Google OAuth consent URL with a base64url state. – `payload`: object – user agent and IP address. – *Returns:* `GetAuthorizationUrlResType` – the authorization URL.
14	`googleCallback(payload)`	A method to exchange a Google code, create or load the account, and issue tokens. – `payload`: object – Google authorization code and state. – *Returns:* `Promise<{ accessToken: string; refreshToken: string }>` – the token pair. – Creates a Client account and an OAuth account on first login; rejects banned accounts.

4.20.2 ClassAttribute

No	Attribute	Type
01	`hashingService`	`HashingService` – private readonly password utility.
02	`emailService`	`EmailService` – private readonly OTP delivery utility.

No	Attribute	Type
03	`sharedRoleRepository`	`SharedRoleRepository` – private readonly system-role lookup.
04	`authRepository`	`AuthRepository` – private readonly authentication persistence.
05	`tokenService`	`TokenService` – private readonly JWT utility.
06	`oauth2Client`	`OAuth2Client` – public Google OAuth client.
07	`logger`	`Logger` – private readonly NestJS logger.

4.21 AuthRepository Class

4.21.1 ClassMethod

No	Method	Description
01	`AuthRepository(prisma: PrismaService)`	A method to initialize the repository with the shared Prisma client. – `prisma`: `PrismaService` – database client. – *Returns:* `void` – constructor, no return value.
02	`findUserByEmail(email: string)`	A method to load a non-deleted account by email. – `email`: `string` – unique account email. – *Returns:* `Promise<UserType | null>` – the account without password timestamps, or `null`.
03	`findUserByEmailIncludeRoles(email: string)`	A method to load an account with its role assignments. – `email`: `string` – unique account email. – *Returns:* `Promise<UserType and roles | null>` – the account and ordered roles, or `null`.
04	`findVerificationCode(uniqueValue)`	A method to find an OTP by email, code, and type. – `uniqueValue`: object – email, code, and verification type. – *Returns:* `Promise<EmailVerificationType | null>` – the matching OTP, or `null`.

No	Method	Description
05	`findVerificationCodeBy EmailAndType(email, type)`	A method to load the current OTP record for an email and type. – `email`: `string` – account email. – `type`: `VerificationCodeType` – registration or password-reset code. – *Returns:* `Promise<EmailVerificationType | null>` – the OTP record, or `null`.
06	`createUserAndRegister( deleteOtp, user)`	A method to delete the consumed OTP and create the account in one transaction. – `deleteOtp`: object – OTP code, type, and email. – `user`: object – email, password, full name, and initial role. – *Returns:* `Promise<UserType>` – the created account.
07	`createUserAndRegister Google(user)`	A method to create a Google account with a Client role and profile. – `user`: object – email, full name, and role identifier. – *Returns:* `Promise<UserType and roles>` – the created account with roles.
08	`upsertVerificationCode( data)`	A method to create or replace an OTP record. – `data`: object – email, type, code, expiry, and attempt count. – *Returns:* `Promise<EmailVerificationType>` – the persisted OTP.
09	`findUserForLogin(email: string)`	A method to load an account and ordered roles for authentication. – `email`: `string` – account email. – *Returns:* `Promise<UserType and roles | null>` – the account, or `null`.
10	`createSession(data)`	A method to persist a refresh-token session. – `data`: object – user id, refresh token, device, IP address, and expiry. – *Returns:* `Promise<Session>` – the created session with its identifier.
11	`deleteSessionByRefresh Token(data)`	A method to delete a session by refresh token and user id. – `data`: object – refresh token and user id. – *Returns:* `Promise<Session>` – the deleted session.

No	Method	Description
12	`findUniqueRefreshToken IncludeUserRole(data)`	A method to load a session, its account, and the account roles by refresh token. – `data.refreshToken`: `string` – refresh token to resolve. – *Returns:* `Promise<Session and user roles | null>` – the session graph, or `null`.
13	`update(where, data)`	A method to update a non-deleted account. – `where`: object – account identifier. – `data`: `Partial<UserType>` – fields to change. – *Returns:* `Promise<UserType | null>` – the updated account.
14	`deleteVerifycationCode( uniqueValue)`	A method to delete an OTP after a successful password reset. – `uniqueValue`: object – email and verification type. – *Returns:* `Promise<EmailVerificationType>` – the deleted OTP.
15	`findUserById(userId: number)`	A method to load an account with profile and ordered roles. – `userId`: `number` – account identifier. – *Returns:* `Promise<UserType and profile and roles | null>` – the account view, or `null`.
16	`switchPrimaryRole(userId, roleName)`	A method to make an assigned role primary in one transaction. – `userId`: `number` – account identifier. – `roleName`: `string` – role to activate. – *Returns:* `Promise<RoleType | null>` – the activated role, or `null` when not assigned.
17	`joinRole(userId, role Name)`	A method to assign a Client or Freelancer role and create its profile row. – `userId`: `number` – account identifier. – `roleName`: `JoinRoleBodyType['role']` – role to join. – *Returns:* `Promise<{ status: 'joined' | ' already-assigned'; role: RoleType } | null>` – the assignment result, or `null` when the role is absent.

No	Method	Description
18	`createOauthAccount(data)`	A method to link a Google identity to an account. – `data`: object – user id, provider, and provider user id. – *Returns:* `Promise<OauthAccount>` – the created OAuth link.

4.21.2 ClassAttribute

No	Attribute	Type
01	`prisma`	`PrismaService` – private readonly database client.

4.22 RolesController Class

4.22.1 ClassMethod

No	Method	Description
01	`RolesController(rolesService: RolesService)`	A method to initialize the controller with the role service. – `rolesService`: `RolesService` – service that owns role use cases. – *Returns:* `void` – constructor, no return value.
02	`getRoles()`	A method to return all active roles. – *Returns:* `Promise<RoleListResponseType>` – the role list. – Route: `GET /roles`.
03	`getRoleById(id: number)`	A method to return one role with its permissions. – `id`: `number` – role identifier parsed by `ParseIntPipe`. – *Returns:* `Promise<RoleDetailResponseType>` – the role detail. – Route: `GET /roles/:id`.
04	`createRole(body: CreateRoleBodyDto)`	A method to create a custom role. – `body`: `CreateRoleBodyDto` – role name and optional description. – *Returns:* `Promise<CreateRoleResponseType>` – the created role. – Route: `POST /roles`.

No	Method	Description
05	`updateRole(id: number, body: UpdateRoleBodyDto)`	A method to update a role's name or description. – `id: number` – role identifier. – `body: UpdateRoleBodyDto` – optional name and description. – *Returns:* `Promise<UpdateRoleResponseType>` – the updated role. – Route: `PATCH /roles/:id`.
06	`setRolePermissions( id: number, body: SetRolePermissions BodyDto)`	A method to replace every permission assigned to a role. – `id: number` – role identifier. – `body.permissionIds: number[]` – the complete replacement set. – *Returns:* `Promise<SetRolePermissionsResponse Type>` – the updated role detail. – Route: `PUT /roles/:id/permissions`.
07	`deleteRole(id: number)`	A method to soft-delete an unused custom role. – `id: number` – role identifier. – *Returns:* `Promise<DeleteRoleResponseType>` – a success message. – Route: `DELETE /roles/:id`.

4.22.2 ClassAttribute

No	Attribute	Type
01	`rolesService`	`RolesService` – private readonly role service.

4.23 RolesService Class

4.23.1 ClassMethod

No	Method	Description
01	`RolesService(roles Repository: Roles Repository)`	A method to initialize the service with the role repository. – `rolesRepository: RolesRepository` – persistence dependency. – *Returns:* `void` – constructor, no return value.

No	Method	Description
02	`isSystemRoleName(name: string)`	A private method to detect the four built-in role names. – `name: string` – role name to test. – *Returns:* `boolean` – true for Admin, Client, Freelancer, or Expert.
03	`getRoles()`	A method to load the active role list. – *Returns:* `Promise<RoleListResponseType>` – the role list. – Failure is mapped to `FailedToLoadRolesException`.
04	`getRoleById(id: number)`	A method to load a role and its permissions. – `id: number` – role identifier. – *Returns:* `Promise<RoleDetailResponseType>` – the role detail. – Prisma errors are mapped to `FailedToLoadRoleDetailException`.
05	`createRole(body: CreateRoleBodyType)`	A method to validate the name and create a custom role. – `body: CreateRoleBodyType` – role name and optional description. – *Returns:* `Promise<CreateRoleResponseType>` – the created role. – Throws `CannotModifySystemRoleException` or `RoleAlreadyExistsException`.
06	`updateRole(id: number, body: UpdateRoleBodyType)`	A method to validate system-role and duplicate-name rules before updating. – `id: number` – role identifier. – `body: UpdateRoleBodyType` – optional name and description. – *Returns:* `Promise<UpdateRoleResponseType>` – the updated role. – Throws `CannotModifySystemRoleException` or `RoleAlreadyExistsException`.
07	`deleteRole(id: number)`	A method to soft-delete a custom role that is not assigned to any user. – `id: number` – role identifier. – *Returns:* `Promise<DeleteRoleResponseType>` – a success message. – Throws `CannotModifySystemRoleException` or `RoleInUseException`.

No	Method	Description
08	`setRolePermissions(id, body)`	A method to validate permission ids and replace the role permission set. – `id`: `number` – role identifier. – `body`: `SetRolePermissionsBodyType` – permission ids to assign. – *Returns:* `Promise<SetRolePermissionsResponseType>` – the updated role detail. – Throws `InvalidPermissionIdsException`.

4.23.2 ClassAttribute

No	Attribute	Type
01	`rolesRepository`	`RolesRepository` – private readonly persistence dependency.
02	`logger`	`Logger` – private readonly NestJS logger.

4.24 RolesRepository Class

4.24.1 ClassMethod

No	Method	Description
01	`RolesRepository(prisma: PrismaService)`	A method to initialize the repository with the shared Prisma client. – `prisma`: `PrismaService` – database client. – *Returns:* `void` – constructor, no return value.
02	`findAll()`	A method to load all non-deleted roles ordered by id. – *Returns:* `Promise<RoleListItemType[]>` – the role list.
03	`findById(id: number)`	A method to load a role and its active permissions. – `id`: `number` – role identifier. – *Returns:* `Promise<RoleDetailResponseType>` – the role detail. – Throws `RoleNotFoundException`.

No	Method	Description
04	`findActiveByName(name, excludeId?)`	A method to find an active role by case-insensitive name. – `name: string` – role name. – `excludeId: number | undefined` – optional role to exclude. – *Returns:* `Promise<RoleListItemType | null>` – the matching role, or `null`.
05	`createRole(body: CreateRoleBodyType)`	A method to insert a role. – `body: CreateRoleBodyType` – name and optional description. – *Returns:* `Promise<CreateRoleResponseType>` – the created role.
06	`updateRole(id: number, body: UpdateRoleBodyType)`	A method to update the provided role fields. – `id: number` – role identifier. – `body: UpdateRoleBodyType` – optional name and description. – *Returns:* `Promise<UpdateRoleResponseType>` – the updated role.
07	`softDeleteRole(id: number)`	A method to set `deletedAt` on a role. – `id: number` – role identifier. – *Returns:* `Promise<void>` – no value.
08	`countAssignedUsers(roleId: number)`	A method to count user-role assignments. – `roleId: number` – role identifier. – *Returns:* `Promise<number>` – the assignment count.
09	`findActivePermissionIds(ids: number[])`	A method to filter permission ids down to active permissions. – `ids: number[]` – requested permission ids. – *Returns:* `Promise<number[]>` – the active ids.
10	`replaceRolePermissions(roleId, permissionIds)`	A method to delete the old role permissions and insert the new set in a transaction. – `roleId: number` – role identifier. – `permissionIds: number[]` – replacement permission ids. – *Returns:* `Promise<void>` – no value.

4.24.2 ClassAttribute

No	Attribute	Type
01	`prisma`	`PrismaService` – private readonly database client.

4.25 ConversationController Class

4.25.1 ClassMethod

No	Method	Description
01	`ConversationController( conversationService: ConversationService)`	A method to initialize the controller with the conversation service. – `conversationService: ConversationService` – service that owns all conversation use cases. – *Returns:* `void` – constructor, no return value.
02	`getConversations(userId: number)`	A method to return the current user's conversation list. – `userId: number` – current user identifier injected by `@UserActive('userId')`. – *Returns:* `Promise<GetConversationsResponseType>` – conversation items with other user, last message, unread count, and pin time. – Route: `GET /conversations`; authenticated endpoint.
03	`createConversation( userId: number, body: CreateConversationBodyType)`	A method to create or reopen a conversation with another user. – `userId: number` – current user identifier. – `body.participantId: number` – the other participant, validated by `CreateConversationBodyDto`. – *Returns:* `Promise<CreateConversationResponseType>` – id, participant1, participant2, and createdAt. – Route: `POST /conversations`; authenticated endpoint.
04	`getMessages( userId: number, id: number)`	A method to return the message history of one conversation. – `userId: number` – current user identifier. – `id: number` – conversation identifier parsed by `ParseIntPipe`. – *Returns:* `Promise<DirectMessageType[]>` – non-deleted messages in ascending creation order. – Route: `GET /conversations/:id/messages`; authenticated endpoint.

No	Method	Description
05	`sendMessage( userId: number, id: number, body: SendMessageBodyType)`	A method to send a message through the REST fallback. – `userId: number` – sender identifier. – `id: number` – conversation identifier. – `body: SendMessageBodyType` – message text and optional attachment, validated by `SendMessageBodyDto`. – *Returns:* `Promise<DirectMessageType>` – the saved message. – Route: `POST /conversations/:id/messages`; authenticated endpoint.
06	`uploadFile( userId: number, id: number, file?: Express.Multer.File)`	A method to upload a message attachment for a conversation participant. – `userId: number` – current user identifier. – `id: number` – conversation identifier. – `file: Express.Multer.File | undefined` – multipart file read into memory by `FileInterceptor`. – *Returns:* `Promise<MessageAttachmentType>` – Cloudinary URL, original name, byte size, and MIME type. – Route: `POST /conversations/:id/files`; authenticated endpoint.
07	`deleteMessage( userId: number, id: number, messageId: number)`	A method to soft-delete a message sent by the current user. – `userId: number` – current user identifier. – `id: number` – conversation identifier. – `messageId: number` – message identifier. – *Returns:* `Promise<DeleteMessageResponseType>` – the deleted message id. – Route: `DELETE /conversations/:id/messages/:messageId`; authenticated endpoint.
08	`markAsRead( userId: number, id: number)`	A method to mark all messages from the other participant as read. – `userId: number` – current user identifier. – `id: number` – conversation identifier. – *Returns:* `Promise<{ count: number }>` – number of updated messages. This route has no response DTO decorator. – Route: `PATCH /conversations/:id/read`; authenticated endpoint.

No	Method	Description
09	`hideConversation( userId: number, id: number)`	A method to hide a conversation only for the current user. – `userId: number` – current user identifier. – `id: number` – conversation identifier. – *Returns:* `Promise<HideConversationResponse Type>` – id and hidden flag. – Route: `PATCH /conversations/:id/hide`; authenticated endpoint.
10	`pinConversation( userId: number, id: number, body: PinConversation BodyType)`	A method to pin or unpin a conversation for the current user. – `userId: number` – current user identifier. – `id: number` – conversation identifier. – `body.pinned: boolean` – target pin state, validated by `PinConversationBodyDto`. – *Returns:* `Promise<PinConversationResponseType>` – id and the new `pinnedAt` value. – Route: `PATCH /conversations/:id/pin`; authenticated endpoint.

4.25.2 ClassAttribute

No	Attribute	Type
01	`conversationService`	`ConversationService` – private readonly service dependency.

4.26 ConversationService Class

4.26.1 ClassMethod

No	Method	Description
01	`ConversationService( conversationRepository: ConversationRepository)`	A method to initialize the service with the conversation repository. – `conversationRepository: Conversation Repository` – persistence dependency. – *Returns:* `void` – constructor, no return value.

No	Method	Description
02	`buildConversationItem( conversation: RawConversationItem, userId: number)`	A private method to map a repository row to the public conversation item. – `conversation`: `RawConversationItem` – row with both participants, last message, and unread count. – `userId`: `number` – current user, used to select the other participant and pin time. – *Returns:* `ConversationListItemType` – normalized item with `otherUser`, `lastMessage`, `unreadCount`, and `pinnedAt`.
03	`getConversations(userId: number)`	A method to load and sort all visible conversations of a user. – `userId`: `number` – current user identifier. – *Returns:* `Promise<GetConversationsResponseType>` – pinned conversations first, then by last-message time or creation time. – Any failure is mapped to `FailedToLoadConversationsException`.
04	`getConversationItem( conversationId: number, userId: number)`	A method to rebuild one conversation item for a realtime update. – `conversationId`: `number` – conversation identifier. – `userId`: `number` – recipient of the realtime item. – *Returns:* `Promise<ConversationListItemType | null>` – the item, or `null` when the conversation is not visible to that user.
05	`getMessages( userId: number, conversationId: number)`	A method to authorize a participant and return the message history. – `userId`: `number` – current user identifier. – `conversationId`: `number` – conversation identifier. – *Returns:* `Promise<DirectMessageType[]>` – non-deleted messages. – Throws `ConversationNotFoundException` when the user is not a participant; other failures become `FailedToLoadConversationsException`.

No	Method	Description
06	`createConversation( userId: number, participantId: number)`	A method to create a conversation or return the existing pair. – `userId: number` – current user identifier. – `participantId: number` – the other user. – *Returns:* `Promise<CreateConversationResponse Type>` – id, participant1, participant2, and createdAt. – Rejects self-conversation with `CannotStart ConversationWithSelfException`; an existing hidden conversation is unhidden.
07	`sendMessage( userId: number, conversationId: number, message: string, attachment?: Message AttachmentType | null)`	A method to save a message and unhide the conversation for both participants. – `userId: number` – sender identifier. – `conversationId: number` – conversation identifier. – `message: string` – text; may be empty when an attachment exists. – `attachment: MessageAttachmentType | null | undefined` – optional file metadata. – *Returns:* `Promise<DirectMessageType>` – the saved message. – Throws `ConversationNotFoundException` or `FailedToSendMessageException`.
08	`uploadFile( userId: number, conversationId: number, file?: Express.Multer. File)`	A method to validate participation and upload a file to Cloudinary. – `userId: number` – current user identifier. – `conversationId: number` – conversation identifier. – `file: Express.Multer.File | undefined` – in-memory multipart file. – *Returns:* `Promise<MessageAttachmentType>` – secure URL, original name, byte size, and MIME type. – Throws `FailedToUploadFileException`, `ConversationNotFoundException`, or passes through a Nest `HttpException`.

No	Method	Description
09	`deleteMessage( userId: number, conversationId: number, messageId: number)`	A method to authorize a participant and soft-delete the sender's message. – `userId: number` – current user identifier; must be the sender. – `conversationId: number` – conversation identifier. – `messageId: number` – message identifier. – *Returns:* `Promise<{ id: number }>` – the deleted message id. – Throws `ConversationNotFoundException`, `MessageNotFoundException`, or `FailedToDeleteMessageException`.
10	`markAsRead( userId: number, conversationId: number)`	A method to mark messages from the other participant as read. – `userId: number` – current user identifier. – `conversationId: number` – conversation identifier. – *Returns:* `Promise<{ count: number }>` – number of updated messages. – Throws `ConversationNotFoundException` or `FailedToMarkAsReadException`.
11	`hideConversation( userId: number, conversationId: number)`	A method to set the current participant's hidden timestamp. – `userId: number` – current user identifier. – `conversationId: number` – conversation identifier. – *Returns:* `Promise<HideConversationResponseType>` – id and hidden flag. – Throws `ConversationNotFoundException` or `FailedToHideConversationException`.
12	`pinConversation( userId: number, conversationId: number, pinned: boolean)`	A method to set or clear the current participant's pin timestamp. – `userId: number` – current user identifier. – `conversationId: number` – conversation identifier. – `pinned: boolean` – true to pin, false to unpin. – *Returns:* `Promise<PinConversationResponseType>` – id and `pinnedAt`. – Throws `ConversationNotFoundException` or `FailedToPinConversationException`.

No	Method	Description
13	`ensureParticipant( conversationId: number, userId: number)`	A method used by the gateway to assert that a socket user belongs to a conversation. – `conversationId`: `number` – conversation identifier. – `userId`: `number` – socket user identifier. – *Returns:* the non-deleted `Conversation` row. – Throws `ConversationAccessDeniedException` when the user is not a participant.

4.26.2 ClassAttribute

No	Attribute	Type
01	`conversationRepository`	`ConversationRepository` – private readonly persistence dependency.
02	`logger`	`Logger` – private readonly NestJS logger; records attachment-upload failures.

4.27 ConversationRepository Class

4.27.1 ClassMethod

No	Method	Description
01	`ConversationRepository( prisma: PrismaService)`	A method to initialize the repository with the shared Prisma service. – `prisma`: `PrismaService` – database client. – *Returns:* `void` – constructor, no return value.
02	`findConversationBetween( userId1: number, user Id2: number)`	A method to find the non-deleted conversation for a user pair in either participant order. – `userId1`, `userId2`: `number` – the two users. – *Returns:* `Promise<Conversation | null>` – the conversation or `null`.

No	Method	Description
03	`getConversationsByUserId( userId: number)`	A method to query all conversations visible to a user with previews and unread counts. – `userId: number` – current user identifier. – *Returns:* conversation rows with both participant profiles, the latest non-deleted message, and a filtered message count. – A conversation is included only when the current participant's hidden timestamp is `null`.
04	`findConversationByIdFor User( conversationId: number, userId: number)`	A method to load a non-deleted conversation only when the user is a participant. – `conversationId: number` – conversation identifier. – `userId: number` – current user identifier. – *Returns:* `Promise<Conversation | null>` – the conversation or `null`.
05	`findConversationById( conversationId: number)`	A method to load a non-deleted conversation without a participant check. – `conversationId: number` – conversation identifier. – *Returns:* `Promise<Conversation | null>` – the conversation or `null`.
06	`hideConversationForUser( conversationId: number, userId: number)`	A method to set the hidden timestamp of the participant who requested the hide. – `conversationId: number` – conversation identifier. – `userId: number` – participant identifier. – *Returns:* `Promise<boolean>` – true when the row was updated, false when the conversation or participant does not match.
07	`unhideConversation( conversationId: number)`	A method to clear both participants' hidden timestamps after new activity. – `conversationId: number` – conversation identifier. – *Returns:* the updated `Conversation` row.
08	`setPinnedForUser( conversationId: number, userId: number, pinned: boolean)`	A method to set or clear the participant's pin timestamp. – `conversationId: number` – conversation identifier. – `userId: number` – participant identifier. – `pinned: boolean` – true sets the current time, false clears it. – *Returns:* `Promise<Date | null>` – the new timestamp, or `null` when unpinned or not a participant.

No	Method	Description
09	`getMessages(conversationId: number)`	A method to query all non-deleted messages of a conversation in ascending time order. – `conversationId: number` – conversation identifier. – *Returns:* `Promise<DirectMessage[]>` – Prisma message rows.
10	`createConversation(participant1: number, participant2: number)`	A method to insert a conversation pair. – `participant1: number` – first participant (the creator in the service flow). – `participant2: number` – second participant. – *Returns:* `Promise<Conversation>` – the inserted row.
11	`createMessage(conversationId: number, senderId: number, message: string, attachment?: MessageAttachmentType)`	A method to insert a message and copy optional attachment metadata. – `conversationId: number` – conversation identifier. – `senderId: number` – sender identifier. – `message: string` – message text. – `attachment: MessageAttachmentType | undefined` – optional file metadata. – *Returns:* `Promise<DirectMessage>` – the inserted message.
12	`softDeleteMessage(conversationId: number, messageId: number, senderId: number)`	A method to set `deletedAt` only when the message belongs to the conversation and sender. – `conversationId: number` – conversation identifier. – `messageId: number` – message identifier. – `senderId: number` – sender identifier. – *Returns:* `Promise<boolean>` – true when one non-deleted row was updated.
13	`markMessagesAsRead(conversationId: number, userId: number)`	A method to mark messages from the other participant as read. – `conversationId: number` – conversation identifier. – `userId: number` – current user; messages sent by this user are excluded. – *Returns:* the Prisma batch payload with the updated row count.
14	`countUnread(conversationId: number, userId: number)`	A method to count unread messages from the other participant. – `conversationId: number` – conversation identifier. – `userId: number` – current user. – *Returns:* `Promise<number>` – count of non-deleted, unread messages not sent by the user.

4.27.2 ClassAttribute

No	Attribute	Type
01	`prisma`	`PrismaService` – private readonly database client.

4.28 ConversationsGateway Class

4.28.1 ClassMethod

No	Method	Description
01	`ConversationsGateway( conversationService: ConversationService, tokenService: TokenService)`	A method to initialize the gateway dependencies. – `conversationService`: `ConversationService` – shared use-case service. – `tokenService`: `TokenService` – verifies the access token during the socket handshake. – *Returns:* `void` – constructor, no return value.
02	`handleConnection(client: Socket)`	A method to authenticate a new socket and join its user room. – `client`: `Socket` – connecting Socket.IO client. – Reads `client.handshake.auth.token`, falling back to the bearer token in the authorization header; verifies it with `TokenService.verifyAccessToken`. – *Returns:* `Promise<void>` – stores `userId` in `client.data` and joins `user:<userId>`; disconnects on failure.
03	`handleDisconnect(client: Socket)`	A method to log a disconnected socket. – `client`: `Socket` – disconnected client. – *Returns:* `void` – no return value.
04	`handleSendMessage( client: Socket, payload: unknown)`	A method to handle the `message:send` event. – `client`: `Socket` – sender socket. – `payload`: `unknown` – validated by `SendMessage PayloadSchema`. – Ensures participation, saves the message, emits `message:new` to both user rooms, and refreshes both conversation lists. – *Returns:* `Promise<{ success: boolean; message?: DirectMessageType }>` – success flag and saved message; emits `message:error` on failure.

No	Method	Description
05	`handleMarkAsRead( client: Socket, payload: unknown)`	A method to handle the `message:read` event. – `client: Socket` – reader socket. – `payload: unknown` – validated by `MarkAsRead PayloadSchema`. – Calls `markAsRead` and emits `conversation:updated` to both participants. – *Returns:* `Promise<{ success: boolean }>` – success flag.
06	`handleDeleteMessage( client: Socket, payload: unknown)`	A method to handle the `message:delete` event. – `client: Socket` – sender socket. – `payload: unknown` – validated by `DeleteMessage PayloadSchema`. – Calls `deleteMessage`, emits `message:deleted` to both user rooms, and refreshes both conversation lists. – *Returns:* `Promise<{ success: boolean }>` – success flag; emits `message:error` on failure.
07	`handleHideConversation( client: Socket, payload: unknown)`	A method to handle the `conversation:hide` event. – `client: Socket` – current user's socket. – `payload: unknown` – validated by `HideConversation PayloadSchema`. – Calls `hideConversation` and emits `conversation: hidden` only to the current user's room. – *Returns:* `Promise<{ success: boolean }>` – success flag.
08	`handleTyping( client: Socket, payload: unknown)`	A method to handle the `typing` event. – `client: Socket` – sender socket. – `payload: unknown` – validated by `TypingPayload Schema`. – Ensures participation and forwards `typing` with `conversationId` and `isTyping` only to the other user's room. – *Returns:* `Promise<{ success: boolean }>` – success flag.

No	Method	Description
09	`emitConversationUpdated( conversationId: number, userId: number)`	A private method to push the latest conversation item to one user. – `conversationId`: `number` – conversation identifier. – `userId`: `number` – target user. – *Returns:* `Promise<void>` – emits `conversation:updated` when `getConversationItem` returns a visible item.

4.28.2 ClassAttribute

No	Attribute	Type
01	`server`	`Server` – Socket.IO server injected by `@WebSocketServer()`; used to address user rooms.
02	`logger`	`Logger` – private readonly NestJS logger for connection and event failures.
03	`conversationService`	`ConversationService` – private readonly service dependency.
04	`tokenService`	`TokenService` – private readonly access-token verifier.

4.29 MilestoneController Class

4.29.1 ClassMethod

No	Method	Description
01	`MilestoneController( milestoneService: MilestoneService)`	A method to initialize the controller with the milestone service dependency. – `milestoneService`: `MilestoneService` – service encapsulating milestone business logic. – *Returns:* `void` – constructor, no return value.

No	Method	Description
02	`getMilestoneList( userId: number, roleName: string, contractId: number, query: GetMilestoneListQueryDTO)`	A method to retrieve a paginated and status-filtered list of milestones for an active contract. – `userId: number` – caller user identifier injected by `@UserActive('userId')`. – `roleName: string` – role name for authorization checks. – `contractId: number` – parent contract identifier parsed by `ParseIntPipe`. – `query: GetMilestoneListQueryDTO` – pagination parameters and optional status filter. – *Returns:* `Promise<GetMilestoneListResponseDTO>` – paginated milestone items. – Route: `GET /contracts/:contractId/milestones`; authenticated endpoint.
03	`getMilestoneDetail( userId: number, roleName: string, contractId: number, milestoneId: number)`	A method to retrieve full detail and dispute linkages of a specific contract milestone. – `userId: number` – caller user identifier. – `roleName: string` – caller role name. – `contractId: number` – contract identifier. – `milestoneId: number` – target milestone identifier. – *Returns:* `Promise<GetMilestoneDetailResponseDTO>` – milestone detail payload. – Route: `GET /contracts/:contractId/milestones/:id`; authenticated endpoint.
04	`createMilestone( userId: number, contractId: number, body: CreateMilestoneBodyDTO)`	A method for the client to define and attach a new milestone to a contract. – `userId: number` – client identifier injected by `@UserActive('userId')`. – `contractId: number` – contract identifier. – `body: CreateMilestoneBodyDTO` – title, description, amount, and optional due date. – *Returns:* `Promise<CreateMilestoneResponseDTO>` – created milestone record. – Route: `POST /contracts/:contractId/milestones`; authenticated endpoint.

No	Method	Description
05	`updateMilestone( userId: number, contractId: number, milestoneId: number, body: UpdateMilestoneBodyDTO)`	A method for the client to update milestone deliverables or budget prior to funding. – `userId: number` – client identifier. – `contractId: number` – contract identifier. – `milestoneId: number` – milestone identifier. – `body: UpdateMilestoneBodyDTO` – mutable fields (title, description, amount, dueDate). – *Returns:* `Promise<UpdateMilestoneResponseDTO>` – updated milestone. – Route: `PATCH /contracts/:contractId/milestones/:id`; authenticated endpoint.
06	`progressMilestone( userId: number, contractId: number, milestoneId: number)`	A method for the freelancer to advance a funded milestone into the in-progress state. – `userId: number` – freelancer identifier. – `contractId: number` – contract identifier. – `milestoneId: number` – target milestone identifier. – *Returns:* `Promise<ProgressMilestoneResponseDTO>` – milestone with updated in-progress status. – Route: `PATCH /contracts/:contractId/milestones/:id/progress`; authenticated endpoint.
07	`deleteMilestone( userId: number, contractId: number, milestoneId: number)`	A method for the client to soft-delete an unfunded pending milestone. – `userId: number` – client identifier. – `contractId: number` – contract identifier. – `milestoneId: number` – milestone identifier. – *Returns:* `Promise<DeleteMilestoneResponseDTO>` – deleted milestone confirmation. – Route: `DELETE /contracts/:contractId/milestones/:id`; authenticated endpoint.

4.29.2 ClassAttribute

No	Attribute	Type
01	`milestoneService`	`MilestoneService` – private readonly milestone service dependency.

4.30 MilestoneService Class

4.30.1 ClassMethod

No	Method	Description
01	`MilestoneService( milestoneRepository: MilestoneRepository)`	A method to initialize the service with the milestone repository dependency. – `milestoneRepository`: `MilestoneRepository` – data access layer for milestone entities. – *Returns:* `void` – constructor, no return value.
02	`getMilestoneList( userId: number, role Name: string, contractId: number, query: GetMilestoneList QueryType)`	A method to verify contract membership and retrieve paginated milestones with dispute metadata. – `userId`: `number` – requesting user identifier. – `roleName`: `string` – role name. – `contractId`: `number` – contract identifier. – `query`: `GetMilestoneListQueryType` – pagination and filtering criteria. – *Returns:* `Promise<MilestoneListResponseType>` – paginated milestone items.
03	`getMilestoneDetail( userId: number, role Name: string, contractId: number, milestoneId: number)`	A method to verify participant authorization and fetch comprehensive milestone details. – `userId`: `number` – user identifier. – `roleName`: `string` – caller role name. – `contractId`: `number` – contract identifier. – `milestoneId`: `number` – milestone identifier. – *Returns:* `Promise<MilestoneDetailType>` – single milestone detail object.
04	`createMilestone( clientId: number, contractId: number, body: CreateMilestone BodyType)`	A method to enforce contract status constraints, verify cumulative budget limits, and create a milestone. – `clientId`: `number` – client user identifier. – `contractId`: `number` – contract identifier. – `body`: `CreateMilestoneBodyType` – milestone parameters. – *Returns:* `Promise<MilestoneType>` – created milestone entity.

No	Method	Description
05	`updateMilestone( clientId: number, contractId: number, milestoneId: number, body: UpdateMilestone BodyType)`	A method to verify that a milestone is still pending, validate cumulative contract ceiling, and update attributes. – `clientId: number` – client user identifier. – `contractId: number` – contract identifier. – `milestoneId: number` – milestone identifier. – `body: UpdateMilestoneBodyType` – revised milestone attributes. – *Returns:* `Promise<MilestoneType>` – updated milestone entity.
06	`deleteMilestone( clientId: number, contractId: number, milestoneId: number)`	A method to verify client ownership, ensure milestone is unfunded and pending, and soft-delete it. – `clientId: number` – client user identifier. – `contractId: number` – contract identifier. – `milestoneId: number` – milestone identifier. – *Returns:* `Promise<MilestoneType>` – deleted milestone record.
07	`progressMilestone( userId: number, contract Id: number, milestoneId: number)`	A method to verify freelancer ownership, ensure milestone is funded, check that no other milestone is currently in progress, and transition status to in-progress. – `userId: number` – freelancer user identifier. – `contractId: number` – contract identifier. – `milestoneId: number` – milestone identifier. – *Returns:* `Promise<MilestoneType>` – updated milestone in progress.

4.30.2 ClassAttribute

No	Attribute	Type
01	`milestoneRepository`	`MilestoneRepository` – private readonly repository dependency.

4.31 MilestoneRepository Class

4.31.1 ClassMethod

No	Method	Description
01	`MilestoneRepository( prisma: PrismaService)`	A method to initialize the repository with the Prisma database client. – `prisma`: `PrismaService` – injected Prisma database service. – *Returns:* `void` – constructor, no return value.
02	`findContractById(contract Id: number)`	A method to fetch contract participants, total amount, and status for permission checks. – `contractId`: `number` – contract identifier. – *Returns:* `Promise<Contract | null>` – found contract record or null.
03	`getTotalMilestoneAmount( contractId: number, excludeMilestoneId?: number)`	A method to calculate the cumulative sum of existing non-deleted milestones within a contract. – `contractId`: `number` – contract identifier. – `excludeMilestoneId`: `number | undefined` – optional milestone ID to exclude from sum when updating. – *Returns:* `Promise<number>` – aggregate sum amount.
04	`findMilestoneById( milestoneId: number)`	A method to retrieve a non-deleted milestone record by primary key. – `milestoneId`: `number` – milestone identifier. – *Returns:* `Promise<Milestone | null>` – milestone record or null.
05	`findInProgressMilestone( contractId: number)`	A method to find any active milestone currently with in-progress status in a contract. – `contractId`: `number` – contract identifier. – *Returns:* `Promise<Milestone | null>` – in-progress milestone record if exists.
06	`getMilestoneList( contractId: number, query: GetMilestoneList QueryType)`	A method to execute a transaction fetching paginated milestone records and total count. – `contractId`: `number` – contract identifier. – `query`: `GetMilestoneListQueryType` – page, limit, and status filters. – *Returns:* `Promise<PaginatedResult<Milestone>>` – paginated milestone collection.

No	Method	Description
07	`getMilestoneDetail( milestoneId: number)`	A method to load complete milestone data including associated dispute status. – `milestoneId: number` – milestone identifier. – *Returns:* `Promise<Milestone | null>` – milestone detail object.
08	`createMilestone( contractId: number, data: CreateMilestone BodyType)`	A method to insert a new milestone record into the database. – `contractId: number` – contract identifier. – `data: CreateMilestoneBodyType` – milestone fields. – *Returns:* `Promise<Milestone>` – newly created milestone entity.
09	`updateMilestone( milestoneId: number, data: UpdateMilestone BodyType)`	A method to update specific fields of an existing milestone. – `milestoneId: number` – milestone identifier. – `data: UpdateMilestoneBodyType` – fields to update. – *Returns:* `Promise<Milestone>` – updated milestone entity.
10	`deleteMilestone(milestone Id: number)`	A method to perform a soft-delete by updating the deletedAt timestamp. – `milestoneId: number` – milestone identifier. – *Returns:* `Promise<Milestone>` – soft-deleted milestone record.
11	`progressMilestone( milestoneId: number)`	A method to transition the status of a milestone to in-progress. – `milestoneId: number` – milestone identifier. – *Returns:* `Promise<Milestone>` – updated milestone record.

4.31.2 ClassAttribute

No	Attribute	Type
01	`prisma`	`PrismaService` – private readonly Prisma connection instance.

4.32 MilestoneFileController Class

4.32.1 ClassMethod

No	Method	Description
01	`MilestoneFileController( milestoneFileService: MilestoneFileService)`	A method to initialize the controller with the milestone file service dependency. – `milestoneFileService`: `MilestoneFileService` – milestone file handling service. – *Returns:* `void` – constructor, no return value.
02	`getMilestoneFiles( userId: number, role Name: string, contractId: number, milestoneId: number)`	A method to retrieve all non-deleted files uploaded for a milestone. – `userId`: `number` – caller user identifier. – `roleName`: `string` – role name. – `contractId`: `number` – contract identifier. – `milestoneId`: `number` – milestone identifier. – *Returns:* `Promise<GetMilestoneFilesResponseDT O>` – list of milestone file items. – Route: `GET /contracts/:contractId/milestones/ :milestoneId/files`; authenticated endpoint.
03	`uploadMilestoneFile( userId: number, contract Id: number, milestoneId: number, file: Express.Multer. File)`	A method to accept a multipart file, upload it to cloud storage, and persist the record. – `userId`: `number` – uploader user identifier. – `contractId`: `number` – contract identifier. – `milestoneId`: `number` – milestone identifier. – `file`: `Express.Multer.File` – validated multipart binary uploaded via `FileInterceptor`. – *Returns:* `Promise<UploadMilestoneFileResponseD TO>` – uploaded file metadata. – Route: `POST /contracts/:contractId/ milestones/:milestoneId/files`; authenticated endpoint.

No	Method	Description
04	`deleteMilestoneFile( userId: number, contractId: number, milestoneId: number, fileId: number)`	A method to soft-delete an uploaded milestone file and remove it from cloud storage. – `userId`: `number` – uploader user identifier. – `contractId`: `number` – contract identifier. – `milestoneId`: `number` – milestone identifier. – `fileId`: `number` – file identifier parsed by `ParseIntPipe`. – *Returns:* `Promise<DeleteMilestoneFileResponseDTO>` – deleted file confirmation. – Route: `DELETE /contracts/:contractId/milestones/:milestoneId/files/:fileId`; authenticated endpoint.

4.32.2 ClassAttribute

No	Attribute	Type
01	`milestoneFileService`	`MilestoneFileService` – private readonly milestone file service dependency.

4.33 MilestoneFileService Class

4.33.1 ClassMethod

No	Method	Description
01	`MilestoneFileService( milestoneFileRepository: MilestoneFileRepository, cloudinaryService: CloudinaryService)`	A method to initialize the service with repository and cloud asset dependencies. – `milestoneFileRepository`: `MilestoneFileRepository` – persistence access layer for files. – `cloudinaryService`: `CloudinaryService` – cloud storage adapter for file hosting. – *Returns:* `void` – constructor, no return value.

No	Method	Description
02	`getMilestoneFiles( userId: number, roleName: string, contractId: number, milestoneId: number)`	A method to verify contract participation and retrieve all files linked to the milestone. – `userId: number` – caller user identifier. – `roleName: string` – role name. – `contractId: number` – contract identifier. – `milestoneId: number` – milestone identifier. – *Returns:* `Promise<MilestoneFileType[]>` – array of milestone file objects.
03	`uploadMilestoneFile( userId: number, contractId: number, milestoneId: number, file: Express.Multer.File)`	A method to verify access, stream file to Cloudinary in milestone folder, and persist file metadata. – `userId: number` – uploader user identifier. – `contractId: number` – contract identifier. – `milestoneId: number` – milestone identifier. – `file: Express.Multer.File` – multipart uploaded file. – *Returns:* `Promise<MilestoneFileType>` – saved milestone file entity.
04	`deleteMilestoneFile( userId: number, contractId: number, milestoneId: number, fileId: number)`	A method to ensure uploader ownership, purge asset from Cloudinary, and soft-delete the database record. – `userId: number` – caller user identifier. – `contractId: number` – contract identifier. – `milestoneId: number` – milestone identifier. – `fileId: number` – target file identifier. – *Returns:* `Promise<MilestoneFileType>` – soft-deleted file record.

4.33.2 ClassAttribute

No	Attribute	Type
01	`milestoneFileRepository`	`MilestoneFileRepository` – private readonly repository dependency.
02	`cloudinaryService`	`CloudinaryService` – private readonly Cloudinary storage provider.

4.34 MilestoneFileRepository Class

4.34.1 ClassMethod

No	Method	Description
01	`MilestoneFileRepository( prisma: PrismaService)`	A method to initialize the repository with the Prisma client. – `prisma`: `PrismaService` – Prisma database service. – *Returns:* `void` – constructor, no return value.
02	`findContractById(contract Id: number)`	A method to query contract participants and status. – `contractId`: `number` – contract identifier. – *Returns:* `Promise<Contract | null>` – found contract record or null.
03	`findMilestoneById( milestoneId: number)`	A method to verify milestone existence and parent contract linkage. – `milestoneId`: `number` – milestone identifier. – *Returns:* `Promise<Milestone | null>` – found milestone or null.
04	`findFileById(fileId: number)`	A method to retrieve a specific milestone file record by ID. – `fileId`: `number` – file identifier. – *Returns:* `Promise<MilestoneFile | null>` – file record or null.
05	`getMilestoneFiles( milestoneId: number)`	A method to fetch all non-deleted files for a milestone in descending order. – `milestoneId`: `number` – milestone identifier. – *Returns:* `Promise<MilestoneFile[]>` – list of file records.
06	`createMilestoneFile(data: CreateMilestoneFileData)`	A method to insert a new milestone file record into the database. – `data`: `CreateMilestoneFileData` – milestoneId, uploaderId, fileUrl, publicId, fileName. – *Returns:* `Promise<MilestoneFile>` – created file record.
07	`softDeleteFile(fileId: number)`	A method to record a soft-deletion timestamp on the file record. – `fileId`: `number` – file identifier. – *Returns:* `Promise<MilestoneFile>` – updated file record.

4.34.2 ClassAttribute

No	Attribute	Type
01	`prisma`	`PrismaService` – private readonly Prisma connection instance.

4.35 MilestoneSubmissionController Class

4.35.1 ClassMethod

No	Method	Description
01	`MilestoneSubmission Controller( submissionService: MilestoneSubmission Service)`	A method to initialize the controller with the milestone submission service. – `submissionService: MilestoneSubmission Service` – deliverable submission service. – *Returns:* `void` – constructor, no return value.
02	`submitMilestone( userId: number, role Name: string, contractId: number, milestoneId: number, body: SubmitMilestone BodyDTO)`	A method for the freelancer to submit completed work packages with comments and file attachments. – `userId: number` – freelancer user identifier. – `roleName: string` – caller role name. – `contractId: number` – contract identifier. – `milestoneId: number` – milestone identifier. – `body: SubmitMilestoneBodyDTO` – work description, external links, and uploaded file IDs. – *Returns:* `Promise<SubmitMilestoneResponseDTO>` – submitted deliverable package. – Route: `POST .../milestones/:milestoneId/ submissions`; authenticated endpoint.
03	`getSubmissions( userId: number, role Name: string, contractId: number, milestoneId: number)`	A method to retrieve the complete submission review history for a milestone. – `userId: number` – caller user identifier. – `roleName: string` – caller role name. – `contractId: number` – contract identifier. – `milestoneId: number` – milestone identifier. – *Returns:* `Promise<GetSubmissionsResponseDTO>` – array of milestone submission packages. – Route: `GET .../milestones/:milestoneId/ submissions`; authenticated endpoint.

No	Method	Description
04	`getSubmissionDetail( userId: number, roleName: string, contractId: number, milestoneId: number, submissionId: number)`	A method to inspect full details and attached file evidence of an individual submission. – `userId: number` – caller user identifier. – `roleName: string` – caller role name. – `contractId: number` – contract identifier. – `milestoneId: number` – milestone identifier. – `submissionId: number` – submission identifier. – *Returns:* `Promise<GetSubmissionResponseDTO>` – submission detail entity. – Route: `GET .../milestones/:milestoneId/submissions/:id`; authenticated endpoint.
05	`requestChanges( userId: number, roleName: string, contractId: number, milestoneId: number, submissionId: number, body: RequestChangesBodyDTO)`	A method for the client to reject a submission package and request deliverable revisions with feedback. – `userId: number` – client user identifier. – `roleName: string` – caller role name. – `contractId: number` – contract identifier. – `milestoneId: number` – milestone identifier. – `submissionId: number` – target submission identifier. – `body: RequestChangesBodyDTO` – revision remarks and requested turnaround due date. – *Returns:* `Promise<RequestChangesResponseDTO>` – updated submission with revision status. – Route: `PATCH .../submissions/:id/request-changes`; authenticated endpoint.
06	`approveMilestone( userId: number, roleName: string, contractId: number, milestoneId: number, submissionId: number)`	A method for the client to approve accepted deliverables and mark milestone as completed. – `userId: number` – client user identifier. – `roleName: string` – caller role name. – `contractId: number` – contract identifier. – `milestoneId: number` – milestone identifier. – `submissionId: number` – submission identifier. – *Returns:* `Promise<ApproveMilestoneResponseDTO>` – approved submission and completed milestone. – Route: `PATCH .../submissions/:id/approve`; authenticated endpoint.

4.35.2 ClassAttribute

No	Attribute	Type
01	`submissionService`	`MilestoneSubmissionService` – private readonly submission service dependency.

4.36 MilestoneSubmissionService Class

4.36.1 ClassMethod

No	Method	Description
01	`MilestoneSubmission Service( submissionRepository: MilestoneSubmission Repository)`	A method to initialize the service with the milestone submission repository. – `submissionRepository: MilestoneSubmission Repository` – submission data access layer. – *Returns:* `void` – constructor, no return value.
02	`resolveAndGuardContext( userId: number, role Name: string, contractId: number, milestoneId: number, role: 'participant' | ' freelancer' | 'client')`	A private helper method to validate contract existence, milestone membership, and enforce strict role-based access. – `userId: number` – caller user identifier. – `roleName: string` – user role name. – `contractId: number` – contract identifier. – `milestoneId: number` – milestone identifier. – `role: 'participant' | 'freelancer' | ' client'` – required caller privilege. – *Returns:* `Promise<{contract, milestone}>` – resolved context entities.
03	`submitMilestone( userId: number, role Name: string, contractId: number, milestoneId: number, body: SubmitMilestone BodyType)`	A method to verify that milestone is funded and in-progress, persist the submission package, and transition status to submitted. – `userId: number` – freelancer user identifier. – `roleName: string` – role name. – `contractId: number` – contract identifier. – `milestoneId: number` – milestone identifier. – `body: SubmitMilestoneBodyType` – message, links, and file IDs. – *Returns:* `Promise<MilestoneSubmission>` – created submission record.

No	Method	Description
04	`getSubmission( userId: number, roleName: string, contractId: number, milestoneId: number)`	A method to retrieve all submission attempts and attached deliverables for a milestone. – `userId: number` – caller user identifier. – `roleName: string` – role name. – `contractId: number` – contract identifier. – `milestoneId: number` – milestone identifier. – *Returns:* `Promise<MilestoneSubmission[]>` – list of submission records.
05	`getSubmissionDetail( userId: number, roleName: string, contractId: number, milestoneId: number, submissionId: number)`	A method to load a single submission package with associated file details. – `userId: number` – caller user identifier. – `roleName: string` – role name. – `contractId: number` – contract identifier. – `milestoneId: number` – milestone identifier. – `submissionId: number` – submission identifier. – *Returns:* `Promise<MilestoneSubmission>` – detailed submission entity.
06	`requestChanges( userId: number, roleName: string, contractId: number, milestoneId: number, submissionId: number, body: RequestChangesBodyType)`	A method to verify client ownership, confirm pending review status, save revision requirements, and set milestone to changes requested. – `userId: number` – client user identifier. – `roleName: string` – role name. – `contractId: number` – contract identifier. – `milestoneId: number` – milestone identifier. – `submissionId: number` – submission identifier. – `body: RequestChangesBodyType` – revision remarks and optional deadline. – *Returns:* `Promise<MilestoneSubmission>` – updated submission entity.
07	`approveMilestone( userId: number, roleName: string, contractId: number, milestoneId: number, submissionId: number)`	A method to verify client ownership, confirm pending review status, mark submission as approved, and complete the milestone. – `userId: number` – client user identifier. – `roleName: string` – role name. – `contractId: number` – contract identifier. – `milestoneId: number` – milestone identifier. – `submissionId: number` – submission identifier. – *Returns:* `Promise<MilestoneSubmission>` – approved submission entity.

4.36.2 ClassAttribute

No	Attribute	Type
01	`submissionRepository`	`MilestoneSubmissionRepository` – private readonly repository dependency.

4.37 MilestoneSubmissionRepository Class

4.37.1 ClassMethod

No	Method	Description
01	`MilestoneSubmission Repository(prisma: PrismaService)`	A method to initialize the repository with the Prisma client. – `prisma`: `PrismaService` – Prisma database service. – *Returns:* `void` – constructor, no return value.
02	`findContractById(contract Id: number)`	A method to retrieve contract participant identifiers and status. – `contractId`: `number` – contract identifier. – *Returns:* `Promise<Contract | null>` – contract record or null.
03	`findMilestoneById( milestoneId: number)`	A method to query milestone status, payment status, and parent contract ID. – `milestoneId`: `number` – milestone identifier. – *Returns:* `Promise<Milestone | null>` – milestone record or null.
04	`findSubmissionById( submissionId: number)`	A method to retrieve a submission record by primary key. – `submissionId`: `number` – submission identifier. – *Returns:* `Promise<MilestoneSubmission | null>` – submission record or null.
05	`getSubmissionDetail( milestoneId: number)`	A method to retrieve all submission records with attached file objects for a milestone. – `milestoneId`: `number` – milestone identifier. – *Returns:* `Promise<MilestoneSubmission[]>` – list of submission entities with files.

No	Method	Description
06	`getSubmission(submissionId: number)`	A method to load an individual submission along with all nested file records. – `submissionId: number` – submission identifier. – *Returns:* `Promise<MilestoneSubmission | null>` – detailed submission or null.
07	`createSubmission(milestoneId: number, freelancerId: number, body: SubmitMilestoneBodyType)`	A method to execute an atomic transaction creating the submission, linking file references, and updating milestone status to submitted. – `milestoneId: number` – milestone identifier. – `freelancerId: number` – freelancer identifier. – `body: SubmitMilestoneBodyType` – message, links, and fileIds array. – *Returns:* `Promise<MilestoneSubmission>` – created submission record.
08	`requestChanges(submissionId: number, milestoneId: number, body: RequestChangesBodyType)`	A method to atomically mark submission as changes requested and transition milestone status to changes requested. – `submissionId: number` – submission identifier. – `milestoneId: number` – milestone identifier. – `body: RequestChangesBodyType` – revision remarks and due date. – *Returns:* `Promise<MilestoneSubmission>` – updated submission entity.
09	`approveSubmission(submissionId: number, milestoneId: number)`	A method to atomically mark submission as approved, record reviewed timestamp, and transition milestone to completed. – `submissionId: number` – submission identifier. – `milestoneId: number` – milestone identifier. – *Returns:* `Promise<MilestoneSubmission>` – approved submission entity.

4.37.2 ClassAttribute

No	Attribute	Type
01	`prisma`	`PrismaService` – private readonly Prisma connection instance.

Supporting DTO classes for milestone, file, and submission management

Class / Element	Attributes or Signature	Responsibility
`CreateMilestoneBodyDTO`	`title: string, amount: number, description?: string, dueDate?: string`	Validates milestone creation payload and budget parameters.
`UpdateMilestoneBodyDTO`	`title?: string, amount?: number, description?: string, dueDate?: string`	Validates mutable fields of pending milestones.
`GetMilestoneListQueryDTO`	`page: number, limit: number, status?: MilestoneStatus`	Validates milestone list pagination and status filter queries.
`UploadMilestoneFileResponseDTO`	`id: number, milestoneId: number, fileUrl: string, fileName?: string`	Serializes uploaded milestone attachment response.
`SubmitMilestoneBodyDTO`	`message?: string, links?: string[], fileIds?: number[]`	Validates deliverable submission package and attachment IDs.
`RequestChangesBodyDTO`	`changeRequestMessage: string, changeRequestDueDate?: string`	Validates deliverable revision remarks and target turnaround date.
`ApproveMilestoneResponseDTO`	`id: number, status: string, reviewedAt: Date`	Serializes deliverable approval result and milestone completion state.

4.38 PermissionsController Class

4.38.1 ClassMethod

No	Method	Description
01	`PermissionsController(permissionsService: PermissionsService)`	A method to initialize the controller with the permission service. – `permissionsService`: `PermissionsService` – service that owns permission queries. – *Returns:* `void` – constructor, no return value.

No	Method	Description
02	`getPermissions(query: PermissionFilterDto)`	A method to return a filtered page of route permissions. – `query: PermissionFilterDto` – page, limit, search, method, module, and sort options. – *Returns:* `Promise<PermissionListResponseType>` – permissions, pagination, and module names. – Route: `GET /permissions`.
03	`getPermissionById(id: number)`	A method to return one permission. – `id: number` – permission identifier parsed by `ParseIntPipe`. – *Returns:* `Promise<PermissionDetailResponseType>` – the permission detail. – Route: `GET /permissions/:id`.

4.38.2 ClassAttribute

No	Attribute	Type
01	`permissionsService`	`PermissionsService` – private readonly permission service.

4.39 PermissionsService Class

4.39.1 ClassMethod

No	Method	Description
01	`PermissionsService(permissionsRepository: PermissionsRepository)`	A method to initialize the service with the permission repository. – `permissionsRepository: PermissionsRepository` – persistence dependency. – *Returns:* `void` – constructor, no return value.

No	Method	Description
02	`getPermissions(filter: PermissionFilterType)`	A method to load a permission page and compute pagination metadata. – `filter`: `PermissionFilterType` – page, limit, search, method, module, and sort options. – *Returns:* `Promise<PermissionListResponseType>` – permissions, pagination, and module names. – Failure is mapped to `FailedToLoadPermissions Exception`.
03	`getPermissionById(id: number)`	A method to load one permission. – `id`: `number` – permission identifier. – *Returns:* `Promise<PermissionDetailResponse Type>` – the permission detail. – Prisma errors are mapped to `FailedToLoad PermissionDetailException`.

4.39.2 ClassAttribute

No	Attribute	Type
01	`permissionsRepository`	`PermissionsRepository` – private readonly persistence dependency.
02	`logger`	`Logger` – private readonly NestJS logger.

4.40 PermissionsRepository Class

4.40.1 ClassMethod

No	Method	Description
01	`PermissionsRepository( prisma: PrismaService)`	A method to initialize the repository with the shared Prisma client. – `prisma`: `PrismaService` – database client. – *Returns:* `void` – constructor, no return value.

No	Method	Description
02	`findAll(filter: PermissionFilterType)`	A method to query permissions, the matching total, and the distinct module list. – `filter`: `PermissionFilterType` – page, limit, search, method, module, and sort options. – *Returns:* `Promise<{ permissions: PermissionListItemType[]; total: number; modules: string[] }>` – the page and supporting data. – Search matches name, path, or module case-insensitively.
03	`findById(id: number)`	A method to load one active permission. – `id`: `number` – permission identifier. – *Returns:* `Promise<PermissionDetailResponseType>` – the permission detail. – Throws `PermissionNotFoundException`.

4.40.2 ClassAttribute

No	Attribute	Type
01	`prisma`	`PrismaService` – private readonly database client.

4.41 SessionsController Class

4.41.1 ClassMethod

No	Method	Description
01	`SessionsController(sessionsService: SessionsService)`	A method to initialize the controller with the session service. – `sessionsService`: `SessionsService` – service that owns session queries. – *Returns:* `void` – constructor, no return value.

No	Method	Description
02	`getMySessions( userId: number, sessionId: number, query: SessionFilterDto)`	A method to return the current user's sessions. – `userId: number` – current user. – `sessionId: number` – current session, used to mark the active row. – `query: SessionFilterDto` – page, limit, search, and sort options. – *Returns:* `Promise<SessionListResponseType>` – sessions and pagination. – Route: `GET /sessions`; authenticated endpoint.
03	`getMySessionById( userId: number, sessionId: number, id: number)`	A method to return one owned session. – `userId: number` – current user. – `sessionId: number` – current session, used to mark the active row. – `id: number` – session identifier. – *Returns:* `Promise<SessionDetailResponseType>` – the session detail. – Route: `GET /sessions/:id`; authenticated endpoint.
04	`revokeMySession( userId: number, sessionId: number, id: number)`	A method to revoke one owned session. – `userId: number` – current user. – `sessionId: number` – current session, used to compute `loggedOut`. – `id: number` – session identifier. – *Returns:* `Promise<RevokeSessionResponseType>` – a success message and logout flag. – Route: `DELETE /sessions/:id`; authenticated endpoint.

4.41.2 ClassAttribute

No	Attribute	Type
01	`sessionsService`	`SessionsService` – private readonly session service.

4.42 SessionsService Class

4.42.1 ClassMethod

No	Method	Description
01	`SessionsService(sessionsRepository: SessionsRepository)`	A method to initialize the service with the session repository. – `sessionsRepository: SessionsRepository` – persistence dependency. – *Returns:* `void` – constructor, no return value.
02	`getMySessions(userId, filter, currentSessionId?)`	A method to load a page of the user's sessions. – `userId: number` – current user. – `filter: SessionFilterType` – page, limit, search, and sort options. – `currentSessionId: number | undefined` – session to mark as current. – *Returns:* `Promise<SessionListResponseType>` – sessions and pagination. – Failure is mapped to `FailedToLoadSessionsException`.
03	`getMySessionById(id, userId, currentSessionId?)`	A method to load one session owned by the user. – `id: number` – session identifier. – `userId: number` – current user. – `currentSessionId: number | undefined` – session to mark as current. – *Returns:* `Promise<SessionDetailResponseType>` – the session detail. – Prisma errors are mapped to `FailedToLoadSessionDetailException`.
04	`revokeMySession(id, userId, currentSessionId?)`	A method to delete an active session owned by the user. – `id: number` – session identifier. – `userId: number` – current user. – `currentSessionId: number | undefined` – current session used to compute `loggedOut`. – *Returns:* `Promise<RevokeSessionResponseType>` – a success message and logout flag. – Prisma errors are mapped to `FailedToRevokeSessionException`.

4.42.2 ClassAttribute

No	Attribute	Type
01	`sessionsRepository`	`SessionsRepository` – private readonly persistence dependency.
02	`logger`	`Logger` – private readonly NestJS logger.

4.43 SessionsRepository Class

4.43.1 ClassMethod

No	Method	Description
01	`SessionsRepository( prisma: PrismaService)`	A method to initialize the repository with the shared Prisma client. – `prisma`: `PrismaService` – database client. – *Returns:* `void` – constructor, no return value.
02	`findAllByUserId(userId, filter, currentSessionId? )`	A method to query the user's sessions and mark the current one. – `userId`: `number` – owner identifier. – `filter`: `SessionFilterType` – page, limit, search, and sort options. – `currentSessionId`: `number | undefined` – session to mark as current. – *Returns:* `Promise<{ sessions: SessionListItem Type[]; total: number }>` – the page and total. – Search matches device information or IP address.
03	`findByIdForUser(id, user Id, currentSessionId?)`	A method to load one session owned by the user. – `id`: `number` – session identifier. – `userId`: `number` – owner identifier. – `currentSessionId`: `number | undefined` – session to mark as current. – *Returns:* `Promise<SessionDetailResponseType>` – the session detail. – Throws `SessionNotFoundException`.

No	Method	Description
04	`deleteActiveByIdForUser(id, userId)`	A method to delete an unexpired session owned by the user. – `id`: `number` – session identifier. – `userId`: `number` – owner identifier. – *Returns:* `Promise<void>` – no value. – Throws `SessionNotFoundException` or `SessionAlreadyExpiredException`.

4.43.2 ClassAttribute

No	Attribute	Type
01	`prisma`	`PrismaService` – private readonly database client.

4.44 Forum Category Administration (Shared Classes)

UC-14.01–14.02 and UC-17.01–17.06 use the shared `ForumAdminController`, `ForumAdminServi` and `ForumAdminRepository`. Their complete specifications, including the category list, detail, create, update, delete, and restore endpoints, are given once in the shared forum administration class specifications for Sprint 3 (see the `ForumAdminController` subsection).

4.45 ForumAdminController Class

4.45.1 ClassMethod

No	Method	Description
01	`ForumAdminController(adminService: ForumAdminService)`	A method to initialize the controller with the administration service. – `adminService`: `ForumAdminService` – service dependency. – *Returns:* `void` – constructor, no return value.

No	Method	Description
02	`getAdminStats(roleName: string)`	A method to return the forum dashboard counters. – `roleName: string` – current role name from `@User Active('roleName')`. – *Returns:* `Promise<ForumAdminStatsType>` – category, post, comment, report, pending-report, and user totals plus the five newest posts. – Route: `GET /forums/admin/stats`; administrator endpoint.
03	`getAdminCategoryLists( roleName: string, page: number, limit: number, search: string, deleted: string, sortBy: 'id' | 'name' | 'createdAt', sortOrder: 'asc' | 'desc')`	A method to list categories with search, deleted-state filtering, and sorting. – `roleName: string` – current role name. – `page, limit: number` – coerced with `Number()` and defaulted to 1 and 10. – `search: string` – optional case-insensitive name search. – `deleted: string` – optional `'true'` or `'false'` filter. – `sortBy, sortOrder: 'id' | 'name' | 'createdAt'` and `'asc' | 'desc'` – optional sort controls. – *Returns:* `Promise<ForumAdminCategoryListResponseType>` – categories and pagination. – Route: `GET /forums/admin/categories`; administrator endpoint.
04	`getAdminCategoryById( roleName: string, id: number)`	A method to return one active category. – `roleName: string` – current role name. – `id: number` – category identifier. – *Returns:* `Promise<ForumCategoryDetailResponseType>` – category detail with public post count. – Route: `GET /forums/admin/categories/:id`; administrator endpoint.
05	`createAdminCategory( roleName: string, body: CreateForumCategoryBodyDto)`	A method to create a category. – `roleName: string` – current role name. – `body: CreateForumCategoryBodyDto` – name and optional description. – *Returns:* `Promise<ForumCategoryType>` – the created category. – Route: `POST /forums/admin/categories`; administrator endpoint.

No	Method	Description
06	`updateAdminCategory( roleName: string, id: number, body: UpdateForum CategoryBodyDto)`	A method to rename or re-describe a category. – `roleName: string` – current role name. – `id: number` – category identifier. – `body: UpdateForumCategoryBodyDto` – optional name and optional description. – *Returns:* `Promise<ForumCategoryType>` – the updated category. – Route: `PATCH /forums/admin/categories/:id`; administrator endpoint.
07	`deleteAdminCategory( roleName: string, id: number)`	A method to soft-delete a category and detach its posts. – `roleName: string` – current role name. – `id: number` – category identifier. – *Returns:* `Promise<{ message: string }>` – deletion confirmation. – Route: `DELETE /forums/admin/categories/:id`; administrator endpoint.
08	`restoreAdminCategory( roleName: string, id: number)`	A method to restore a soft-deleted category. – `roleName: string` – current role name. – `id: number` – category identifier. – *Returns:* `Promise<ForumCategoryType>` – the restored category. – Route: `PATCH /forums/admin/categories/:id/ restore`; administrator endpoint.
09	`getAdminCommentLists( roleName: string, page: number, limit: number, search: string)`	A method to list active comments for administration. – `roleName: string` – current role name. – `page, limit: number` – pagination, defaulted to 1 and 10. – `search: string` – optional case-insensitive content search. – *Returns:* `Promise<ForumAdminCommentList ResponseType>` – comments with author and parent-post summaries, plus pagination. – Route: `GET /forums/admin/comments`; administrator endpoint.

No	Method	Description
10	`getPendingPosts( roleName: string, page: number, limit: number)`	A method to list posts waiting for moderation. – `roleName: string` – current role name. – `page, limit: number` – pagination, defaulted to 1 and 10. – *Returns:* `Promise<PendingForumPostListResponse Type>` – non-deleted `PENDING` posts, oldest first, with pagination. – Route: `GET /forums/admin/pending-posts`; administrator endpoint.
11	`getTrashPosts( roleName: string, page: number, limit: number)`	A method to list deleted or rejected posts. – `roleName: string` – current role name. – `page, limit: number` – pagination, defaulted to 1 and 10. – *Returns:* `Promise<ForumTrashPostListResponse Type>` – posts with `deletedAt` set or moderation status `R EJECTED`. – Route: `GET /forums/admin/trash/posts`; administrator endpoint.
12	`getTrashComments( roleName: string, page: number, limit: number)`	A method to list deleted comments. – `roleName: string` – current role name. – `page, limit: number` – pagination, defaulted to 1 and 10. – *Returns:* `Promise<ForumTrashCommentList ResponseType>` – soft-deleted comments with author and parent-post summaries. – Route: `GET /forums/admin/trash/comments`; administrator endpoint.
13	`restoreTrashPost( roleName: string, id: number)`	A method to restore a post from the trash. – `roleName: string` – current role name. – `id: number` – post identifier. – *Returns:* `Promise<ForumPostType>` – the restored post. – Route: `PATCH /forums/admin/trash/posts/:id/ restore`; administrator endpoint.

No	Method	Description
14	`restoreTrashComment(roleName: string, id: number)`	A method to restore a comment from the trash. – `roleName: string` – current role name. – `id: number` – comment identifier. – *Returns:* `Promise<ForumAdminCommentType>` – the restored comment. – Route: `PATCH /forums/admin/trash/comments/:id/restore`; administrator endpoint.
15	`approvePendingPost(roleName: string, userId: number, id: number)`	A method to approve a pending post. – `roleName: string` – current role name. – `userId: number` – reviewing administrator recorded in `reviewedById`. – `id: number` – post identifier. – *Returns:* `Promise<ForumPostType>` – the approved post. – Route: `PATCH /forums/admin/posts/:id/approve`; administrator endpoint.
16	`rejectPendingPost(roleName: string, userId: number, id: number)`	A method to reject a pending post and move it to the trash. – `roleName: string` – current role name. – `userId: number` – reviewing administrator. – `id: number` – post identifier. – *Returns:* `Promise<ForumPostType>` – the rejected post, now soft-deleted. – Route: `PATCH /forums/admin/posts/:id/reject`; administrator endpoint.

4.45.2 ClassAttribute

No	Attribute	Type
01	`adminService`	`ForumAdminService` – private readonly service dependency.

4.46 ForumAdminService Class

4.46.1 ClassMethod

No	Method	Description
01	`ForumAdminService( adminRepository: ForumAdminRepository)`	A method to initialize the service with the administration repository. – `adminRepository`: `ForumAdminRepository` – persistence dependency. – *Returns:* `void` – constructor, no return value.
02	`getAdminStats(roleName: string)`	A method to authorize an administrator and load dashboard counters. – `roleName`: `string` – must equal `RoleName.ADMIN`. – *Returns:* `Promise<ForumAdminStatsType>` – counters and the five newest posts. – Throws `ForumReportForbiddenException` for a non-admin or a repository failure.
03	`getAdminCommentLists( roleName: string, page: number, limit: number, search?: string)`	A method to authorize an administrator and return a page of active comments. – `roleName`: `string` – must equal `RoleName.ADMIN`. – `page`, `limit`: `number` – pagination. – `search`: `string | undefined` – optional content filter. – *Returns:* `Promise<ForumAdminCommentList ResponseType>` – comments and pagination with `total Pages`.
04	`getAdminCategoryLists( roleName: string, page: number, limit: number, search?: string, sortBy?: 'id' | 'name' | 'createdAt', sortOrder?: 'asc' | 'desc', deleted?: string)`	A method to authorize an administrator and return a page of categories. – `roleName`: `string` – must equal `RoleName.ADMIN`. – `page`, `limit`: `number` – pagination. – `search`: `string | undefined` – optional name filter. – `sortBy`, `sortOrder`: optional sort controls; repository defaults are `id` and `desc`. – `deleted`: `string | undefined` – `'true'` lists only deleted, `'false'` only active. – *Returns:* `Promise<ForumAdminCategoryList ResponseType>` – categories and pagination.

No	Method	Description
05	`getAdminCategoryById( roleName: string, categoryId: number)`	A method to authorize an administrator and load one active category. – `roleName: string` – must equal `RoleName.ADMIN`. – `categoryId: number` – category identifier. – *Returns:* `Promise<ForumCategoryDetailResponse Type>` – category detail. – Throws `ForumCategoryNotFoundException` when absent, or `ForumReportForbiddenException` for other failures.
06	`createAdminCategory( roleName: string, body: CreateForum CategoryBodyType)`	A method to authorize an administrator and create a category. – `roleName: string` – must equal `RoleName.ADMIN`. – `body: CreateForumCategoryBodyType` – name and optional description. – *Returns:* `Promise<ForumCategoryType>` – the created category. – Unknown repository errors are rethrown unchanged.
07	`updateAdminCategory( roleName: string, categoryId: number, body: UpdateForum CategoryBodyType)`	A method to authorize an administrator and update a category. – `roleName: string` – must equal `RoleName.ADMIN`. – `categoryId: number` – category identifier. – `body: UpdateForumCategoryBodyType` – optional name and description. – *Returns:* `Promise<ForumCategoryType>` – the updated category. – Unknown repository errors are rethrown unchanged.
08	`deleteAdminCategory( roleName: string, categoryId: number)`	A method to authorize an administrator and soft-delete a category. – `roleName: string` – must equal `RoleName.ADMIN`. – `categoryId: number` – category identifier. – *Returns:* `Promise<{ message: string }>` – deletion confirmation. – Throws `FailedToDeleteForumCategoryException` for an unexpected failure.

No	Method	Description
09	`restoreAdminCategory( roleName: string, categoryId: number)`	A method to authorize an administrator and restore a deleted category. – `roleName: string` – must equal `RoleName.ADMIN`. – `categoryId: number` – category identifier. – *Returns:* `Promise<ForumCategoryType>` – the restored category. – Throws `FailedToRestoreForumCategoryException` for an unexpected failure.
10	`getPendingPosts( roleName: string, page: number, limit: number)`	A method to authorize an administrator and load the moderation queue. – `roleName: string` – must equal `RoleName.ADMIN`. – `page, limit: number` – pagination. – *Returns:* `Promise<PendingForumPostListResponseType>` – pending posts and pagination. – Throws `ForumReportForbiddenException` for a non-admin or a repository failure.
11	`reviewPendingPost( roleName: string, postId: number, status: 'APPROVED' | 'REJECTED', adminId: number)`	A method to authorize an administrator and apply a moderation decision. – `roleName: string` – must equal `RoleName.ADMIN`. – `postId: number` – post identifier. – `status: 'APPROVED' | 'REJECTED'` – decision passed by the approve/reject routes. – `adminId: number` – reviewing administrator. – *Returns:* `Promise<ForumPostType>` – the reviewed post. – A Prisma `P2025` error is mapped to `ForumPostNotFoundException`; other Prisma errors become `FailedToReviewForumPostException`.
12	`getTrashPosts( roleName: string, page: number, limit: number)`	A method to authorize an administrator and load the post trash. – `roleName: string` – must equal `RoleName.ADMIN`. – `page, limit: number` – pagination. – *Returns:* `Promise<ForumTrashPostListResponseType>` – deleted or rejected posts and pagination. – Throws `ForumReportForbiddenException` for a non-admin or a repository failure.

No	Method	Description
13	`getTrashComments( roleName: string, page: number, limit: number)`	A method to authorize an administrator and load the comment trash. – `roleName: string` – must equal `RoleName.ADMIN`. – `page, limit: number` – pagination. – *Returns:* `Promise<ForumTrashCommentList ResponseType>` – deleted comments and pagination. – Throws `ForumReportForbiddenException` for a non-admin or a repository failure.
14	`restoreTrashPost( roleName: string, post Id: number)`	A method to authorize an administrator and restore a post. – `roleName: string` – must equal `RoleName.ADMIN`. – `postId: number` – post identifier. – *Returns:* `Promise<ForumPostType>` – the restored post. – Throws `FailedToRestoreForumPostException` for an unexpected failure.
15	`restoreTrashComment( roleName: string, commentId: number)`	A method to authorize an administrator and restore a comment. – `roleName: string` – must equal `RoleName.ADMIN`. – `commentId: number` – comment identifier. – *Returns:* `Promise<ForumAdminCommentType>` – the restored comment. – A Prisma `P2025` error is mapped to `ForumCommentNot FoundException`; other Prisma errors become `FailedTo RestoreForumCommentException`.

4.46.2 ClassAttribute

No	Attribute	Type
01	`adminRepository`	`ForumAdminRepository` – private readonly persistence dependency.

4.47 ForumAdminRepository Class

4.47.1 ClassMethod

No	Method	Description
01	`ForumAdminRepository( prisma: PrismaService)`	A method to initialize the repository with the shared Prisma service. – `prisma`: `PrismaService` – database client. – *Returns:* `void` – constructor, no return value.
02	`getPendingPosts(page: number, limit: number)`	A method to query the moderation queue and its total in one transaction. – `page, limit`: `number` – pagination. – *Returns:* `Promise<{ posts: PendingForumPostType[]; total: number }>` – non-deleted `PENDING` posts, oldest first, with author summary. – `moderationCategories` is normalized to `string[] | null`.
03	`reviewForumPost( postId: number, status: 'APPROVED' | 'REJECTED', adminId: number)`	A method to apply a moderation decision only to a non-deleted `PENDING` post. – `postId`: `number` – post identifier. – `status`: `'APPROVED' | 'REJECTED'` – target moderation status. – `adminId`: `number` – reviewer recorded in `reviewedById`. – *Returns:* `Promise<ForumPostType>` – the reviewed post. – `REJECTED` also sets `deletedAt`; a zero-row update throws `ForumPostNotFoundException`.
04	`getAdminStats()`	A method to count forum entities and load the five newest posts in one transaction. – *Returns:* `Promise<ForumAdminStatsType>` – active category, post, comment, and user counts; total and pending reports; five recent posts. – Posts, comments, categories, and users exclude soft-deleted rows; report totals include all rows.
05	`getAdminCommentLists( page: number, limit: number, search?: string)`	A method to query a page of active comments and the matching total in one transaction. – `page, limit`: `number` – pagination. – `search`: `string | undefined` – optional case-insensitive content filter. – *Returns:* `Promise<{ comments: ForumAdminCommentType[]; total: number }>` – comments with author and parent-post summaries, newest first.

No	Method	Description
06	`getAdminCategoryLists( page: number, limit: number, search?: string, sortBy?: 'id' | 'name' | 'createdAt', sortOrder?: 'asc' | 'desc', deleted?: string)`	A method to query a page of categories and the matching total in one transaction. – `page`, `limit`: `number` – pagination. – `search`: `string | undefined` – optional case-insensitive name filter. – `sortBy`, `sortOrder`: optional sort controls; defaults are `'id'` and `'desc'`. – `deleted`: `string | undefined` – `'true'` selects only deleted rows, `'false'` only active rows, omitted selects both. – *Returns:* `Promise<{ categories: ForumAdminCategoryType[]; total: number }>` – categories with `postCount` and pagination total.
07	`getAdminCategoryById(id: number)`	A method to load one active category with its public post count. – `id`: `number` – category identifier. – *Returns:* `Promise<ForumCategoryType | null>` – the category detail or `null`.
08	`createAdminCategory(data: { name: string; description?: string | null })`	A method to create a category with a unique name and generated slug. – `data.name`: `string` – category name. – `data.description`: `string | null | undefined` – optional description. – *Returns:* `Promise<ForumCategoryType>` – the created category with `postCount = 0`. – Throws `ForumCategoryAlreadyExistsException` for a case-insensitive active-name conflict; slug collisions receive a numeric suffix.
09	`updateAdminCategory( id: number, data: UpdateForumCategoryBodyType)`	A method to update a category and regenerate its slug when the name changes. – `id`: `number` – category identifier. – `data`: `UpdateForumCategoryBodyType` – optional name and description. – *Returns:* `Promise<ForumCategoryType>` – the updated category. – Throws `ForumCategoryNotFoundException` or `ForumCategoryAlreadyExistsException`.

No	Method	Description
10	`deleteAdminCategory(id: number)`	A method to detach posts and soft-delete a category in one transaction. – `id: number` – category identifier. – *Returns:* `Promise<{ message: string }>` – deletion confirmation. – Non-deleted posts in the category are reassigned to `categoryId = null` before the category receives `deletedAt`.
11	`restoreAdminCategory(id: number)`	A method to restore a soft-deleted category after a name-conflict check. – `id: number` – category identifier. – *Returns:* `Promise<ForumCategoryType>` – the restored category. – Throws `ForumCategoryNotFoundException` or `ForumCategoryAlreadyExistsException`.
12	`getTrashPosts(page: number, limit: number)`	A method to query deleted or rejected posts and the matching total in one transaction. – `page, limit: number` – pagination. – *Returns:* `Promise<{ posts: ForumTrashPostType[]; total: number }>` – trash posts with `deletedAt` and author summary. – Ordering is by `deletedAt` descending with null values last.
13	`getTrashComments(page: number, limit: number)`	A method to query soft-deleted comments and the matching total in one transaction. – `page, limit: number` – pagination. – *Returns:* `Promise<{ comments: ForumTrashCommentType[]; total: number }>` – trash comments with `deletedAt` and parent-post summary, newest deletion first.
14	`restoreTrashPost(postId: number)`	A method to restore a post from the trash. – `postId: number` – post identifier. – *Returns:* `Promise<ForumPostType>` – the restored post. – A deleted `APPROVED` post becomes visible again; a `REJECTED` post returns to `PENDING` and its reviewer fields are cleared.

No	Method	Description
15	`restoreTrashComment( commentId: number)`	A method to clear `deletedAt` on a trashed comment. – `commentId`: `number` – comment identifier. – *Returns:* `Promise<ForumAdminCommentType>` – the restored comment with author and parent-post summaries. – Throws `ForumCommentNotFoundException` when no deleted row matches.

4.47.2 ClassAttribute

No	Attribute	Type
01	`prisma`	`PrismaService` – private readonly database client.

4.48 IdentityVerificationsAdminController Class

4.48.1 ClassMethod

No	Method	Description
01	`IdentityVerifications AdminController( service: Identity VerificationsAdmin Service)`	A method to initialize the controller with the administration service. – `service`: `IdentityVerificationsAdminService` – service dependency. – *Returns:* `void` – constructor, no return value.
02	`list(query: Identity VerificationAdminFilter Dto)`	A method to return a filtered, paginated document list. – `query`: `IdentityVerificationAdminFilterDto` – page, limit, optional status, and optional search. – *Returns:* `Promise<IdentityVerificationAdmin ListResponseType>` – documents and pagination. – Route: `GET /admin/identity-verifications`.

No	Method	Description
03	`detail(id: number)`	A method to return one document with user and reviewer detail. – `id: number` – document identifier parsed by `ParseIntPipe`. – *Returns:* `Promise<IdentityVerificationAdminDetailType>` – document, user profile, freelancer verification state, and reviewing administrator. – Route: `GET /admin/identity-verifications/:id`.
04	`file(id: number, side: string | undefined, response: Response)`	A method to stream a legacy `local://` document image. – `id: number` – document identifier. – `side: string | undefined` – `'back'` selects the back image; any other value selects the front. – `response: Response` – Express response used to set headers. – *Returns:* `Promise<StreamableFile>` – file stream; sets inline `Content-Disposition` and `application/octet-stream`. – Route: `GET /admin/identity-verifications/:id/file`.
05	`approve(id: number, adminId: number, body: ApproveIdentityVerificationBodyDto)`	A method to approve a pending document. – `id: number` – document identifier. – `adminId: number` – reviewing administrator from `@UserActive('userId')`. – `body: ApproveIdentityVerificationBodyDto` – optional `reviewNotes` up to 1000 characters. – *Returns:* `Promise<IdentityVerificationAdminActionResponseType>` – the reviewed document detail. – Route: `PATCH /admin/identity-verifications/:id/approve`.
06	`reject(id: number, adminId: number, body: RejectIdentityVerificationBodyDto)`	A method to reject a pending document. – `id: number` – document identifier. – `adminId: number` – reviewing administrator. – `body: RejectIdentityVerificationBodyDto` – optional trimmed `reviewNotes` up to 1000 characters. – *Returns:* `Promise<IdentityVerificationAdminActionResponseType>` – the reviewed document detail. – Route: `PATCH /admin/identity-verifications/:id/reject`.

4.48.2 ClassAttribute

No	Attribute	Type
01	`service`	`IdentityVerificationsAdmin Service` – private readonly service dependency.

4.49 IdentityVerificationsAdminService Class

4.49.1 ClassMethod

No	Method	Description
01	`IdentityVerifications AdminService( repository: IdentityVerificationsAdmin Repository)`	A method to initialize the service with the identity-verification repository. – `repository`: `IdentityVerificationsAdmin Repository` – persistence dependency. – *Returns:* `void` – constructor, no return value.
02	`presentFileUrl(document: T)`	A private generic method to replace legacy `local://` file paths with API URLs. – `document`: `T` – object with `id`, `fileUrl`, and optional `backFileUrl`. – *Returns:* the same object with `fileUrl` set to the file route and, when needed, `backFileUrl` set to the same route with `?side=back`. – Non-local URLs are returned unchanged.
03	`presentList(documents: T[])`	A private method to map `presentFileUrl` over a document list. – `documents`: `T[]` – list of document objects. – *Returns:* the mapped list.
04	`list(params: { page: number; limit: number; status?: string; search?: string })`	A method to load a filtered page of documents. – `params`: object – page, limit, optional status string, and optional search. – *Returns:* `Promise<IdentityVerificationAdmin ListResponseType>` – documents with presentation URLs and pagination. – `status` is cast to the Prisma `VerificationStatus` union before the repository call.

No	Method	Description
05	`detail(id: number)`	A method to load one document and present its file URLs. – `id: number` – document identifier. – *Returns:* `Promise<IdentityVerificationAdminDetailType>` – the document detail. – Throws `IdentityVerificationNotFoundException`.
06	`review( id: number, adminId: number, status: ReviewableStatus, reviewNotes: string | null)`	A method to approve or reject a pending document. – `id: number` – document identifier. – `adminId: number` – reviewing administrator. – `status: ReviewableStatus` – `APPROVED` or `REJECTED`. – `reviewNotes: string | null` – optional reviewer note. – *Returns:* `Promise<IdentityVerificationAdminActionResponseType>` – the reviewed document detail. – Throws `IdentityVerificationNotFoundException` or `IdentityVerificationAlreadyReviewedException`.
07	`getLocalFile( id: number, side: 'front' | 'back' = 'front' )`	A method to resolve and validate a legacy local document file. – `id: number` – document identifier. – `side: 'front' | 'back'` – which stored URL to use; default front. – *Returns:* `Promise<{ absolutePath: string; fileName: string }>` – resolved path and base filename. – Throws `IdentityVerificationNotFoundException` or `IdentityVerificationFileInvalidException`; rejects paths escaping the uploads root.

4.49.2 ClassAttribute

No	Attribute	Type
01	`repository`	`IdentityVerificationsAdminRepository` – private readonly persistence dependency.

4.50 IdentityVerificationsAdminRepository Class

4.50.1 ClassMethod

No	Method	Description
01	`IdentityVerifications AdminRepository( prisma: PrismaService)`	A method to initialize the repository with the shared Prisma service. – `prisma`: `PrismaService` – database client. – *Returns:* `void` – constructor, no return value.
02	`findDocuments(params: IdentityVerificationAdmin ListParams)`	A method to query a filtered page of active documents and the matching total. – `params`: `IdentityVerificationAdminListParams` – page, limit, optional status, and optional search. – *Returns:* `Promise<{ documents: IdentityVerificationAdminListItemType[]; total: number }>` – documents with user summary, newest first. – Search matches user email or profile display name case-insensitively; `deletedAt` must be `null`.
03	`findDocumentById(id: number)`	A method to load the full detail projection of an active document. – `id`: `number` – document identifier. – *Returns:* `Promise<IdentityVerificationAdmin DetailType | null>` – document, reviewer, user, and freelancer summary, or `null`.
04	`findDocumentRaw(id: number)`	A method to load the minimal fields needed for review and file streaming. – `id`: `number` – document identifier. – *Returns:* `Promise<{ id: number; userId: number; fileUrl: string; backFileUrl: string | null; status: VerificationStatus } | null>` – raw document fields, or `null`.

No	Method	Description
05	`reviewDocument( id: number, adminId: number, status: VerificationStatus, reviewNotes: string | null)`	A method to persist a review and synchronize the freelancer verification flag in one transaction. – `id`: `number` – document identifier. – `adminId`: `number` – reviewing administrator. – `status`: `VerificationStatus` – `APPROVED` or `REJECTED`. – `reviewNotes`: `string | null` – reviewer note. – *Returns:* `Promise<IdentityVerificationAdminDetailType | null>` – the updated detail, or `null`. – Updates the document status, reviewer, notes, and `reviewedAt`; sets `FreelancerProfile.idVerified` to true only for `APPROVED`. Transaction limits are 10s wait and 15s timeout.

4.50.2 ClassAttribute

No	Attribute	Type
01	`prisma`	`PrismaService` – private readonly database client.

4.51 DisputeController Class

4.51.1 ClassMethod

No	Method	Description
01	`DisputeController( disputeService: DisputeService)`	A method to initialize the controller with the dispute service dependency. – `disputeService`: `DisputeService` – service encapsulating arbitration business logic. – *Returns:* `void` – constructor, no return value.

No	Method	Description
02	`listMyDisputes( userId: number, query: GetDisputeList QueryDTO)`	A method to retrieve a paginated list of disputes in which the current user is an initiator or respondent. – `userId: number` – caller user identifier injected by `@ UserActive('userId')`. – `query: GetDisputeListQueryDTO` – search keyword, status filter, page, and limit. – *Returns:* `Promise<DisputeListResponseDTO>` – user's dispute cases with pagination metadata. – Route: `GET /disputes`; authenticated endpoint.
03	`createDispute( userId: number, body: CreateDisputeBodyD TO)`	A method for a contract participant to open an official dispute case on an eligible funded milestone. – `userId: number` – user opening the dispute. – `body: CreateDisputeBodyDTO` – milestoneId, dispute reason, detailed narrative, and evidence file references. – *Returns:* `Promise<DisputeDetailResponseDTO>` – created dispute record with fee assessment. – Route: `POST /disputes`; authenticated endpoint.
04	`payFee( userId: number, dispute Id: number, body: PayDisputeFeeBodyD TO)`	A method for a disputing party to record payment of their assigned arbitration fee. – `userId: number` – user paying the fee. – `disputeId: number` – target dispute case identifier parsed by `ParseIntPipe`. – `body: PayDisputeFeeBodyDTO` – payment method reference and payment intent identifier. – *Returns:* `Promise<{success: boolean, fee: DisputeFee}>` – updated fee status. – Route: `POST /disputes/:id/fee`; authenticated endpoint.
05	`submitResponse( userId: number, dispute Id: number, body: SubmitDispute ResponseBodyDTO)`	A method for the respondent to submit their formal rebuttal explanation and supporting evidence. – `userId: number` – respondent user identifier. – `disputeId: number` – dispute identifier. – `body: SubmitDisputeResponseBodyDTO` – defense description and counter-evidence file IDs. – *Returns:* `Promise<DisputeDetailResponseDTO>` – updated dispute with submitted response. – Route: `POST /disputes/:id/response`; authenticated endpoint.

No	Method	Description
06	`getDisputeByMilestoneId( userId: number, roleName: string, milestoneId: number)`	A method to retrieve dispute information associated with a specific contract milestone. – `userId: number` – caller user identifier. – `roleName: string` – caller role name. – `milestoneId: number` – milestone identifier. – *Returns:* `Promise<DisputeDetailResponseDTO>` – dispute dossier entity. – Route: `GET /disputes/milestone/:milestoneId`; authenticated endpoint.
07	`getDisputeDetail( userId: number, roleName: string, disputeId: number)`	A method to retrieve complete dispute dossier including claims, evidence, fees, and arbitrator decisions. – `userId: number` – caller user identifier. – `roleName: string` – caller role name. – `disputeId: number` – dispute case identifier. – *Returns:* `Promise<DisputeDetailResponseDTO>` – comprehensive dispute details. – Route: `GET /disputes/:id`; authenticated endpoint.
08	`submitDecisionReview( userId: number, disputeId: number, body: ReviewDisputeDecisionBodyDTO)`	A method for either disputing party to accept or contest an arbitrator's proposed split resolution. – `userId: number` – participant submitting decision review. – `disputeId: number` – dispute identifier. – `body: ReviewDisputeDecisionBodyDTO` – response flag (ACCEPTED/REJECTED) and objection justification. – *Returns:* `Promise<DisputeDetailResponseDTO>` – updated dispute or finalized settlement. – Route: `POST /disputes/:id/review`; authenticated endpoint.

4.51.2 ClassAttribute

No	Attribute	Type
01	`disputeService`	`DisputeService` – private readonly dispute service dependency.

4.52 DisputeAdminController Class

4.52.1 ClassMethod

No	Method	Description
01	`DisputeAdminController( disputeService: DisputeService)`	A method to initialize the administrative controller with the dispute service dependency. – `disputeService`: `DisputeService` – dispute service handling administrator actions. – *Returns:* `void` – constructor, no return value.
02	`listDisputes( roleName: string, query: GetDisputeListQueryDTO)`	A method for platform administrators to list all system disputes with status and date filtering. – `roleName`: `string` – administrator role name verified via `@UserActive('roleName')`. – `query`: `GetDisputeListQueryDTO` – pagination parameters and dispute status filters. – *Returns:* `Promise<DisputeListResponseDTO>` – full dispute list and count. – Route: `GET /admin/disputes`; administrator-only endpoint.
03	`getDisputeDetail( userId: number, roleName: string, disputeId: number)`	A method for administrators to examine all dispute claims, submissions, and payment logs. – `userId`: `number` – administrator user identifier. – `roleName`: `string` – role name. – `disputeId`: `number` – dispute identifier. – *Returns:* `Promise<DisputeDetailResponseDTO>` – comprehensive dispute dossier. – Route: `GET /admin/disputes/:id`; administrator-only endpoint.

No	Method	Description
04	`adminMakeDecision( adminId: number, roleName: string, disputeId: number, body: AdminDisputeDecisionBodyDTO)`	A method for an arbitrator to issue the initial proposed fund split between client and freelancer. – `adminId: number` – presiding administrator identifier. – `roleName: string` – caller role name. – `disputeId: number` – dispute identifier. – `body: AdminDisputeDecisionBodyDTO` – freelancerAmount, clientAmount, and legal reasoning. – *Returns:* `Promise<DisputeDetailResponseDTO>` – dispute transitioned to DECISION_MADE. – Route: `POST /admin/disputes/:id/decision`; administrator-only endpoint.
05	`adminFinalDecision( adminId: number, roleName: string, disputeId: number, body: AdminDisputeFinalDecisionBodyDTO)`	A method for an arbitrator to issue a binding final decision when a party rejected the initial proposal. – `adminId: number` – presiding administrator identifier. – `roleName: string` – caller role name. – `disputeId: number` – dispute identifier. – `body: AdminDisputeFinalDecisionBodyDTO` – final fund distribution and binding explanation. – *Returns:* `Promise<DisputeDetailResponseDTO>` – finalized dispute triggering financial settlement. – Route: `POST /admin/disputes/:id/final-decision`; administrator-only endpoint.

4.52.2 ClassAttribute

No	Attribute	Type
01	`disputeService`	`DisputeService` – private readonly dispute service dependency.

4.53 DisputeService Class

4.53.1 ClassMethod

No	Method	Description
01	`DisputeService( disputeRepo: DisputeRepository)`	A method to initialize the service with the dispute repository dependency. – `disputeRepo`: `DisputeRepository` – database repository for dispute operations. – *Returns:* `void` – constructor, no return value.
02	`createDispute( userId: number, body: CreateDisputeBodyType)`	A method to validate participant ownership, verify milestone funding, enforce evidence limits, and initialize dispute case with arbitration fee schedules. – `userId`: `number` – initiator identifier. – `body`: `CreateDisputeBodyType` – milestone ID, reason, narrative, and attached files. – *Returns:* `Promise<DisputeDetailType>` – created dispute entity.
03	`payFee( userId: number, disputeId: number, body: PayDisputeFeeBodyType)`	A method to verify participant authorization, check pending fee status, and record paid fee status. – `userId`: `number` – payer user identifier. – `disputeId`: `number` – dispute case identifier. – `body`: `PayDisputeFeeBodyType` – payment method reference. – *Returns:* `Promise<{success: boolean, fee: DisputeFee}>` – paid fee confirmation.
04	`submitResponse( userId: number, disputeId: number, body: SubmitDisputeResponseBodyType)`	A method to ensure only the respondent can reply, verify dispute status, enforce single-response policy, and store rebuttal evidence. – `userId`: `number` – respondent user identifier. – `disputeId`: `number` – dispute identifier. – `body`: `SubmitDisputeResponseBodyType` – rebuttal text and evidence file IDs. – *Returns:* `Promise<DisputeDetailType>` – updated dispute entity.
05	`getDisputeDetail( userId: number, roleName: string, disputeId: number)`	A method to verify access rights, detect overdue unpaid arbitration fees, and return complete case data. – `userId`: `number` – caller user identifier. – `roleName`: `string` – role name. – `disputeId`: `number` – dispute identifier. – *Returns:* `Promise<DisputeDetailType>` – dispute dossier entity.

No	Method	Description
06	`getDisputeByMilestoneId( userId: number, roleName: string, milestoneId: number)`	A method to look up dispute case linked to a specific milestone with permission validation. – `userId: number` – caller user identifier. – `roleName: string` – role name. – `milestoneId: number` – milestone identifier. – *Returns:* `Promise<DisputeDetailType>` – dispute entity.
07	`listMyDisputes( userId: number, query: GetDisputeListQueryType)`	A method to return all disputes where caller is either initiator or defendant. – `userId: number` – user identifier. – `query: GetDisputeListQueryType` – pagination and filter options. – *Returns:* `Promise<PaginatedResult<Dispute>>` – paginated dispute list.
08	`adminListDisputes( roleName: string, query: GetDisputeListQueryType)`	A method to enforce admin privileges and return system-wide dispute cases. – `roleName: string` – caller role name. – `query: GetDisputeListQueryType` – status filter and pagination. – *Returns:* `Promise<PaginatedResult<Dispute>>` – all system disputes.
09	`adminMakeDecision( adminId: number, roleName: string, disputeId: number, body: AdminDisputeDecisionBodyType)`	A method to validate that the fund split equals the milestone total, record proposed split amounts, and set status to DECISION_MADE. – `adminId: number` – administrator identifier. – `roleName: string` – caller role name. – `disputeId: number` – dispute identifier. – `body: AdminDisputeDecisionBodyType` – freelancer and client distribution. – *Returns:* `Promise<DisputeDetailType>` – updated dispute with proposal.

No	Method	Description
10	`submitDecisionReview( userId: number, disputeId: number, body: ReviewDisputeDecisionBodyType)`	A method to process party feedback: if rejected, moves dispute to REVIEW_REQUESTED; if both accept, automatically executes finalization. – `userId: number` – participant identifier. – `disputeId: number` – dispute identifier. – `body: ReviewDisputeDecisionBodyType` – accept/reject decision and reason. – *Returns:* `Promise<DisputeDetailType>` – dispute status after review.
11	`adminFinalDecision( adminId: number, roleName: string, disputeId: number, body: AdminDisputeFinalDecisionBodyType)`	A method for the administrator to render a final, unappealable decision and trigger payout execution. – `adminId: number` – administrator identifier. – `roleName: string` – role name. – `disputeId: number` – dispute identifier. – `body: AdminDisputeFinalDecisionBodyType` – final amounts and justification. – *Returns:* `Promise<DisputeDetailType>` – finalized dispute.
12	`executeFinalization( dispute: Dispute, freelancerAmount: number, clientAmount: number, decisionReason: string, adminId?: number)`	A private helper method to atomically record dispute finalization and transition milestone status. – `dispute: Dispute` – dispute record being finalized. – `freelancerAmount: number` – final freelancer entitlement. – `clientAmount: number` – final client refund entitlement. – `decisionReason: string` – recorded legal summary. – `adminId: number | undefined` – deciding admin ID. – *Returns:* `Promise<Dispute>` – updated dispute entity.

4.53.2 ClassAttribute

No	Attribute	Type
01	`disputeRepo`	`DisputeRepository` – private readonly repository dependency.
02	`logger`	`Logger` – private readonly NestJS system logger.

4.54 DisputeRepository Class

4.54.1 ClassMethod

No	Method	Description
01	`DisputeRepository(prisma: PrismaService)`	A method to initialize the repository with the Prisma client. – `prisma`: `PrismaService` – database service instance. – *Returns:* `void` – constructor, no return value.
02	`findMilestoneWithContract(milestoneId: number)`	A method to fetch a milestone along with its parent contract and existing dispute. – `milestoneId`: `number` – milestone identifier. – *Returns:* `Promise<MilestoneWithContract | null>` – milestone with contract or null.
03	`findDisputeById(disputeId: number)`	A method to query full dispute entity with claims, fees, evidence, and milestone data. – `disputeId`: `number` – dispute identifier. – *Returns:* `Promise<DisputeDetail | null>` – complete dispute record.
04	`findDisputeByMilestoneId(milestoneId: number)`	A method to query the active dispute record for a given milestone. – `milestoneId`: `number` – milestone identifier. – *Returns:* `Promise<DisputeDetail | null>` – dispute record or null.
05	`findSharedFilesByIds(fileIds: number[], contractId: number)`	A method to verify that referenced evidence files exist within the contract boundary. – `fileIds`: `number[]` – array of file identifiers. – `contractId`: `number` – contract identifier. – *Returns:* `Promise<SharedFile[]>` – valid file records.
06	`createDisputeTransaction(data: CreateDisputeTransactionData)`	A method to atomically insert dispute record, initialize arbitration fees for both parties, and attach initial evidence. – `data`: `CreateDisputeTransactionData` – dispute parameters and evidence files. – *Returns:* `Promise<Dispute>` – created dispute entity.

No	Method	Description
07	`findFeeByDisputeAndUser( disputeId: number, user Id: number)`	A method to retrieve the arbitration fee invoice for a specific party in a dispute. – `disputeId`: `number` – dispute identifier. – `userId`: `number` – party user identifier. – *Returns:* `Promise<DisputeFee | null>` – dispute fee record.
08	`updateFeeStatus( feeId: number, status: DisputeFeeStatus, paymentReference?: string, paidAt?: Date)`	A method to update fee payment state and record payment gateway transaction reference. – `feeId`: `number` – fee identifier. – `status`: `DisputeFeeStatus` – new fee status (e.g. PAID). – `paymentReference`: `string | undefined` – payment tracking identifier. – `paidAt`: `Date | undefined` – payment timestamp. – *Returns:* `Promise<DisputeFee>` – updated fee entity.
09	`markFeeOverdue(feeId: number)`	A method to set an expired unpaid fee to OVERDUE status. – `feeId`: `number` – fee identifier. – *Returns:* `Promise<DisputeFee>` – updated fee entity.
10	`hasUserSubmittedEvidence( disputeId: number, user Id: number, type: string)`	A method to check whether a participant has already submitted evidence for a specific dispute phase. – `disputeId`: `number` – dispute identifier. – `userId`: `number` – user identifier. – `type`: `string` – submission stage (e.g. RESPONSE). – *Returns:* `Promise<boolean>` – true if already submitted.
11	`submitResponse Transaction( disputeId: number, respondentId: number, description: string, evidenceFiles?: Dispute EvidenceFileInput[])`	A method to atomically persist respondent statement, save evidence attachments, and transition dispute status to UNDER_REVIEW. – `disputeId`: `number` – dispute identifier. – `respondentId`: `number` – respondent user ID. – `description`: `string` – response argument text. – `evidenceFiles`: `DisputeEvidenceFileInput[] | undefined` – evidence files. – *Returns:* `Promise<Dispute>` – updated dispute record.

No	Method	Description
12	`listDisputes( query: GetDisputeList QueryType, userId?: number)`	A method to retrieve paginated disputes with status filtering and optional participant scoping. – `query`: `GetDisputeListQueryType` – pagination parameters. – `userId`: `number | undefined` – optional user ID filter. – *Returns:* `Promise<PaginatedResult<Dispute>>` – paginated dispute cases.
13	`makeDecisionTransaction( disputeId: number, admin Id: number, freelancerAmount: number, clientAmount: number, decisionReason: string, clientId: number, freelancerId: number)`	A method to record proposed split amounts, set dispute status to DECISION_MADE, and initialize pending decision review entries for both parties. – `disputeId`: `number` – dispute identifier. – `adminId`: `number` – administrator user ID. – `freelancerAmount`: `number` – allocated freelancer amount. – `clientAmount`: `number` – allocated client refund amount. – `decisionReason`: `string` – justification summary. – `clientId`: `number` – client identifier. – `freelancerId`: `number` – freelancer identifier. – *Returns:* `Promise<Dispute>` – updated dispute record.
14	`upsertDecisionReview( disputeId: number, user Id: number, response: Dispute DecisionResponse, reason? : string)`	A method to persist a participant's accept or reject review vote. – `disputeId`: `number` – dispute identifier. – `userId`: `number` – participant user ID. – `response`: `DisputeDecisionResponse` – ACCEPTED or REJECTED. – `reason`: `string | undefined` – optional objection feedback. – *Returns:* `Promise<DisputeDecisionReview>` – saved review record.
15	`findDecisionReviews( disputeId: number)`	A method to query all recorded decision review votes for a dispute. – `disputeId`: `number` – dispute identifier. – *Returns:* `Promise<DisputeDecisionReview[]>` – review vote entries.

No	Method	Description
16	`updateDisputeStatus( disputeId: number, status: DisputeStatus)`	A method to transition dispute state directly in the database. – `disputeId`: `number` – dispute identifier. – `status`: `DisputeStatus` – target dispute status. – *Returns:* `Promise<Dispute>` – updated dispute record.
17	`finalizeDispute Transaction( data: FinalizeDispute Data)`	A method to atomically set final dispute distribution, mark milestone as completed, and record resolution timestamp. – `data`: `FinalizeDisputeData` – final amounts, milestone status, and timestamps. – *Returns:* `Promise<Dispute>` – finalized dispute record.

4.54.2 ClassAttribute

No	Attribute	Type
01	`prisma`	`PrismaService` – private readonly Prisma connection instance.

Supporting DTO classes for dispute management and arbitration

Class / Element	Attributes or Signature	Responsibility
`CreateDisputeBodyDTO`	`milestoneId: number, reason: string, description: string, evidenceFiles?: object[]`	Validates formal dispute filing payload and attached evidence.
`PayDisputeFeeBodyDTO`	`paymentMethodId?: string, paymentIntentId?: string`	Validates arbitration fee payment transaction payload.
`SubmitDisputeResponse BodyDTO`	`description: string, evidenceFiles?: object[]`	Validates respondent rebuttal statement and supporting counter-evidence.
`ReviewDisputeDecision BodyDTO`	`response: DisputeDecision Response, reason?: string`	Validates party acceptance or objection to arbitrator proposed split.
`AdminDisputeDecision BodyDTO`	`freelancerAmount: number, clientAmount: number, decisionReason: string`	Validates arbitrator initial proposed fund split and justification.

Class / Element	Attributes or Signature	Responsibility
AdminDisputeFinal DecisionBodyDTO	freelancerAmount: number, clientAmount: number, decisionReason: string	Validates arbitrator binding final fund distribution verdict.
GetDisputeListQueryDTO	page: number, limit: number, status?: Dispute Status, q?: string	Validates dispute dossier query filters and pagination parameters.

4.55 UsersController Class

4.55.1 ClassMethod

No	Method	Description
01	UsersController(service: UsersService)	A method to initialize the controller with the user administration service. – service: UsersService – service that owns admin user use cases. – *Returns:* void – constructor, no return value.
02	createUser(body: AdminCreateUserBodyDto)	A method to create an account with an initial role. – body: AdminCreateUserBodyDto – email, password, full name, and role. – *Returns:* Promise<AdminCreateUserResponseType> – the created account. – Route: POST /users.
03	updateUser(id: number, actorId: number, body: AdminUpdateUser BodyDto)	A method to update email, display name, or ban state. – id: number – target user identifier. – actorId: number – current administrator, used for the self-ban check. – body: AdminUpdateUserBodyDto – optional email, full name, and ban flag. – *Returns:* Promise<AdminUpdateUserResponseType> – the updated account. – Route: PATCH /users/:id.

No	Method	Description
04	`updateClientProfile( id: number, body: AdminUpdateClient ProfileBodyDto)`	A method to update the Client profile attached to an account. – `id: number` – target user identifier. – `body: AdminUpdateClientProfileBodyDto` – company name, description, and website. – *Returns:* `Promise<AdminClientProfileResponse Type>` – the updated Client profile. – Route: `PATCH /users/:id/client-profile`.
05	`updateFreelancerProfile( id: number, body: AdminUpdate FreelancerProfileBodyDto)`	A method to update the Freelancer headline and biography. – `id: number` – target user identifier. – `body: AdminUpdateFreelancerProfileBodyDto` – title, bio, languages, education, and certifications. – *Returns:* `Promise<MessageResType>` – a success message. – Route: `PATCH /users/:id/freelancer-profile`.
06	`replaceFreelancerSkills( id: number, body: AdminReplace FreelancerSkillsBodyDto)`	A method to replace every skill on a Freelancer profile. – `id: number` – target user identifier. – `body.skills`: array of skill name and proficiency pairs. – *Returns:* `Promise<MessageResType>` – a success message. – Route: `PUT /users/:id/freelancer-profile/ skills`.
07	`createPortfolioItem( id: number, body: AdminCreate PortfolioItemBodyDto)`	A method to add a portfolio item to a Freelancer profile. – `id: number` – target user identifier. – `body: AdminCreatePortfolioItemBodyDto` – title, description, technologies, and project URL. – *Returns:* `Promise<MessageResType>` – a success message. – Route: `POST /users/:id/freelancer-profile/ portfolio-items`.

No	Method	Description
08	`updatePortfolioItem( id: number, itemId: number, body: AdminUpdate PortfolioItemBodyDto)`	A method to update one portfolio item owned by the Freelancer profile. – `id: number` – target user identifier. – `itemId: number` – portfolio item identifier. – `body: AdminUpdatePortfolioItemBodyDto` – optional item fields. – *Returns:* `Promise<MessageResType>` – a success message. – Route: `PATCH /users/:id/freelancer-profile/ portfolio-items/:itemId`.
09	`deletePortfolioItem( id: number, itemId: number)`	A method to soft-delete one portfolio item. – `id: number` – target user identifier. – `itemId: number` – portfolio item identifier. – *Returns:* `Promise<MessageResType>` – a success message. – Route: `DELETE /users/:id/freelancer-profile/ portfolio-items/:itemId`.
10	`getUsers(query: Admin UserQueryDto)`	A method to return a filtered, paginated user list. – `query: AdminUserQueryDto` – page, limit, search, role, sort, and status filters. – *Returns:* `Promise<AdminUserListResponseType>` – users and pagination. – Route: `GET /users`.
11	`getSkillCatalog()`	A method to return the active skill catalogue for the profile editor. – *Returns:* `Promise<AdminSkillCatalogListType>` – active skills ordered by name. – Route: `GET /users/skills-catalog`.
12	`getUserById(id: number)`	A method to return a consolidated user detail. – `id: number` – target user identifier. – *Returns:* `Promise<AdminUserDetailResponseType>` – account, roles, and profile summaries. – Route: `GET /users/:id`.

4.55.2 ClassAttribute

No	Attribute	Type
01	service	UsersService – private readonly user administration service.

4.56 UsersService Class

4.56.1 ClassMethod

No	Method	Description
01	UsersService(repository: UsersRepository, hashingService: HashingService)	A method to initialize the service with persistence and password hashing. – repository: UsersRepository – user persistence dependency. – hashingService: HashingService – password hashing utility. – *Returns:* void – constructor, no return value.
02	createUser(body: AdminCreateUserBodyType)	A method to validate the email and role, hash the password, and create the account. – body: AdminCreateUserBodyType – email, password, full name, and role id. – *Returns:* Promise<AdminCreateUserResponseType> – the created account. – Throws EmailAlreadyExistsException, CreateUserRoleNotFoundException, CannotAssignAdminRoleException, or FailedToCreateUserException.
03	getUsers(query: AdminUserQueryType)	A method to load a user page. – query: AdminUserQueryType – page, limit, search, role, sort, and status filters. – *Returns:* Promise<AdminUserListResponseType> – users and pagination. – Failure is mapped to FailedToGetUsersException.
04	getUserById(id: number)	A method to load a consolidated account detail. – id: number – target user identifier. – *Returns:* Promise<AdminUserDetailResponseType> – account, roles, and profiles. – Throws UserNotFoundException when absent or FailedToGetUserException on an unexpected read failure.

No	Method	Description
05	`listSkillCatalog()`	A method to return the active skill catalogue. – *Returns:* `Promise<AdminSkillCatalogListType>` – active skills ordered by name.
06	`updateUser(id, actorId, body)`	A method to update account fields while enforcing the self-ban and email rules. – `id`: `number` – target user identifier. – `actorId`: `number` – current administrator. – `body`: `AdminUpdateUserBodyType` – optional email, full name, and ban flag. – *Returns:* `Promise<AdminUpdateUserResponseType>` – the updated account. – Throws `CannotBanSelfException`, `EmailAlreadyExistsException`, `UserNotFoundException`, or `FailedToUpdateUserException`.
07	`updateClientProfile(userId, body)`	A method to validate the Client profile context and upsert the profile. – `userId`: `number` – target user identifier. – `body`: `AdminUpdateClientProfileBodyType` – company fields. – *Returns:* `Promise<AdminClientProfileResponseType>` – the updated Client profile. – Throws `UserNotFoundException`, `NoClientRoleForClientProfileException`, or `FailedToUpdateClientProfileException`.
08	`updateFreelancerProfile(userId, body)`	A method to validate the Freelancer context and upsert profile fields. – `userId`: `number` – target user identifier. – `body`: `AdminUpdateFreelancerProfileBodyType` – title, bio, languages, education, and certifications. – *Returns:* `Promise<MessageResType>` – a success message. – Throws `NoFreelancerRoleForProfileException` or `FailedToUpdateFreelancerProfileException`.

No	Method	Description
09	`replaceFreelancerSkills(userId, body)`	A method to replace all Freelancer skills and create missing catalogue skills. – `userId`: `number` – target user identifier. – `body`: `AdminReplaceFreelancerSkillsBodyType` – skill name and proficiency pairs. – *Returns:* `Promise<MessageResType>` – a success message. – Throws `NoFreelancerRoleForProfileException` or `FailedToSaveSkillsException`.
10	`createPortfolioItem(userId, body)`	A method to create a portfolio item on the Freelancer profile. – `userId`: `number` – target user identifier. – `body`: `AdminCreatePortfolioItemBodyType` – item fields. – *Returns:* `Promise<MessageResType>` – a success message. – Throws `FailedToCreatePortfolioItemException`.
11	`updatePortfolioItem(userId, itemId, body)`	A method to update an owned portfolio item. – `userId`: `number` – target user identifier. – `itemId`: `number` – portfolio item identifier. – `body`: `AdminUpdatePortfolioItemBodyType` – optional item fields. – *Returns:* `Promise<MessageResType>` – a success message. – Throws `PortfolioItemNotFoundException` or `FailedToUpdatePortfolioItemException`.
12	`deletePortfolioItem(userId, itemId)`	A method to soft-delete an owned portfolio item. – `userId`: `number` – target user identifier. – `itemId`: `number` – portfolio item identifier. – *Returns:* `Promise<MessageResType>` – a success message. – Throws `PortfolioItemNotFoundException` or `FailedToDeletePortfolioItemException`.

No	Method	Description
13	`getFreelancerProfile Context(userId)`	A private method to load the profile and role context for a Freelancer edit. – `userId`: `number` – target user identifier. – *Returns:* `Promise<context>` – profile id, Freelancer profile id, and role flag. – Throws `UserNotFoundException` or `NoFreelancer RoleForProfileException`.
14	`ensureFreelancerProfile Row(userId, context)`	A private method to create the Profile and FreelancerProfile rows when missing. – `userId`: `number` – target user identifier. – `context`: object – profile id and role context. – *Returns:* `Promise<number | null>` – the Freelancer profile id. – Used by skill and portfolio editors before writing child rows.

4.56.2 ClassAttribute

No	Attribute	Type
01	`repository`	`UsersRepository` – private readonly persistence dependency.
02	`hashingService`	`HashingService` – private readonly password utility.

4.57 UsersRepository Class

4.57.1 ClassMethod

No	Method	Description
01	`UsersRepository(prisma: PrismaService)`	A method to initialize the repository with the shared Prisma client. – `prisma`: `PrismaService` – database client. – *Returns:* `void` – constructor, no return value.

No	Method	Description
02	`getUsers(query: AdminUserQueryType)`	A method to query users with search, role, status, sorting, and pagination. – `query`: `AdminUserQueryType` – list filters. – *Returns:* `Promise<AdminUserListResponseType>` – users and pagination.
03	`createUserByAdmin(data)`	A method to create the account, profile, and primary role in one write. – `data`: object – email, hashed password, full name, role id, and role name. – *Returns:* `Promise<AdminCreateUserResponseType>` – the created account. – Creates an empty Expert profile when the role is Expert.
04	`findUserByEmail(email)`	A method to find a non-deleted account by email. – `email`: `string` – account email. – *Returns:* `Promise<{ id: number } | null>` – the account id, or `null`.
05	`findActiveRoleById(id)`	A method to load an active role by id. – `id`: `number` – role identifier. – *Returns:* `Promise<{ id: number; name: string } | null>` – the role, or `null`.
06	`updateUserByAdmin(id, data)`	A method to update account fields, profile name, and sessions on ban. – `id`: `number` – target account. – `data`: object – optional email, full name, and ban flag. – *Returns:* `Promise<AdminUpdateUserResponseType | null>` – the updated account, or `null` when absent. – Setting `isBanned` to true deletes every session in the same transaction.
07	`findClientProfileEditContext(id)`	A method to load the profile context required by the Client editor. – `id`: `number` – target account. – *Returns:* `Promise<context | null>` – profile id, Client profile id, and role flag.
08	`upsertClientProfileByAdmin(userId, data)`	A method to create or update the Client profile. – `userId`: `number` – target account. – `data`: object – company name, description, and website. – *Returns:* `Promise<ClientProfile | null>` – the persisted Client profile.

No	Method	Description
09	`findFreelancerProfileEditContext(id)`	A method to load the profile context required by the Freelancer editor. – `id`: `number` – target account. – *Returns:* `Promise<context | null>` – profile id, Freelancer profile id, and role flag.
10	`updateFreelancerProfileByAdmin(userId, data)`	A method to update Profile biography and Freelancer profile fields. – `userId`: `number` – target account. – `data`: object – title, bio, languages, education, and certifications. – *Returns:* `Promise<void>` – no value.
11	`ensureFreelancerProfile(userId, profileId)`	A method to create the missing Profile or FreelancerProfile row. – `userId`: `number` – target account. – `profileId`: `number | null` – existing profile id, or `null`. – *Returns:* `Promise<number | null>` – the Freelancer profile id.
12	`replaceFreelancerSkills(freelancerProfileId, skills)`	A method to replace all skills and create missing catalogue skills in a transaction. – `freelancerProfileId`: `number` – Freelancer profile identifier. – `skills`: array of skill name and proficiency pairs. – *Returns:* `Promise<void>` – no value. – Free-text names are matched case-insensitively; missing skills receive a generated slug.
13	`createPortfolioItem(freelancerProfileId, data)`	A method to insert a portfolio item. – `freelancerProfileId`: `number` – Freelancer profile identifier. – `data`: object – title, description, technologies, and project URL. – *Returns:* `Promise<PortfolioItem>` – the created item.
14	`findPortfolioItemOwned(id)`	A method to load a portfolio item with its owner and delete state. – `id`: `number` – portfolio item identifier. – *Returns:* `Promise<{ id: number; freelancerProfileId: number; deletedAt: Date | null } | null>` – the ownership record, or `null`.

No	Method	Description
15	`updatePortfolioItemBy Admin(id, data)`	A method to update only the provided portfolio item fields. – `id`: `number` – portfolio item identifier. – `data`: object – optional item fields. – *Returns:* `Promise<PortfolioItem>` – the updated item.
16	`softDeletePortfolioItem( id)`	A method to set the portfolio item delete timestamp. – `id`: `number` – portfolio item identifier. – *Returns:* `Promise<PortfolioItem>` – the soft-deleted item.
17	`listActiveSkillCatalog()`	A method to return active skills ordered by name. – *Returns:* `Promise<AdminSkillCatalogListType>` – the catalogue.
18	`getUserById(id)`	A method to load the full account graph for the detail page. – `id`: `number` – target account. – *Returns:* `Promise<AdminUserDetailResponseType | null>` – the detail view, or `null`.

4.57.2 ClassAttribute

No	Attribute	Type
01	`prisma`	`PrismaService` – private readonly database client.

4.58 PaymentController Class

4.58.1 ClassMethod

No	Method	Description
01	`PaymentController( paymentService: Payment Service)`	A method to initialize the controller with the payment service dependency. – `paymentService`: `PaymentService` – service encapsulating escrow and Stripe gateway logic. – *Returns:* `void` – constructor, no return value.

No	Method	Description
02	createOnboardingLink(userId: number, body?: CreateOnboarding LinkBodyDTO)	A method to generate a Stripe Connect Express onboarding URL for freelancer payout setup. – userId: number – user identifier injected by @User Active('userId'). – body: CreateOnboardingLinkBodyDTO | undefined – optional custom returnUrl. – *Returns:* Promise<StripeOnboardingLinkResponse DTO> – onboarding redirect URL. – Route: POST /payments/connect/onboarding-link; authenticated endpoint.
03	getConnectStatus(userId: number)	A method to inspect the connected payout account status, onboarding completion, and payoutsEnabled flag. – userId: number – user identifier. – *Returns:* Promise<StripeConnectStatusResponseD TO> – connect capability state. – Route: GET /payments/connect/status; authenticated endpoint.
04	getSavedPaymentMethods(userId: number)	A method to retrieve customer saved credit cards from Stripe for off-session 1-click payments. – userId: number – customer user identifier. – *Returns:* Promise<GetSavedPaymentMethods ResponseDTO> – list of saved payment cards. – Route: GET /payments/payment-methods; authenticated endpoint.
05	createCustomerPortal Session(userId: number, return Url?: string)	A method to create a Stripe Billing Customer Portal session for managing billing details. – userId: number – customer user identifier. – returnUrl: string | undefined – redirect URL upon exiting portal. – *Returns:* Promise<CreateCustomerPortalSession ResponseDTO> – portal session URL. – Route: POST /payments/customer-portal; authenticated endpoint.

No	Method	Description
06	`createPlatformFeeIntent( userId: number, contractId: number)`	A method to initialize a Stripe PaymentIntent for paying contract activation platform fees. – `userId: number` – client user identifier. – `contractId: number` – contract identifier parsed by `ParseIntPipe`. – *Returns:* `Promise<CreatePaymentIntentResponseDTO>` – clientSecret and paymentIntentId. – Route: `POST /payments/contracts/:contractId/platform-fee/intent`; authenticated endpoint.
07	`createPlatformFeeCheckoutSession( userId: number, contractId: number)`	A method to create a hosted Stripe Checkout session for paying contract platform fees. – `userId: number` – client user identifier. – `contractId: number` – contract identifier. – *Returns:* `Promise<CreateCheckoutSessionResponseDTO>` – checkout session URL. – Route: `POST /payments/contracts/:contractId/platform-fee/checkout-session`; authenticated endpoint.
08	`chargeSavedCardForPlatformFee( userId: number, contractId: number, body?: ChargeSavedCardBodyDTO)`	A method to execute off-session platform fee charging using a pre-saved credit card. – `userId: number` – client user identifier. – `contractId: number` – contract identifier. – `body: ChargeSavedCardBodyDTO | undefined` – optional paymentMethodId. – *Returns:* `Promise<ChargeSavedCardResponseDTO>` – payment status and transaction record. – Route: `POST /payments/contracts/:contractId/platform-fee/saved-card`; authenticated endpoint.
09	`createMilestoneFundIntent( userId: number, contractId: number, milestoneId: number)`	A method to initialize a Stripe PaymentIntent for depositing milestone escrow funds. – `userId: number` – client user identifier. – `contractId: number` – contract identifier. – `milestoneId: number` – milestone identifier. – *Returns:* `Promise<CreatePaymentIntentResponseDTO>` – clientSecret and intent ID. – Route: `POST .../milestones/:milestoneId/fund/intent`; authenticated endpoint.

No	Method	Description
10	createMilestoneFund CheckoutSession(userId: number, contract Id: number, milestoneId: number)	A method to create a hosted Stripe Checkout session for depositing milestone escrow funds. – userId: number – client user identifier. – contractId: number – contract identifier. – milestoneId: number – milestone identifier. – *Returns:* Promise<CreateCheckoutSession ResponseDTO> – checkout session URL. – Route: POST .../milestones/:milestoneId/fund/ checkout-session; authenticated endpoint.
11	chargeSavedCardFor MilestoneFund(userId: number, contract Id: number, milestoneId: number, body?: ChargeSavedCard BodyDTO)	A method to fund a milestone escrow using an existing saved payment card off-session. – userId: number – client user identifier. – contractId: number – contract identifier. – milestoneId: number – milestone identifier. – body: ChargeSavedCardBodyDTO | undefined – optional specific paymentMethodId. – *Returns:* Promise<ChargeSavedCardResponseDTO> – funding confirmation and transaction record. – Route: POST .../milestones/:milestoneId/fund/ saved-card; authenticated endpoint.
12	releaseMilestone(userId: number, contract Id: number, milestoneId: number)	A method for the client to release deposited escrow funds to the freelancer's connected Stripe account. – userId: number – client user identifier. – contractId: number – contract identifier. – milestoneId: number – target milestone identifier. – *Returns:* Promise<ReleaseMilestoneResponseDTO> – payout transfer result and milestone status. – Route: POST .../milestones/:milestoneId/ release; authenticated endpoint.
13	refundMilestone(userId: number, contract Id: number, milestoneId: number)	A method to refund deposited milestone escrow funds back to the client's original payment source. – userId: number – client or admin user identifier. – contractId: number – contract identifier. – milestoneId: number – milestone identifier. – *Returns:* Promise<RefundMilestoneResponseDTO> – refund transaction confirmation. – Route: POST .../milestones/:milestoneId/ refund; authenticated endpoint.

No	Method	Description
14	createDisputeFeeIntent(userId: number, disputeId: number)	A method to initialize a Stripe PaymentIntent for paying arbitration filing fees. – userId: number – party user identifier. – disputeId: number – dispute identifier. – *Returns:* Promise<CreatePaymentIntentResponseDTO> – fee payment client secret. – Route: POST /payments/disputes/:disputeId/fee/intent; authenticated endpoint.
15	createDisputeFeeCheckoutSession(userId: number, disputeId: number)	A method to create a hosted Stripe Checkout session for paying dispute arbitration fees. – userId: number – party user identifier. – disputeId: number – dispute identifier. – *Returns:* Promise<CreateCheckoutSessionResponseDTO> – checkout session URL. – Route: POST /payments/disputes/:disputeId/fee/checkout-session; authenticated endpoint.
16	settleDispute(userId: number, roleName: string, disputeId: number, body: DisputeSettlementBodyDTO)	A method for administrators to execute multi-party financial settlement payouts following dispute resolution. – userId: number – administrator user identifier. – roleName: string – caller role name. – disputeId: number – dispute identifier. – body: DisputeSettlementBodyDTO – freelancerAmount, clientAmount, and winningParty. – *Returns:* Promise<DisputeSettlementResponseDTO> – transfer IDs, refund IDs, and status. – Route: POST /payments/disputes/:disputeId/settle; administrator-only endpoint.
17	getTransactions(userId: number, roleName: string, query: GetTransactionListQueryDTO)	A method to retrieve audit logs of financial transactions, escrow events, and fee charges. – userId: number – caller user identifier. – roleName: string – caller role name. – query: GetTransactionListQueryDTO – pagination and filtering criteria. – *Returns:* Promise<GetTransactionListResponseDTO> – paginated transaction items. – Route: GET /payments/transactions; authenticated endpoint.

No	Method	Description
18	`syncPaymentIntent( userId: number, paymentIntentId: string)`	A method to synchronize local database state with the actual status of a Stripe PaymentIntent. – `userId: number` – caller user identifier. – `paymentIntentId: string` – Stripe payment intent ID. – *Returns:* `Promise<SyncStatusResponse>` – synced transaction entity. – Route: `POST /payments/intents/:paymentIntentId/sync`; authenticated endpoint.
19	`syncCheckoutSession( userId: number, sessionId: string)`	A method to synchronize local escrow records upon client return from a hosted Stripe Checkout page. – `userId: number` – caller user identifier. – `sessionId: string` – Stripe checkout session ID. – *Returns:* `Promise<SyncCheckoutSessionResponseDTO>` – synchronization outcome. – Route: `POST /payments/checkout-sessions/:sessionId/sync`; authenticated endpoint.
20	`handleWebhook( req: Request, signature: string)`	A public endpoint to process cryptographically verified asynchronous webhook events dispatched by Stripe. – `req: Request & {rawBody?: Buffer}` – incoming raw HTTP request with signature. – `signature: string` – cryptographic signature header `stripe-signature`. – *Returns:* `Promise<{received: boolean}>` – acknowledgement receipt. – Route: `POST /payments/webhook`; public webhook endpoint.

4.58.2 ClassAttribute

No	Attribute	Type
01	`paymentService`	`PaymentService` – private readonly payment service dependency.

4.59 PaymentService Class

4.59.1 ClassMethod

No	Method	Description
01	`PaymentService( paymentRepo: PaymentRepo, stripeService: StripeService)`	A method to initialize the service with repository and Stripe payment gateway client. – `paymentRepo`: `PaymentRepo` – financial ledger data access layer. – `stripeService`: `StripeService` – Stripe API integration service. – *Returns:* `void` – constructor, no return value.
02	`getOnboardingLink( userId: number, returnUrl?: string)`	A method to create or retrieve a Stripe Express account and produce an onboarding link for account verification. – `userId`: `number` – freelancer user identifier. – `returnUrl`: `string | undefined` – destination URL upon onboarding completion. – *Returns:* `Promise<{url: string}>` – account onboarding link.
03	`getConnectStatus(userId: number)`	A method to check whether the freelancer has fully completed Stripe Connect Express KYC verification and can receive payouts. – `userId`: `number` – user identifier. – *Returns:* `Promise<ConnectStatusType>` – status with payout readiness flags.
04	`getCustomerPortalLink( userId: number, returnUrl?: string)`	A method to generate a self-service Stripe billing portal link for managing saved payment cards. – `userId`: `number` – customer user identifier. – `returnUrl`: `string | undefined` – exit redirect URL. – *Returns:* `Promise<{url: string}>` – billing portal URL.
05	`getSavedPaymentMethods( userId: number)`	A method to query Stripe for card payment methods attached to the customer's Stripe Customer ID. – `userId`: `number` – customer user identifier. – *Returns:* `Promise<{paymentMethods: object[]}>` – list of card tokens and brands.

No	Method	Description
06	`chargeSavedCardFor PlatformFee( userId: number, contract Id: number, paymentMethodId?: string)`	A method to execute an off-session charge on a saved card for contract activation fees. – `userId: number` – client user identifier. – `contractId: number` – contract identifier. – `paymentMethodId: string | undefined` – specific payment method ID. – *Returns:* `Promise<ChargeSavedCardResponseType>` – charge transaction status.
07	`chargeSavedCardFor MilestoneFund( userId: number, contract Id: number, milestoneId: number, paymentMethodId?: string)`	A method to charge a customer's stored card to fund a milestone into escrow. – `userId: number` – client user identifier. – `contractId: number` – contract identifier. – `milestoneId: number` – milestone identifier. – `paymentMethodId: string | undefined` – card payment method ID. – *Returns:* `Promise<ChargeSavedCardResponseType>` – escrow deposit confirmation.
08	`createPlatformFeeIntent( userId: number, contract Id: number)`	A method to validate signed contract terms and create a Stripe PaymentIntent for platform fees. – `userId: number` – client user identifier. – `contractId: number` – contract identifier. – *Returns:* `Promise<PaymentIntentResponseType>` – client secret and intent ID.
09	`createPlatformFeeCheckout Session( userId: number, contract Id: number)`	A method to construct a hosted Stripe Checkout session for payment of platform fees. – `userId: number` – client user identifier. – `contractId: number` – contract identifier. – *Returns:* `Promise<{url: string, sessionId: string}>` – checkout session metadata.
10	`createMilestoneFund Intent( userId: number, contract Id: number, milestoneId: number)`	A method to initialize a PaymentIntent for milestone escrow funding. – `userId: number` – client user identifier. – `contractId: number` – contract identifier. – `milestoneId: number` – milestone identifier. – *Returns:* `Promise<PaymentIntentResponseType>` – client secret and intent identifier.

No	Method	Description
11	`createMilestoneFund CheckoutSession( userId: number, contract Id: number, milestoneId: number)`	A method to construct a hosted Stripe Checkout session for funding a milestone. – `userId: number` – client user identifier. – `contractId: number` – contract identifier. – `milestoneId: number` – milestone identifier. – *Returns:* `Promise<{url: string, sessionId: string}>` – checkout session parameters.
12	`releaseMilestone( userId: number, contract Id: number, milestoneId: number)`	A method to execute a Stripe transfer of escrowed funds to the freelancer's connected Stripe account. – `userId: number` – client user identifier. – `contractId: number` – contract identifier. – `milestoneId: number` – milestone identifier. – *Returns:* `Promise<ReleaseMilestoneResponse Type>` – release transfer ID and completion status.
13	`refundMilestone( userId: number, contract Id: number, milestoneId: number)`	A method to issue a Stripe refund for an escrowed milestone back to the client's original card. – `userId: number` – caller user identifier. – `contractId: number` – contract identifier. – `milestoneId: number` – milestone identifier. – *Returns:* `Promise<RefundMilestoneResponseType>` – refund record and status.
14	`createDisputeFeeIntent( userId: number, dispute Id: number)`	A method to initialize a Stripe PaymentIntent for paying an assessed dispute fee. – `userId: number` – party user identifier. – `disputeId: number` – dispute identifier. – *Returns:* `Promise<PaymentIntentResponseType>` – fee payment client secret.
15	`createDisputeFeeCheckout Session( userId: number, dispute Id: number)`	A method to construct a hosted Stripe Checkout session for paying dispute fees. – `userId: number` – party user identifier. – `disputeId: number` – dispute identifier. – *Returns:* `Promise<{url: string, sessionId: string}>` – checkout session URL.

No	Method	Description
16	`settleDispute( userId: number, roleName: string, disputeId: number, body: DisputeSettlementBodyType)`	A method to disburse resolved funds: transfers freelancer portion via Stripe Connect, refunds client portion, and refunds prevailing party's fee. – `userId: number` – administrator user identifier. – `roleName: string` – role name. – `disputeId: number` – dispute identifier. – `body: DisputeSettlementBodyType` – split amounts and winner. – *Returns:* `Promise<DisputeSettlementResponseType>` – settlement disbursement breakdown.
17	`getTransactions( userId: number, roleName: string, query: GetTransactionListQueryType)`	A method to query transaction audit records with role-based scoping and pagination. – `userId: number` – caller user identifier. – `roleName: string` – role name. – `query: GetTransactionListQueryType` – pagination and filter options. – *Returns:* `Promise<PaginatedResult<Transaction>>` – list of transaction records.
18	`syncPaymentIntent( userId: number, paymentIntentId: string)`	A method to verify payment status with Stripe and transition corresponding local escrow records if succeeded. – `userId: number` – caller user identifier. – `paymentIntentId: string` – Stripe payment intent ID. – *Returns:* `Promise<SyncResponseType>` – updated transaction and entity state.
19	`syncCheckoutSession( userId: number, sessionId: string)`	A method to retrieve checkout session details from Stripe and update database records. – `userId: number` – caller user identifier. – `sessionId: string` – Stripe checkout session ID. – *Returns:* `Promise<SyncCheckoutSessionResponseType>` – updated records.
20	`handleWebhook( payload: Buffer | string, signature: string)`	A method to cryptographically verify incoming Stripe events and trigger idempotent ledger transitions. – `payload: Buffer | string` – raw event payload. – `signature: string` – cryptographic signature header. – *Returns:* `Promise<{received: boolean}>` – acknowledgement receipt.

4.59.2 ClassAttribute

No	Attribute	Type
01	`paymentRepo`	`PaymentRepo` – private readonly repository dependency.
02	`stripeService`	`StripeService` – private readonly Stripe API integration service.

4.60 PaymentRepo Class

4.60.1 ClassMethod

No	Method	Description
01	`PaymentRepo(prisma: PrismaService)`	A method to initialize the repository with the Prisma client. – `prisma`: `PrismaService` – Prisma database service instance. – *Returns:* `void` – constructor, no return value.
02	`findUserById(userId: number)`	A method to query user account details and Stripe customer/connect metadata. – `userId`: `number` – user identifier. – *Returns:* `Promise<User | null>` – user entity or null.
03	`updateUserStripe( userId: number, data: UpdateUserStripeData)`	A method to record Stripe customer IDs, connected account IDs, and onboarding completion flags. – `userId`: `number` – user identifier. – `data`: `UpdateUserStripeData` – stripeCustomerId, stripeAccountId, stripeOnboardingCompleted. – *Returns:* `Promise<User>` – updated user entity.
04	`findContractById(contract Id: number)`	A method to load contract participant identities, budget, and fee status. – `contractId`: `number` – contract identifier. – *Returns:* `Promise<Contract | null>` – contract record or null.

No	Method	Description
05	`updateContractPlatformFee( contractId: number, status: PlatformFeeStatus)`	A method to update contract platform fee payment state (PAID/UNPAID). – `contractId`: `number` – contract identifier. – `status`: `PlatformFeeStatus` – target platform fee status. – *Returns:* `Promise<Contract>` – updated contract record.
06	`findMilestoneById( milestoneId: number)`	A method to load milestone data, parent contract details, and participant Stripe accounts. – `milestoneId`: `number` – milestone identifier. – *Returns:* `Promise<Milestone | null>` – milestone record with relations.
07	`updateMilestonePaymentStatus( milestoneId: number, status: MilestonePaymentStatus)`	A method to update milestone payment state (FUNDED, RELEASED, REFUNDED). – `milestoneId`: `number` – milestone identifier. – `status`: `MilestonePaymentStatus` – target payment status. – *Returns:* `Promise<Milestone>` – updated milestone entity.
08	`findDisputeById(disputeId: number)`	A method to load dispute arbitration details, fees, and related milestone escrow. – `disputeId`: `number` – dispute identifier. – *Returns:* `Promise<Dispute | null>` – dispute entity or null.
09	`updateDisputeStatus( disputeId: number, status: DisputeStatus)`	A method to update dispute lifecycle status. – `disputeId`: `number` – dispute identifier. – `status`: `DisputeStatus` – target dispute status. – *Returns:* `Promise<Dispute>` – updated dispute record.
10	`findDisputeFee(disputeId: number, userId: number)`	A method to query arbitration fee record for a specific participant in a dispute. – `disputeId`: `number` – dispute identifier. – `userId`: `number` – party user identifier. – *Returns:* `Promise<DisputeFee | null>` – dispute fee record or null.

No	Method	Description
11	`updateDisputeFeeStatus( feeId: number, status: DisputeFeeStatus, paymentReference?: string, paidAt?: Date)`	A method to mark dispute fee as paid and link transaction gateway references. – `feeId`: `number` – fee identifier. – `status`: `DisputeFeeStatus` – new fee status. – `paymentReference`: `string | undefined` – reference string. – `paidAt`: `Date | undefined` – payment timestamp. – *Returns:* `Promise<DisputeFee>` – updated fee entity.
12	`createTransaction(data: CreateTransactionData)`	A method to persist an immutable financial transaction entry in the system ledger. – `data`: `CreateTransactionData` – amount, type, status, userId, contractId, milestoneId, stripePaymentIntentId. – *Returns:* `Promise<Transaction>` – created transaction record.
13	`updateTransactionStatus( id: number, status: TransactionStatus)`	A method to update transaction lifecycle state (PENDING, COMPLETED, FAILED). – `id`: `number` – transaction primary key. – `status`: `TransactionStatus` – new transaction status. – *Returns:* `Promise<Transaction>` – updated transaction entity.
14	`findTransactionByPayment IntentId( paymentIntentId: string)`	A method to query a transaction record matching a Stripe PaymentIntent ID. – `paymentIntentId`: `string` – Stripe payment intent ID. – *Returns:* `Promise<Transaction | null>` – transaction record or null.
15	`findTransactionBySession Id(sessionId: string)`	A method to query a transaction record matching a Stripe Checkout Session ID. – `sessionId`: `string` – Stripe session ID. – *Returns:* `Promise<Transaction | null>` – transaction record or null.
16	`findEscrowDeposit Transaction(milestoneId: number)`	A method to locate the original completed escrow deposit transaction for a milestone. – `milestoneId`: `number` – milestone identifier. – *Returns:* `Promise<Transaction | null>` – escrow deposit transaction.

No	Method	Description
17	`getTransactions( query: GetTransaction ListQueryType, userId?: number)`	A method to retrieve paginated ledger transactions with user and type filters. – `query`: `GetTransactionListQueryType` – pagination parameters. – `userId`: `number | undefined` – optional user ID filter. – *Returns:* `Promise<PaginatedResult<Transaction> >` – paginated transactions.

4.60.2 ClassAttribute

No	Attribute	Type
01	`prisma`	`PrismaService` – private readonly Prisma connection instance.

4.61 StripeService Class

4.61.1 ClassMethod

No	Method	Description
01	`StripeService()`	A method to initialize the Stripe SDK client using configured secret key and API version. – *Returns:* `void` – constructor, no return value.
02	`createCustomer( email: string, name?: string)`	A method to create a new Customer resource in Stripe. – `email`: `string` – customer email address. – `name`: `string | undefined` – customer name. – *Returns:* `Promise<Stripe.Customer>` – Stripe customer object.
03	`createExpressAccount( email: string, country?: string)`	A method to create a Stripe Connect Express account with transfer capabilities. – `email`: `string` – freelancer email. – `country`: `string` – country code (defaults from config). – *Returns:* `Promise<Stripe.Account>` – created Stripe connected account.

No	Method	Description
04	`createAccountLink( stripeAccountId: string, returnUrl: string, refreshUrl: string)`	A method to generate an onboarding URL for Stripe Express account setup. – `stripeAccountId: string` – Stripe account ID. – `returnUrl: string` – return redirect URL. – `refreshUrl: string` – refresh redirect URL. – *Returns:* `Promise<Stripe.AccountLink>` – account onboarding link.
05	`getAccount(stripeAccount Id: string)`	A method to retrieve connected account capabilities and verification status. – `stripeAccountId: string` – Stripe account ID. – *Returns:* `Promise<Stripe.Account>` – Stripe account details.
06	`createPaymentIntent( params: CreatePayment IntentParams)`	A method to create a Stripe PaymentIntent with automatic payment methods. – `params: CreatePaymentIntentParams` – amountInCents, currency, customerId, metadata. – *Returns:* `Promise<Stripe.PaymentIntent>` – Stripe payment intent object.
07	`createTransfer(params: CreateTransferParams)`	A method to transfer funds from the platform balance to a connected Express account. – `params: CreateTransferParams` – amountInCents, currency, destination, transferGroup, metadata. – *Returns:* `Promise<Stripe.Transfer>` – Stripe transfer object.
08	`createRefund(params: CreateRefundParams)`	A method to issue a full or partial refund on a PaymentIntent back to the payer. – `params: CreateRefundParams` – paymentIntentId, amountInCents, metadata. – *Returns:* `Promise<Stripe.Refund>` – Stripe refund object.
09	`retrievePaymentIntent( paymentIntentId: string)`	A method to query current status and charges of a Stripe PaymentIntent. – `paymentIntentId: string` – Stripe payment intent ID. – *Returns:* `Promise<Stripe.PaymentIntent>` – retrieved payment intent.

No	Method	Description
10	`createCheckoutSession( params: CreateCheckout SessionParams)`	A method to create a hosted Stripe Checkout page session. – `params`: `CreateCheckoutSessionParams` – amountInCents, currency, customerId, title, successUrl, cancelUrl. – *Returns:* `Promise<Stripe.Checkout.Session>` – Stripe checkout session.
11	`retrieveCheckoutSession( sessionId: string)`	A method to inspect payment status of a completed checkout session. – `sessionId`: `string` – Stripe session ID. – *Returns:* `Promise<Stripe.Checkout.Session>` – retrieved session.
12	`createBillingPortal Session( customerId: string, returnUrl: string)`	A method to initialize a Stripe customer portal session for card management. – `customerId`: `string` – Stripe customer ID. – `returnUrl`: `string` – redirect URL upon exiting portal. – *Returns:* `Promise<Stripe.BillingPortal. Session>` – portal session object.
13	`listPaymentMethods( customerId: string)`	A method to query all saved card payment methods for a customer. – `customerId`: `string` – Stripe customer ID. – *Returns:* `Promise<Stripe.ApiList<Stripe. PaymentMethod>>` – payment method list.
14	`chargeOffSession(params: ChargeOffSessionParams)`	A method to execute an automatic off-session charge using an existing saved payment method. – `params`: `ChargeOffSessionParams` – amountInCents, customerId, paymentMethodId, metadata. – *Returns:* `Promise<Stripe.PaymentIntent>` – confirmed payment intent.
15	`constructWebhookEvent( payload: Buffer | string, signature: string)`	A method to cryptographically verify and parse an incoming Stripe webhook payload. – `payload`: `Buffer | string` – raw request payload. – `signature`: `string` – value of `stripe-signature` header. – *Returns:* `Stripe.Event` – validated Stripe webhook event.

4.61.2 ClassAttribute

No	Attribute	Type
01	stripe	Stripe – private readonly Stripe official SDK client instance.
02	logger	Logger – private readonly NestJS system logger.

Supporting DTO classes for payment and Stripe integration

Class / Element	Attributes or Signature	Responsibility
CreateOnboardingLinkBodyDTO	returnUrl?: string	Validates custom return URL for Stripe Connect Express onboarding.
StripeConnectStatusResponseDTO	connected: boolean, onboardingCompleted: boolean, payoutsEnabled: boolean	Serializes freelancer Stripe Connect verification and payout capability state.
CreatePaymentIntentResponseDTO	clientSecret: string, paymentIntentId: string	Serializes Stripe PaymentIntent client credentials for frontend SDK.
CreateCheckoutSessionResponseDTO	url: string, sessionId: string	Serializes hosted Stripe Checkout redirect URL and session identifier.
ChargeSavedCardBodyDTO	paymentMethodId?: string	Validates stored payment method identifier for off-session card charges.
ReleaseMilestoneResponseDTO	milestoneId: number, status: string, transferId?: string	Serializes milestone escrow payout release confirmation.
RefundMilestoneResponseDTO	milestoneId: number, status: string, refundId?: string	Serializes milestone escrow refund execution result.
DisputeSettlementBodyDTO	freelancerAmount: number, clientAmount: number, winningParty: string	Validates multi-party dispute resolution financial disbursement instructions.
GetTransactionListQueryDTO	page: number, limit: number, type?: TransactionType, status?: TransactionStatus	Validates financial ledger audit query and pagination filters.

Class / Element	Attributes or Signature	Responsibility
SyncCheckoutSession ResponseDTO	success: boolean, status: string, transactionId?: number	Serializes status reconciliation result following Stripe Checkout completion.

4.62 BannersAdminController Class

4.62.1 ClassMethod

No	Method	Description
01	BannersAdminController(service: BannersAdmin Service)	A method to initialize the controller with the banner administration service. – service: BannersAdminService – service that owns banner use cases. – *Returns:* void – constructor, no return value.
02	listBanners(query: BannerAdminQueryDto)	A method to return a filtered, paginated banner list. – query: BannerAdminQueryDto – page, limit, search, position, active, and sort options. – *Returns:* Promise<BannerAdminListResponseType> – banners and pagination. – Route: GET /admin/banners.
03	createBanner(body: CreateBannerBodyDto)	A method to create an advertisement banner. – body: CreateBannerBodyDto – title, image URL, link, position, active state, and dates. – *Returns:* Promise<BannerAdminDetailResponse Type> – the created banner. – Route: POST /admin/banners.
04	updateBanner(id: number, body: UpdateBannerBody Dto)	A method to update a banner. – id: number – banner identifier. – body: UpdateBannerBodyDto – optional banner fields. – *Returns:* Promise<BannerAdminDetailResponse Type> – the updated banner. – Route: PATCH /admin/banners/:id.
05	deleteBanner(id: number)	A method to permanently delete a banner. – id: number – banner identifier. – *Returns:* Promise<BannerAdminDeleteResponse Type> – a success message. – Route: DELETE /admin/banners/:id.

No	Method	Description
06	`uploadImage(file?: Express.Multer.File)`	A method to upload a banner image and return its URL. – `file`: `Express.Multer.File | undefined` – multipart image field named `file`. – *Returns:* `Promise<{ imageUrl: string; publicId: string }>` – the stored image reference. – Route: `POST /admin/banners/upload-image`; multipart field `file`; maximum 10 MB.
07	`getBannerDetail(id: number)`	A method to return one banner. – `id`: `number` – banner identifier. – *Returns:* `Promise<BannerAdminDetailResponseType>` – the banner detail. – Route: `GET /admin/banners/:id`.

4.62.2 ClassAttribute

No	Attribute	Type
01	`service`	`BannersAdminService` – private readonly banner administration service.

4.63 BannersAdminService Class

4.63.1 ClassMethod

No	Method	Description
01	`BannersAdminService(repository: BannersAdminRepository, cloudinaryService: CloudinaryService)`	A method to initialize the service with persistence and image storage. – `repository`: `BannersAdminRepository` – banner persistence dependency. – `cloudinaryService`: `CloudinaryService` – remote image storage. – *Returns:* `void` – constructor, no return value.

No	Method	Description
02	`uploadImage(file?: Express.Multer.File)`	A method to validate an image and store it on Cloudinary or the local upload directory. – `file: Express.Multer.File | undefined` – multipart image. – *Returns:* `Promise<{ imageUrl: string; publicId: string }>` – the stored image reference. – Throws `BannerImageRequiredException`, `BannerImageInvalidTypeException`, `BannerImageTooLargeException`, or `FailedToUploadBannerImageException`.
03	`listBanners(query: BannerAdminQueryType)`	A method to load a banner page. – `query: BannerAdminQueryType` – list filters. – *Returns:* `Promise<BannerAdminListResponseType>` – banners and pagination. – Failure is mapped to `FailedToLoadBannerListException`.
04	`getBannerDetail(id: number)`	A method to load one banner. – `id: number` – banner identifier. – *Returns:* `Promise<BannerAdminDetailResponseType>` – the banner detail. – Throws `BannerAdminNotFoundException` or `FailedToLoadBannerDetailException`.
05	`createBanner(body: BannerCreateBodyType)`	A method to create a banner after validating its position. – `body: BannerCreateBodyType` – banner fields. – *Returns:* `Promise<BannerAdminDetailResponseType>` – the created banner. – Throws `BannerPositionTakenException` or `FailedToCreateBannerException`.
06	`updateBanner(id: number, body: BannerUpdateBodyType)`	A method to update a banner after validating its position. – `id: number` – banner identifier. – `body: BannerUpdateBodyType` – optional banner fields. – *Returns:* `Promise<BannerAdminDetailResponseType>` – the updated banner. – Throws `BannerAdminNotFoundException`, `BannerPositionTakenException`, or `FailedToUpdateBannerException`.

No	Method	Description
07	`deleteBanner(id: number)`	A method to permanently delete a banner. – `id: number` – banner identifier. – *Returns:* `Promise<BannerAdminDeleteResponse Type>` – a success message. – Throws `BannerAdminNotFoundException` or `FailedToDeleteBannerException`.

4.63.2 ClassAttribute

No	Attribute	Type
01	`repository`	`BannersAdminRepository` – private readonly persistence dependency.
02	`cloudinaryService`	`CloudinaryService` – private readonly image storage dependency.

4.64 BannersAdminRepository Class

4.64.1 ClassMethod

No	Method	Description
01	`BannersAdminRepository( prisma: PrismaService)`	A method to initialize the repository with the shared Prisma client. – `prisma: PrismaService` – database client. – *Returns:* `void` – constructor, no return value.
02	`listBanners(query: BannerAdminQueryType)`	A method to query banners with search, position, active-state, and sorting filters. – `query: BannerAdminQueryType` – list filters. – *Returns:* `Promise<BannerAdminListResponseType>` – banners and pagination.
03	`getBannerDetail(id: number)`	A method to load one banner. – `id: number` – banner identifier. – *Returns:* `Promise<BannerAdminDetailResponse Type | null>` – the banner, or `null`.

No	Method	Description
04	`createBanner(data: BannerCreateBodyType)`	A method to validate the position constraint and insert a banner. – `data`: `BannerCreateBodyType` – banner fields. – *Returns:* `Promise<BannerAdminDetailResponse Type>` – the created banner. – Throws `BannerPositionTakenException`.
05	`updateBanner(id, data)`	A method to validate the position constraint and update a banner. – `id`: `number` – banner identifier. – `data`: `BannerUpdateBodyType` – optional banner fields. – *Returns:* `Promise<BannerAdminDetailResponse Type>` – the updated banner. – Throws `BannerAdminNotFoundException` or `Banner PositionTakenException`.
06	`deleteBanner(id: number)`	A method to delete a banner and report whether a row was removed. – `id`: `number` – banner identifier. – *Returns:* `Promise<BannerAdminDeleteResponse Type>` – a success message. – Throws `BannerAdminNotFoundException`.

4.64.2 ClassAttribute

No	Attribute	Type
01	`prisma`	`PrismaService` – private readonly database client.

4.65 SkillsAdminController Class

4.65.1 ClassMethod

No	Method	Description
01	`SkillsAdminController(service: SkillsAdminService)`	A method to initialize the controller with the skill administration service. – `service`: `SkillsAdminService` – service that owns skill use cases. – *Returns:* `void` – constructor, no return value.
02	`listSkills(query: SkillAdminQueryDto)`	A method to return a filtered, paginated skill list. – `query`: `SkillAdminQueryDto` – page, limit, search, active, and sort options. – *Returns:* `Promise<SkillAdminListResponseType>` – skills and pagination. – Route: `GET /admin/skills`.
03	`createSkill(body: CreateSkillBodyDto)`	A method to create a skill. – `body`: `CreateSkillBodyDto` – name and optional description. – *Returns:* `Promise<SkillAdminDetailResponseType>` – the created skill. – Route: `POST /admin/skills`.
04	`updateSkill(id: number, body: UpdateSkillBodyDto)`	A method to update a skill. – `id`: `number` – skill identifier. – `body`: `UpdateSkillBodyDto` – optional name, description, and active state. – *Returns:* `Promise<SkillAdminDetailResponseType>` – the updated skill. – Route: `PATCH /admin/skills/:id`.
05	`restoreSkill(id: number)`	A method to restore a soft-deleted skill. – `id`: `number` – skill identifier. – *Returns:* `Promise<SkillAdminDetailResponseType>` – the restored skill. – Route: `PATCH /admin/skills/:id/restore`.
06	`deleteSkill(id: number)`	A method to soft-delete a skill. – `id`: `number` – skill identifier. – *Returns:* `Promise<SkillAdminDeleteResponseType>` – a success message. – Route: `DELETE /admin/skills/:id`.

No	Method	Description
07	`getSkillDetail(id: number)`	A method to return one skill. – `id: number` – skill identifier. – *Returns:* `Promise<SkillAdminDetailResponse Type>` – the skill detail. – Route: `GET /admin/skills/:id`.

4.65.2 ClassAttribute

No	Attribute	Type
01	`service`	`SkillsAdminService` – private readonly skill administration service.

4.66 SkillsAdminService Class

4.66.1 ClassMethod

No	Method	Description
01	`SkillsAdminService( repository: SkillsAdmin Repository)`	A method to initialize the service with the skill repository. – `repository: SkillsAdminRepository` – persistence dependency. – *Returns:* `void` – constructor, no return value.
02	`listSkills(query: Skill AdminQueryType)`	A method to load a skill page. – `query: SkillAdminQueryType` – list filters. – *Returns:* `Promise<SkillAdminListResponseType>` – skills and pagination. – Failure is mapped to `FailedToLoadSkillList Exception`.
03	`getSkillDetail(id: number)`	A method to load one skill. – `id: number` – skill identifier. – *Returns:* `Promise<SkillAdminDetailResponse Type>` – the skill detail. – Throws `SkillAdminNotFoundException` or `Failed ToLoadSkillDetailException`.

No	Method	Description
04	`createSkill(body: AdminCreateSkillBodyType)`	A method to create a skill and handle a concurrent duplicate. – `body: AdminCreateSkillBodyType` – name and optional description. – *Returns:* `Promise<SkillAdminDetailResponseType>` – the created skill. – Throws `SkillNameAlreadyExistsException` or `FailedToCreateSkillException`.
05	`updateSkill(id, body)`	A method to update a skill and handle a concurrent duplicate. – `id: number` – skill identifier. – `body: AdminUpdateSkillBodyType` – optional name, description, and active state. – *Returns:* `Promise<SkillAdminDetailResponseType>` – the updated skill. – Throws `SkillAdminNotFoundException`, `SkillNameAlreadyExistsException`, or `FailedToUpdateSkillException`.
06	`deleteSkill(id: number)`	A method to soft-delete a skill. – `id: number` – skill identifier. – *Returns:* `Promise<SkillAdminDeleteResponseType>` – a success message. – Throws `SkillAdminNotFoundException` or `FailedToDeleteSkillException`.
07	`restoreSkill(id: number)`	A method to restore a soft-deleted skill. – `id: number` – skill identifier. – *Returns:* `Promise<SkillAdminDetailResponseType>` – the restored skill. – Throws `SkillAdminNotFoundException`, `SkillNameAlreadyExistsException`, or `FailedToRestoreSkillException`.

4.66.2 ClassAttribute

No	Attribute	Type
01	`repository`	`SkillsAdminRepository` – private readonly persistence dependency.

4.67 SkillsAdminRepository Class

4.67.1 ClassMethod

No	Method	Description
01	`SkillsAdminRepository(prisma: PrismaService)`	A method to initialize the repository with the shared Prisma client. – `prisma`: `PrismaService` – database client. – *Returns:* `void` – constructor, no return value.
02	`listSkills(query: SkillAdminQueryType)`	A method to query skills with search, active-state, sorting, and pagination. – `query`: `SkillAdminQueryType` – list filters. – *Returns:* `Promise<SkillAdminListResponseType>` – skills and pagination. – Search matches name, slug, or description.
03	`getSkillDetail(id: number)`	A method to load one skill. – `id`: `number` – skill identifier. – *Returns:* `Promise<SkillAdminDetailResponseType | null>` – the skill, or `null`.
04	`createSkill(data: AdminCreateSkillBodyType)`	A method to generate a unique slug and insert a skill. – `data`: `AdminCreateSkillBodyType` – name and optional description. – *Returns:* `Promise<SkillAdminDetailResponseType>` – the created skill. – Throws `SkillNameAlreadyExistsException`.
05	`updateSkill(id, data)`	A method to update a skill and regenerate its slug when the name changes. – `id`: `number` – skill identifier. – `data`: `AdminUpdateSkillBodyType` – optional name, description, and active state. – *Returns:* `Promise<SkillAdminDetailResponseType>` – the updated skill. – Throws `SkillAdminNotFoundException` or `SkillNameAlreadyExistsException`.
06	`deleteSkill(id: number)`	A method to set the skill delete timestamp. – `id`: `number` – skill identifier. – *Returns:* `Promise<SkillAdminDeleteResponseType>` – a success message. – Throws `SkillAdminNotFoundException`.

No	Method	Description
07	`restoreSkill(id: number)`	A method to clear the skill delete timestamp after checking name conflicts. – `id: number` – skill identifier. – *Returns:* `Promise<SkillAdminDetailResponse Type>` – the restored skill. – Throws `SkillAdminNotFoundException` or `Skill NameAlreadyExistsException`.
08	`generateUniqueSlug(name: string)`	A private method to derive a slug and append a numeric suffix on collision. – `name: string` – skill name. – *Returns:* `Promise<string>` – the unique slug.

4.67.2 ClassAttribute

No	Attribute	Type
01	`prisma`	`PrismaService` – private readonly database client.

4.68 AdminExpertProfileController Class

4.68.1 ClassMethod

No	Method	Description
01	`AdminExpertProfile Controller(service: ExpertProfileService)`	A method to initialize the controller with the expert profile service. – `service: ExpertProfileService` – service that owns expert profile use cases. – *Returns:* `void` – constructor, no return value.

No	Method	Description
02	`update( userId: number, adminId: number, body: AdminUpdateExpert ProfileDto)`	A method for an administrator to update an expert profile directly. – `userId: number` – expert account identifier. – `adminId: number` – current administrator, recorded on superseded revisions. – `body: AdminUpdateExpertProfileDto` – display name, bio, title, expertise, experience, education, certifications, website, and active state. – *Returns:* `Promise<MessageResType>` – a success message. – Route: `PATCH /users/:id/expert-profile`.

4.68.2 ClassAttribute

No	Attribute	Type
01	`service`	`ExpertProfileService` – private readonly expert profile service.

4.69 ExpertProfileService Class

4.69.1 ClassMethod

No	Method	Description
01	`ExpertProfileService( repository: Expert ProfileRepository, revisions: Profile RevisionService)`	A method to initialize the service with persistence and revision handling. – `repository: ExpertProfileRepository` – expert profile persistence. – `revisions: ProfileRevisionService` – self-service revision workflow. – *Returns:* `void` – constructor, no return value.
02	`cleanList(values: string[])`	A private method to trim, de-duplicate, and preserve the first casing of a string list. – `values: string[]` – raw list values. – *Returns:* `string[]` – the cleaned list.

No	Method	Description
03	`sanitize(input)`	A private method to clean list fields and normalize the website value. – `input`: `UpdateExpertProfileType` – profile input. – *Returns:* `UpdateExpertProfileType` – the sanitized input.
04	`context(userId: number)`	A private method to load an expert account and verify its Expert role. – `userId`: `number` – target account. – *Returns:* `Promise<user context>` – account, profile, and roles. – Throws `ExpertProfileNotFoundException` or `NotAnExpertAccountException`.
05	`getMine(userId: number)`	A method to return the authenticated expert's profile. – `userId`: `number` – current expert account. – *Returns:* `Promise<ExpertProfileType>` – profile and expert fields. – Used by the self-service profile page, not by the admin endpoint.
06	`submitUpdate(userId, input)`	A method to submit a self-service expert profile revision. – `userId`: `number` – current expert account. – `input`: `UpdateExpertProfileType` – requested profile changes. – *Returns:* `Promise<revision result>` – the pending revision. – Throws `ExpertProfilePendingRevisionException` when a revision is already pending.
07	`updateByAdmin(userId, adminId, input)`	A method to update an expert profile directly and supersede pending revisions. – `userId`: `number` – target expert account. – `adminId`: `number` – current administrator. – `input`: `AdminUpdateExpertProfileType` – administrative profile changes. – *Returns:* `Promise<MessageResType>` – a success message. – Throws `ExpertProfileNotFoundException` or `NotAnExpertAccountException`.

4.69.2 ClassAttribute

No	Attribute	Type
01	`repository`	`ExpertProfileRepository` – private readonly persistence dependency.
02	`revisions`	`ProfileRevisionService` – private readonly revision workflow dependency.

4.70 ExpertProfileRepository Class

4.70.1 ClassMethod

No	Method	Description
01	`ExpertProfileRepository( prisma: PrismaService)`	A method to initialize the repository with the shared Prisma client. – `prisma`: `PrismaService` – database client. – *Returns:* `void` – constructor, no return value.
02	`findByUserId(userId: number)`	A method to load an expert account with its profile and roles. – `userId`: `number` – target account. – *Returns:* `Promise<user context | null>` – the account graph, or `null`.
03	`updateDirect(userId, adminId, input, profile CompletionPercent)`	A method to supersede pending revisions, update the profile, and upsert the expert row in one transaction. – `userId`: `number` – target expert account. – `adminId`: `number` – reviewing administrator. – `input`: `AdminUpdateExpertProfileType` – sanitized profile changes. – `profileCompletionPercent`: `number` – recalculated completeness. – *Returns:* `Promise<void>` – no value. – Creates a system notification when a pending revision was superseded.

No	Method	Description
04	`hasPendingRevision(userId: number)`	A method to check for a pending expert revision. – `userId: number` – target account. – *Returns:* `Promise<{ id: number } | null>` – the pending revision, or `null`.
05	`findPublicMany(query: PublicExpertQueryType)`	A method to query active public expert profiles. – `query: PublicExpertQueryType` – expertise, search, and pagination filters. – *Returns:* `Promise<public expert page>` – public expert cards.
06	`findPublicById(id: number)`	A method to load one active public expert profile. – `id: number` – expert identifier. – *Returns:* `Promise<public expert | null>` – the public profile, or `null`.

4.70.2 ClassAttribute

No	Attribute	Type
01	`prisma`	`PrismaService` – private readonly database client.

4.71 JobCategoriesAdminController Class

4.71.1 ClassMethod

No	Method	Description
01	`JobCategoriesAdminController(service: JobCategoriesAdminService)`	A method to initialize the controller with the job category service. – `service: JobCategoriesAdminService` – service that owns job category use cases. – *Returns:* `void` – constructor, no return value.

No	Method	Description
02	`listJobCategories(query: JobCategoryAdminQueryDto)`	A method to return a filtered, paginated job category list. – `query: JobCategoryAdminQueryDto` – page, limit, search, status, deleted, and sort options. – *Returns:* `Promise<JobCategoryAdminListResponseType>` – categories and pagination. – Route: `GET /admin/job-categories`.
03	`createJobCategory(body: CreateJobCategoryBodyDto)`	A method to create a job category. – `body: CreateJobCategoryBodyDto` – name, description, icon, and status. – *Returns:* `Promise<JobCategoryAdminDetailResponseType>` – the created category. – Route: `POST /admin/job-categories`.
04	`updateJobCategory(id: number, body: UpdateJobCategoryBodyDto)`	A method to update a job category. – `id: number` – category identifier. – `body: UpdateJobCategoryBodyDto` – optional category fields. – *Returns:* `Promise<JobCategoryAdminDetailResponseType>` – the updated category. – Route: `PATCH /admin/job-categories/:id`.
05	`restoreJobCategory(id: number)`	A method to restore a soft-deleted job category. – `id: number` – category identifier. – *Returns:* `Promise<JobCategoryAdminDetailResponseType>` – the restored category. – Route: `PATCH /admin/job-categories/:id/restore`.
06	`deleteJobCategory(id: number)`	A method to soft-delete a job category that has no active jobs. – `id: number` – category identifier. – *Returns:* `Promise<JobCategoryAdminDeleteResponseType>` – a success message. – Route: `DELETE /admin/job-categories/:id`.
07	`getJobCategoryDetail(id: number)`	A method to return one job category. – `id: number` – category identifier. – *Returns:* `Promise<JobCategoryAdminDetailResponseType>` – the category detail. – Route: `GET /admin/job-categories/:id`.

4.71.2 ClassAttribute

No	Attribute	Type
01	`service`	`JobCategoriesAdminService` – private readonly job category service.

4.72 JobCategoriesAdminService Class

4.72.1 ClassMethod

No	Method	Description
01	`JobCategoriesAdmin Service( repository: Job CategoriesAdmin Repository)`	A method to initialize the service with the job category repository. – `repository: JobCategoriesAdminRepository` – persistence dependency. – *Returns:* `void` – constructor, no return value.
02	`listJobCategories(query: JobCategoryAdminQuery Type)`	A method to load a job category page. – `query: JobCategoryAdminQueryType` – list filters. – *Returns:* `Promise<JobCategoryAdminListResponse Type>` – categories and pagination. – Failure is mapped to `FailedToLoadJobCategoryList Exception`.
03	`getJobCategoryDetail(id: number)`	A method to load one job category. – `id: number` – category identifier. – *Returns:* `Promise<JobCategoryAdminDetail ResponseType>` – the category detail. – Throws `JobCategoryAdminNotFoundException` or `FailedToLoadJobCategoryDetailException`.
04	`createJobCategory(body: AdminCreateJobCategory BodyType)`	A method to create a category and handle a concurrent duplicate. – `body: AdminCreateJobCategoryBodyType` – category fields. – *Returns:* `Promise<JobCategoryAdminDetail ResponseType>` – the created category. – Throws `JobCategoryNameAlreadyExistsException` or `FailedToCreateJobCategoryException`.

No	Method	Description
05	`updateJobCategory(id, body)`	A method to update a category and handle a concurrent duplicate. – `id: number` – category identifier. – `body: AdminUpdateJobCategoryBodyType` – optional category fields. – *Returns:* `Promise<JobCategoryAdminDetailResponseType>` – the updated category. – Throws `JobCategoryAdminNotFoundException`, `JobCategoryNameAlreadyExistsException`, or `FailedToUpdateJobCategoryException`.
06	`deleteJobCategory(id: number)`	A method to soft-delete a category after checking active job references. – `id: number` – category identifier. – *Returns:* `Promise<JobCategoryAdminDeleteResponseType>` – a success message. – Throws `JobCategoryAdminNotFoundException`, `JobCategoryHasJobsException`, or `FailedToDeleteJobCategoryException`.
07	`restoreJobCategory(id: number)`	A method to restore a soft-deleted category. – `id: number` – category identifier. – *Returns:* `Promise<JobCategoryAdminDetailResponseType>` – the restored category. – Throws `JobCategoryAdminNotFoundException`, `JobCategoryNameAlreadyExistsException`, or `FailedToRestoreJobCategoryException`.

4.72.2 ClassAttribute

No	Attribute	Type
01	`repository`	`JobCategoriesAdminRepository` – private readonly persistence dependency.

4.73 JobCategoriesAdminRepository Class

4.73.1 ClassMethod

No	Method	Description
01	`JobCategoriesAdmin Repository(prisma: PrismaService)`	A method to initialize the repository with the shared Prisma client. – `prisma`: `PrismaService` – database client. – *Returns:* `void` – constructor, no return value.
02	`listJobCategories(query: JobCategoryAdminQuery Type)`	A method to query categories with search, status, deleted-state, sorting, and pagination. – `query`: `JobCategoryAdminQueryType` – list filters. – *Returns:* `Promise<JobCategoryAdminListResponse Type>` – categories and pagination. – Search matches name, slug, or description.
03	`getJobCategoryDetail(id: number)`	A method to load one category with its job count. – `id`: `number` – category identifier. – *Returns:* `Promise<JobCategoryAdminDetail ResponseType | null>` – the category, or `null`.
04	`createJobCategory(data)`	A method to generate a unique slug and insert a category. – `data`: `AdminCreateJobCategoryBodyType` – category fields. – *Returns:* `Promise<JobCategoryAdminDetail ResponseType>` – the created category. – Throws `JobCategoryNameAlreadyExistsException`.
05	`updateJobCategory(id, data)`	A method to update a category and regenerate its slug when the name changes. – `id`: `number` – category identifier. – `data`: `AdminUpdateJobCategoryBodyType` – optional category fields. – *Returns:* `Promise<JobCategoryAdminDetail ResponseType>` – the updated category. – Throws `JobCategoryAdminNotFoundException` or `JobCategoryNameAlreadyExistsException`.
06	`deleteJobCategory(id: number)`	A method to check active job references and set the delete timestamp. – `id`: `number` – category identifier. – *Returns:* `Promise<JobCategoryAdminDelete ResponseType>` – a success message. – Throws `JobCategoryAdminNotFoundException` or `JobCategoryHasJobsException`.

No	Method	Description
07	`restoreJobCategory(id: number)`	A method to clear the delete timestamp after checking name conflicts. – `id: number` – category identifier. – *Returns:* `Promise<JobCategoryAdminDetail ResponseType>` – the restored category. – Throws `JobCategoryAdminNotFoundException` or `JobCategoryNameAlreadyExistsException`.
08	`generateUniqueSlug(name: string)`	A private method to derive a slug and append a numeric suffix on collision. – `name: string` – category name. – *Returns:* `Promise<string>` – the unique slug.

4.73.2 ClassAttribute

No	Attribute	Type
01	`prisma`	`PrismaService` – private readonly database client.